

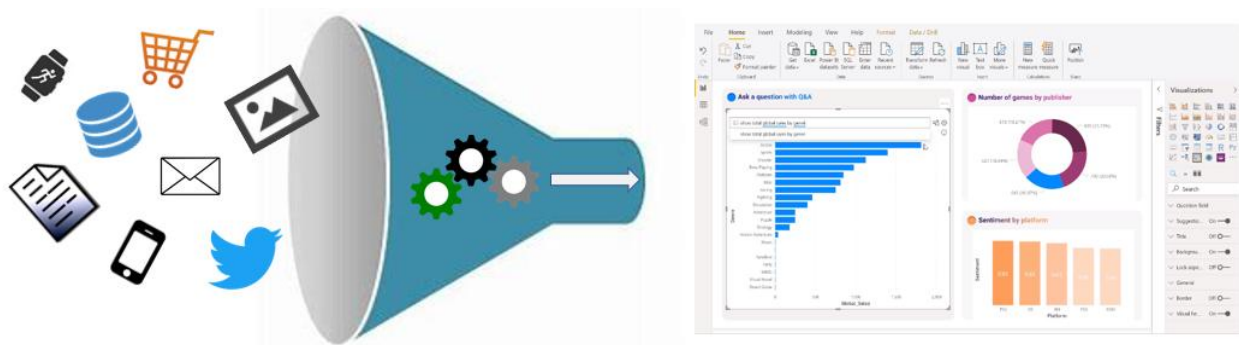
Power BI Data Analyst Associate

Get started with Microsoft data analytics

Discover data analysis

Introduction

As a data analyst, you are on a journey. **Think about all the data that is being generated** each day and that is available in an organization, from transactional data in a traditional database, telemetry data from services that you use, to signals that you get from different areas like social media.



For example, today's **retail businesses collect** and store massive amounts of **data** that track the items you **browsed and purchased**, the **pages you've visited** on their site, the **aisles you purchase** products from, your **spending habits**, and much more.

With data and information as the most strategic asset of a business, the underlying **challenge that organizations have today is understanding and using their data to positively effect change within the business**. Businesses continue to struggle to use their data in a meaningful and productive way, which impacts their ability to act.

A **retail business** should be able to use their vast amounts of data and information in such a way that impacts the business, including:

- **Tracking inventory**
- **Identifying purchase habits**
- **Detecting user trends and patterns**
- **Recommending purchases**
- **Determining price optimizations**
- **Identifying and stopping fraud**

Additionally, you might be looking for daily/monthly sale patterns. Common data segments that you might want to examine include [day-over-day](#), [week-over-week](#), and [month-over-month](#) so that you can compare how sales have been to where they were in the same week last year, for example.

[The key to unlocking this data is being able to tell a story with it.](#) In today's highly competitive and fast-paced business world, crafting reports that tell that story is what helps business leaders take action on the data. [Business decision makers depend on an accurate story to drive better business decisions.](#) The faster a business can make precise decisions, the more competitive they will be and the better advantage they will have. Without the story, it is difficult to understand what the data is trying to tell you.

[However, having data alone is not enough. You need to be able to act on the data](#) to effect change within the business. That action could [involve reallocating resources](#) within the business to accommodate a need, or it could be [identifying a failing campaign](#) and [knowing when to change course](#). These situations are where telling a story with your data is important.

The underlying [challenge that businesses face today is understanding and using their data](#) in such a way that impacts their business and ultimately their bottom line. You need to be able to look at the data and facilitate trusted business decisions. Then, [you need the ability to look at metrics and clearly understand the meaning behind those metrics.](#)

This requirement might seem daunting, but it's a task that you can accomplish. [Your first step is to partner with data experts](#) within your organization, such as [data engineers and data scientists](#), to help get the data that you need to tell that story. Ask these experts to participate in that data journey with you.

Your journey of telling a story with data also ties into building that data culture within your organization. [While telling the story is important, where that story is told is also crucial,](#) ensuring that the story is told to the right people. Also, [make sure that people can discover the story, that they know where to find it, and that it is part of the regular interactions.](#)

Data analysis exists to help overcome these challenges and pain points, ultimately [assisting businesses in finding insights and uncovering hidden value in troves of data through storytelling.](#) As you read on, you will learn how to use and apply analytical skills to go beyond a single report and help impact and influence your organization by telling stories with data and driving that data culture.

Overview of data analysis

Before data can be used to tell a story, it must be run through a process that makes it usable in the story. [Data analysis](#) is the [process of identifying, cleaning, transforming, and modeling](#)

data to discover meaningful and useful information. The data is then crafted into a story through reports for analysis to support the critical decision-making process.



As the world becomes more data-driven, storytelling through data analysis is becoming a vital component and aspect of large and small businesses. It is the reason that organizations continue to hire data analysts.

Data-driven businesses make decisions based on the story that their data tells, and in today's data-driven world, data is not being used to its full potential, a challenge that most businesses face. Data analysis is, and should be, a critical aspect of all organizations to help determine the impact to their business, including evaluating customer sentiment, performing market and product research, and identifying trends or other data insights.

While the process of data analysis focuses on the tasks of cleaning, modeling, and visualizing data, the concept of data analysis and its importance to business should not be understated. To analyze data, core components of analytics are divided into the following categories:

- Descriptive
- Diagnostic
- Predictive
- Prescriptive
- Cognitive

Descriptive analytics

Descriptive analytics help answer questions about what has happened based on historical data. Descriptive analytics techniques summarize large semantic models to describe outcomes to stakeholders.

By developing key performance indicators (KPIs), these strategies can help track the success or failure of key objectives. Metrics such as return on investment (ROI) are used in many industries, and specialized metrics are developed to track performance in specific industries.

An example of descriptive analytics is generating reports to provide a view of an organization's sales and financial data.

Diagnostic analytics

Diagnostic analytics help answer questions about why events happened. Diagnostic analytics techniques supplement basic descriptive analytics, and they use the findings from descriptive analytics to discover the cause of these events. Then, performance indicators are further investigated to discover why these events improved or became worse. Generally, this process occurs in three steps:

1. Identify anomalies in the data. These anomalies might be unexpected changes in a metric or a particular market.
2. Collect data that's related to these anomalies.
3. Use statistical techniques to discover relationships and trends that explain these anomalies.

Predictive analytics

Predictive analytics help answer questions about what will happen in the future. Predictive analytics techniques use historical data to identify trends and determine if they're likely to recur. Predictive analytical tools provide valuable insight into what might happen in the future. Techniques include a variety of statistical and machine learning techniques such as neural networks, decision trees, and regression.

Prescriptive analytics

Prescriptive analytics help answer questions about which actions should be taken to achieve a goal or target. By using insights from prescriptive analytics, organizations can make data-driven decisions. This technique allows businesses to make informed decisions in the face of uncertainty. Prescriptive analytics techniques rely on machine learning as one

of the strategies to find patterns in large semantic models. By analyzing past decisions and events, organizations can estimate the likelihood of different outcomes.

Cognitive analytics

Cognitive analytics attempt to draw inferences from existing data and patterns, derive conclusions based on existing knowledge bases, and then add these findings back into the knowledge base for future inferences, a self-learning feedback loop. Cognitive analytics help you learn what might happen if circumstances change and determine how you might handle these situations.

Inferences aren't structured queries based on a rules database; rather, they're unstructured hypotheses that are gathered from several sources and expressed with varying degrees of confidence. Effective cognitive analytics depend on machine learning algorithms, and will use several natural language processing concepts to make sense of previously untapped data sources, such as call center conversation logs and product reviews.

Example

By enabling reporting and data visualizations, a retail business uses descriptive analytics to look at patterns of purchases from previous years to determine what products might be popular next year. The company might also look at supporting data to understand why a particular product was popular and if that trend is continuing, which will help them determine whether to continue stocking that product.

A business might determine that a certain product was popular over a specific timeframe. Then, they can use this analysis to determine whether certain marketing efforts or online social activities contributed to the sales increase.

An underlying facet of data analysis is that a business needs to trust its data. As a practice, the data analysis process will capture data from trusted sources and shape it into something that is consumable, meaningful, and easily understood to help with the decision-making process. Data analysis enables businesses to fully understand their data through data-driven processes and decisions, allowing them to be confident in their decisions.

As the amount of data grows, so does the need for data analysts. A data analyst knows how to organize information and distill it into something relevant and comprehensible. A data analyst knows how to gather the right data and what to do with it, in other words, making sense of the data in your data overload.

Roles in data

Telling a story with the data is a journey that usually doesn't start with you. The data must come from somewhere. **Getting that data** into a place that is usable by you takes effort that **is likely out of your scope**, especially in consideration of the enterprise.

Today's applications and projects can be large and intricate, often involving the use of skills and knowledge from numerous individuals. Each person brings a unique talent and expertise, sharing in the effort of working together and coordinating tasks and responsibilities to see a project through from concept to production.

In the recent past, roles such as business analysts and business intelligence developers were the standard for data processing and understanding. However, excessive expansion of the size and different types of data has caused these roles to evolve into more specialized sets of skills that modernize and streamline the processes of data engineering and analysis.



Business Analyst



Data Analyst



Data Engineer



Data Scientist



Database
Administrator

The following sections highlight these different roles in data and the specific responsibility in the overall spectrum of data discovery and understanding:

- Business analyst
- Data analyst
- Data engineer
- Data scientist
- Database administrator

Business analyst

While some similarities exist between a data analyst and business analyst, the key differentiator between the two roles is what they do with data. A **business analyst** is **closer to the business and is a specialist in interpreting the data that comes from the visualization.**

Often, the roles of data analyst and business analyst could be the responsibility of a single person.

Data analyst

A data analyst enables businesses to maximize the value of their data assets through visualization and reporting tools such as Microsoft Power BI. Data analysts are responsible for profiling, cleaning, and transforming data. Their responsibilities also include designing and building scalable and effective semantic models, and enabling and implementing the advanced analytics capabilities into reports for analysis. A data analyst works with the pertinent stakeholders to identify appropriate and necessary data and reporting requirements, and then they are tasked with turning raw data into relevant and meaningful insights.

A data analyst is also responsible for the management of Power BI assets, including reports, dashboards, workspaces, and the underlying semantic models that are used in the reports. They are tasked with implementing and configuring proper security procedures, in conjunction with stakeholder requirements, to ensure the safekeeping of all Power BI assets and their data.

Data analysts work with data engineers to determine and locate appropriate data sources that meet stakeholder requirements. Additionally, data analysts work with the data engineer and database administrator to ensure that the analyst has proper access to the needed data sources. The data analyst also works with the data engineer to identify new processes or improve existing processes for collecting data for analysis.

Data engineer

Data engineers provision and set up data platform technologies that are on-premises and in the cloud. They manage and secure the flow of structured and unstructured data from multiple sources. The data platforms that they use can include relational databases, nonrelational databases, data streams, and file stores. Data engineers also ensure that data services securely and seamlessly integrate across data platforms.

Primary responsibilities of data engineers include the use of on-premises and cloud data services and tools to ingest, egress, and transform data from multiple sources. Data engineers collaborate with business stakeholders to identify and meet data requirements. They design and implement solutions.

While some alignment might exist in the tasks and responsibilities of a data engineer and a database administrator, a data engineer's scope of work goes well beyond looking after a

database and the server where it's hosted and likely doesn't include the overall operational data management.

A data engineer adds tremendous value to business intelligence and data science projects. When the data engineer brings data together, often described as data wrangling, projects move faster because data scientists can focus on their own areas of work.

As a data analyst, you would work closely with a data engineer in making sure that you can access the variety of structured and unstructured data sources because they will support you in optimizing semantic models, which are typically served from a modern data warehouse or data lake.

Both database administrators and business intelligence professionals can transition to a data engineer role; they need to learn the tools and technology that are used to process large amounts of data.

Data scientist

Data scientists perform advanced analytics to extract value from data. Their work can vary from descriptive analytics to predictive analytics. Descriptive analytics evaluate data through a process known as exploratory data analysis (EDA). Predictive analytics are used in machine learning to apply modeling techniques that can detect anomalies or patterns. These analytics are important parts of forecast models.

Descriptive and predictive analytics are only partial aspects of data scientists' work. Some data scientists might work in the realm of deep learning, performing iterative experiments to solve a complex data problem by using customized algorithms.

Anecdotal evidence suggests that most of the work in a data science project is spent on data wrangling and feature engineering. Data scientists can speed up the experimentation process when data engineers use their skills to successfully wrangle data.

On the surface, it might seem that a data scientist and data analyst are far apart in the work that they do, but this conjecture is untrue. A data scientist looks at data to determine the questions that need answers and will often devise a hypothesis or an experiment and then turn to the data analyst to assist with the data visualization and reporting.

Database administrator

A database administrator implements and manages the operational aspects of cloud-native and hybrid data platform solutions that are built on Microsoft Azure data services and Microsoft SQL Server. A database administrator is responsible for the overall availability and consistent performance and optimizations of the database solutions. They work with

stakeholders to identify and implement the policies, tools, and processes for data backup and recovery plans.

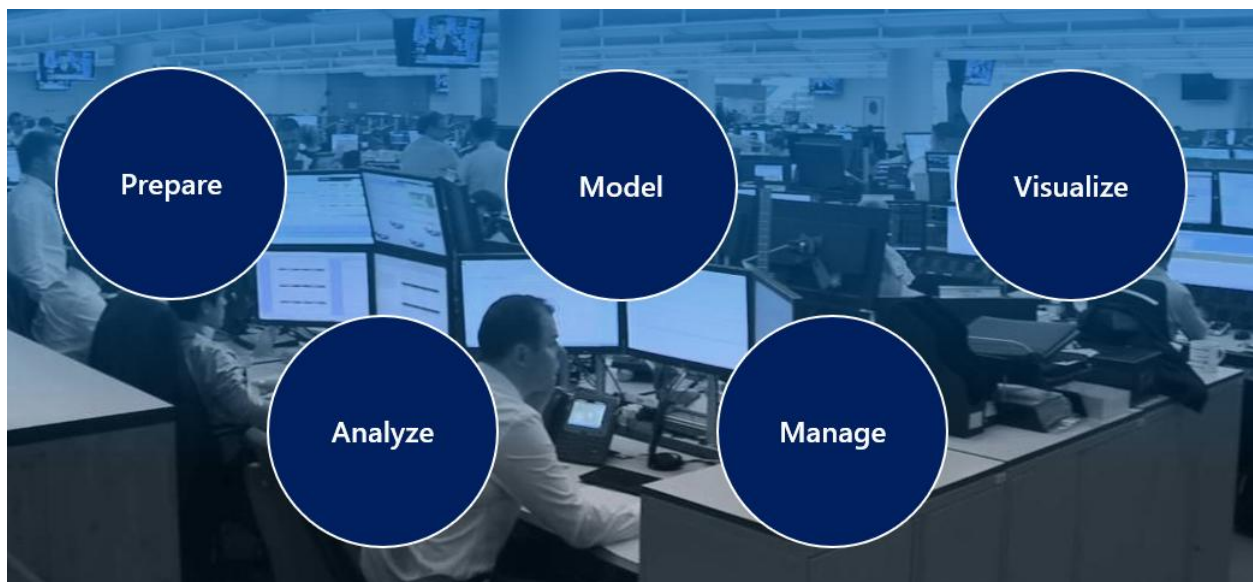
The role of a database administrator is different from the role of a data engineer. A database administrator monitors and manages the overall health of a database and the hardware that it resides on, whereas a data engineer is involved in the process of data wrangling, in other words, ingesting, transforming, validating, and cleaning data to meet business needs and requirements.

The database administrator is also responsible for managing the overall security of the data, granting and restricting user access and privileges to the data as determined by business needs and requirements.

Tasks of a data analyst

A data analyst is one of several critical roles in an organization, who help uncover and make sense of information to keep the company balanced and operating efficiently. Therefore, it's vital that a data analyst clearly understands their responsibilities and the tasks that are performed on a near-daily basis. Data analysts are essential in helping organizations gain valuable insights into the expanse of data that they have, and they work closely with others in the organization to help reveal valuable information.

The following figure shows the five key areas that you'll engage in during the data analysis process.



Prepare

As a data analyst, you'll likely divide most of your time between the prepare and model tasks. Deficient or incorrect data can have a major impact that results in invalid reports, a loss of trust, and a negative effect on business decisions, which can lead to loss in revenue, a negative business impact, and more.

Before a report can be created, data must be prepared. Data preparation is the process of profiling, cleaning, and transforming your data to get it ready to model and visualize.

Data preparation is the process of taking raw data and turning it into information that is trusted and understandable. It involves, among other things, ensuring the integrity of the data, correcting wrong or inaccurate data, identifying missing data, converting data from one structure to another or from one type to another, or even a task as simple as making data more readable.

Data preparation also involves understanding how you're going to get and connect to the data and the performance implications of the decisions. When connecting to data, you need to make decisions to ensure that models and reports meet, and perform to, acknowledged requirements and expectations.

Privacy and security assurances are also important. These assurances can include anonymizing data to avoid oversharing or preventing people from seeing personally identifiable information when it isn't needed. Alternatively, helping to ensure privacy and security can involve removing that data completely if it doesn't fit in with the story that you're trying to shape.

Data preparation can often be a lengthy process. Data analysts follow a series of steps and methods to prepare data for placement into a proper context and state that eliminate poor data quality and allow it to be turned into valuable insights.

Model

When the data is in a proper state, it's ready to be modeled. Data modeling is the process of determining how your tables are related to each other. This process is done by defining and creating relationships between the tables. From that point, you can enhance the model by defining metrics and adding custom calculations to enrich your data.

Creating an effective and proper semantic model is a critical step in helping organizations understand and gain valuable insights into the data. An effective semantic model makes reports more accurate, allows the data to be explored faster and efficiently, decreases time for the report writing process, and simplifies future report maintenance.

The model is another critical component that has a direct effect on the performance of your report and overall data analysis. A poorly designed model can have a drastically negative impact on the general accuracy and performance of your report. Conversely, a well-designed model with well-prepared data will ensure a properly efficient and trusted report. This notion is more prevalent when you are working with data at scale.

From a Power BI perspective, if your report is performing slowly, or your refreshes are taking a long time, you will likely need to revisit the data preparation and modeling tasks to optimize your report.

The process of preparing data and modeling data is an iterative process. Data preparation is the first task in data analysis. Understanding and preparing your data before you model it will make the modeling step much easier.

Visualize

The visualization task is where you get to bring your data to life. The ultimate goal of the visualize task is to solve business problems. A well-designed report should tell a compelling story about that data, which will enable business decision makers to quickly gain needed insights. By using appropriate visualizations and interactions, you can provide an effective report that guides the reader through the content quickly and efficiently, therefore allowing the reader to follow a narrative into the data.

The reports that are created during the visualization task help businesses and decision makers understand what that data means so that accurate and vital decisions can be made. Reports drive the overall actions, decisions, and behaviors of an organization that is trusting and relying on the information that is discovered in the data.

The business might communicate that they need all data points on a given report to help them make decisions. As a data analyst, you should take the time to fully understand the problem that the business is trying to solve. Determine whether all their data points are necessary because too much data can make detecting key points difficult. Having a small and concise data story can help find insights quickly.

With the built-in AI capabilities in Power BI, data analysts can build powerful reports, without writing any code, that enable users to get insights and answers and find actionable objectives. The AI capabilities in Power BI, such as the built-in AI visuals, enable the discovering of data by asking questions, using the Quick Insights feature, or creating machine learning models directly within Power BI.

An important aspect of visualizing data is designing and creating reports for accessibility. As you build reports, it is important to think about people who will be accessing and reading the

reports. Reports should be designed with accessibility in mind from the outset so that no special modifications are needed in the future.

Many components of your report will help with storytelling. From a color scheme that is complementary and accessible, to fonts and sizing, to picking the right visuals for what is being displayed, they all come together to tell that story.

Analyze

The analyze task is the important step of understanding and interpreting the information that is displayed on the report. In your role as a data analyst, you should understand the analytical capabilities of Power BI and use those capabilities to find insights, identify patterns and trends, predict outcomes, and then communicate those insights in a way that everyone can understand.

Advanced analytics enables businesses and organizations to ultimately drive better decisions throughout the business and create actionable insights and meaningful results. With advanced analytics, organizations can drill into the data to predict future patterns and trends, identify activities and behaviors, and enable businesses to ask the appropriate questions about their data.

Previously, analyzing data was a difficult and intricate process that was typically performed by data engineers or data scientists. Today, Power BI makes data analysis accessible, which simplifies the data analysis process. Users can quickly gain insights into their data by using visuals and metrics directly from their desktop and then publish those insights to dashboards so that others can find needed information.

This feature is another area where AI integrations within Power BI can take your analysis to the next level. Integrations with Azure Machine Learning, cognitive services, and built-in AI visuals will help to enrich your data and analysis.

Manage

Power BI consists of many components, including reports, dashboards, workspaces, semantic models, and more. As a data analyst, you are responsible for the management of these Power BI assets, overseeing the sharing and distribution of items, such as reports and dashboards, and ensuring the security of Power BI assets.

Apps can be a valuable distribution method for your content and allow easier management for large audiences. This feature also allows you to have custom navigation experiences and link to other assets within your organization to complement your reports.

The management of your content helps to foster collaboration between teams and individuals. Sharing and discovery of your content is important for the right people to get the answers that they need. It is also important to help ensure that items are secure. You want to make sure that the right people have access and that you are not leaking data past the correct stakeholders.

Proper management can also help reduce data silos within your organization. Data duplication can make managing and introducing data latency difficult when resources are overused. Power BI helps reduce data silos with the use of shared semantic models, and it allows you to reuse data that you have prepared and modeled. For key business data, endorsing a semantic model as certified can help to ensure trust in that data.

The management of Power BI assets helps reduce the duplication of efforts and helps ensure security of the data.

Knowledge check

Summary

In this module, you learned that the role of data analyst is vital to the success of an organization. Additionally, the tasks that data analysts perform help ensure that the business decisions are based on trusted data. You also learned about the different roles in data and how the people in these roles work closely with a data analyst to deliver valuable insights into a business's data assets.

Get started building with Power BI

Introduction

Microsoft Power BI is a complete reporting solution that offers data preparation, data visualization, distribution, and management through development tools and an online platform.

Power BI can scale from simple reports using a single data source to reports requiring complex data modeling and consistent themes. Use Power BI to create visually stunning, interactive reports to serve as the analytics and decision engine behind group projects, divisions, or entire organizations.

Power BI is an essential tool to data analysts and their organization; however, all data professionals benefit from understanding how Power BI works to explore and present data insights within organizations.

Use Power BI

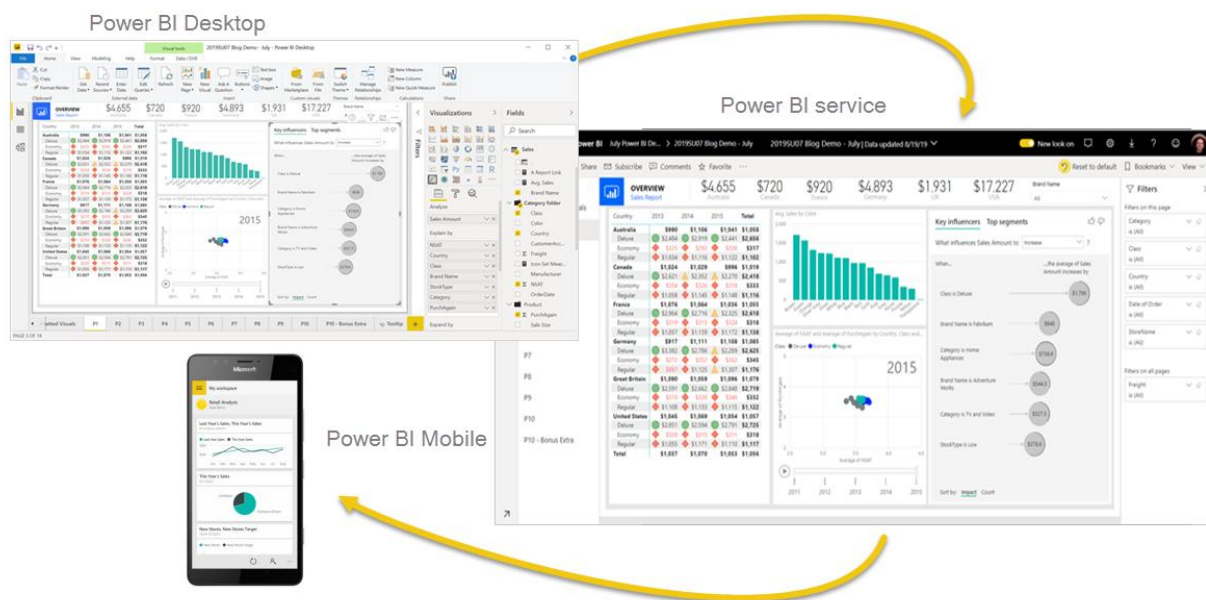
In order to create reports with Power BI, you must first understand the tools necessary. There are **three primary components** to Power BI:

- Power BI Desktop (desktop application)
- Power BI service (online platform)
- Power BI Mobile (cross-platform mobile app)

Power BI Desktop is the development tool **available to data analysts and other report creators**. While the **Power BI service** allows you to **organize, manage, and distribute your reports** and other Power BI items. Power BI Desktop is available to download for free either through the Windows store or directly online.

You can **access the Power BI service at app.powerbi.com** with a school or work account. If your organization doesn't already use Power BI, you can still explore the service by getting a free trial or signing up for a free **Microsoft 365 Developer account**.

Power BI Mobile **allows consumers to view reports in a mobile-optimized format**. You can create these optimized report views in Power BI Desktop.



Explore the flow of Power BI

There's a common flow when creating reports with Power BI. First, you **start with Power BI Desktop** to connect to data and create the report. Then you **publish the report to the Power BI service** and **distribute to consumers**.

The flow of Power BI is:

1. **Connect to data** with **Power BI Desktop**.
2. **Transform and model data** with Power BI Desktop.
3. **Create visualizations and reports** with Power BI Desktop.
4. **Publish** report to **Power BI service**.
5. **Distribute and manage** reports in the Power BI service.

The **Power BI service** also allows you to create high-level dashboards that drill down to reports, and apps to easily group related reports to users in a simple format.

Building blocks of Power BI

The **building blocks of Power BI** are **semantic models** and **visualizations**. Create a semantic model and then use visuals to build a report. Let's explore these items in more detail and how they relate to the flow of Power BI.

Create a semantic model

A **semantic model** consists of all connected data, transformations, relationships, and calculations. To follow the **flow of Power BI**, you first connect to data, transform data, and create relationships and calculations to create a semantic model.

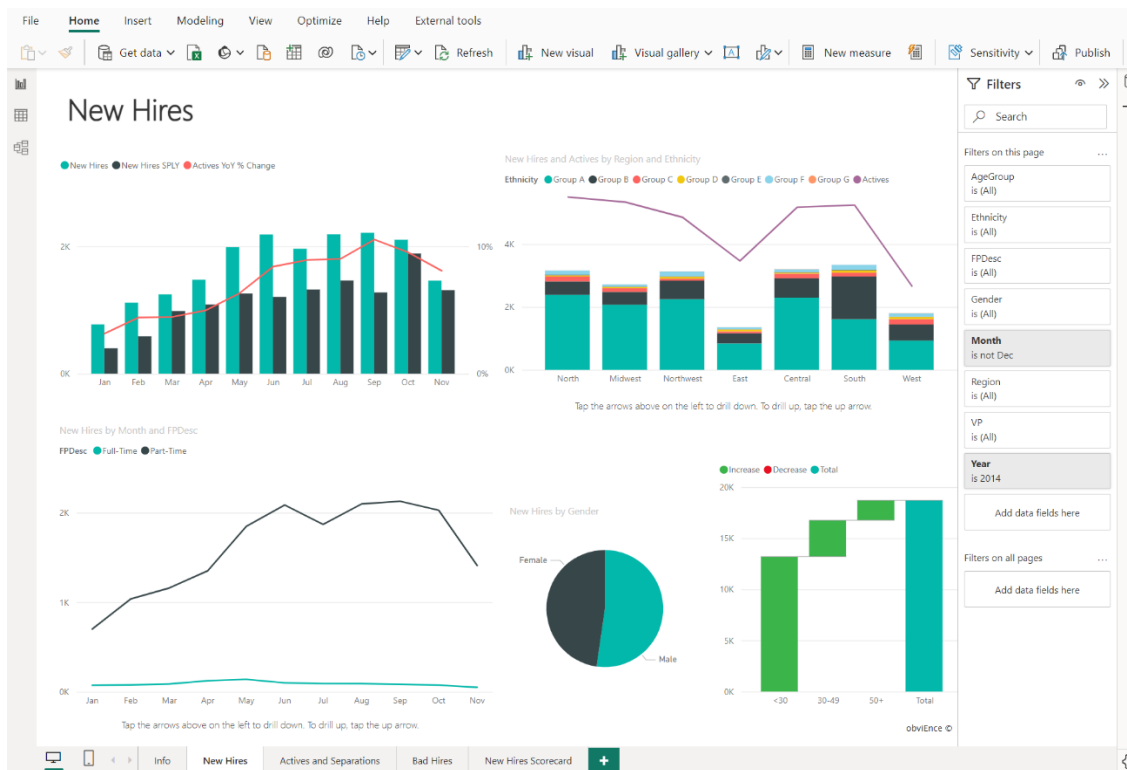
First, **connect to as many data sources you need**. Then **clean and transform** the data to your needs. **Add relationships between tables and calculations** to extend the semantic model. After all of that, **now you can create a report**.

Create visualizations in a report

In Power BI Desktop, when **you create a visualization** (also called *visual*), you **add it to the canvas** for a report page. **Choose your visualizations to build pages** in your report. It's ideal to **keep each page simple** with related data, **so consumers can easily see the insights**.

Power BI is a **low-code solution**, which means that you can **"drag and drop"** data field directly onto the canvas. **Power BI will choose a visual for your data field**. You can **easily change** between **visuals** for the same fields, and **add or remove data fields** to the visual.

One of the **most valuable features** of Power BI reports is the **interactivity between visuals**. **Consumers can select different data points in the visual and see how that affects the other visuals**. Depending on your design, they can also drillthrough from one visual to more detail or filter based on different fields in the report.

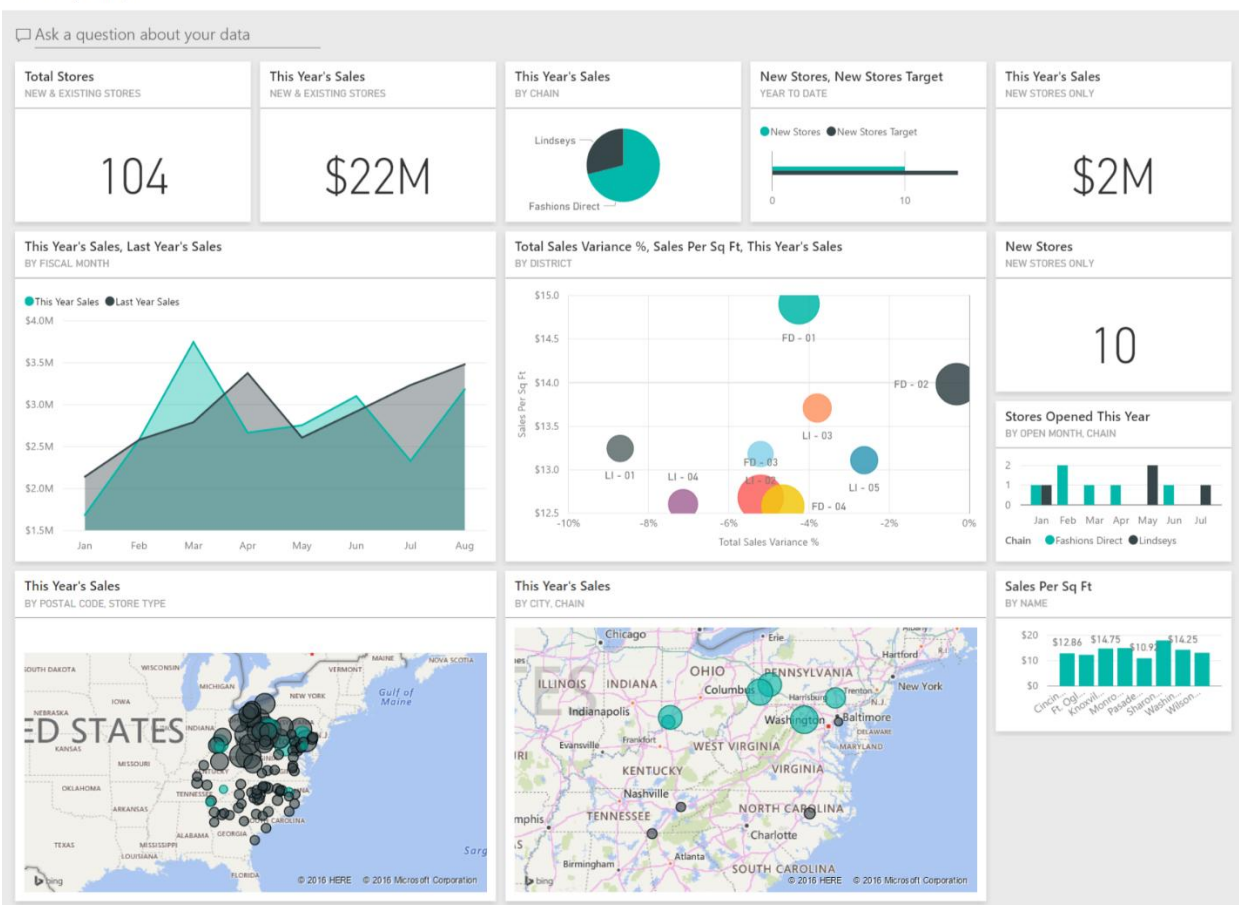


Once you're satisfied with your report, you publish it to the Power BI service.

Create a dashboard

In the Power BI service, you can also create **dashboards** after you've published a report. **Dashboards** consist of a single page made up of **tiles**. Add tiles to a dashboard by pinning a visual in a report to the dashboard. Tiles aren't interactive like visuals, so when a user interacts with the tile, they go to the underlying report for more information.

Dashboards are an excellent way to provide high-level information to consumers. Similar to a dashboard in a vehicle, include the most important information in a dashboard. Then consumers can go to the report for more details.



To recap, the building blocks of Power BI are **semantic models** and **visuals**. Using **Power BI Desktop**, you create the semantic model and use visuals to create reports.

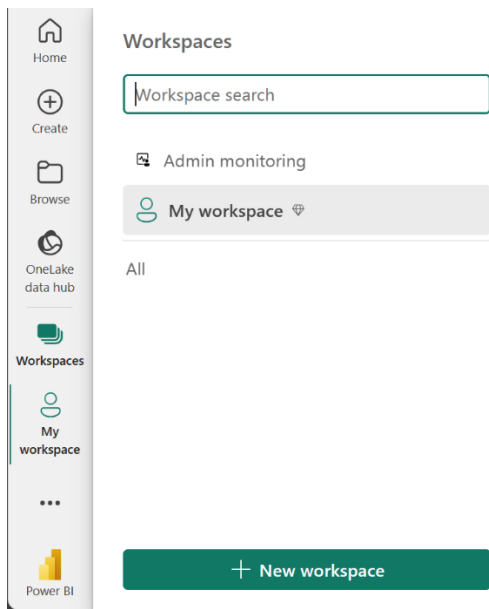
In the **Power BI service**, you can **distribute content to your consumers** and use reports to **create dashboards**.

Tour and use the Power BI service

Now that you understand how to create a report, let's explore the Power BI service. The **Power BI service** provides a **simple and interactive user experience** to take your data analytics to the next level.

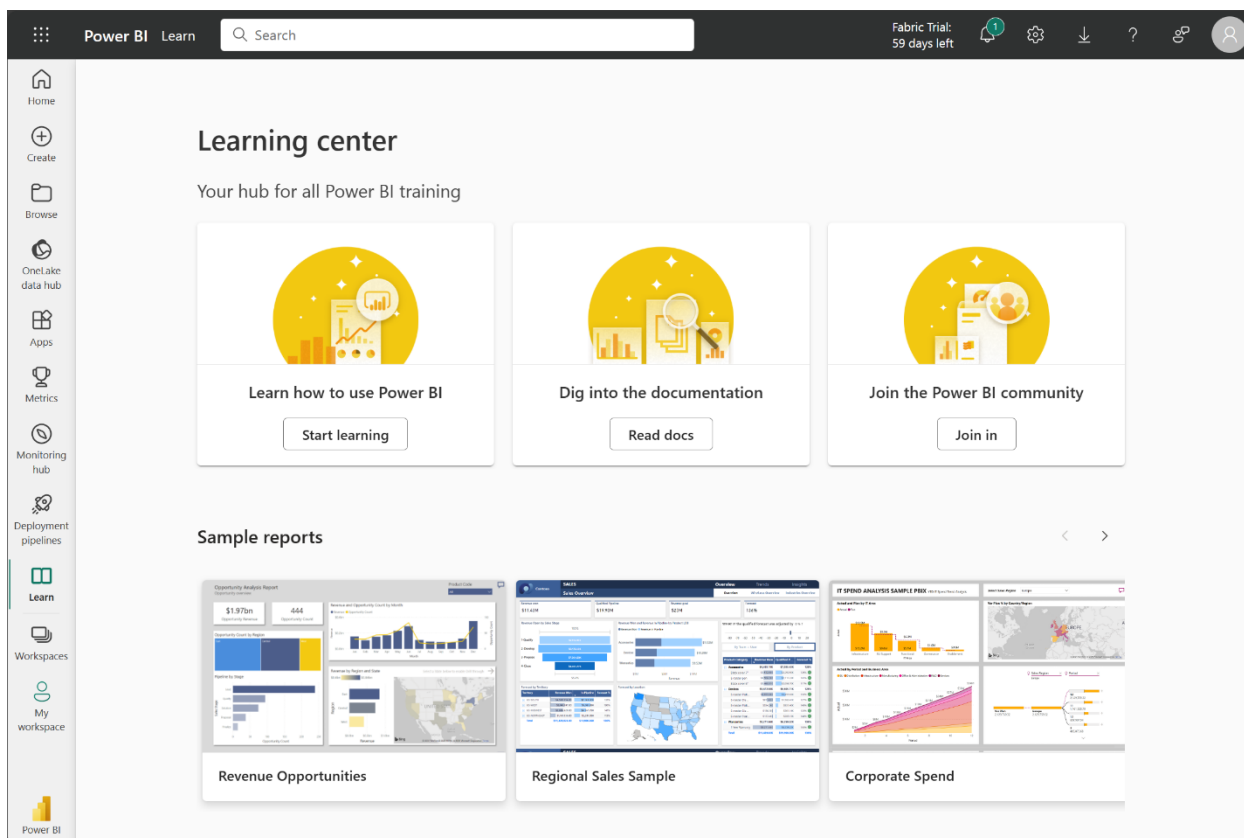
Organize items with workspaces

Workspaces are the **foundation of the Power BI service**. When publishing any report, you **must choose a workspace**. By default, every user has access to **My workspace**, which is **ideal only for testing**. When you **want to share content** with others, **always** create and use a **shared workspace**.



Explore sample reports

If you haven't created a report yet, **Power BI offers several sample reports** for you to explore. These reports **load to My workspace** so you can explore privately. You can **access sample reports in the Learn section** of the navigation pane.



Distribute content

In a workspace, you can create an **app**, which provides consumers a **simplified interface to access reports and dashboards**. In the app configuration, you set up the app, **select the content to include** (limited to the current workspace), and **choose your audience**.

Once you create an app, you **must update the app after each change** to items in the workspace. The requirement to update the app allows you **to control what version of the content is visible to your audience**.

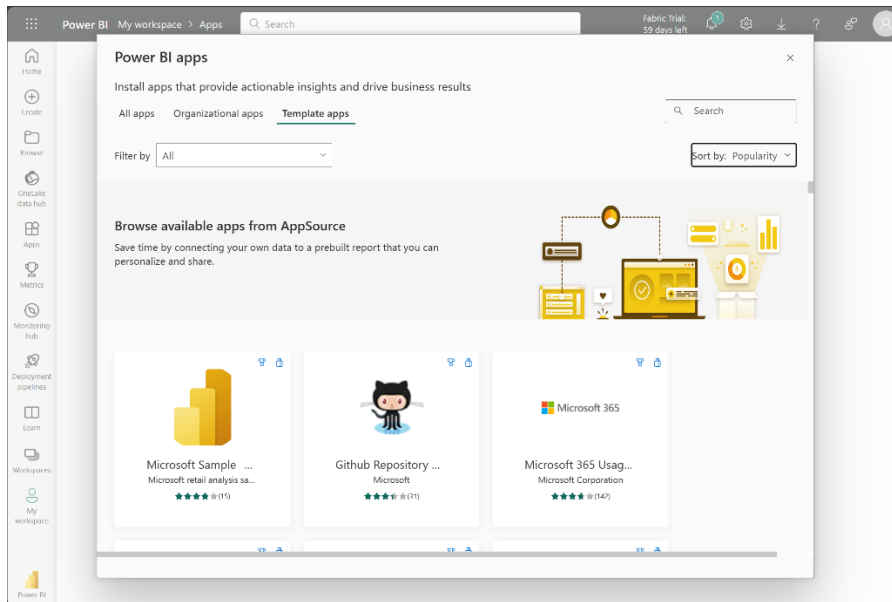
The screenshot shows the 'Power BI Apps' configuration interface. The top navigation bar includes 'Power BI Apps' and a search bar. The left sidebar lists various Power BI features: Home, Create, Browse, OneLake data hub, Apps, Metrics, Monitoring hub, Deployment pipelines, Learn, and Workspaces. The main content area is titled 'Build your app' and is divided into three steps: 1. Setup*, 2. Content, and 3. Audience. The 'Setup' step includes the following fields and options:

- App name ***: A text input field containing 'Demo App'.
- Description ***: A text area with the placeholder 'Enter a summary'.
- Describe your app.**: A red text prompt below the description field.
- App logo**: A section with a logo icon and buttons for 'Upload' and 'Delete'.
- App theme color**: A color selection dropdown menu.
- Contact information**: A section with three radio button options:
 - ☒ Show app publisher
 - ☐ Show items contacts from the workspace
 - ☐ Show specific individuals or groups
- Search bar**: A text input field with the placeholder 'Enter a name or email address'.
- Copy link**: A button with a link icon and the text 'Copy link'.
- Advanced settings**: A dropdown menu.

Apps are the **ideal sharing solution within any organization**. While you can grant access to the workspace, **workspace permissions may grant users access to more content than desired**. Sharing individual items also presents a problem **if you make changes you don't want consumers to see yet**.

Explore template apps

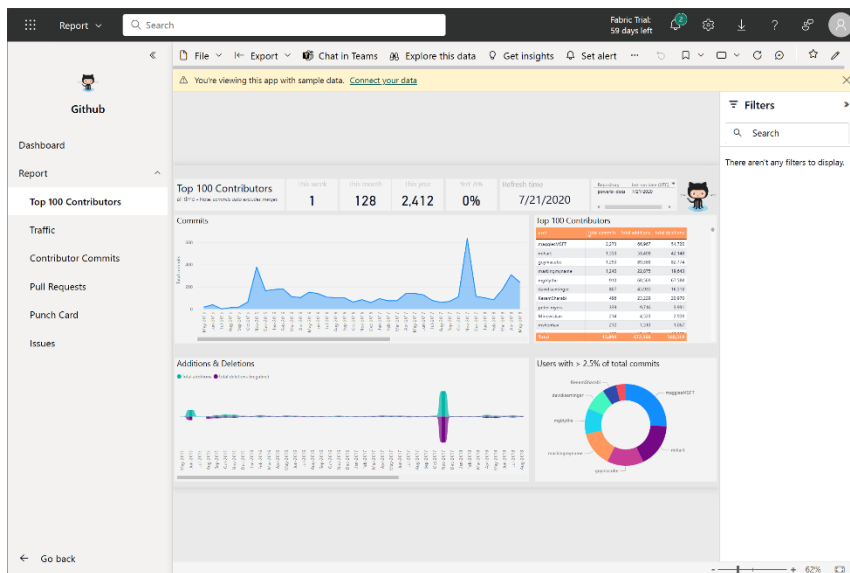
Now that you understand what an app is, let's look at **template apps**. Template apps allow you to **find an existing app that suits your needs** and then you connect your data. These apps can be a great way to quickly share insights with minimal effort.



Tip

To access template apps, select the Apps icon from the left **navigation pane > Get apps > Template apps**.

In the following screenshot, we've installed the GitHub template app and have expanded the report. We can see different report pages, including *Top 100 Contributors* and *Pull Requests*. If your organization is using GitHub, using this template app can easily support your needs without starting from the beginning.



Refresh a semantic model

In order to **support your ever-changing data**, you can **configure scheduled refreshes** of your semantic models in the Power BI service. **On-demand refreshes are also available.**

Home | General | Dashboards | **Semantic models** | Workbooks | Dataflows | Datamarts (Preview) | App

Home | Create | Browse | OneLake data hub | App | Metrics | Monitoring hub | Deployment pipelines | Learn | Workspaces | PL 300 | Power BI

Demo
Sales Analysis

Settings for Sales Analysis
[View semantic model](#)
 This semantic model has been configured by [angie@beepboopbop.onmicrosoft.com](#).
[Refresh history](#)
Semantic model description
 Gateway and cloud connections
 Data source credentials
 Parameters
 Query Caching
Refresh
 Configure a refresh schedule
 Define a data refresh schedule to import data from the data source into the semantic model. [Learn more](#)
☐ Off
 Refresh frequency
 Daily
 Time zone
 (UTC-05:00) Eastern Time (US and Ca...
 Time
[Add another time](#)
 Send refresh failure notifications to
☒ Semantic model owner
☐ These contacts:
 Enter email addresses
 Apply Discard

Prepare data for analysis with Power BI

Get data in Power BI

Introduction

Like most of us, you work for a company where you're required to build Microsoft Power BI reports. The **data resides in several different databases and files**. These data repositories are different from each other, some are in Microsoft SQL Server, some are in Microsoft Excel, but all the data is related.

[Video](#)

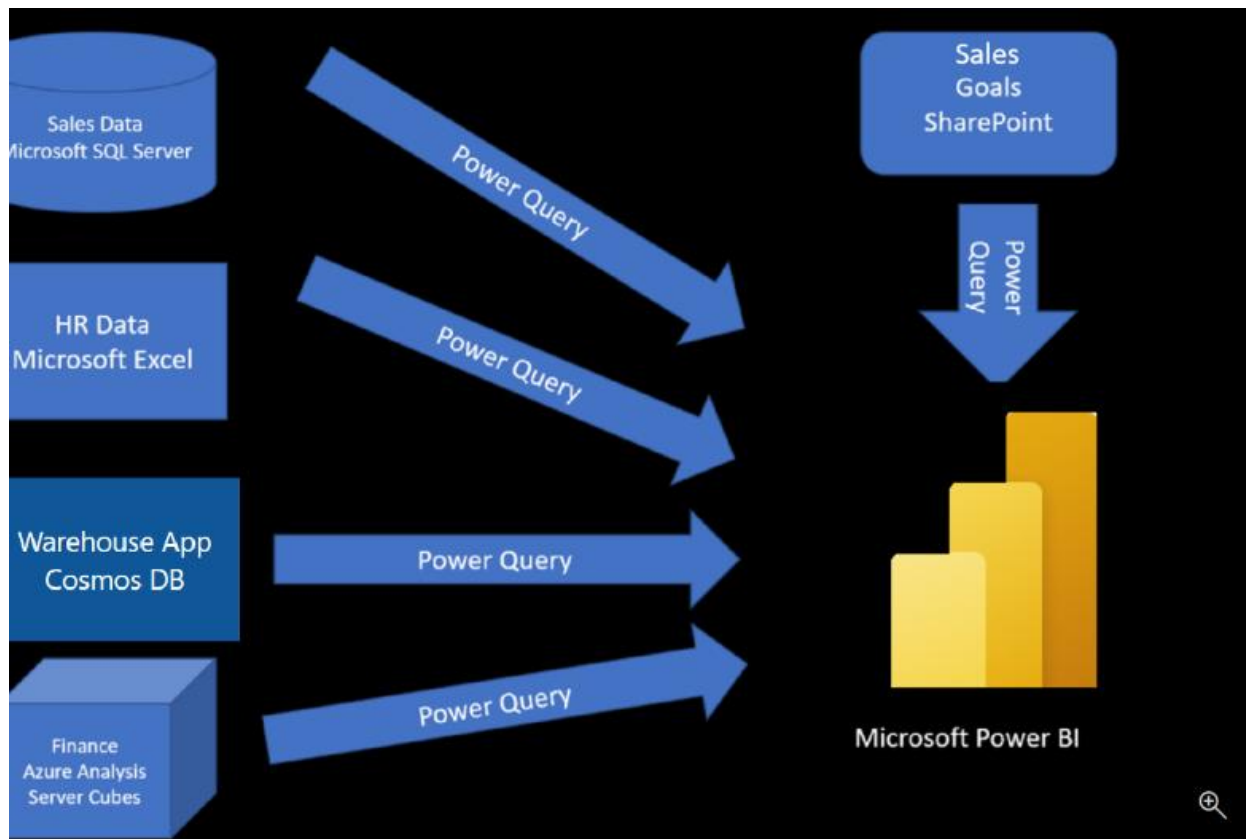
Note

The module sections prior to the lab are purely informational. You will be given the opportunity to work with real data during the lab.

In this module's scenario, you **work for Tailwind Traders**. You've been tasked by **senior leadership to create a suite of reports** that are dependent on data in several different locations. The database that **tracks sales transactions is in SQL Server, a relational database** that contains **what items each customer bought** and when. It **also tracks which employee made the sale**, along **with the employee name and employee ID**. However, that database **doesn't contain the employee's hire date, their title, or who their manager is**. For that information, you **need to access files that Human Resources keeps in Excel**. You've been **consistently requesting that they use an SQL database**, but **they haven't yet** had the **chance** to implement it.

When an item ships, **the shipment is recorded in the warehousing application**, which is new to the company. The developers chose to **store data in Cosmos DB, as a set of JSON documents**.

Tailwind Traders has an application that helps with financial projections, so that they can **predict what their sales will be in future months and years**, based on past trends. Those **projections are stored in Microsoft Azure Analysis Services**. Here's a view of the many data sources you're asked to combine data from.



Before you can create reports, you must first extract data from the various data sources. Interacting with SQL Server is different from Excel, so you should learn the nuances of both systems. After gaining understanding of the systems, you can use Power Query to help you clean the data, such as renaming columns, replacing values, removing errors, and combining query results. Power Query is also available in Excel. After the data has been cleaned and organized, you're ready to build reports in Power BI. Finally, you'll publish your combined semantic model and reports to Power BI service. From there, other people can use your semantic model and build their own reports or they can use the reports you've already built. Additionally, if someone else built a semantic model you'd like to use, you can build reports from that too!

This module will focus on the first step of getting the data from the different data sources and importing it into Power BI by using Power Query.

By the end of this module, you'll be able to:

- Identify and connect to a data source
- Get data from a relational database, such as Microsoft SQL Server
- Get data from a file, such as Microsoft Excel

- Get data from applications
- Get data from Azure Analysis Services
- Select a storage mode
- Fix performance issues
- Resolve data import errors

Get data from files

Organizations often export and store data in files. One possible file format is a flat file. A flat file is a type of file that has only one data table and every row of data is in the same structure. The file doesn't contain hierarchies. Likely, you're familiar with the most common types of flat files, which are comma-separated values (.csv) files, delimited text (.txt) files, and fixed width files. Another type of file would be the output files from different applications, like Microsoft Excel workbooks (.xlsx).



Power BI Desktop allows you to get data from many types of files. You can find a list of the available options when you use the **Get data** feature in Power BI Desktop. The following sections explain how you can import data from an Excel file that is stored on a local computer.

Scenario

The Human Resources (HR) team at Tailwind Traders has prepared a flat file that contains some of your organization's employee data, such as employee name, hire date, position, and

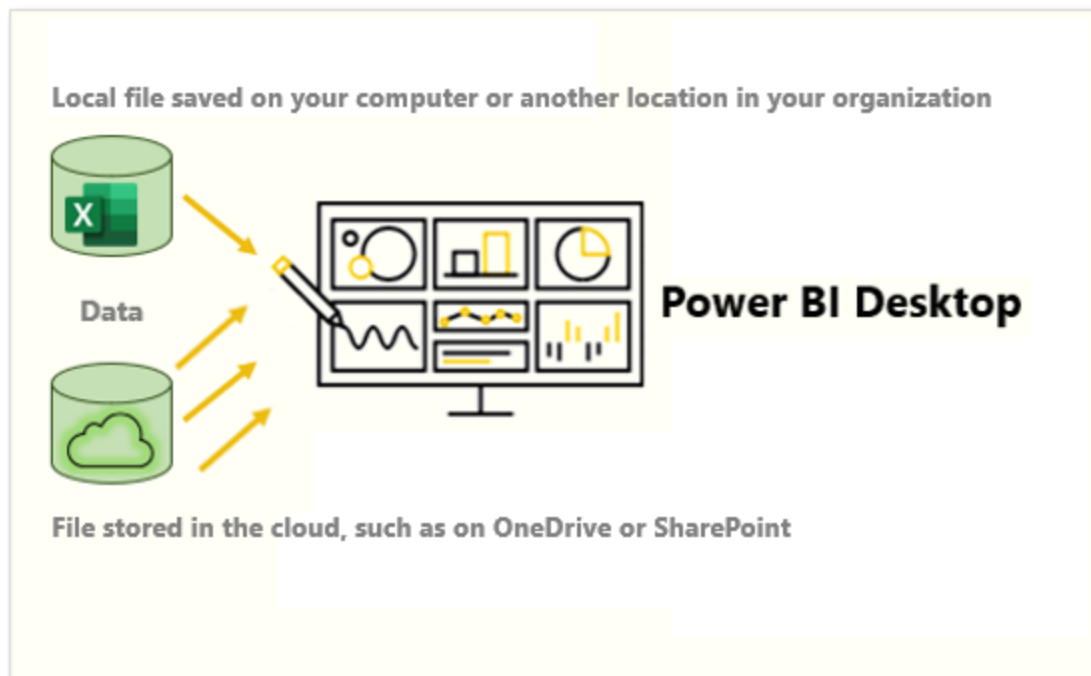
manager. They've requested that you build Power BI reports by using this data, and data that is located in several other data sources.

Flat file location

The first step is to determine which file location you want to use to export and store your data.

Your Excel files might exist in one of the following locations:

- **Local** - You can import data from a local file into Power BI. The file isn't moved into Power BI, and a link doesn't remain to it. Instead, a new semantic model is created in Power BI, and data from the Excel file is loaded into it. Accordingly, changes to the original Excel file aren't reflected in your Power BI semantic model. You can use local data import for data that doesn't change.
- **OneDrive for Business** - You can pull data from OneDrive for Business into Power BI. This method is effective in keeping an Excel file and your semantic model, reports, and dashboards in Power BI synchronized. Power BI connects regularly to your file on OneDrive. If any changes are found, your semantic model, reports, and dashboards are automatically updated in Power BI.
- **OneDrive - Personal** - You can use data from files on a personal OneDrive account, and get many of the same benefits that you would with OneDrive for Business. However, you'll need to sign in with your personal OneDrive account, and select the **Keep me signed in** option. Check with your system administrator to determine whether this type of connection is allowed in your organization.
- **SharePoint - Team Sites** - Saving your Power BI Desktop files to SharePoint Team Sites is similar to saving to OneDrive for Business. The main difference is how you connect to the file from Power BI. You can specify a URL or connect to the root folder.



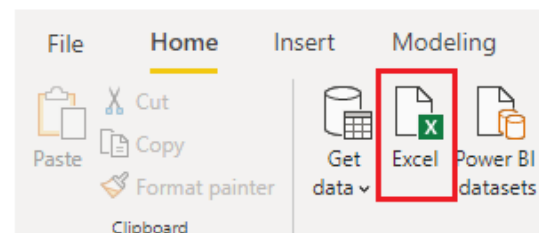
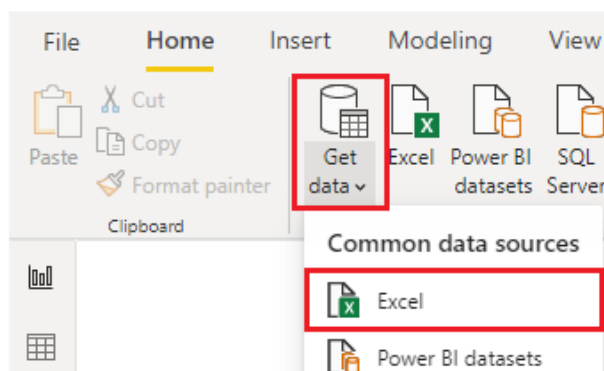
Using a cloud option such as [OneDrive or SharePoint Team Sites](#) is the most effective way to [keep your file](#) and your semantic model, reports, and dashboards in Power BI [in-sync](#). However, [if your data doesn't change regularly, saving files on a local computer](#) is a suitable option.

Connect to data in a file

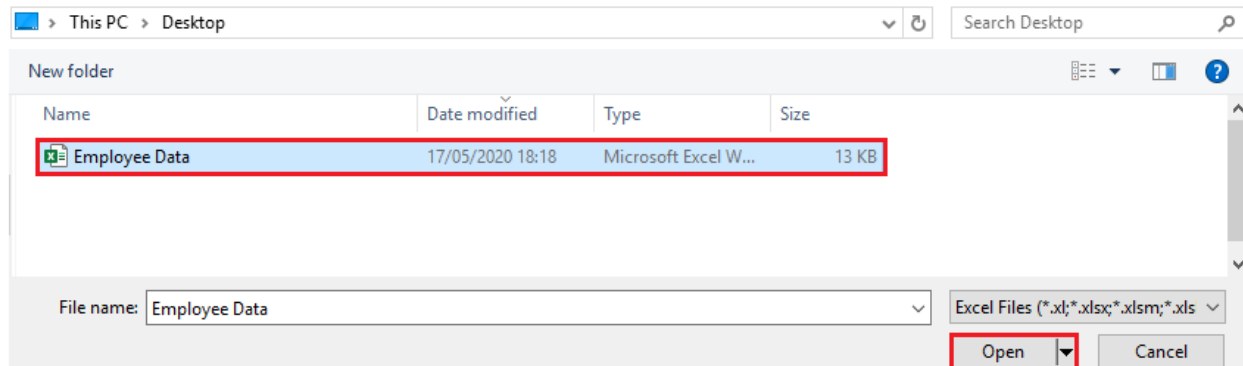
In Power BI, on the **Home** tab, select **Get data**. In the list that displays, **select the option that you require**, such as **Text/CSV** or **XML**. For this example, you'll select **Excel**.

Tip

The **Home** tab contains quick access data source options, such as **Excel**, next to the **Get data** button.



Depending on your selection, you need to find and open your data source. You might be prompted to sign into a service, such as OneDrive, to authenticate your request. In this example, you'll open the **Employee Data** Excel workbook that is stored on the Desktop (Remember, no files are provided for practice, these are hypothetical steps).

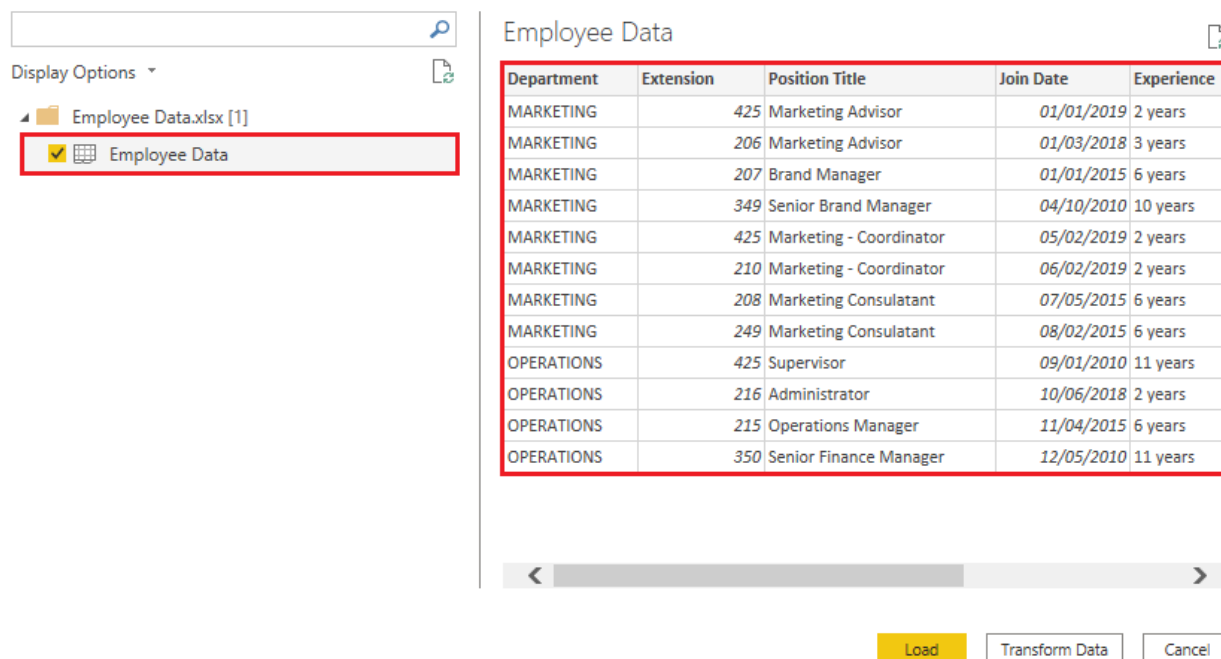


Select the file data to import

After the file has connected to Power BI Desktop, the **Navigator** window opens. This window shows you the data that is available in your data source (the Excel file in this example). You can select a table or entity to preview its contents, to ensure that the correct data is loaded into the Power BI model.

Select the check box(es) of the table(s) that you want to bring in to Power BI. This selection activates the **Load** and **Transform Data** buttons as shown in the following image.

Navigator



Now you can select the **Load** button to automatically load your data into the Power BI model or select the **Transform Data** button to launch the Power Query Editor, where you can review and clean your data before loading it into the Power BI model.

We often recommend that you transform data, but that process will be discussed later in this module. For this example, you can select **Load**.

Change the source file

You might have to change the location of a source file for a data source during development, or if a file storage location changes. To keep your reports up to date, you'll need to update your file connection paths in Power BI.

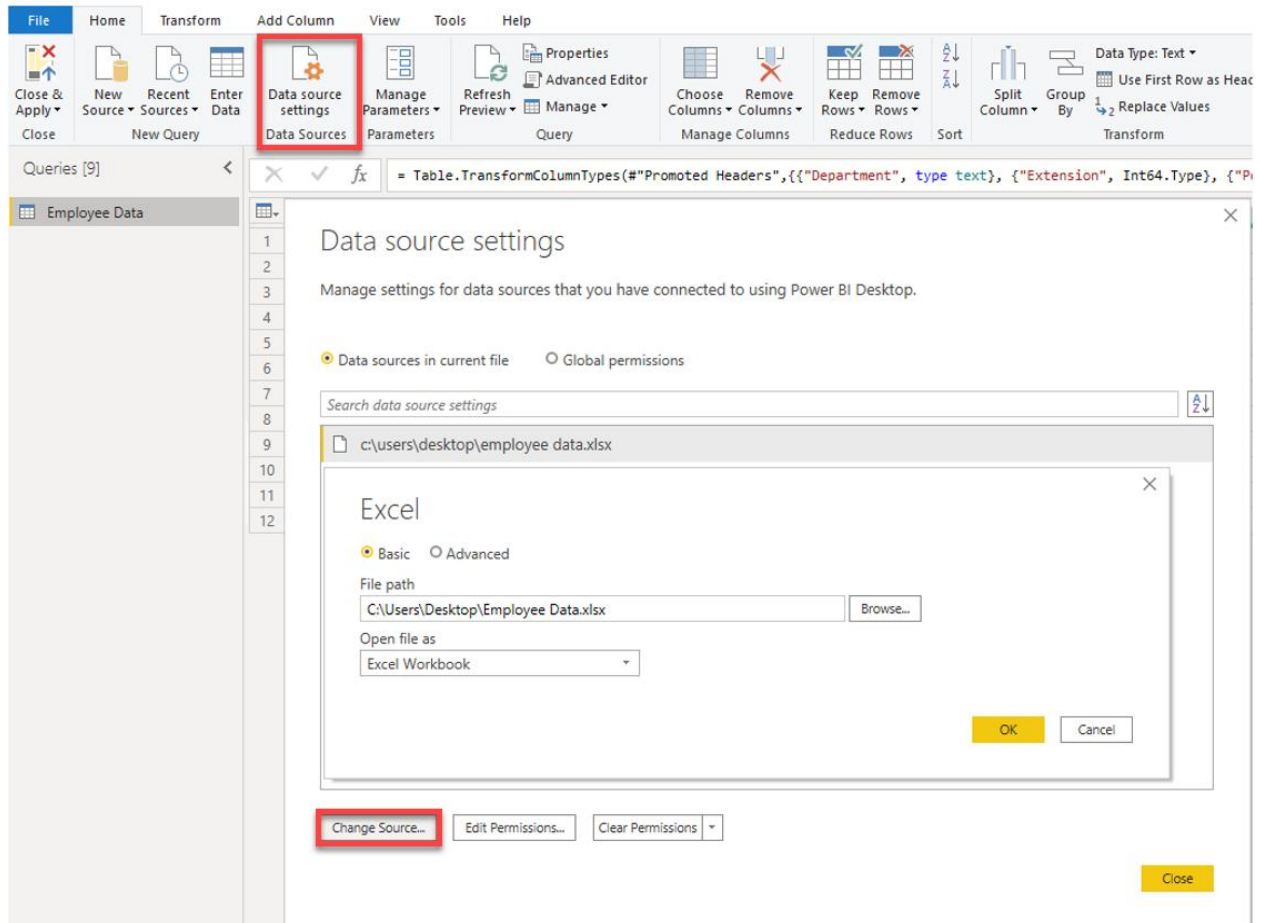
Power Query provides many ways for you to accomplish this task, so that you can make this type of change when needed.

1. Data source settings
2. Query settings
3. Advanced Editor

Warning

If you are changing a file path, make sure that you reconnect to the same file with the same file structure. Any structural changes to a file, such as deleting or renaming columns in the source file, will break the reporting model.

For example, try changing the data source file path in the data source settings. Select **Data source settings** in Power Query. In the **Data source settings** window, select your file and then select **Change Source**. Update the **File path** or use the **Browse** option to locate your file, select **OK**, and then select **Close**.



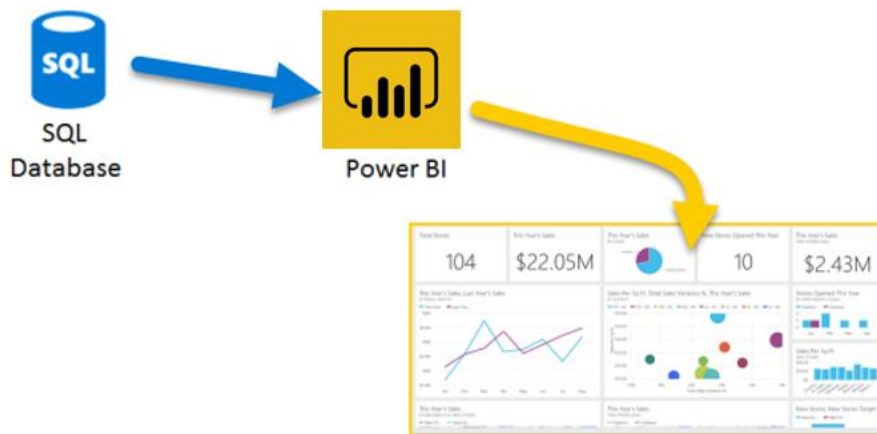
Get data from relational data sources

If your organization uses a [relational database](#) for sales, you can use Power BI Desktop to [connect directly to the database](#) instead of using exported flat files.

Connecting Power BI to your database will [help you to monitor the progress of your business and identify trends](#), so you can forecast sales figures, plan budgets and set performance indicators and targets. Power BI Desktop can connect to many relational databases that are either in the cloud or on-premises.

Scenario

The Sales team at Tailwind Traders has requested that you [connect to the organization's on-premises SQL Server database and get the sales data into Power BI Desktop](#) so you can build sales reports.

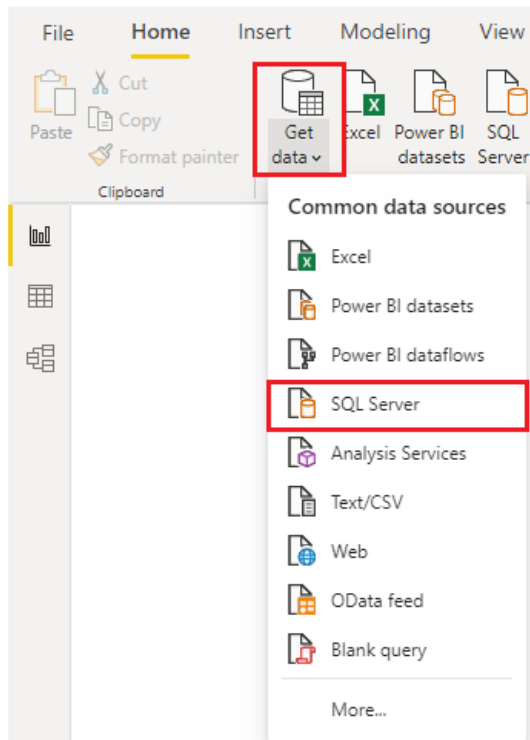


Connect to data in a relational database

You can use the **Get data** feature in Power BI Desktop and select the applicable option for your relational database. For this example, you would select the **SQL Server** option, as shown in the following screenshot.

Tip

Next to the **Get Data** button are quick access data source options, such as **SQL Server**.



Your next step is to enter your database server name and a database name in the **SQL Server database** window. The two options in data connectivity mode are: **Import** (selected by default, recommended) and **DirectQuery**. Mostly, you select **Import**. Other advanced options are also available in the **SQL Server database** window, but you can ignore them for now.

SQL Server database

Server ①

Database (optional)

Data Connectivity mode ①

☒ Import

☐ DirectQuery

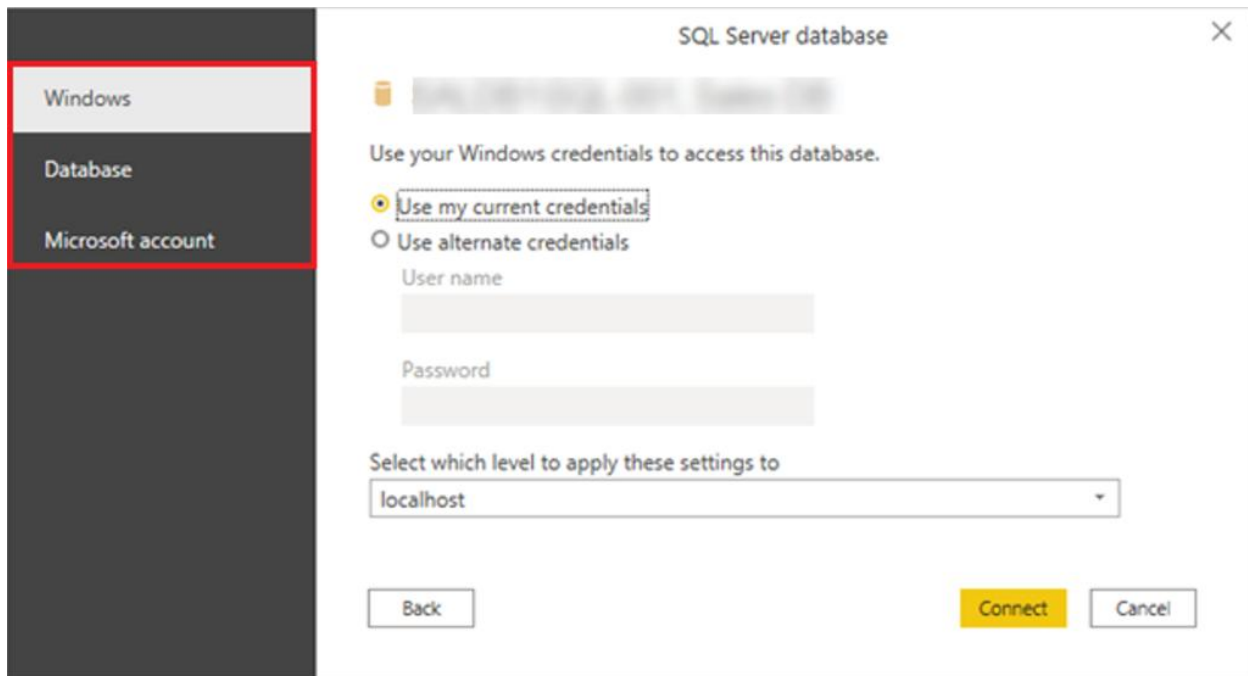
▸ Advanced options

OK Cancel

After you've added your server and database names, you'll be prompted to sign in with a username and password. You'll have three sign-in options:

- **Windows** - Use your **Windows account** (Azure Active Directory credentials).
- **Database** - Use your **database credentials**. For instance, [SQL Server has its own sign-in and authentication system](#) that is sometimes used. [If the database administrator gave you a unique sign-in](#) to the database, you might need to [enter those credentials](#) on the **Database** tab.
- **Microsoft account** - Use your **Microsoft account credentials**. This option is [often used for Azure services](#).

Select a sign-in option, [enter your username and password](#), and then select **Connect**.



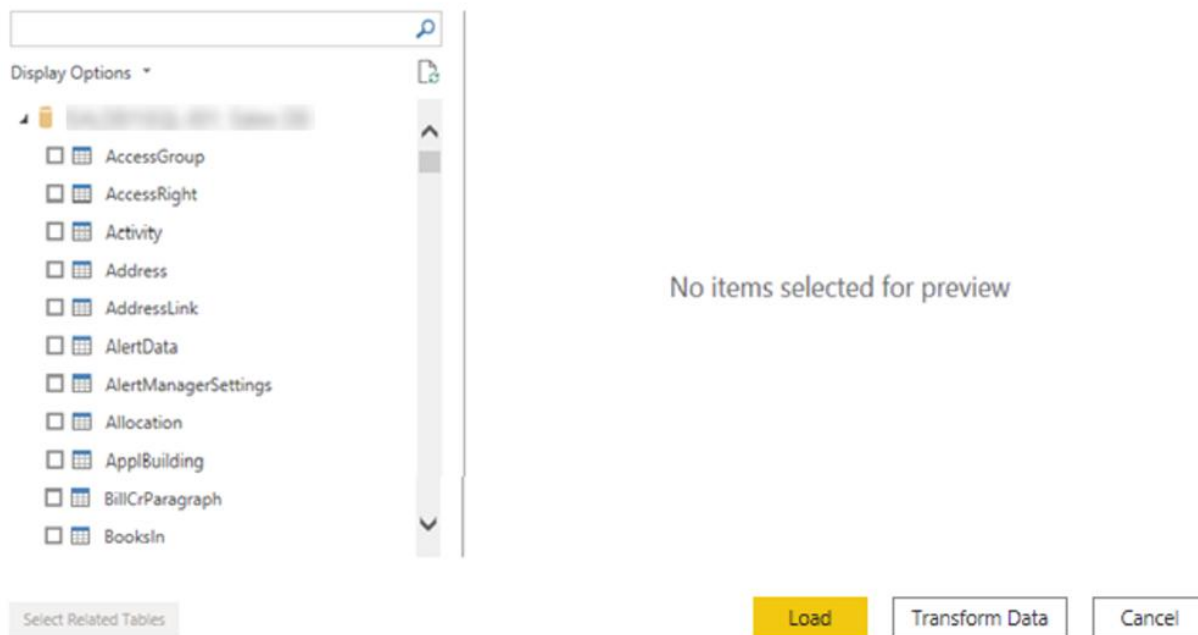
Select data to import

After the database has been connected to Power BI Desktop, the **Navigator** window displays the data that is available in your data source (the SQL database in this example). You can select a table or entity to preview its contents and make sure that the correct data will be loaded into the Power BI model.

Select the check box(es) of the table(s) that you want to bring in to Power BI Desktop, and then select either the **Load** or **Transform Data** option.

- **Load** - Automatically load your data into a Power BI model in its current state.
- **Transform Data** - Open your data in Microsoft Power Query, where you can perform actions such as deleting unnecessary rows or columns, grouping your data, removing errors, and many other data quality tasks.

Navigator



Import data by writing an SQL query

Another way you can import data is to write an SQL query to specify only the tables and columns that you need.

To write your SQL query, on the **SQL Server database** window, enter your server and database names, and then select the arrow next to **Advanced options** to expand this section and view your options. In the **SQL statement** box, write your query statement, and then select **OK**. In this example, you'll use the **Select SQL statement** to load the ID, NAME and SALESAMOUNT columns from the SALES table.

SQL Server database

Server ①

Database (optional)

▲ Advanced options

Command timeout in minutes (optional)

SQL statement (optional, requires database)

```
SELECT
    ID
, NAME
, SALESAMOUNT
FROM SALES
```

☒ Include relationship columns

☐ Navigate using full hierarchy

☐ Enable SQL Server Failover support

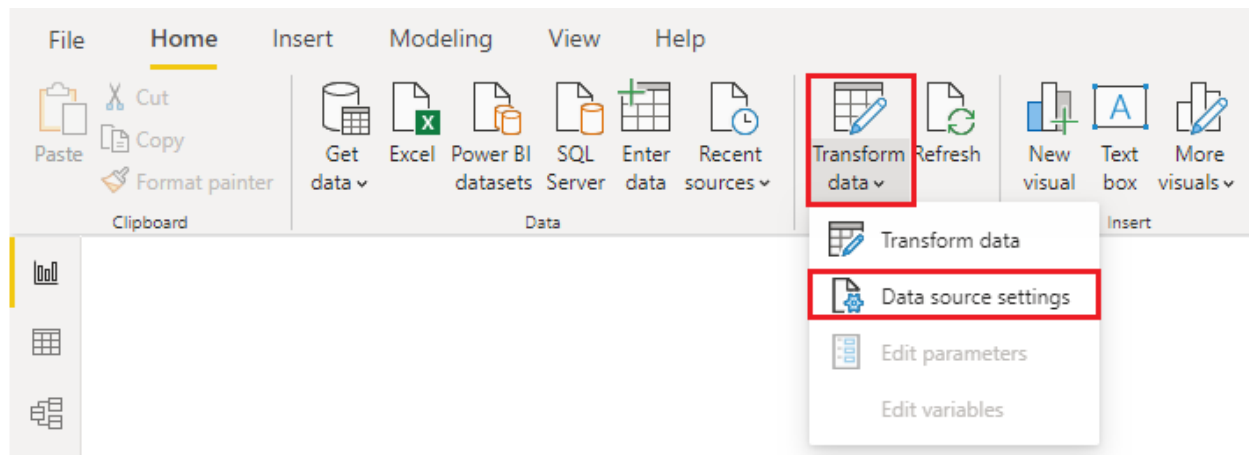
OK

Cancel

Change data source settings

After you create a data source connection and load data into Power BI Desktop, you can return and change your connection settings at any time. This action is often required due to a security policy within the organization, for example, when the password needs to be updated every 90 days. You can change the data source, edit permissions or clear permissions.

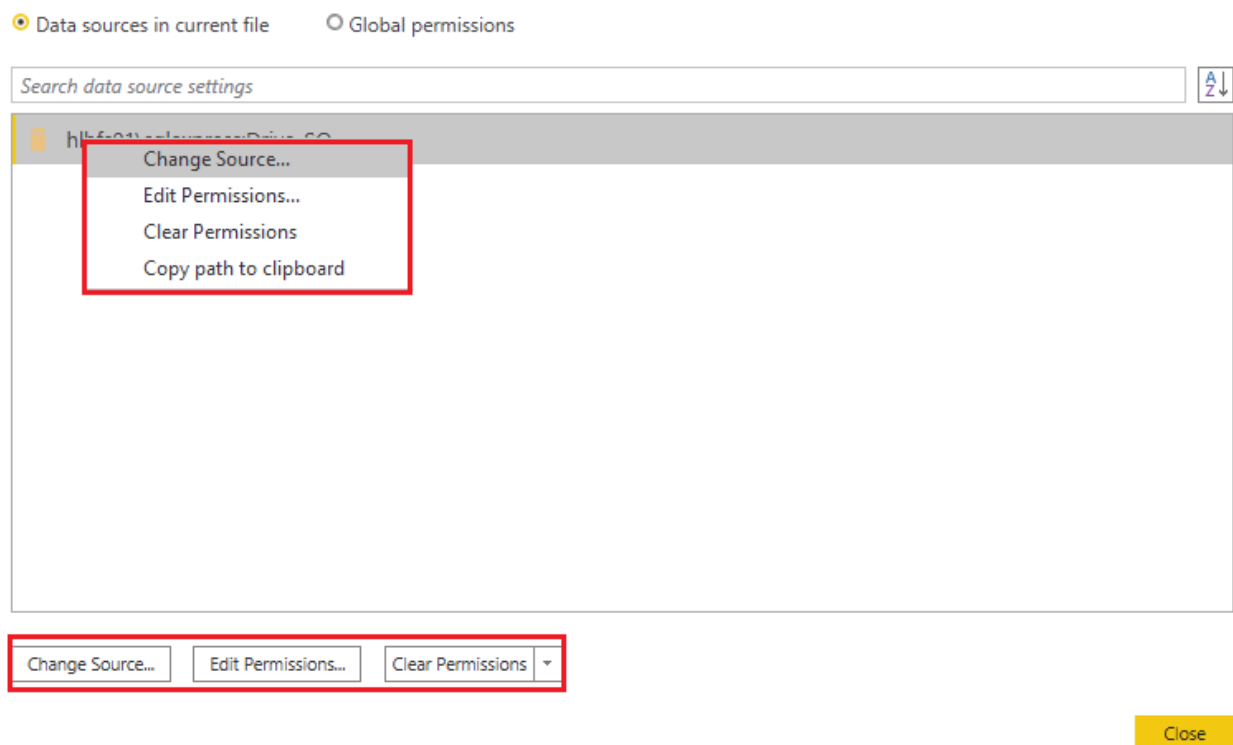
On the **Home** tab, select **Transform data**, and then select the **Data source settings** option.



From the list of data sources that displays, select the data source that you want to update. Then, you can right-click that data source to view the available update options or you can use the update option buttons on the lower left of the window. Select the update option that you need, change the settings as required, and then apply your changes.

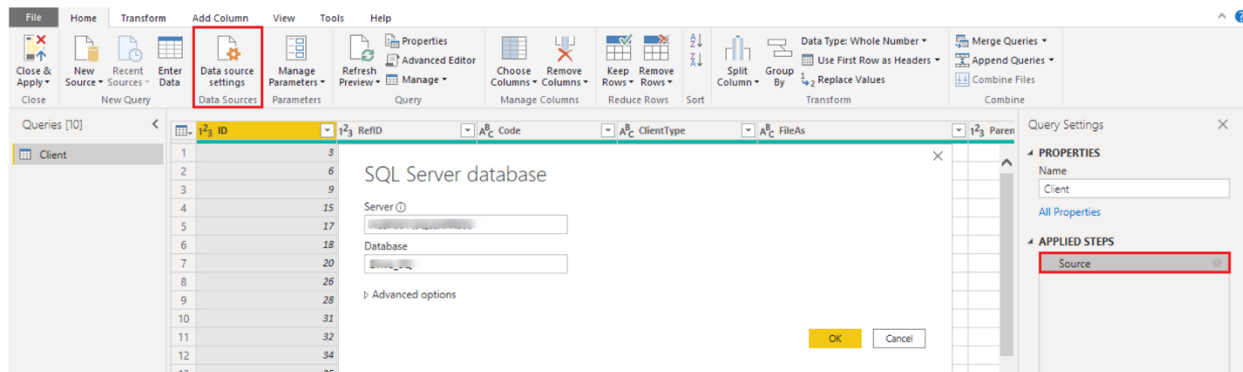
Data source settings

Manage settings for data sources that you have connected to using Power BI Desktop.



You can also change your data source settings from within Power Query. Select the table, and then select the **Data source settings** option on the **Home** ribbon. Alternatively, you can

go to the **Query Settings** panel on the right side of the screen and select the settings icon next to **Source** (or double Select Source). In the window that displays, update the server and database details, and then select **OK**.



After you have made the changes, select **Close and Apply** to apply those changes to your data source settings.

Write an SQL statement

As previously mentioned, you can import data into your Power BI model by using an SQL query. **SQL stands for Structured Query Language** and is a standardized programming language that is **used to manage relational databases and perform various data management operations**.

Consider the scenario where **your database has a large table that is comprised of sales data over several years**. **Sales data from 2009 isn't relevant to the report that you're creating**. This situation is where **SQL is beneficial because it allows you to load only the required set of data by specifying exact columns and rows in your SQL statement and then importing them into your semantic model**. **You can also join different tables, run specific calculations, create logical statements, and filter data in your SQL query**.

The following example shows **a simple query where the ID, NAME and SALESAMOUNT are selected from the SALES table**.

The SQL query **starts with a Select statement**, which allows you to **choose the specific fields that you want to pull from your database**. In this example, you want to load the ID, NAME, and SALESAMOUNT columns.

SQL

SELECT

ID

, NAME

```
, SALESAMOUNT  
FROM
```

FROM specifies the name of the table that you want to pull the data from. In this case, it's the **SALES** table. The following example is the full SQL query:

SQL

```
SELECT  
ID  
, NAME  
, SALESAMOUNT  
FROM  
SALES
```

When using an SQL query to import data, try to [avoid using the wildcard character \(*\)](#) in your query. If you use the wildcard character (*) in your SELECT statement, you [import all columns](#) that you don't need from the specified table.

The following example shows the query using the wildcard character.

SQL

```
SELECT *  
FROM  
SALES
```

The wildcard character (*) will import all columns within the **Sales** table. [This method isn't recommended](#) because it will lead to redundant data in your semantic model, which will [cause performance issues and require extra steps to normalize](#) your data for reporting.

[All queries should also have a **WHERE** clause](#). This clause will [filter the rows to pick only filtered records](#) that you want. In this example, if you want to get recent sales data after January 1st, 2020, add a **WHERE** clause. The evolved query would look like the following example.

SQL

```
SELECT  
ID  
, NAME
```

```
, SALESAMOUNT  
FROM  
SALES  
WHERE  
OrderDate >= '1/1/2020'
```

It's a best practice to avoid doing this directly in Power BI. Instead, consider writing a query like this in a **view**. A view is **an object in a relational database, similar to a table**. Views **have rows and columns, and can contain almost every operator in the SQL language**. If Power BI uses a view, when it retrieves data, it participates in query folding, a feature of **Power Query**. Query folding will be explained later, but in short, Power Query **will optimize data retrieval according to how the data is being used later**.

Create dynamic reports with parameters

Dynamic reports are reports in which **the data can be changed by a developer according to user specifications**. Dynamic reports are valuable because **a single report can be used for multiple purposes**. If you use dynamic reports, **you'll have fewer individual reports to create, which will save organizational time and resources**.

You can **use parameters by determining the values that you want to see data for in the report, and the report updates accordingly by filtering the data for you**.

Creating dynamic reports allows you to **give users more power over the data** that is displayed in your reports; they **can change the data source and filter the data by themselves**.

In the following example, you've created a report for the Sales team at Tailwind Traders that **displays the sales data in the SQL Server database**. The report gives a holistic view of how the Sales team is performing. While the report is useful, the **Sales team members want to be able to filter the report so that they can view their own data only and track their performance against their sales targets**.

Create dynamic reports for individual values

To create a dynamic report, you first need to write your SQL query. Then use the **Get data** feature in Power BI Desktop to connect to the database.

In this example, you **connect to your database** on SQL Server by following these steps:

1. After you have entered your server details, in the **SQL Server database** window, select **Advanced options**.
2. Paste the SQL query into the **SQL statement box** and then select **OK**.

```

SELECT [SalesOrderID]
      ,[OrderDate]
      ,[OnlineOrderFlag]
      ,[SalesOrderNumber]
      ,[CustomerID]
      ,[SalesPersonID]
      ,[TerritoryID]
      ,[SubTotal]
      ,[TaxAmt]
      ,[Freight]
      ,[TotalDue]
FROM [TailwindTraders].[Sales].[SalesOrderHeader] s
WHERE s.[SalesPersonID] = 279

```

SQL Server database

Server ⓘ

localhost\SQLEXPRESS

Database (optional)

TailwindTraders

Data Connectivity mode ⓘ

☒ Import
 ☐ DirectQuery

Advanced options

Command timeout in minutes (optional)

SQL statement (optional, requires database)

```

      ,[TaxAmt]
      ,[Freight]
      ,[TotalDue]
FROM [TailwindTraders].[Sales].[SalesOrderHeader] s
WHERE s.[SalesPersonID] = 279

```

☒ Include relationship columns
 ☐ Navigate using full hierarchy
 ☐ Enable SQL Server Failover support

OK

Cancel

When the connection is made, the data is shown in the preview window.

3. Select **Edit** to open the data in Power Query Editor.

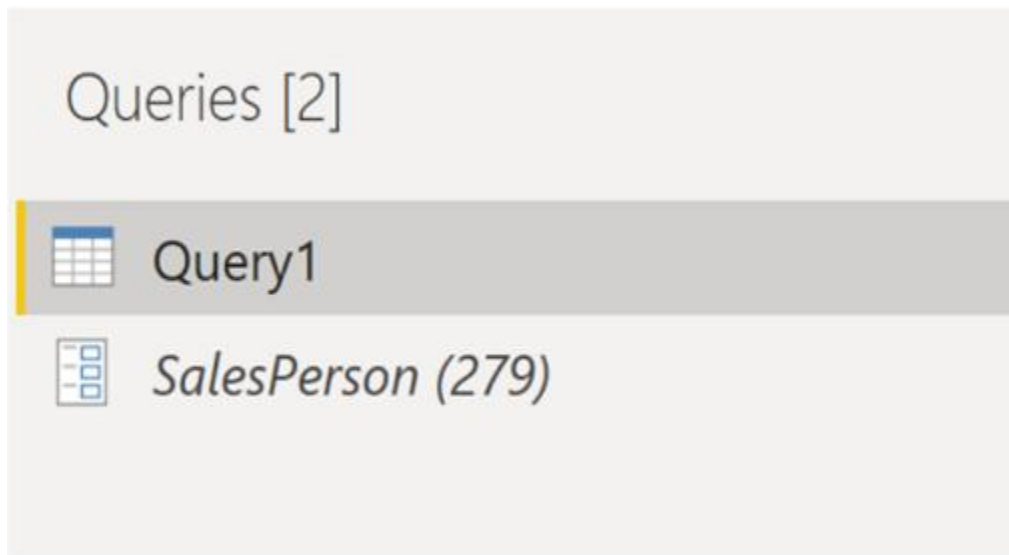
Next, you create the parameter by following these steps:

1. On the **Home** tab, select **Manage parameters > New parameter**.
2. On the **Parameters** window, change the default parameter name to something more **descriptive** so that its purpose is clear. In this case, you change the name to **SalesPerson**.
3. Select **Text** from the **Type** list and then select **Any value** from the **Suggested value** list.
4. Select **OK**.

The screenshot shows the 'Manage Parameters' dialog box. On the left, there is a list of parameters with a tab labeled 'SalesPerson'. A 'New' button is visible above the list. On the right, the configuration for the selected parameter is shown:

- Name:** SalesPerson
- Description:** (empty text box)
- Required:** ☒
- Type:** Text (dropdown menu)
- Suggested Values:** Any value (dropdown menu)
- Current Value:** 279 (text box)

A new query is shown for the parameter that you created.



Now, you need to adjust the code in SQL query to assess your new parameter:

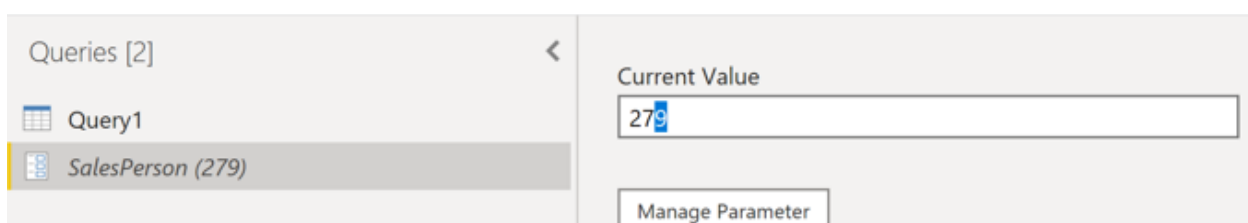
1. Right-click **Query1** and then select **Advanced editor**.
2. Replace the existing value in the execute statement with an ampersand (&) followed by your parameter name (**SalesPerson**), as illustrated in the following image.



3. Make sure that no errors are shown at bottom of the window and then select **Done**.

Though you don't see a difference on the screen, Power BI ran the query.

4. To confirm that the query was run, you can run a test by selecting the parameter query and entering a new value in the **Current Value** box.



5. A warning icon might display next to the query. If so, select that query to view the warning message, which states that permission is required to run this native database query. Select **Edit Permission** and then select **Run**.

When the query runs successfully, the parameter displays the new value.

	AB SalesOrderNumber	123 SalesOrderID	123 SalesPersonID	OrderDate
1	SO43659	43659	279	31/05/2011 00:00:00
2	SO43660	43660	279	31/05/2011 00:00:00
3	SO43681	43681	279	31/05/2011 00:00:00
4	SO43684	43684	279	31/05/2011 00:00:00
5	SO43685	43685	279	31/05/2011 00:00:00
6	SO43694	43694	279	31/05/2011 00:00:00
7	SO43695	43695	279	31/05/2011 00:00:00
8	SO43696	43696	279	31/05/2011 00:00:00

6. Select **Close and Apply** to return to the report editor.

Now, you can apply the parameter to the report:

1. Select **Edit queries > Edit parameters**.
2. On the **Edit Parameters** window, **enter a new value** and then select **OK**.
3. Select **Apply changes** and then run the native query again.

Now, when you view the data, you see the data for the new value that was passed through the parameter.

SalesOrderNumber	SalesOrderID	SalesPersonID	OrderDate	CustomerID	TerritoryID	SubTotal	TaxAmt	Freight	TotalDue
SO43659	43659	279	31/05/2011 00:00:00	29825	5	20565.6206	1971.5149	616.0984	23153.2339
SO43660	43660	279	31/05/2011 00:00:00	29672	5	1294.2529	124.2483	38.8276	1457.3288
SO43681	43681	279	31/05/2011 00:00:00	29661	5	13787.5434	1323.0668	413.4584	15524.0686
SO43684	43684	279	31/05/2011 00:00:00	29912	5	5596.4705	537.2612	167.8941	6301.6258
SO43685	43685	279	31/05/2011 00:00:00	30084	5	2736.5678	263.201	82.2503	3082.0191
SO43694	43694	279	31/05/2011 00:00:00	29549	5	20645.634	1978.3257	618.2268	23242.1865
SO43695	43695	279	31/05/2011 00:00:00	29958	5	39373.781	3787.4632	1183.5823	44344.8265
SO43696	43696	279	31/05/2011 00:00:00	29849	5	419.4589	40.2681	12.5838	472.3108
SO43845	43845	279	01/07/2011 00:00:00	29888	5	8580.0739	823.6669	257.3959	9661.1367
SO43861	43861	279	01/07/2011 00:00:00	29749	5	23401.1062	2244.4088	701.3777	26346.8927
SO43862	43862	279	01/07/2011 00:00:00	29945	5	31000.7804	2987.8703	933.7095	34922.3602

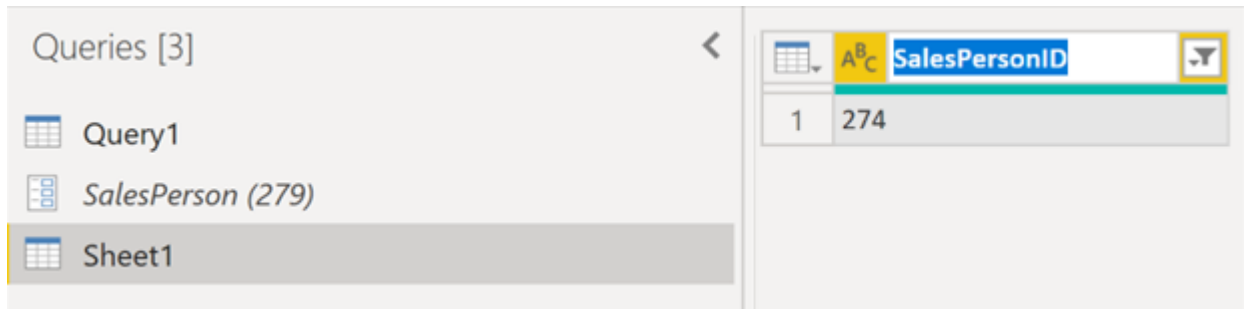
You can now create a report that displays data for one particular value at a time. More steps are needed to display data for multiple values at the same time.

Create dynamic reports for multiple values

To accommodate multiple values at a time, you first need to create a Microsoft Excel worksheet that has a table consisting of one column that contains the list of values.

Next, use the **Get data** feature in Power BI Desktop to connect to the data in that Excel worksheet, and then follow these steps:

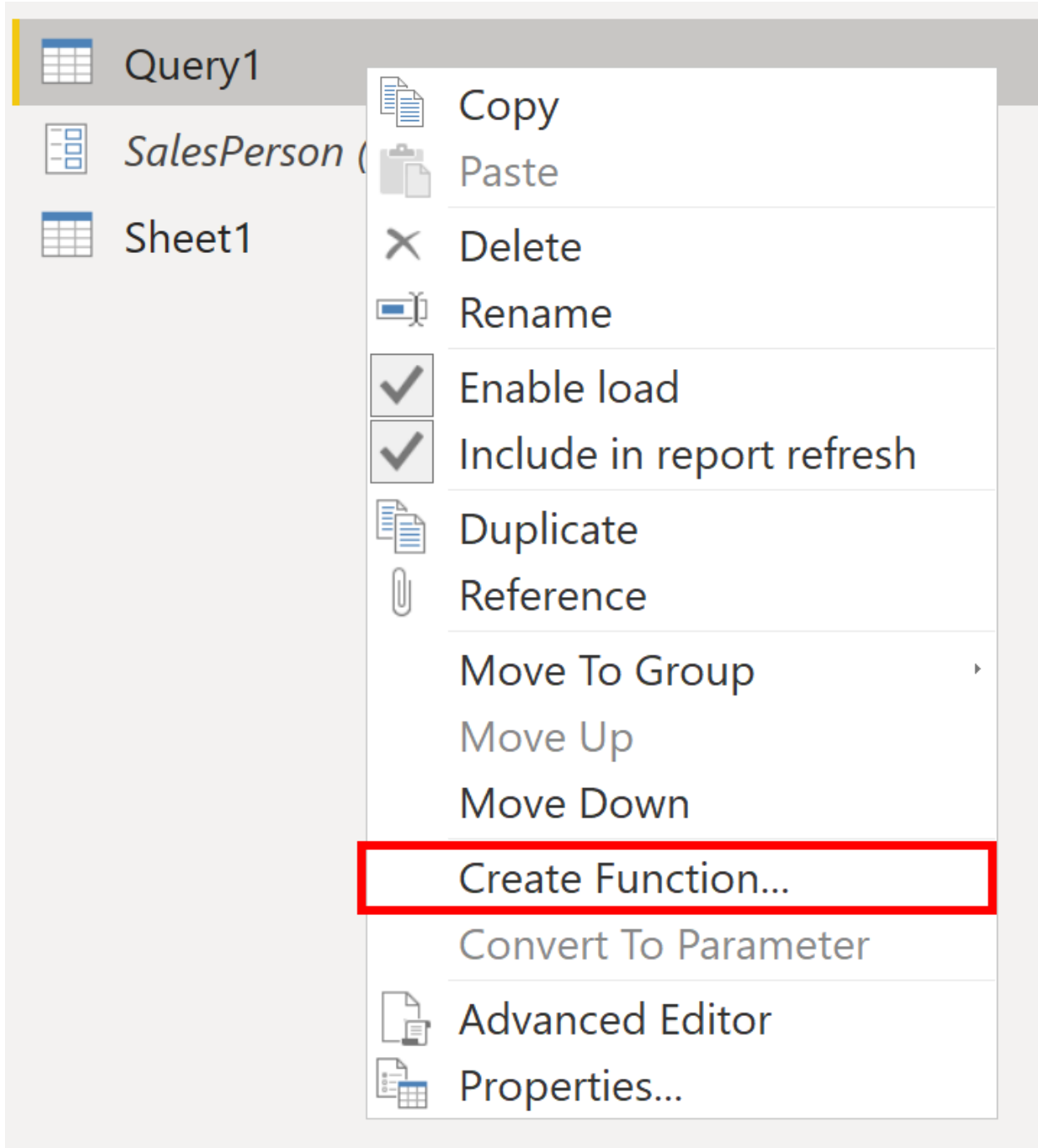
1. On the **Navigator** window, select **Edit** to open the data in Power Query Editor, where you see a new query for the data table.



2. Rename the column in the table to something more descriptive.
3. Change the column data type to **Text** so that it matches the parameter type and you avoid data conversion problems.
4. In the query **Properties** section, change the name of the data source to something more descriptive. For this example, enter **SalesPersonID**.

Next, you need to create a function that passes the new **SalesPersonID** query into **Query1**:

1. Right-click **Query1** and then select **Create function**.



2. Enter a name for the function and then select **OK**.

Create Function

Enter a name for the new function.

Function name

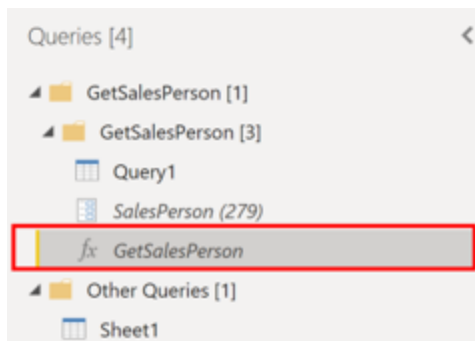
GetSalesPerson

Parameters: SalesPerson

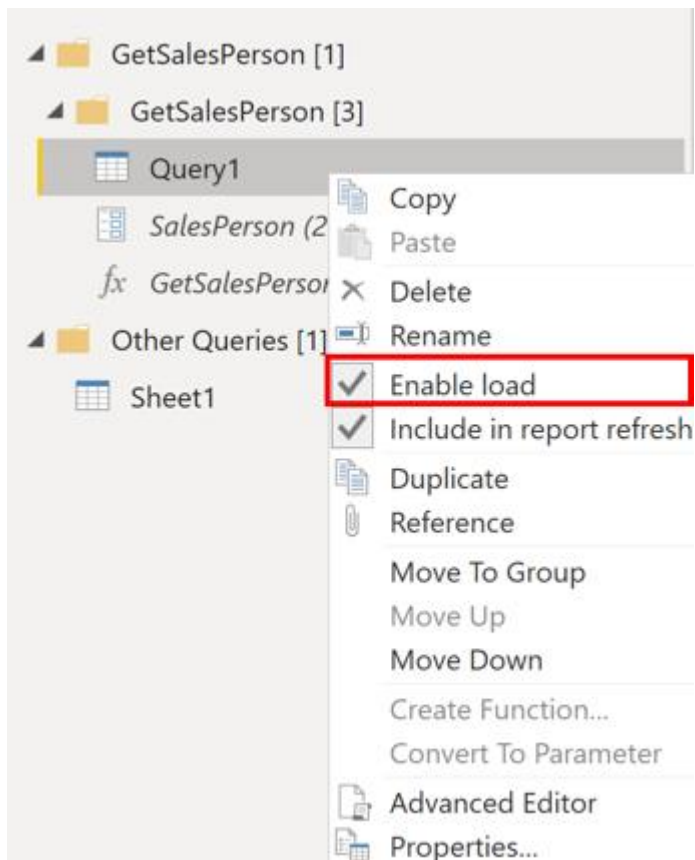
OK

Cancel

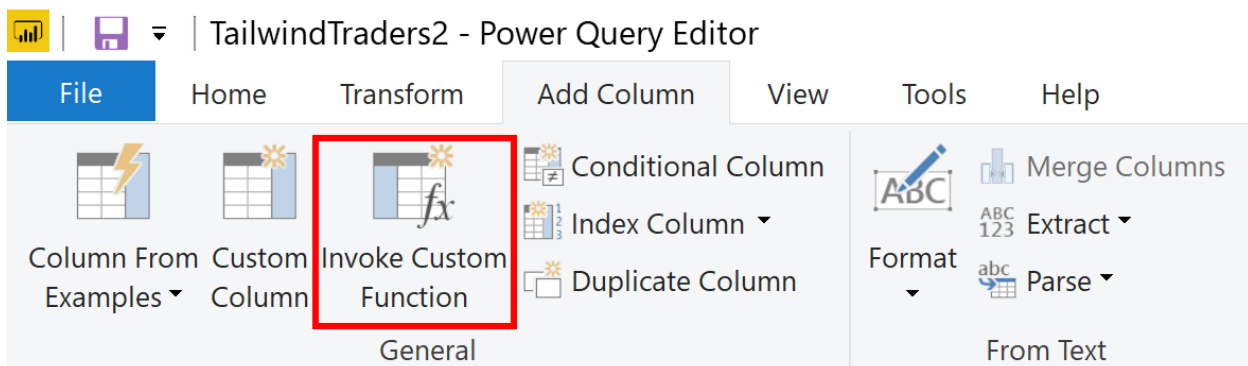
Your new function appears in the **Queries** pane.



3. To ensure that **Query1** doesn't show up in the field list for the **report**, which could potentially **confuse users**, you can **disable it** from loading in the report **by right-clicking Query1** again and then selecting **Enable load** (selected by default) to disable the feature.



4. Select the **SalesPersonID** query that you loaded from the Excel worksheet and then, on the **Add Column** tab, select **Invoke custom function** to run the custom function that you created.



5. On the **Invoke Custom Function** window, select your function from the **Function query** list.

The **New column name** updates automatically and the table that contains the values that you're going to pass through the parameter is selected by default.

6. Select **OK** and, if necessary, run the native query.

×

Invoke Custom Function

Invoke a custom function defined in this file for each row.

New column name

Function query

SalesPerson

A new column for your **GetSalesFromSalesPerson** function appears next to the **SalesPersonID** column.

	ABC SalesPersonID	ABC 123 GetSalesFromSalesPerson
1	274	Table

- Select the two arrows icon in the new column header and then select the check boxes of the columns that you want to load. This section is where you determine the details that are available in the report for each value (sales person ID).
- Clear the **Use original column name as prefix** check box at the bottom of the screen because you don't need to see a prefix with the column names in the report.
- Select **OK**.

The image shows a software interface with a dialog box titled "GetSalesFromSalesPerson". The dialog box has a header bar with a grid icon, a dropdown menu showing "SalesPersonID", and a yellow button labeled "GetSalesFromSalesPerson". Below the header, there is a text input field with a vertical cursor and a small icon with "A Z" and a downward arrow. The main area of the dialog box contains two radio buttons: "Expand" (selected) and "Aggregate". Below these is a list of columns, each with a yellow checkmark in a box: "(Select All Columns)", "SalesOrderNumber", "SalesOrderID", "SalesPersonID", "OrderDate", "CustomerID", "TerritoryID", "SubTotal", "TaxAmt", "Freight", "TotalDue", and "OnlineOrderFlag". At the bottom of the list is an unchecked checkbox labeled "Use original column name as prefix". At the bottom right of the dialog box are two buttons: a yellow "OK" button and a white "Cancel" button with a grey border.

1

ABC 123 GetSalesFromSalesPerson

Expand Aggregate

- ✓ (Select All Columns)
- ✓ SalesOrderNumber
- ✓ SalesOrderID
- ✓ SalesPersonID
- ✓ OrderDate
- ✓ CustomerID
- ✓ TerritoryID
- ✓ SubTotal
- ✓ TaxAmt
- ✓ Freight
- ✓ TotalDue
- ✓ OnlineOrderFlag

☐ Use original column name as prefix

OK Cancel

You should be able to view the data for the columns that you selected, for each value (sales person ID).

Queries [4]		= Table.Distinct("#Expanded GetSalesFromSalesPerson ", {"Sa	
GetSalesPerson [1]		SalesPersonID	SalesOrderNumber
GetSalesPerson [3]			SalesOrderID
Query1		1 274	SO43849 43849
SalesPerson (279)		2 275	SO43670 43670
GetSalesPerson		3 276	SO43663 43663
		4 277	SO43667 43667
		5 278	SO43677 43677
		6 279	SO43659 43659
		7 280	SO43664 43664

If necessary, you can add more values (sales people IDs) to the **SalesPersonID** column in the Excel worksheet, or you can change the existing values.

10. Save your changes and then return to Power Query Editor.

11. On the **Home** tab, select **Refresh Preview**, and then run the native query again (if necessary). You should see the sales from the new sales people IDs that you added into the worksheet.

12. Select **Close and Apply** to return to the report editor, where you see the new column names in the Fields pane.

Now, you can start building your report.

Get data from a NoSQL database

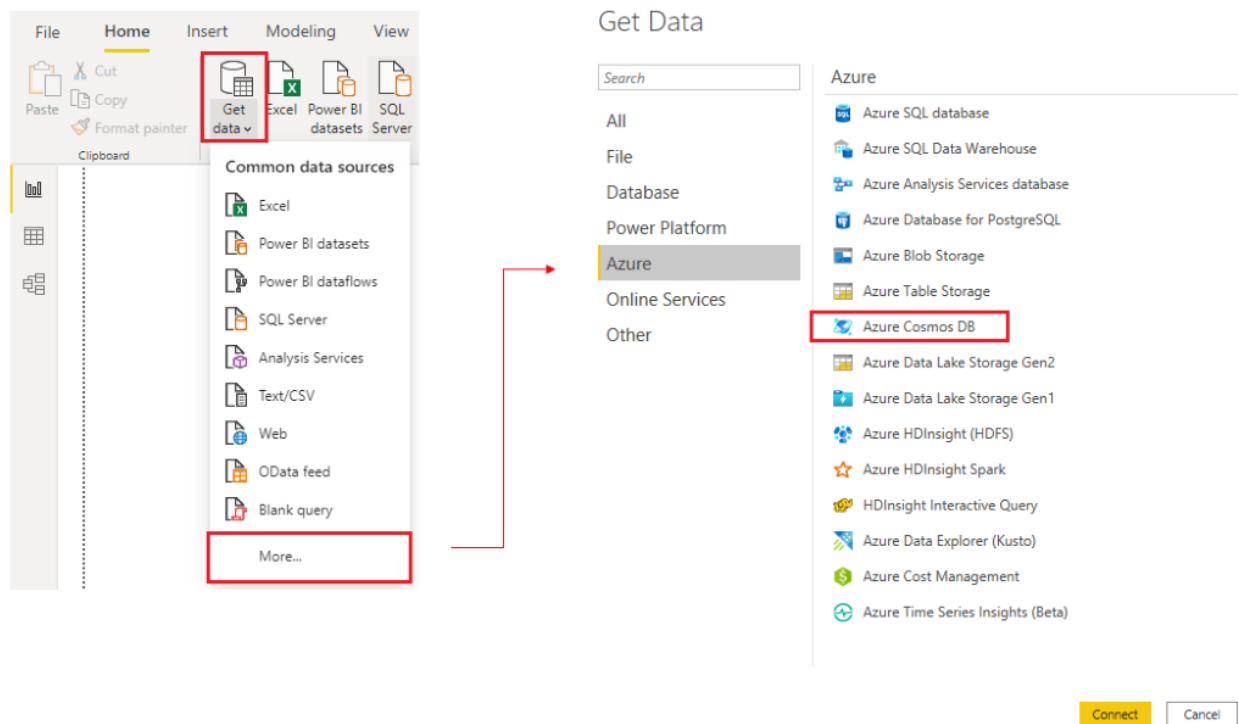
Some organizations don't use a relational database but instead use a **NoSQL** database. A **NoSQL database** (also referred to as non-SQL, not only SQL or *non-relational*) is a flexible type of database that doesn't use tables to store data.

Scenario

Software developers at Tailwind Traders created an application to manage shipping and tracking products from their warehouses. The application uses **Cosmos DB**, a NoSQL database, as the data repository. Data is stored as **JSON documents**, which are open standard file formats that are primarily used to transmit data between a server and web application. You need to import this data into a Power BI semantic model for reporting.

Connect to a NoSQL database (Azure Cosmos DB)

In this scenario, you'll use the **Get data** feature in Power BI Desktop. However, this time you'll select the **More...** option to locate and connect to the type of database that you use. In this example, you'll select the **Azure** category, select **Azure Cosmos DB**, and then select **Connect**.



On the **Preview Connector** window, select **Continue** and then enter your database credentials. In this example, on the **Azure Cosmos DB** window, you can enter the database details. You can specify the Azure Cosmos DB account endpoint URL that you want to get the data from (you can get the URL from the **Keys** blade of your Azure portal). Alternatively, you can enter the database name, collection name or use the navigator to select the database and collection to identify the data source.

If you're connecting to an endpoint for the first time, as you are in this example, make sure that you enter your account key. You can find this key in the **Primary Key** box in the **Read-only Keys** blade of your Azure portal.

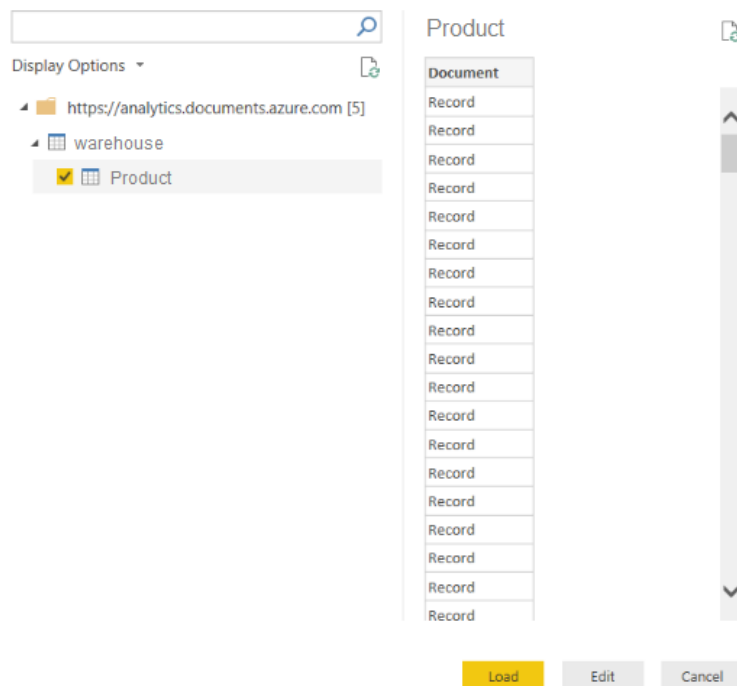
Import a JSON file

If you're working with data stored in JSON format, it's often necessary to extract and normalize the data first. This is because JSON data is often stored in a nested or unstructured format, which makes it difficult to analyze or report on directly.

In this example, the data must be extracted and normalized before you can report on them, so you need to transform the data before loading it into Power BI Desktop.

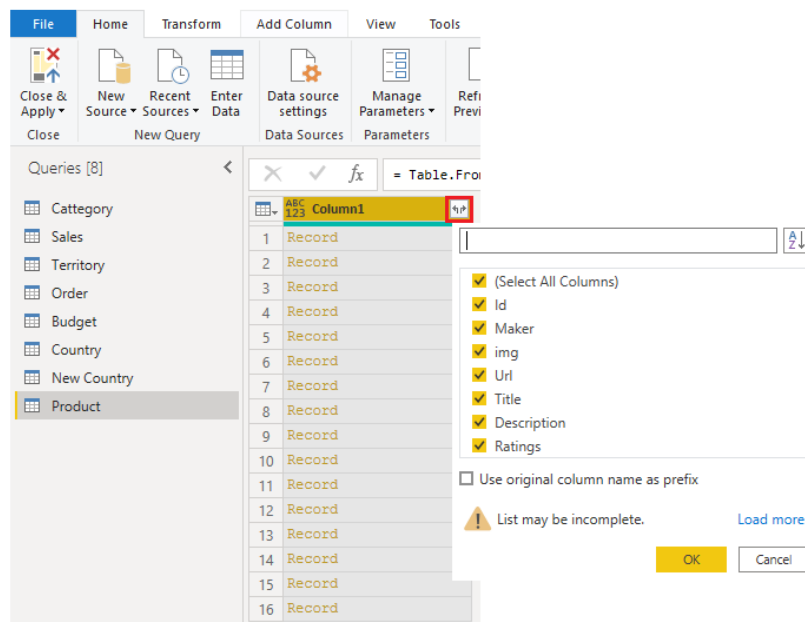
After you've connected to the database account, the **Navigator** window opens, showing a list of databases under that account. Select the table that you want to import. In this example, you'll select the Product table. The preview pane only shows **Record** items because all records in the document are represented as a Record type in Power BI.

Navigator

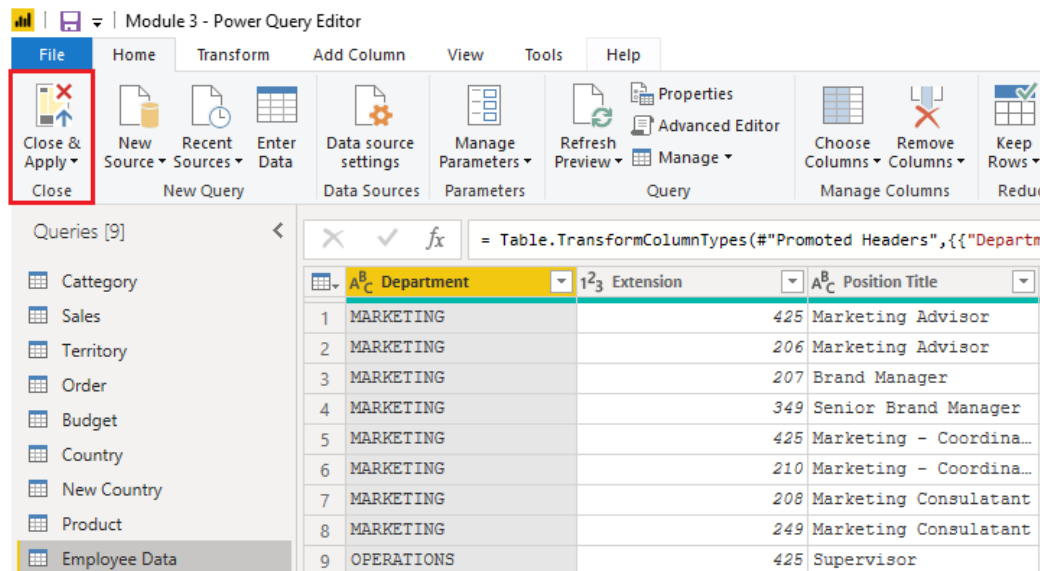


Select the **Edit** button to open the records in Power Query.

In Power Query, select the **Expander** button to the right side of the **Column1** header, which displays the context menu with a list of fields. Select the fields that you want to load into Power BI Desktop, clear the **Use original column name as prefix** checkbox, and then select **OK**.



Review the selected data to ensure that you're satisfied with it, then select **Close & Apply** to load the data into Power BI Desktop.



The data now resembles a table with rows and columns. Data from Cosmos DB can now be related to data from other data sources and can eventually be used in a Power BI report.

Get data from online services

To support their daily operations, organizations frequently use a range of software applications, such as [SharePoint](#), [OneDrive](#), [Dynamics 365](#), [Google Analytics](#) and so on. These applications produce their own data. Power BI can combine the data from multiple applications to produce more meaningful insights and reports.

Scenario

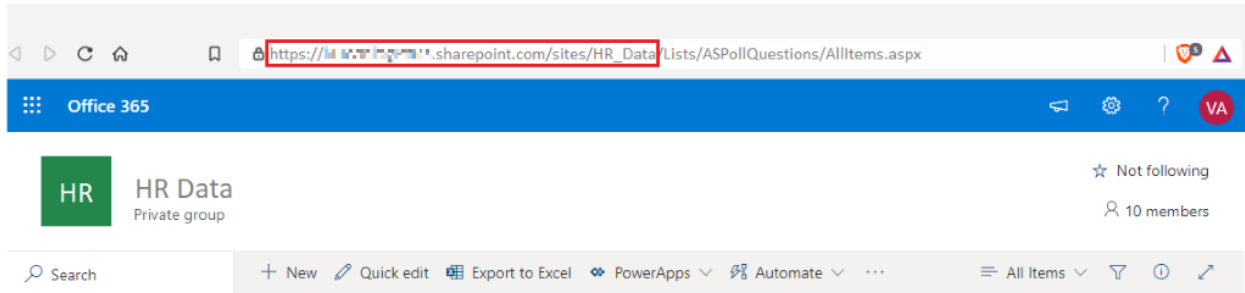
Tailwind Traders uses [SharePoint](#) to collaborate and store sales data. It's the start of the new financial year and the sales managers want to enter new goals for the sales team. The form that the leadership uses exists in [SharePoint](#). You're required to establish a connection to this data within Power BI Desktop, so that the sales goals can be used alongside other sales data to determine the health of the sales pipeline.

The following sections examine how to use the Power BI Desktop **Get Data** feature to connect to data sources that are produced by external applications. To illustrate this process, we've provided an example that shows how to connect to a [SharePoint](#) site and import data from an online list.

Connect to data in an application

When connecting to data in an application, you would begin in the same way as you would when connecting to the other data sources: by selecting the **Get data** feature in Power BI Desktop. Then, select the option that you need from the **Online Services** category. In this example, you select **SharePoint Online List**.

After you've selected **Connect**, you'll be asked for your **SharePoint URL**. This URL is the one that you use to sign into your SharePoint site through a web browser. You **can copy the URL from your SharePoint site and paste it** into the connection window in Power BI. You don't need to **enter your full URL file path**; you only need to load your site URL because, when you're connected, **you can select the specific list** that you want to load. Depending on the URL that you copied, you **might need to delete the last part of your URL**, as illustrated in the following image.



After you've entered your URL, select **OK**. Power BI needs to authorize the connection to SharePoint, so **sign in with your Microsoft account and then select **Connect****.



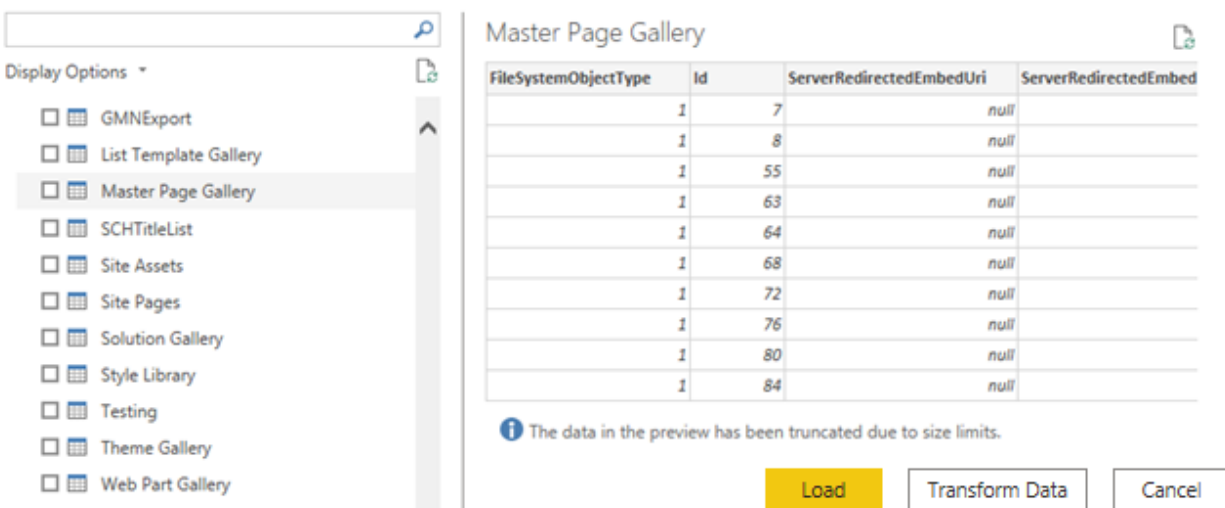
Choose the application data to import

After Power BI has made the **connection** with SharePoint, the **Navigator window** appears, as it does when you connect to other data sources. The window **displays the tables and entities within your SharePoint site**. **Select the list that you want to load** into Power BI Desktop. Similar to when you import from other data sources, you have the option to automatically

load your data into a Power BI model or launch the Power Query Editor to transform your data before loading it.

For this example, you select the **Load option**.

Navigator



Select a storage mode

The most popular way to use data in Power BI is to import it into a Power BI semantic model. Importing the data means that the data is stored in the Power BI file and gets published along with the Power BI reports. This process helps make it easier for you to interact directly with your data. However, this approach might not work for all organizations.

To continue with the scenario, you're building Power BI reports for the Sales department at Tailwind Traders, where importing the data isn't an ideal method. The first task you need to accomplish is to create your semantic models in Power BI so you can build visuals and other report elements. The Sales department has many different semantic models of varying sizes. For security reasons, you aren't allowed to import local copies of the data into your reports, so directly importing data is no longer an option. Therefore, you need to create a direct connection to the Sales department's data source. The following section describes how you can ensure that these business requirements are satisfied when you're importing data into Power BI.

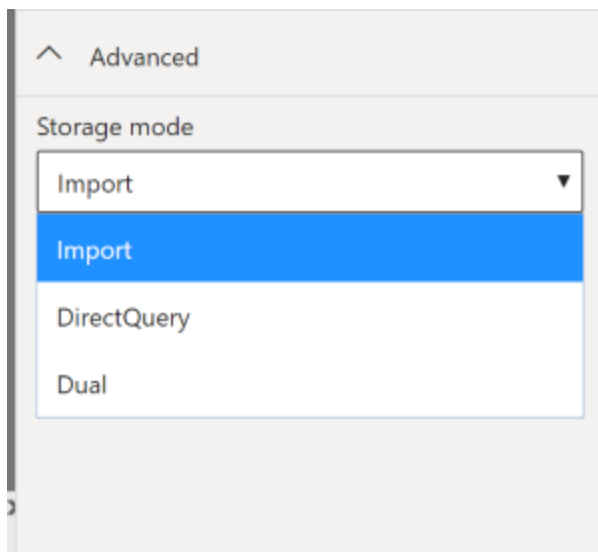
However, sometimes there may be security requirements around your data that make it impossible to directly import a copy. Or your semantic models may simply be too large and would take too long to load into Power BI, and you want to avoid creating a performance bottleneck. Power BI solves these problems by using the DirectQuery storage mode, which allows you to query the data in the data source directly and not import a copy into Power BI.

DirectQuery is useful because it ensures you're always viewing the most recent version of the data.

The three different types of storage modes you can choose from:

- Import
- DirectQuery
- Dual (Composite)

You can access storage modes by switching to the **Model view**, selecting a **data table**, and in the **resulting Properties pane**, selecting which mode that you want to use from the **Storage mode drop-down list**, as shown in the following visual.



Let's take a closer look at the different types of Storage Modes.

Import mode

The **Import mode** allows you to create a local Power BI copy of your semantic models from your data source. You can use **all Power BI service features** with this storage mode, including **Q&A and Quick Insights**. **Data refreshes can be scheduled or on-demand**. Import mode is the default for creating new Power BI reports.

DirectQuery mode

The DirectQuery option is useful when you don't want to save local copies of your data because your data **won't be cached**. Instead, you **can query the specific tables** that you'll need by **using native Power BI queries**, and the required data will be retrieved from the underlying data source. Essentially, you're **creating a direct connection to the data source**. Using this model **ensures that you're always viewing the most up-to-date data**, and that all

security requirements are satisfied. Additionally, this mode is suited for when you have large semantic models to pull data from. Instead of slowing down performance by having to load large amounts of data into Power BI, you can use DirectQuery to create a connection to the source, solving data latency issues as well.

Dual (Composite mode)

In Dual mode, you can identify some data to be directly imported and other data that must be queried. Any table that is brought in to your report is a product of both Import and DirectQuery modes. Using the Dual mode allows Power BI to choose the most efficient form of data retrieval.

For more information regarding Storage Modes, refer to [Storage Modes](#).

Get data from Azure Analysis Services

Azure Analysis Services is a fully managed platform as a service (PaaS) that provides enterprise-grade semantic models in the cloud. You can use advanced mashup and modeling features to combine data from multiple data sources, define metrics, and secure your data in a single, trusted tabular semantic model. The semantic model provides an easier and faster way for users to perform ad hoc data analysis using tools like Power BI.

To resume the scenario, Tailwind Traders uses Azure Analysis Services to store financial projection data. You've been asked to compare this data with actual sales data in a different database. Getting data from Azure Analysis Services server is similar to getting data from SQL Server, in that you can:

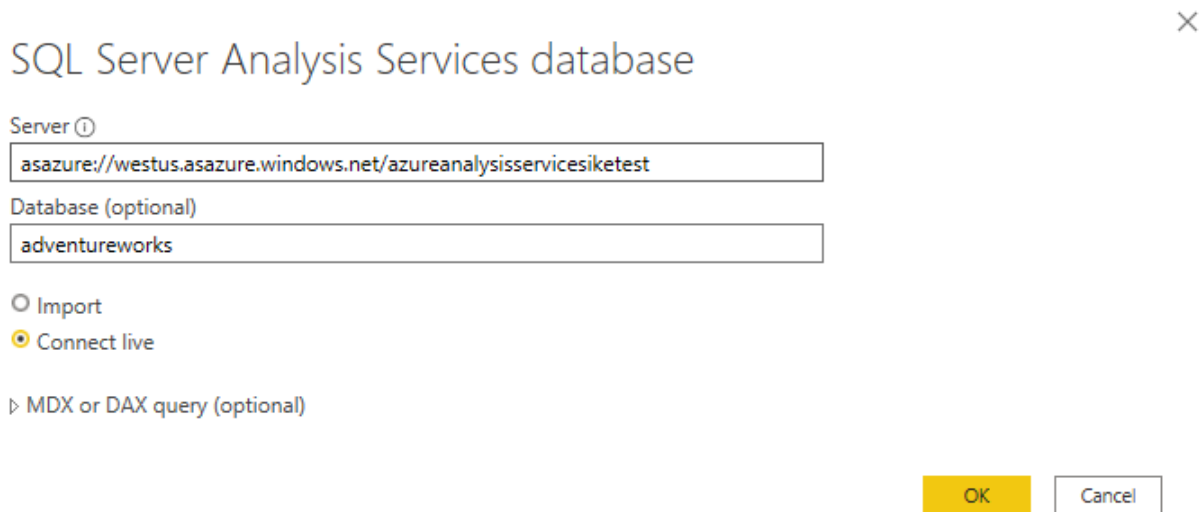
- Authenticate to the server.
- Pick the model you want to use.
- Select which tables you need.

Notable differences between Azure Analysis Services and SQL Server are:

- Analysis Services models have calculations already created.
- If you don't need an entire table, you can query the data directly. Instead of using Transact-SQL (T-SQL) to query the data, like you would in SQL Server, you can use multi-dimensional expressions (MDX) or data analysis expressions (DAX).

Connect to data in Azure Analysis Services

As previously mentioned, you use the **Get data** feature in Power BI Desktop. When you select **Analysis Services**, you're prompted for the **server address and the database name** with two options: **Import** and **Connect live**.



SQL Server Analysis Services database

Server ⓘ
asazure://westus.asazure.windows.net/azureanalysiservicesiketest

Database (optional)
adventureworks

☐ Import
☒ Connect live

▸ MDX or DAX query (optional)

OK Cancel

Connect live is an option for Azure Analysis Services. Azure Analysis Services **uses the tabular model and DAX to build calculations**, similar to Power BI. These models are **compatible with one another**. Using the **Connect live option helps you keep the data and DAX calculations in their original location**, without having to import them all into Power BI. Azure Analysis Services can **have a fast refresh schedule**, which means that when data is refreshed in the service, Power BI reports will **immediately be updated, without the need to initiate a Power BI refresh schedule**. This process can improve the timeliness of the data in your report.

Similar to a relational database, you **can choose the tables that you want** to use. If you want to directly query the Azure Analysis Services model, you can **use DAX or MDX**.

You'll likely import the data directly into Power BI. **An acceptable alternative is to import all other data that you want (from Excel, SQL Server, and so on) into the Azure Analysis Services model and then use a live connection**. This approach simplifies your solution by **keeping the semantic modeling and DAX measures in one place**.

For more information on connecting Power BI to Azure Analysis Services, see [Connect with Power BI documentation](#).

Fix performance issues

Occasionally, organizations will need to address performance issues when running reports. Power BI provides the Performance Analyzer tool to help fix problems and streamline the process.

Consider the scenario where you're building reports for the Sales team in your organization. You've imported your data, which is in several tables within the Sales team's SQL database, by creating a data connection to the database through DirectQuery. When you create preliminary visuals and filters, you notice that some tables are queried faster than others, and some filters are taking longer to process compared to others.

Optimize performance in Power Query

The performance in Power Query depends on the performance at the data source level. The variety of data sources that Power Query offers is wide, and the performance tuning techniques for each source are equally wide. For instance, if you extract data from a Microsoft SQL Server, you should follow the performance tuning guidelines for the product. Good SQL Server performance tuning techniques include index creation, hardware upgrades, execution plan tuning, and data compression. These topics are beyond the scope here, and are covered only as an example to build familiarity with your data source and reap the benefits when using Power BI and Power Query.

Power Query takes advantage of good performance at the data source through a technique called Query Folding.

Query folding

The query folding within Power Query Editor helps you increase the performance of your Power BI reports. Query folding is the process by which the transformations and edits that you make in Power Query Editor are simultaneously tracked as native queries, or simple **Select** SQL statements, while you're actively making transformations. The reason for implementing this process is to ensure that these transformations can take place in the original data source server and don't overwhelm Power BI computing resources.

You can use Power Query to load data into Power BI. Then use Power Query Editor to transform your data, such as renaming or deleting columns, appending, parsing, filtering, or grouping your data.

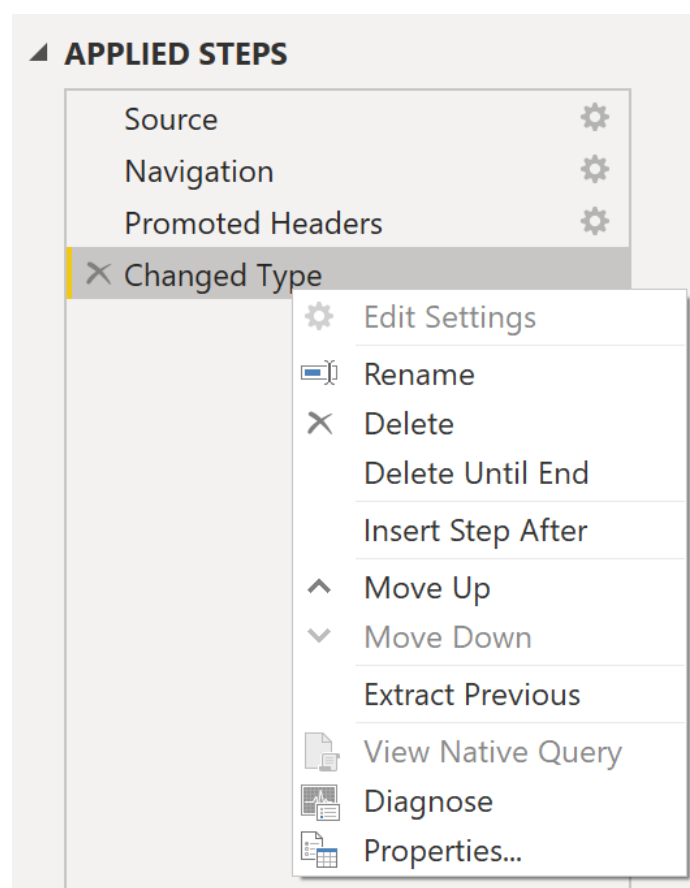
Consider a scenario where you've renamed a few columns in the Sales data and merged a city and state column together in the "city state" format. Meanwhile, the query folding feature tracks those changes in native queries. Then, when you load your data, the

transformations take place independently in the original source, this ensures that performance is optimized in Power BI.

The benefits to query folding include:

- **More efficiency in data refreshes and incremental refreshes.** When you import data tables by using query folding, Power BI is better able to allocate resources and refresh the data faster because Power BI doesn't have to run through each transformation locally.
- **Automatic compatibility with DirectQuery and Dual storage modes.** All DirectQuery and Dual storage mode data sources must have the back-end server processing abilities to create a direct connection, which means that query folding is an automatic capability that you can use. If all transformations can be reduced to a single **Select** statement, then query folding can occur.

The following scenario shows query folding in action. In this scenario, you apply a set of queries to multiple tables. After you add a new data source by using Power Query, and you're directed to the Power Query Editor, you go to the **Query Settings** pane and right-click the last applied step, as shown in the following figure.



If the **View Native Query** option isn't available (not displayed in bold type), then query folding isn't possible for this step, and you'll have to work backward in the **Applied Steps** area until you reach the step in which **View Native Query** is available (displays in bold type). This process will reveal the native query that is used to transform the semantic model.

Native queries aren't possible for the following transformations:

- Adding an index column
- Merging and appending columns of different tables with two different sources
- Changing the data type of a column

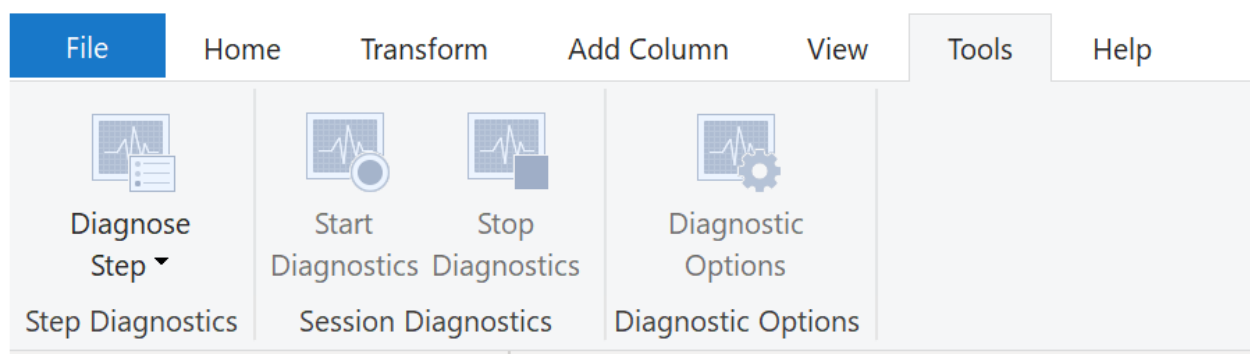
A good guideline to remember is that if you can translate a transformation into a **Select SQL statement**, which includes operators and clauses such as GROUP BY, SORT BY, WHERE, UNION ALL, and JOIN, you can use query folding.

While query folding is one option to optimize performance when retrieving, importing, and preparing data, another option is query diagnostics.

Query diagnostics

Another tool that you can use to study query performance is **query diagnostics**. You can determine what bottlenecks may exist while loading and transforming your data, refreshing your data in Power Query, running SQL statements in Query Editor, and so on.

To access query diagnostics in Power Query Editor, go to **Tools** in the Home ribbon. When you're ready to begin transforming your data or making other edits in Power Query Editor, select **Start Diagnostics** in the **Session Diagnostics** section. When you're finished, make sure that you select **Stop Diagnostics**.



Selecting **Diagnose Step** shows you the length of time that it takes to run that step, as shown in the following image. This selection can tell you if a step takes longer to complete than others, which then serves as a starting point for further investigation.

Queries [17]	Id	Query	Step	Exclusive Duration	Category	Data Source Kind
1	1.1	Product	Changed Type	0.00:00:00.0457545	Evaluator	null
2	1.2	Product	Source	0.00:00:01.9567741	Evaluator	null

This tool is useful when you want to analyze performance on the Power Query side for tasks such as loading semantic models, running data refreshes, or running other transformative tasks.

Other techniques to optimize performance

Other ways to optimize query performance in Power BI include:

- **Process as much data as possible in the original data source.** Power Query and Power Query Editor allow you to process the data; however, the processing power that is required to complete this task might lower performance in other areas of your reports. Generally, a good practice is to process, as much as possible, in the native data source.
- **Use native SQL queries.** When using DirectQuery for SQL databases, such as the case for our scenario, make sure that you aren't pulling data from stored procedures or common table expressions (CTEs).
- **Separate date and time, if bound together.** If any of your tables have columns that combine date and time, make sure that you separate them into distinct columns before importing them into Power BI. This approach will increase compression abilities.

For more information, refer to [Query Folding Guidance](#) and [Query Folding](#).

Resolve data import errors

While importing data into Power BI, you may encounter errors resulting from factors such as:

- Power BI imports from numerous data sources.
- Each data source might have dozens (and sometimes hundreds) of different error messages.
- Other components can cause errors, such as hard drives, networks, software services, and operating systems.
- Data often can't comply with any specific schema.

The following sections cover some of the more common error messages that you might encounter in Power BI.

Query timeout expired

Relational source systems often have many people who are concurrently using the same data in the same database. Some relational systems and their administrators seek to limit a user from monopolizing all hardware resources by setting a query timeout. These timeouts can be configured for any timespan, from as little as five seconds to as much as 30 minutes or more.

For instance, if you're pulling data from your organization's SQL Server, you might see the error shown in the following figure.



Power BI Query Error: Timeout expired

This error indicates that you've pulled too much data according to your organization's policies. Administrators incorporate this policy to avoid slowing down a different application or suite of applications that might also be using that database.

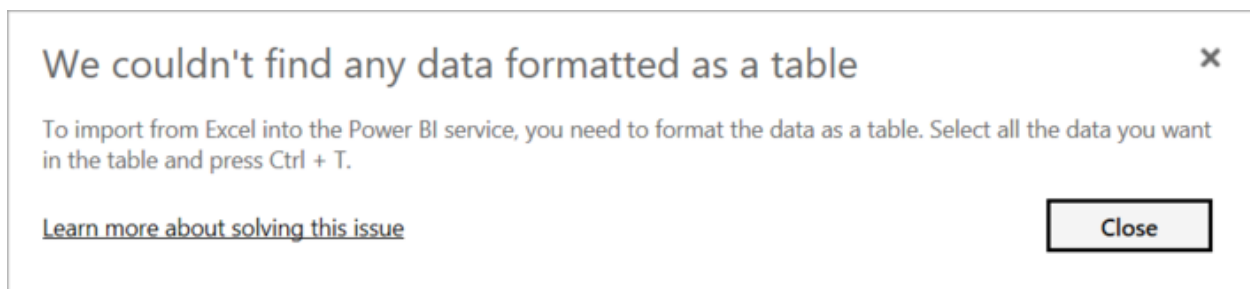
You can resolve this error by pulling fewer columns or rows from a single table. While you're writing SQL statements, it might be a common practice to include groupings and aggregations. You can also join multiple tables in a single SQL statement. Additionally, you can perform complicated subqueries and nested queries in a single statement. These complexities add to the query processing requirements of the relational system and can greatly elongate the time of implementation.

If you need the rows, columns, and complexity, consider **taking small chunks of data** and **then bringing them back together by using Power Query**. For instance, you can **combine half the columns in one query and the other half in a different query**. Power Query can **merge those two queries back together after you're finished**.

We couldn't find any data formatted as a table

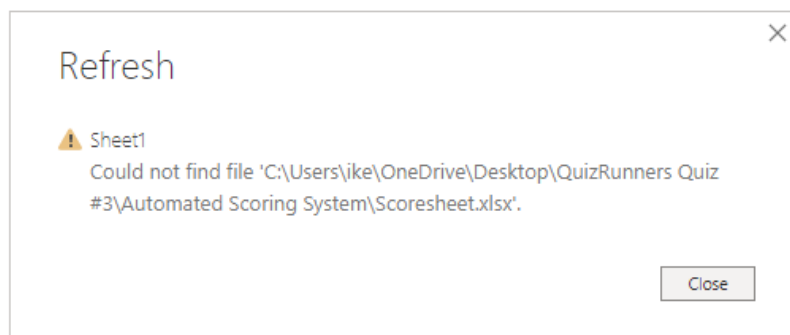
Occasionally, you may encounter the “We couldn’t find any data formatted as a table” error **while importing data from Microsoft Excel**. Fortunately, this error is self-explanatory. **Power BI expects to find data formatted as a table from Excel**. The error even tells you the resolution. Perform the following steps to resolve the issue:

1. Open your Excel workbook, and **highlight the data that you want to import**.
2. Press the **Ctrl-T** keyboard shortcut. The **first row will likely be your column headers**.
3. **Verify that the column headers reflect how you want to name your columns. Then, try to import data** from Excel again. This time, it should work.



Couldn't find file

While importing data from a file, you may get the "Couldn't find file" error.

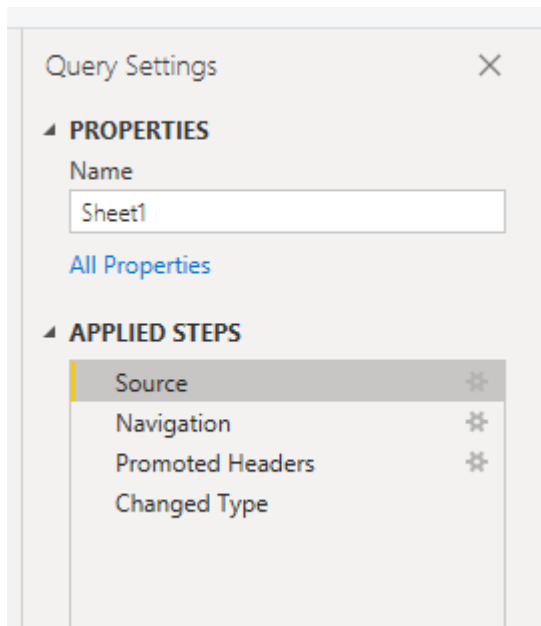


Usually, this error is **caused by the file moving locations or the permissions to the file changing**. If the cause is the former, you need to find the file and change the source settings.

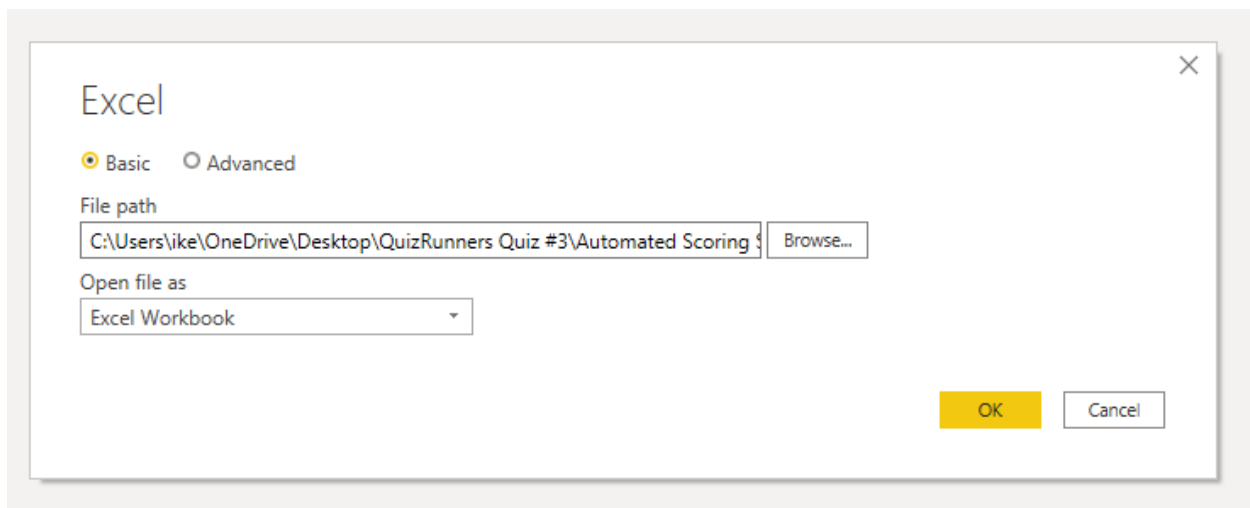
1. Open Power Query by selecting the **Transform Data button** in Power BI.

2. Highlight the query that is creating the error.

3. On the left, under **Query Settings**, select the gear icon next to **Source**.



4. Change the file location to the new location.



Data type errors

Sometimes, when you import data into Power BI, the columns appear blank. This situation happens because of an error in interpreting the data type in Power BI. The resolution to this error is unique to the data source. For instance, if you're importing data from SQL Server and see blank columns, you could try to convert to the correct data type in the query.

Instead of using this query:

```
SELECT CustomerPostalCode FROM Sales.Customers
```

Use this query:

```
SELECT CAST(CustomerPostalCode as varchar(10)) FROM Sales.Customers
```

By specifying the correct type at the data source, you eliminate many of these common data source errors.

You may encounter different types of errors in Power BI that are caused by the diverse data source systems where your data resides.

If you experience an error not covered, you can search [Microsoft documentation](#) for the error message, and the resolution you need.

Exercise - Prepare data in Power BI Desktop

This unit includes a lab to complete.

Use the free resources provided in the lab to complete the exercises in this unit. You will **not be charged for the lab environment**; however, you may need to bring your own subscription depending on the lab.

Microsoft provides this lab experience and related content for educational purposes. All presented information is owned by Microsoft and intended solely for learning about the covered products and services in this Microsoft Learn module.

[Launch lab](#)

The estimated time to complete the exercise is 30 minutes.

Note

A virtual machine containing the client tools you need is provided, along with the exercise instructions. Use the button above to launch the virtual machine.

A limited number of concurrent sessions are available - if the hosted environment is unavailable, try again later.

Alternatively, you can use these [setup instructions](#) to create your own lab environment, then follow these [exercise instructions](#).

Get Data in Power BI Desktop

Lab story

This lab is designed to introduce you to Power BI Desktop application and how to connect to data and how to use data preview techniques to understand the characteristics and quality of the source data. The learning objectives are:

- Open Power BI Desktop
- Connect to different data sources
- Preview source data with Power Query
- Use data profiling features in Power Query

This lab should take approximately 30 minutes.

Get started with Power BI Desktop

To complete this exercise, first open a web browser and enter the following URL to download the zip folder:

<https://github.com/MicrosoftLearning/PL-300-Microsoft-Power-BI-Data-Analyst/raw/Main/Allfiles/Labs/01-prepare-data-with-power-query-in-power-bi-desktop/01-prepare-data.zip>

Extract the folder to the **C:\Users\Student\Downloads\01-prepare-data** folder.

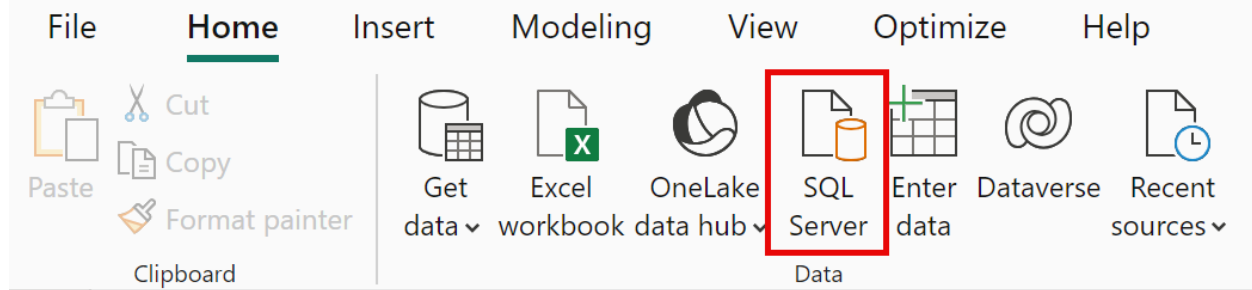
Open the **01-Starter-Sales Analysis.pbix** file.

- This starter file has been specially configured to help you complete the lab. The following report-level settings have been disabled in the starter file:
 - Data Load > Import relationships from data sources on first load
 - Data Load > Autodetect new relationships after data is loaded

Get data from SQL Server

This task teaches you how to connect to a SQL Server database and import tables, which create queries in Power Query.

1. On the **Home** ribbon tab, from inside the **Data** group, select **SQL Server**.



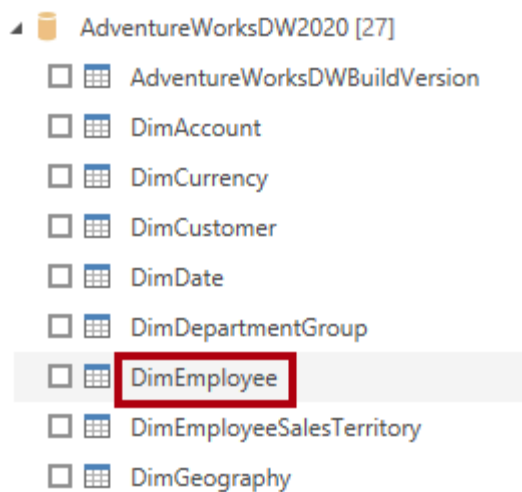
2. In the **SQL Server Database** window, in the **Server** box, enter **localhost** and leave **Database** blank, then select **OK**.

Note: In this lab, you'll connect to the SQL Server database by using **localhost** because gateway data sources can't resolve **localhost**. This isn't a recommended practice when creating your own solutions.

3. If prompted for credentials, select **Windows > Use my current credentials**, and then **Connect**.
4. Select **OK** if you receive a warning that an encrypted connection cannot be established.
5. In the **Navigator** pane, expand the **AdventureWorksDW2020** database.

Note: The **AdventureWorksDW2020** database is based on the **AdventureWorksDW2017** sample database. It has been modified to support the learning objectives of the course labs.

6. Select the **DimEmployee** table, and notice the preview of the table data.



Note: The preview data allows you to see the columns and a sample of rows.

7. To import the table data, **select the checkbox next to the following tables:**

- DimEmployee
- DimEmployeeSalesTerritory
- DimProduct
- DimReseller
- DimSalesTerritory
- FactResellerSales

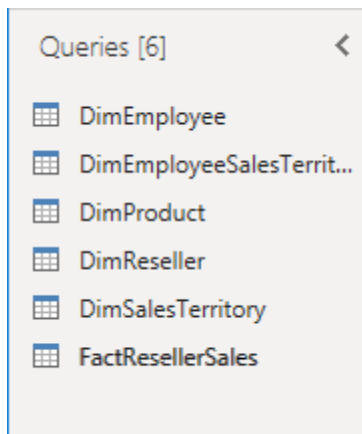
8. Complete this task by selecting **Transform Data**, which will open Power Query Editor - leave this open for the next task.

You've now connected to six tables from a SQL Server database.

Preview Data in Power Query Editor

This task introduces the Power Query Editor and allows you to review and profile the data. This helps you determine how to clean and transform the data later. You'll also review both dimension tables prefixed with "Dim" and fact tables prefixed with "Fact".

1. In the **Power Query Editor** window, at the left, notice the **Queries** pane. The **Queries** pane contains one query for each table you checked.



2. Select the first query—**DimEmployee**.

*The **DimEmployee** table in the SQL Server database stores one row for each employee. A subset of the rows from this table represents the salespeople, which will be relevant to the model you'll develop.*

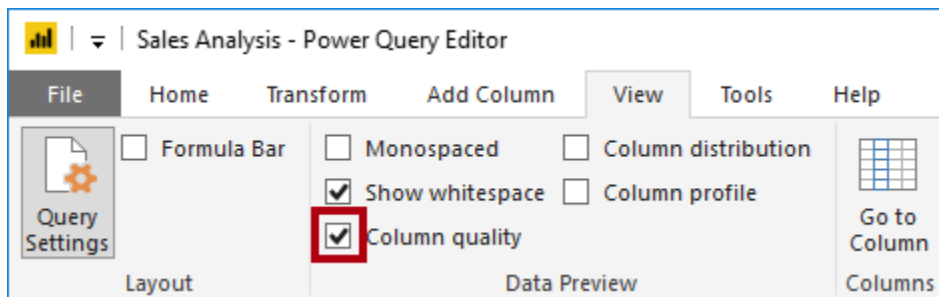
- At the bottom left corner of the status bar, some table statistics are provided—the table has 33 columns, and 296 rows.

33 COLUMNS, 296 ROWS Column profiling based on top 1000 rows

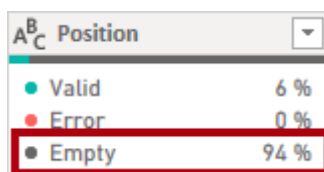
- In the data preview pane, scroll horizontally to review all columns. Notice that the last five columns contain **Table** or **Value** links.

*These five columns represent relationships to other tables in the database. They can be used to join tables together. You'll join tables in the **Load Transformed Data in Power BI Desktop** lab.*

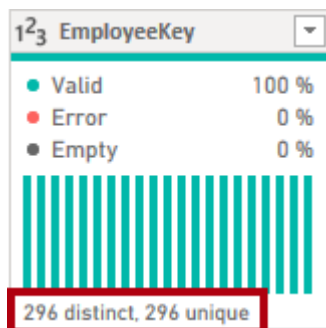
- To assess column quality, on the **View** ribbon tab, from inside the **Data Preview** group, check **Column Quality**. The column quality feature allows you to easily determine the percentage of valid, error, or empty values found in columns.



- Notice that the **Position** column has 94% empty (null) rows.



- To assess column distribution, on the **View** ribbon tab, from inside the **Data Preview** group, check **Column Distribution**.
- Review the **Position** column again, and notice that there are four distinct values, and one unique value.
- Review the column distribution for the **EmployeeKey** column—there are 296 distinct values, and 296 unique values.



Note: When the distinct and unique counts are the same, it means the column contains unique values. When modeling, it's important that some model tables have unique columns. These unique columns can be used to create one-to-many relationships, which you'll do in the **Model Data in Power BI Desktop** lab.

10. In the **Queries** pane, select the **DimProduct** query.

The **DimProduct** table contains one row per product sold by the company.

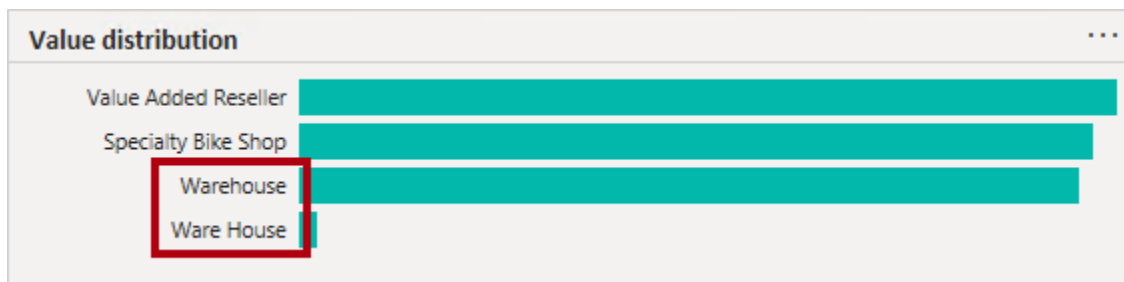
11. In the **Queries** pane, select the **DimReseller** query.

The **DimReseller** table contains one row per reseller. Resellers sell, distribute, or value add to the Adventure Works products.

12. To view column values, on the **View** ribbon tab, from inside the **Data Preview** group, check **Column Profile**.

13. Select the **BusinessType** column header, and notice the new pane beneath the data preview pane. Review the column statistics and value distribution in the data preview pane.

Notice the data quality issue: there are two labels for warehouse (**Warehouse**, and the misspelled **Ware House**).



14. Hover the cursor over the **Ware House** bar, and notice that there are five rows with this value.

15. In the **Queries** pane, select the **DimSalesTerritory** query.

The **DimSalesTerritory** table contains one row per sales region, including **Corporate HQ** (headquarters). Regions are assigned to a country, and countries are assigned to groups. In the **Model Data in Power BI Desktop** lab, you'll create a hierarchy to support analysis at region, country, or group level.

16. In the **Queries** pane, select the **FactResellerSales** query.

The **FactResellerSales** table contains one row per sales order line—a sales order contains one or more line items.

17. Review the column quality for the **TotalProductCost** column, and notice that 8% of the rows are empty.

Missing **TotalProductCost** column values is a data quality issue.

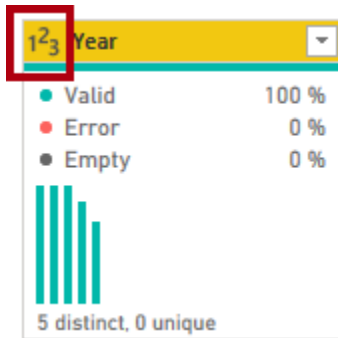
Get data from a CSV file

In this task, you'll create a new query based on CSV files.

1. To add a new query, in the **Power Query Editor** window, on the **Home** ribbon tab, from inside the **New Query** group, select the **New Source** down-arrow, and then select **Text/CSV**.
2. Navigate to the **01-prepare-data > ResellerSalesTargets.csv** file. Select **Open**.
3. In the **ResellerSalesTargets.csv** window, review the preview data. Select **OK**.
4. In the **Queries** pane, notice the addition of the **ResellerSalesTargets** query.

The **ResellerSalesTargets** CSV file contains one row per salesperson, per year. Each row records 12 monthly sales targets (expressed in thousands). The business year for the Adventure Works company commences on July 1.

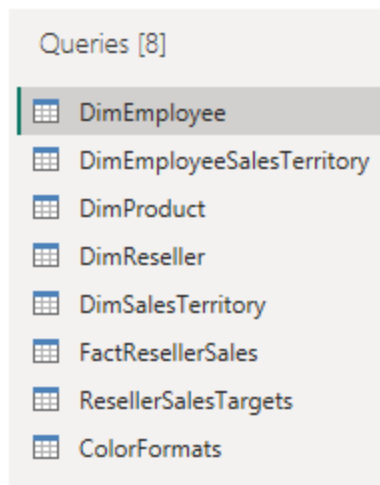
5. Notice that no column contains empty values. When there isn't a monthly sales target, a hyphen character is stored instead.
6. Review the icons in each column header, to the left of the column name. The icons represent the column data type. **123** is whole number, and **ABC** is text.



- Repeat the steps to create a query based on the **ColorFormats.csv** file.

The **ColorFormats** CSV file contains one row per product color. Each row records the HEX codes to format background and font colors.

You should now have two new queries, **ResellerSalesTargets** and **ColorFormats**.



Lab complete

Knowledge Check

Summary

In this module, you learned about pulling data from many different data sources and into Power BI. You can pull data from files, relational databases, Azure Analysis Services, cloud-based applications, websites, and more. Retrieving data from different data sources requires treating each data source differently. For instance, Microsoft Excel data should be pulled in from an Excel table. Relational databases often have query timeouts. You can

connect to SQL Server Analysis Services with **Connect live**, which allows you to see data changes in real-time.

It's important to select the correct **storage mode** for your data. Do you require that visuals **interact quickly but don't mind possibly refreshing** the data when the underlying data source changes? If so, **select Import** to import data into Power BI. If you prefer to see **updates to data as soon as they happen** at the cost of interactivity performance, then **choose Direct Query** for your data instead.

In addition, you learned how to **solve performance problems and data import errors**. You learned that Power BI gives you tooling to identify **where performance problems may exist**. **Data import errors can be alarming at first**, but you can see that the resolution is easily implemented.

Clean, transform, and load data in Power BI

Introduction

Consider the **scenario where you have imported data** into Power BI from several different sources and, when you examine the data, it is **not prepared for analysis**. What could make the data unprepared for analysis?

When examining the data, you discover several issues, including:

- A **column** called **Employment status** **only contains numerals**.
- Several columns **contain errors**.
- Some columns **contain null** values.
- The customer ID in some columns appears as if it was **duplicated repeatedly**.
- A single address column has **combined street address**, city, state, and zip code.

You start working with the data, but **every time you create visuals** on reports, **you get bad data, incorrect results**, and simple reports about sales totals are wrong.

Dirty data can be overwhelming and, though you might feel frustrated, you decide to get to work and figure out how to make this semantic model as pristine as possible.

Fortunately, **Power BI and Power Query offer you a powerful environment to clean and prepare the data**. Clean data has the following advantages:

- Measures and columns **produce more accurate results** when they perform aggregations and calculations.

- **Tables are organized**, where users can find the data in an intuitive manner.
- **Duplicates are removed**, making data **navigation simpler**. It will also produce **columns that can be used in slicers and filters**.
- A **complicated column can be split into two**, simpler columns. **Multiple columns can be combined** into one column for readability.
- **Codes and integers can be replaced** with **human readable values**.

In this module, you will learn how to:

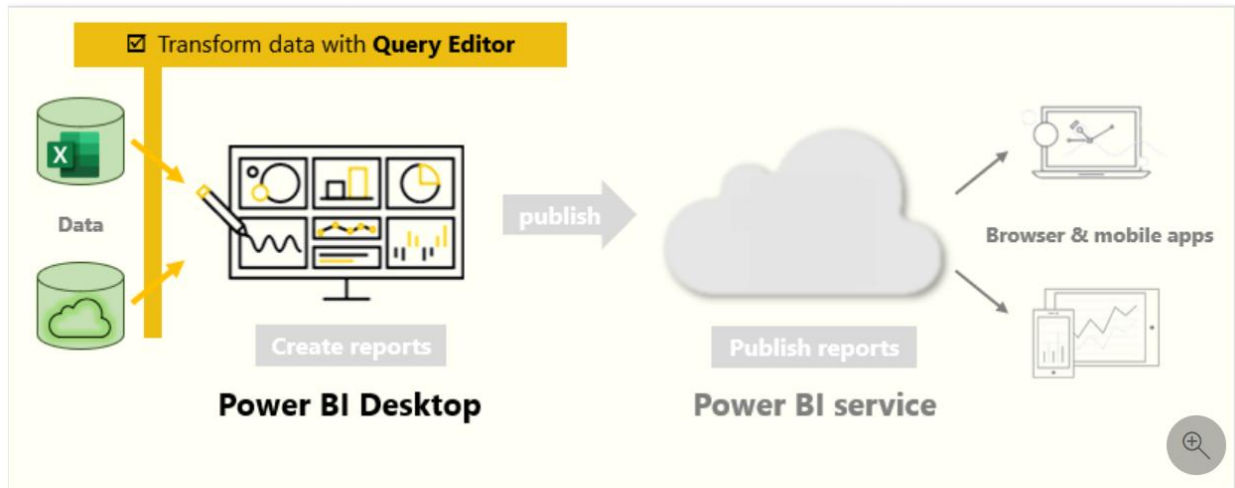
- Resolve inconsistencies, unexpected or null values, and **data quality issues**.
- Apply **user-friendly value replacements**.
- **Profile data so you can learn more about a specific column** before using it.
- **Evaluate and transform column data types**.
- **Apply data shape transformations** to table structures.
- **Combine queries**.
- **Apply user-friendly naming conventions** to columns and queries.
- **Edit M code** in the Advanced Editor.

Shape the initial data

Power Query Editor in Power BI Desktop **allows you to shape** (transform) your imported data. You can accomplish actions **such as renaming columns or tables, changing text to numbers, removing rows, setting the first row as headers**, and much more. It is important to shape your data to ensure that it meets your needs and is suitable for use in reports.

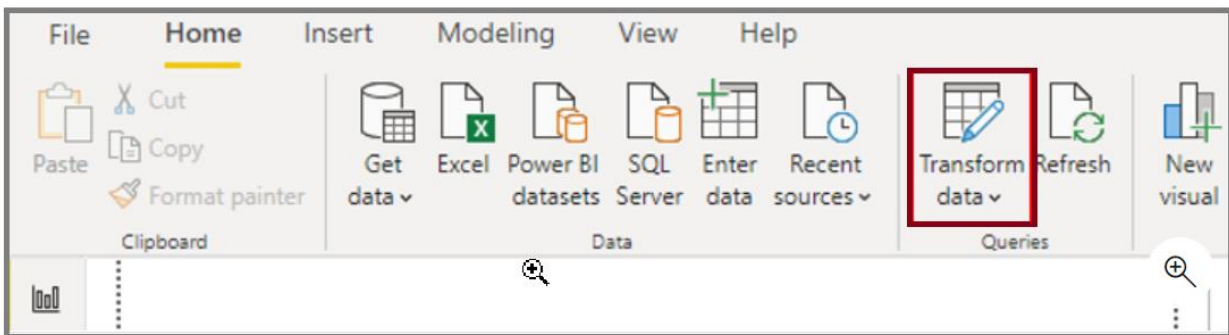
You have **loaded raw sales data from two sources** into a Power BI model. Some of the data came from a **.csv file** that was created **manually in Microsoft Excel by the Sales team**. The **other data** was **loaded through a connection** to your organization's Enterprise Resource Planning (ERP) system. Now, when you look at the data in Power BI Desktop, you notice that it's in disarray; **some data that you don't need and some data that you do need are in the wrong format**.

You need to use **Power Query Editor to clean up and shape** this data before you can start building reports.



Get started with Power Query Editor

To start shaping your data, open Power Query Editor by selecting the **Transform data** option on the **Home** tab of Power BI Desktop.



In Power Query Editor, the data in your selected query displays in the middle of the screen and, on the left side, the **Queries pane** lists the available queries (tables).

When you work in Power Query Editor, **all steps that you take** to shape your data **are recorded**. Then, **each time the query connects to the data source, it automatically applies your steps**, so your **data is always shaped** the way that you specified. Power Query Editor **only makes changes to a particular view** of your data, so you can feel confident about changes that are being made to your original data source. You can **see a list of your steps on the right side of the screen**, in the **Query Settings** pane, along with the query's properties.

The Power Query Editor ribbon contains many buttons you can use to select, view, and shape your data.

To learn more about the available features and functions, see [The query ribbon](#).

Note

In Power Query Editor, the right-click context menus and **Transform** tab in the ribbon provide many of the same options.

Identify column headers and names

The **first step** in shaping your initial data is to **identify the column headers and names** within the data and then evaluate where they are located to ensure that they are in the right place.

In the following screenshot, the source data in the csv file for SalesTarget (sample not provided) had a **target categorized by products and a subcategory** split by months, both of which are organized into columns.

A	B	C	D	E	F	G	H	I	J	K	L	M	N
ProductSubcategoryID	Name	January	February	March	April	May	June	July	August	September	October	November	December
1	Mountain Bikes	780000	790000	800000	810000	820000	830000	840000	850000	860000	870000	880000	890000
2	Road Bikes	4500	5000	5500	6000	6500	7000	7500	8000	8500	9000	9500	10000
3	Touring Bikes	501000	502000	503000	504000	505000	506000	507000	508000	509000	510000	511000	512000
4	Handlebars	1100	1200	1300	1400	1500	1600	1700	1800	1900	2000	2100	2200
5	Bottom Brackets	1100	1200	1300	1400	1500	1600	1700	1800	1900	2000	2100	2200
6	Brakes	1100	1200	1300	1400	1500	1600	1700	1800	1900	2000	2100	2200
7	Chains	1100	1200	1300	1400	1500	1600	1700	1800	1900	2000	2100	2200
8	Cranksets	1100	1200	1300	1400	1500	1600	1700	1800	1900	2000	2100	2200
9	Derailleurs	1100	1200	1300	1400	1500	1600	1700	1800	1900	2000	2100	2200

However, you notice that the data did not import as expected.

A ^B Column1	A ^B Column2	A ^B Column3	A ^B Column4	A ^B Column5	A ^B Column6	A ^B Column7
1		January	February	March	April	May
2						
3	ProductSubcategoryID	Name				
4	1	Mountain Bikes	780000	790000	800000	810000
5	2	Road Bikes	4500	5000	5500	6000
6	3	Touring Bikes	501000	502000	503000	504000
7	4	Handlebars	1100	1200	1300	1400
8	5	Bottom Brackets	1100	1200	1300	1400
9	6	Brakes	1100	1200	1300	1400
10	7	Chains	1100	1200	1300	1400
11	8	Cranksets	1100	1200	1300	1400
12	9	Derailleurs	1100	1200	1300	1400

Consequently, the **data is difficult to read**. A problem has occurred with the data in its current state because **column headers are in different rows** (marked in red), and **several columns have undescriptive names**, such as **Column1**, **Column2**, and so on.

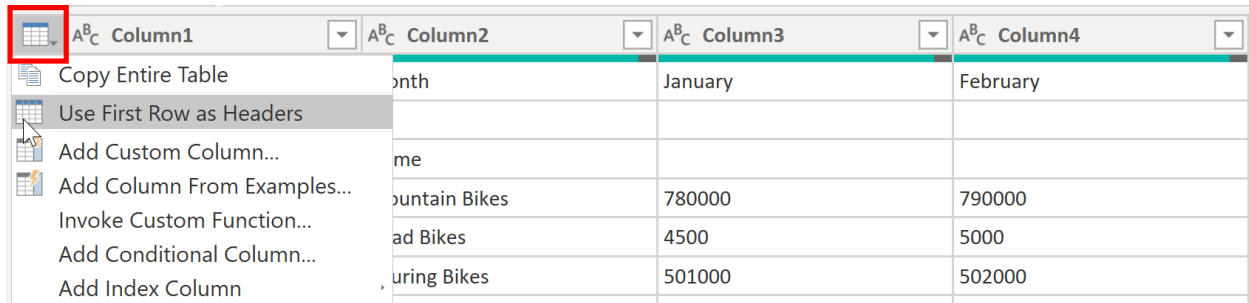
When you have **identified where the column headers and names are located**, you can **make changes** to reorganize the data.

Promote headers

When a table is created in Power BI Desktop, **Power Query Editor assumes that all data belongs in table rows**. However, a data source might have a **first row that contains column**

names, which is what happened in the previous SalesTarget example. To correct this inaccuracy, you need to promote the first table row into column headers.

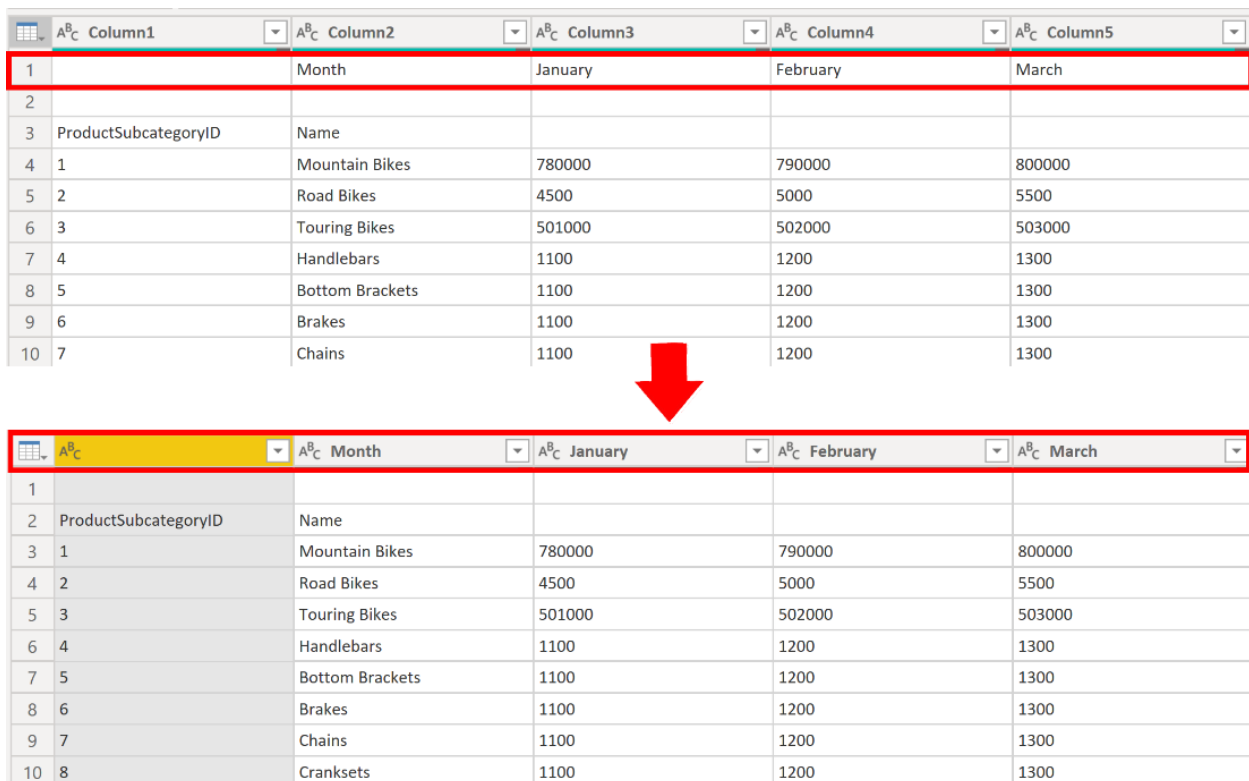
You can promote headers in two ways: by selecting the **Use First Row as Headers** option on the **Home** tab or by selecting the drop-down button next to **Column1** and then selecting **Use First Row as Headers**.



The screenshot shows a table with four columns: Column1, Column2, Column3, and Column4. The first row of data is highlighted in red. The dropdown menu for Column1 is open, and the 'Use First Row as Headers' option is selected.

Column1	Column2	Column3	Column4
Month	January	February	
Mountain Bikes	780000	790000	
Road Bikes	4500	5000	
Touring Bikes	501000	502000	

The following image illustrates how the **Use First Row as Headers** feature impacts the data:



The top screenshot shows a table with five columns: Column1, Column2, Column3, Column4, and Column5. The first row of data is highlighted in red. A red arrow points down to the bottom screenshot, which shows the same table after the first row has been promoted to the header row. The first row is now the header, and the data rows start from the second row.

Column1	Column2	Column3	Column4	Column5
Month	January	February	March	
ProductSubcategoryID	Name			
1	Mountain Bikes	780000	790000	800000
2	Road Bikes	4500	5000	5500
3	Touring Bikes	501000	502000	503000
4	Handlebars	1100	1200	1300
5	Bottom Brackets	1100	1200	1300
6	Brakes	1100	1200	1300
7	Chains	1100	1200	1300
8	Cranksets	1100	1200	1300

Rename columns

The next step in shaping your data is to examine the column headers. You might discover that one or more columns have the wrong headers, a header has a spelling error, or the header naming convention is not consistent or user-friendly.

Refer to the previous screenshot, which shows the impact of the **Use First Row as Headers** feature. Notice that the column that contains the subcategory Name data now has **Month** as its column header. This column header is incorrect, so it needs to be renamed.

You can rename column headers in **two ways**. One approach is to **right-click the header, select Rename**, edit the name, and then press **Enter**. Alternatively, you **can double-click the column header and overwrite the name** with the correct name.

You can **also work around this issue by removing (skipping) the first two rows and then renaming** the columns to the correct name.

Remove top rows

When shaping your data, you **might need to remove some of the top rows**, for example, if they are blank or if they contain data that you do not need in your reports.

Continuing with the SalesTarget example, notice that the **first row is blank** (it has no data) and the **second row has data that is no longer required**.

	ProductSubcategoryID	Subcategory Name	January	February	March
1					
2	ProductSubcategoryID	Name			
3	1	Mountain Bikes	780000	790000	800000
4	2	Road Bikes	4500	5000	5500
5	3	Touring Bikes	501000	502000	503000
6	4	Handlebars	1100	1200	1300

To remove these excess rows, select **Remove Rows > Remove Top Rows** on the **Home** tab.

The screenshot shows the Power Query ribbon with the 'Home' tab selected. The 'Remove Rows' button is highlighted, and its dropdown menu is open, showing options like 'Remove Top Rows', 'Remove Bottom Rows', 'Remove Alternate Rows', 'Remove Duplicates', 'Remove Blank Rows', and 'Remove Errors'. The 'Remove Top Rows' option is selected. Below the ribbon, a portion of the data table is visible, showing the first three rows of the 'SalesTarget' example.

Remove columns

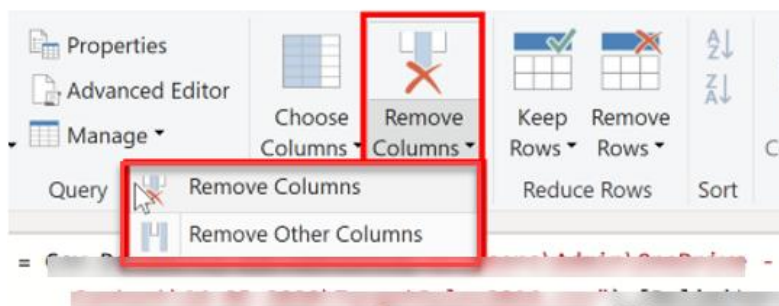
A key step in the data shaping process is to **remove unnecessary columns**. It is much better to **remove columns as early as possible**. One way to remove columns would be to **limit the column when you get data from data source**. For instance, if you are extracting data from a

relational database by using **SQL**, you would want to limit the column that you extract by using a column list in the **SELECT** statement.

Removing columns at an early stage in the process rather than later is best, especially when you have established relationships between your tables. Removing unnecessary columns will help you to focus on the data that you need and help improve the overall performance of your Power BI Desktop semantic models and reports.

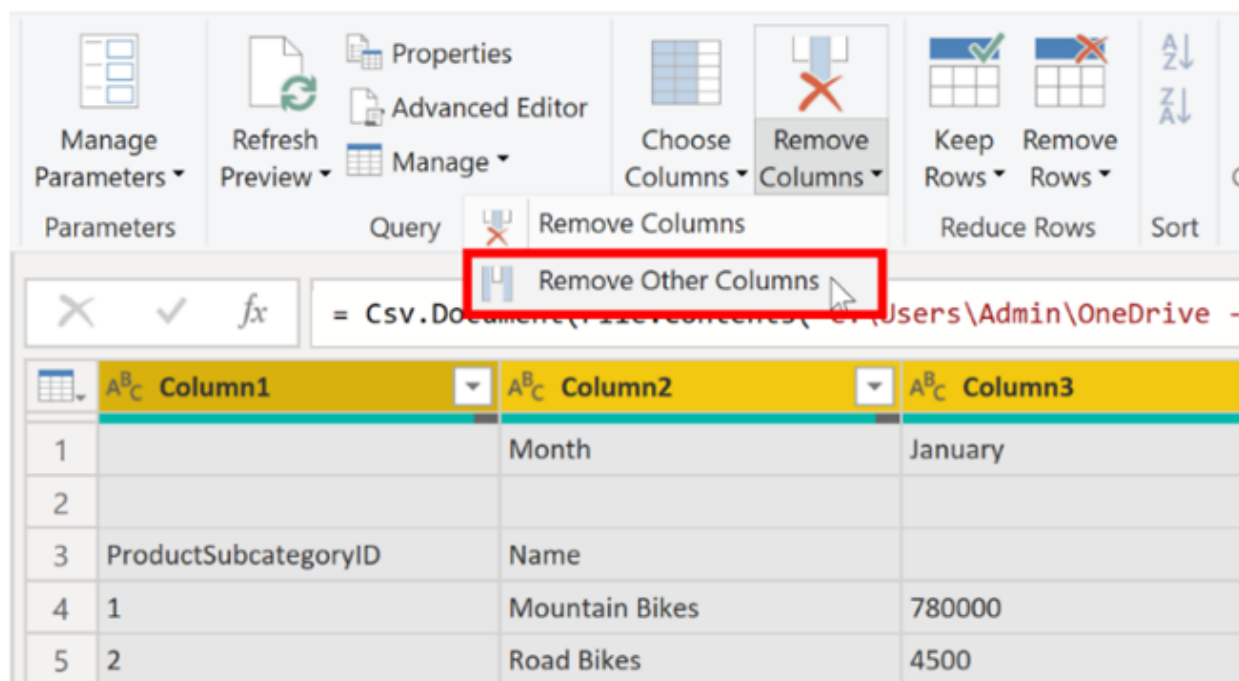
Examine each column and ask yourself if you really need the data that it contains. If you don't plan on using that data in a report, the column adds no value to your semantic model. Therefore, the column should be removed. You can always add the column later, if your requirements change over time.

You can remove columns in two ways. The first method is to select the columns that you want to remove and then, on the **Home** tab, select **Remove Columns**.



2	ABC Column13	ABC Column14
	November	December
	880000	890000
	9500	10000
	511000	512000
	2100	2200
	2100	2200
	---	---

Alternatively, you can select the columns that you want to keep and then, on the **Home** tab, select **Remove Columns > Remove Other Columns**.



Unpivot columns

Unpivoting is a useful feature of Power BI. You can use this feature with data from any data source, but you would most often use it when importing data from Excel. The following example shows a sample Excel document with sales data.

Year	2018		2019	
January	\$	15,370	\$	16,063
February	\$	15,950	\$	12,161
March	\$	13,862	\$	14,180
April	\$	18,530	\$	6,516
May	\$	5,203	\$	19,395
June	\$	5,928	\$	19,324
July	\$	14,736	\$	15,939
August	\$	6,243	\$	15,390
September	\$	15,178	\$	17,832
October	\$	18,148	\$	5,185
November	\$	8,014	\$	9,299
December	\$	19,470	\$	14,082

Though the data might initially make sense, it would be difficult to create a total of all sales combined from 2018 and 2019. Your goal would then be to use this data in Power BI with three columns: **Month**, **Year**, and **SalesAmount**.

When you import the data into Power Query, it will look like the following image.

	Year	2018	2019
1	January	15370	16063
2	February	15950	12161
3	March	13862	14180
4	April	18530	6516
5	May	5203	19395
6	June	5928	19324
7	July	14736	15939
8	August	6243	15390
9	September	15178	17832
10	October	18148	5185
11	November	8014	9299
12	December	19470	14082

Next, rename the first column to **Month**. This column was mislabeled because that header in Excel was labeling the 2018 and 2019 columns. Highlight the 2018 and 2019 columns, select the **Transform** tab in Power Query, and then select **Unpivot**.

	Year	Attribute	Value
1	January	2018	15370
1	January	2019	16063
1	February	2018	15950
1	February	2019	12161
1	March	2018	13862
1	March	2019	14180
1	April	2018	18530
1	April	2019	6516
1	May	2018	5203
1	May	2019	19395
1	June	2018	5928
2	June	2019	19324
3	July	2018	14736
4	July	2019	15939
5	August	2018	6243
5	August	2019	15390
7	September	2018	15178
7	September	2019	17832
9	October	2018	18148
3	October	2019	5185
1	November	2018	8014
2	November	2019	9299
3	December	2018	19470
4	December	2019	14082

You can rename the **Attribute** column to **Year** and the **Value** column to **SalesAmount**.

Unpivoting streamlines the process of creating DAX measures on the data later. By completing this process, you have now created a simpler way of slicing the data with the **Year** and **Month** columns.

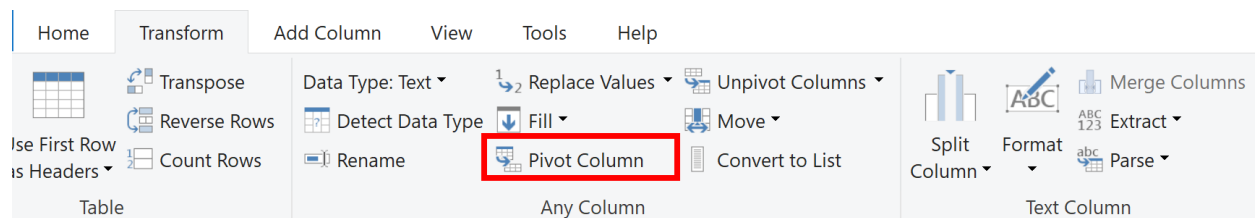
Pivot columns

If the data that you are shaping is flat (in other words, it has lot of detail but is not organized or grouped in any way), the lack of structure can complicate your ability to identify patterns in the data.

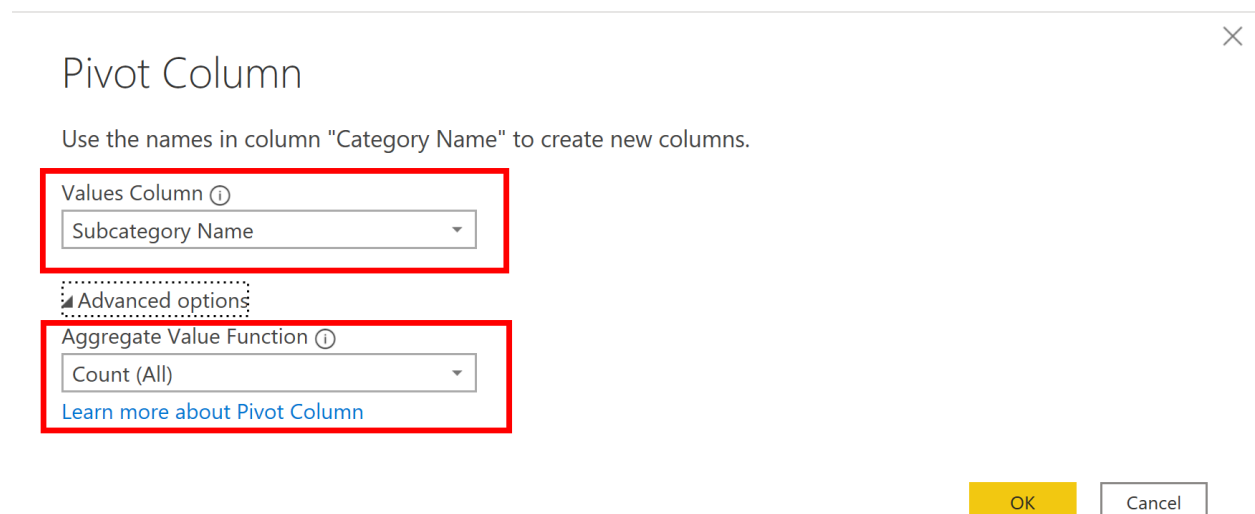
You can use the **Pivot Column** feature to convert your flat data into a table that contains an aggregate value for each unique value in a column. For example, you might want to use this feature to summarize data by using different math functions such as **Count**, **Minimum**, **Maximum**, **Median**, **Average**, or **Sum**.

In the SalesTarget example, you can pivot the columns to get the quantity of product subcategories in each product category.

On the **Transform** tab, select **Transform > Pivot Columns**.



On the **Pivot Column** window that displays, select a column from the **Values Column** list, such as **Subcategory name**. Expand the **advanced options** and select an option from the **Aggregate Value Function** list, such as **Count (All)**, and then select **OK**.



The following image illustrates how the **Pivot Column** feature changes the way that the data is organized.

	A ^B _C Category Name	A ^B _C Subcategory Name
1	Bikes	Mountain Bikes
2	Bikes	Road Bikes
3	Bikes	Touring Bikes
4	Clothing	Bib-Shorts
5	Clothing	Caps
6	Clothing	Gloves
7	Clothing	Jerseys
8	Clothing	Shorts
9	Clothing	Socks
10	Clothing	Tights
11	Clothing	Vests
12	Accessories	Bike Racks
13	Accessories	Bike Stands
14	Accessories	Bottles and Cages

	1.2 Bikes	1.2 Components	1.2 Clothing	1.2 Accessories
1	3	14	8	12

Power Query Editor records all steps that you take to shape your data, and the list of steps are shown in the **Query Settings** pane. If you have made all the required changes, select **Close & Apply** to close Power Query Editor and apply your changes to your semantic model. However, before you select **Close & Apply**, you can take further steps to clean up and transform your data in Power Query Editor. These additional steps are covered later in this module.

Simplify the data structure

When you import data from multiple sources into Power BI Desktop, the data retains its predefined table and column names. You might want to change some of these names so that they are in a consistent format, easier to work with, and more meaningful to a user. You can use Power Query Editor in Power BI Desktop to make these name changes and simplify your data structure.

To continue with the previous scenario where you shaped the initial data in your model, you need to take further action to simplify the structure of the sales data and get it ready for developing reports for the Sales team. You have already renamed the columns, but now you need to examine the names of the queries (tables) to determine if any improvements can be made. You also need to review the contents of the columns and replace any values that require correction.

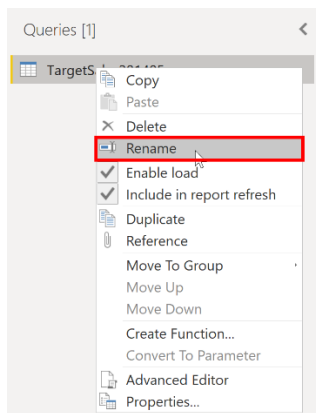
Rename a query

It's good practice to change uncommon or unhelpful query names to names that are more obvious or that the user is more familiar with. For instance, if you import a product fact table into Power BI Desktop and the query name displays as *FactProductTable*, you might want to change it to a more user-friendly name, such as *Products*. Similarly, if you import a view, the

view might have a name that contains a prefix of **v**, such as **vProduct**. People might find this name unclear and confusing, so you might want to remove the prefix.

In this example, you have examined the name of the **TargetSales** query and realize that this name is unhelpful because you'll have a query with this name for every year. To avoid confusion, you want to add the year to the query name.

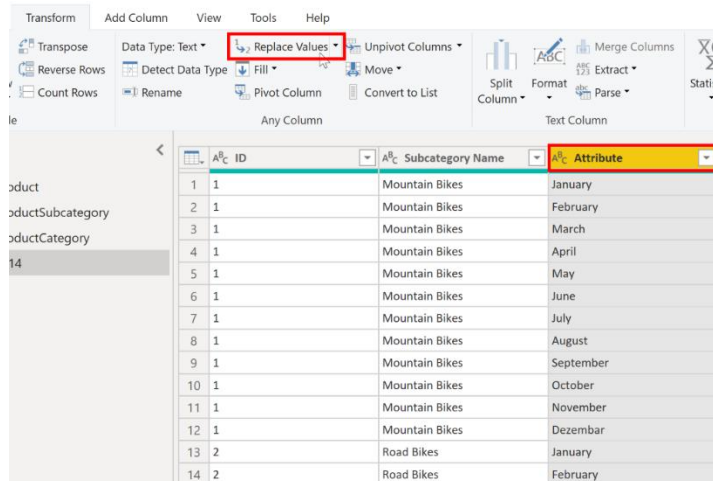
In Power Query Editor, in the **Queries pane** to the left of your data, select the query that you want to rename. Right-click the query and select **Rename**. Edit the current name or type a new name, and then press **Enter**.



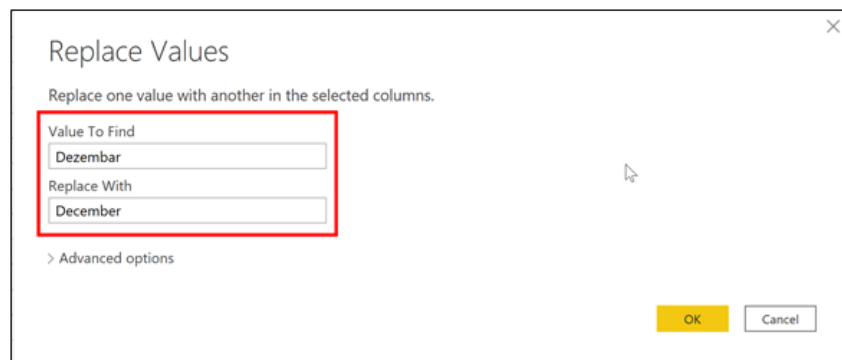
Replace values

You can use the **Replace Values** feature in Power Query Editor to replace any value with another value in a selected column.

In this example, you notice that, in the **Attribute** column, the month **December** is misspelled. You need to correct this spelling mistake. Select the column that contains the value that you want to replace (**Attribute** in this case), and then select **Replace Values** on the **Transform** tab.



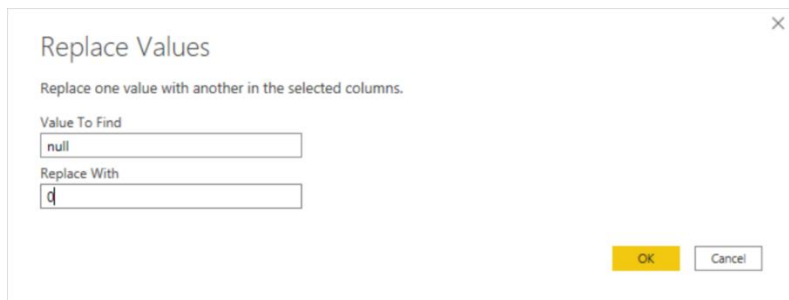
In the **Value to Find** box, enter the name of the value that you want to replace, and then in the **Replace With** box, enter the correct value name and then select **OK**. In Power Query, you can't select one cell and change one value, like you might have done in Excel.



You can review the list of steps that you took to restructure and correct your data in the **Query Settings pane**. When you have completed all steps that you want to take, you can select **Close & Apply** to close Power Query Editor and apply your changes to your data model. However, you can take further action to clean and transform your data.

Replace null values

Occasionally, you might find that your data sources contain null values. For example, a freight amount on a sales order might have a null value if it's synonymous with zero. If the value stays null, the averages will not calculate correctly. One solution would be to change the nulls to zero, which will produce the more accurate freight average. In this instance, using the same steps that you followed previously will help you replace the null values with zero.



Replace Values

Replace one value with another in the selected columns.

Value To Find
null

Replace With
q

OK Cancel

Remove duplicates

You can also remove duplicates from columns to only keep unique names in a selected column by using the **Remove Duplicates** feature in Power Query.

In this example, notice that the **Category Name** column contains duplicates for each category. As a result, you want to create a table with unique categories and use it in your semantic model. You can achieve this action by selecting a column, right-clicking on the header of the column, and then selecting the **Remove Duplicates** option.

You might consider copying the table before removing the duplicates. The **Copy** option is at the top of the context menu, as shown in the following screenshot. Copying the table before removing duplicates will give you a comparison of the tables and will let you use both tables, if needed.

	A ^B C Category Name
1	Components
2	Accessories
3	Components
4	Components
5	Components
6	Components
7	Components
8	Components
9	Components
10	Components
11	Components
12	Components
13	Components
14	Bikes
15	Bikes
16	Bikes
17	Bikes
18	Bikes
19	Bikes
20	Bikes
21	Bikes
22	Bikes
23	Bikes



	A ^B C Category Name
1	Components
2	Accessories
3	Bikes
4	Clothing

Best practices for naming tables, columns, and values

Naming conventions for tables, columns, and values have **no fixed rules**; however, we recommend that you **use the language and abbreviations that are commonly used within your organization** and that everyone agrees on and considers them as common terminology.

A **best practice is to give your tables, columns, and measures descriptive business terms and replace underscores ("_") with spaces**. Be consistent with abbreviations, prefixes, and words like "number" and "ID." **Excessively short abbreviations can cause confusion** if they are **not commonly used within the organization**.

Also, by **removing prefixes or suffixes** that you might use in table names and instead naming them in a simple format, you will **help avoid confusion**.

When replacing values, **try to imagine how those values will appear on the report**. Values that are **too long might be difficult to read** and fit on a visual. Values that are **too short might be difficult to interpret**. **Avoiding acronyms in values is also a good idea**, provided that the text will fit on the visual.

Evaluate and change column data types

When you import a table from any data source, Power BI Desktop automatically starts scanning the first 1,000 rows (default setting) and tries to detect the type of data in the columns. Some situations might occur where Power BI Desktop doesn't detect the correct data type. Where incorrect data types occur, you'll experience performance issues.

You have a higher chance of getting data type errors when you're dealing with flat files, such as comma-separated values (.CSV) files and Excel workbooks (.XLSX), because data was entered manually into the worksheets and mistakes were made. Conversely, in databases, the data types are predefined when tables or views are created.

A best practice is to evaluate the column data types in Power Query Editor before you load the data into a Power BI semantic model. If you determine that a data type is incorrect, you can change it. You might also want to apply a format to the values in a column and change the summarization default for a column.

To continue with the scenario where you're cleaning and transforming sales data in preparation for reporting, you now need to evaluate the columns to ensure that they have the correct data type. You need to correct any errors that you identify.

You evaluate the OrderDate column. As expected, it contains numeric data, but Power BI Desktop has incorrectly set the column data type to Text. To report on this column, you need to change the data type from Text to Date.

	1 ² SalesOrderID	2 ³ OrderDate	3 ⁴ Sort_of_Sales	4 ⁵ ProductID	5 ⁶ OrderQty
1	52242	07/07/2013	Internet	870	1
2	52592	14/07/2013	Internet	870	1
3	52694	16/07/2013	Internet	870	1
4	52799	18/07/2013	Internet	870	1
5	53799	03/08/2013	Internet	870	1
6	54058	08/08/2013	Internet	870	1
7	54059	08/08/2013	Internet	870	1
8	54063	08/08/2013	Internet	870	1
9	54158	10/08/2013	Internet	870	1
10	54281	12/08/2013	Internet	870	1

Implications of incorrect data types

The following information provides insight into problems that can arise when Power BI doesn't detect the correct data type.

Incorrect data types will prevent you from creating certain calculations, deriving hierarchies, or creating proper relationships with other tables. For example, if you try to calculate the Quantity of Orders YTD, you'll get the following error stating that the OrderDate column data type isn't Date, which is required in time-based calculations.

```
Quantity of Orders YTD = TOTALYTD(SUM('Sales'[OrderQty]), 'Sales'[OrderDate])
```

Couldn't load the data for this visual

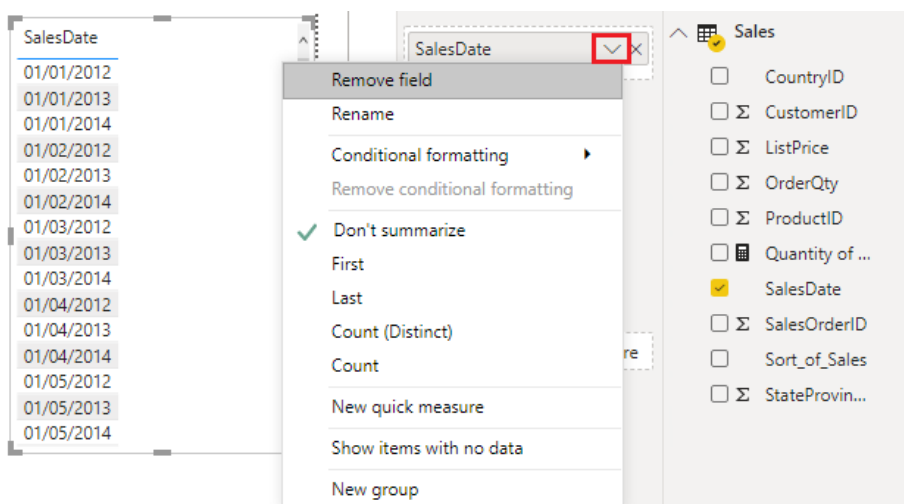
MdxScript(Model) (19, 40) Calculation error in measure
'Sales'[Quantity of Orders YTD]: A column specified in the call to function 'TOTALYTD' is not of type DATE. This is not supported.

[Copy details](#)

Send a Frown

Close

Another issue with having an incorrect data type applied on a date field is the inability to create a date hierarchy, which would allow you to analyze your data on a yearly, monthly, or weekly basis. The following screenshot shows that the SalesDate field isn't recognized as type Date and will only be presented as a list of dates in the Table visual. However, It's a best practice to use a date table and turn off the auto date/time to get rid of the auto generated hierarchy. For more information about this process, see [Auto generated data type](#) documentation.

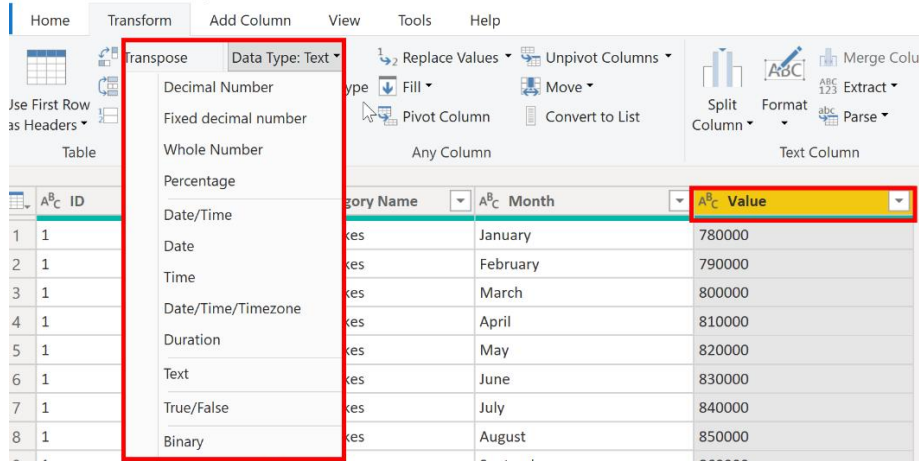


Change the column data type

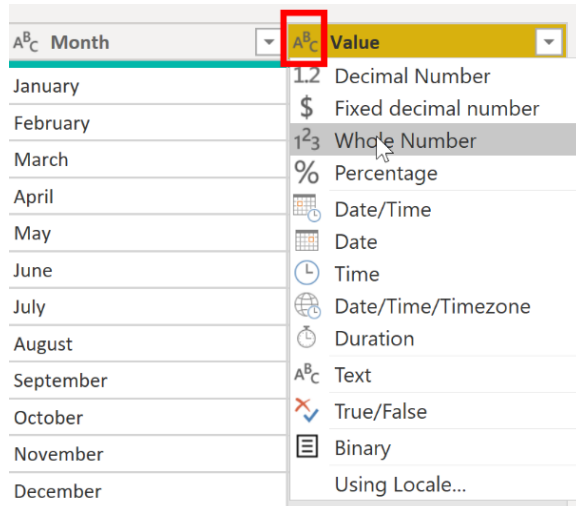
You can change the data type of a column in two places: in Power Query Editor and in the Power BI Desktop Report view by using the column tools. It is best to change the data type in the Power Query Editor before you load the data.

Change the column data type in Power Query Editor

In Power Query Editor, you can change the column data type in two ways. One way is to select the column that has the issue, select **Data Type** in the **Transform** tab, and then select the correct data type from the list.



Another method is to select the data type icon next to the column header and then select the correct data type from the list.



As with any other changes that you make in Power Query Editor, the change that you make to the column data type is saved as a programmed step. This step is called **Changed Type** and it will be iterated every time the data is refreshed.

After you have completed all steps to clean and transform your data, select **Close & Apply** to close Power Query Editor and apply your changes to your semantic model. At this stage, your data should be in great shape for analysis and reporting.

For more information, see [Data types in Power BI Desktop](#).

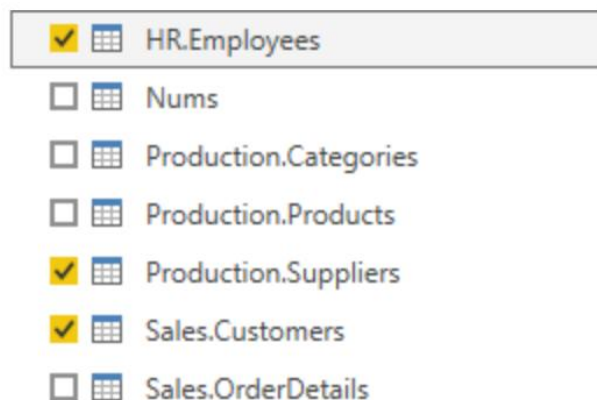
Combine multiple tables into a single table

The ability to [combine queries is powerful](#) because it allows you to append or merge different tables or queries together. You can combine tables into a single table in the following circumstances:

- [Too many tables exist](#), making it [difficult to navigate](#) an overly complicated semantic model.
- Several [tables have a similar role](#).
- A table [has only a column or two](#) that can fit into a different table.
- You [want to use several columns from different tables in a custom column](#).

You can combine the tables in [two different ways](#): [merging and appending](#).

Assume that you're [developing Power BI reports for the Sales and HR teams](#). They have asked you [to create a contact information report that contains the contact information and location of every employee, supplier, and customer](#). The data is in the HR.Employees, Production.Suppliers, and the Sales.Customers tables, as shown in the following image.



However, this [data comes from multiple tables](#), so the dilemma is determining [how you can merge the data in these multiple tables and create one source-of-truth table](#) to create a report from. The [inherent functionality of Power BI allows you to combine and merge queries into a single table](#).

Append queries

When you [append queries](#), you'll be [adding rows of data to another table or query](#). For example, you could have [two tables](#), one with [300 rows](#) and another with [100 rows](#), and when you append queries, you'll [end up with 400 rows](#). When you [merge queries](#), you'll be [adding](#)

columns from one table (or query) into another. To merge two tables, you must have a column that is the key between the two tables.

For the previously mentioned scenario, you'll append the HR.Employees table with the Production.Suppliers and Sales.Customers tables so that you have one master list of contact information. Because you want to create one table that has all contact information for employees, suppliers, and customers, when you combine the queries, the pertinent columns that you require in your combined table must be named the same in your original data tables to see one consolidated view.

Before you begin combining queries, you can remove extraneous columns that you don't need for this task from your tables. To complete this task, format each table to have only four columns with your pertinent information, and rename them so they all have the same column headers: ID, company, name, and phone. The following images are snippets of the reformatted Sales.Customers, Production.Suppliers, and HR.Employees tables.

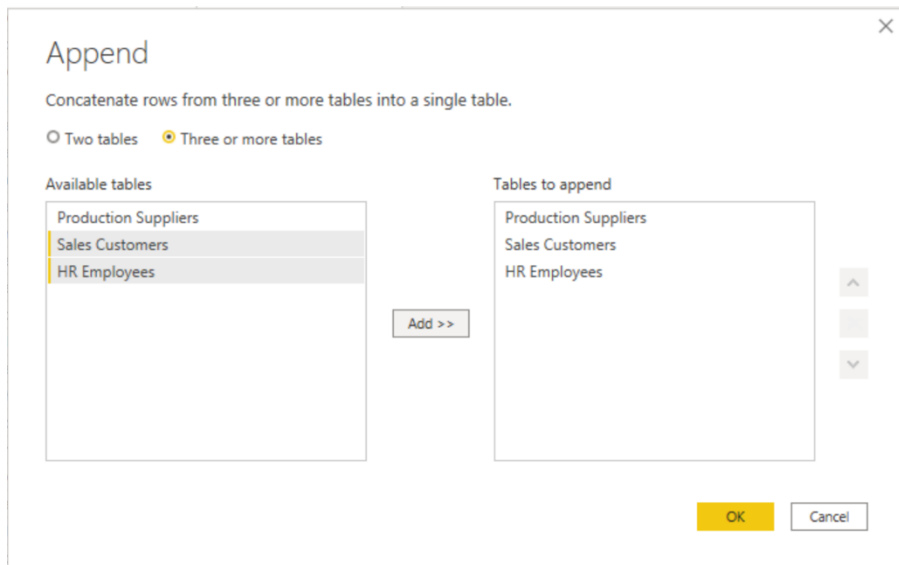
1 ² 3 id	A ^B C company	A ^B C name	1 ² 3 phone
1	96351 cus145	Mike Poi	1626258811
2	64826 cus134	John Likec	1363718361
3	92647 cus018	Theresa Yulia	2467355553

1 ² 3 id	A ^B C company	A ^B C name	1 ² 3 phone
1	86563 sup126	Mike Firt	7163615121
2	96352 sup889	Josie Lind	1831635671
3	48256 sup761	Joanna Threven	2936827338

1 ² 3 id	A ^B C company	A ^B C name	1 ² 3 phone
1	12347 emp124	John Kate	1625535511
2	76478 emp273	Luke John	1028777351
3	82482 emp291	Michael Uji	1245245672

After you have finished reformatting, you can combine the queries. On the **Home** tab on the **Power Query Editor ribbon**, select the drop-down list for **Append Queries**. You can select **Append Queries as New**, which means that the output of appending will result in a new query or table, or you can select **Append Queries**, which will add the rows from an existing table into another.

Your next task is to create a new master table, so you need to select **Append Queries as New**. This selection will bring you to a window where you can add the tables that you want to append from **Available Tables to Tables to Append**, as shown in the following image.



After you have added the tables that you want to append, select **OK**. You'll be routed to a new query that contains all rows from all three of your tables, as shown in the following image.

	id	company	name	phone
1	12347	emp124	John Kate	1625535511
2	76478	emp273	Luke John	1028777351
3	82482	emp291	Michael Uji	1245245672
4	97436	emp173	Kate Fitch	2352467634
5	12462	emp270	Eve Jun	3578999554
6	35237	emp715	Don Potre	3579006677
7	23467	emp183	Marc Wetb	2245789954
8	13892	emp163	Sara Scotts	2388367234
9	56356	emp172	Mitch Potter	1234683673
10	23478	emp812	Lilly Kithc	4567800522
11	45783	emp818	Ren Swrte	2357997515
12	86563	sup126	Mike Firt	7163615121
13	96352	sup889	Josie Lind	1831635671
14	48256	sup761	Joanna Threven	2936827338
15	28461	sup163	Michael Bob	1937293165
16	83613	sup162	Mimi Jukth	2916384462
17	96351	cus145	Mike Poi	1626258811
18	64826	cus134	John Likec	1363718361
19	92647	cus018	Theresa Yulia	2467355553
20	91661	cus182	Ren Thibe	3345783234
21	1736	cus104	Ron Mikel	1235799789
22	1835	cus103	Joy Qui	2345689411
23	1745	cus141	Cat Yate	2345678986

You have now succeeded in creating a master table that contains the information for the employees, suppliers, and customers. You can exit Power Query Editor and build any report elements surrounding this master table.

However, if you wanted to merge tables instead of appending the data from one table to another, the process would be different.

Merge queries

When you merge queries, you're combining the data from multiple tables into one based on a column that is common between the tables. This process is similar to the JOIN clause in SQL. Consider a scenario where the Sales team now wants you to consolidate orders and

their corresponding details (which are currently in two tables) into a single table. You can accomplish this task by merging the two tables, **Orders** and **OrderDetails**, as shown in the following image. The column that is shared between these two tables is **OrderID**.

	1 ² orderid	orderdate	1 ² shipperid
1	1	4/23/2018	12
2	2	4/25/2018	24
3	3	6/12/2018	19
4	4	6/13/2018	13
5	5	7/23/2018	11
6	6	7/25/2018	33

	1 ² orderid	1 ² productid	1 ² qty	1.2 unitprice
1	1	124	12	14
2	2	134	55	11.2
3	3	641	57	45
4	4	98	5	112.5
5	5	312	23	11.1
6	6	124	78	11.2

Go to **Home** on the Power Query Editor ribbon and select the **Merge Queries** drop-down menu, where you can select **Merge Queries as New**. This selection will open a new window, where you can choose the tables that you want to merge from the drop-down list, and then select the column that is matching between the tables, which in this case is **orderid**.

Merge

Select a table and matching columns to create a merged table.

Sales Orders

orderid	custid	empid	orderdate	requireddate	shippeddate	shipperid	freight	shipname
10248	85	5	7/4/2014	8/1/2014	7/16/2014	3	32.38	Ship to 85-B
10249	79	6	7/5/2014	8/16/2014	7/10/2014	1	11.61	Ship to 79-C
10250	34	4	7/8/2014	8/5/2014	7/12/2014	2	65.83	Destination SCQ
10251	84	3	7/8/2014	8/5/2014	7/15/2014	1	41.34	Ship to 84-A

Sales OrderDetails

orderid	productid	unitprice	qty	discount
10248	11	14.00	12	0
10248	42	9.80	10	0
10248	72	34.80	5	0
10249	14	18.60	9	0
10249	51	42.40	40	0

Join Kind

Left Outer (all from first, matching from second)

☐ Use fuzzy matching to perform the merge

Fuzzy matching options

The selection matches 830 of 830 rows from the first table.

OK Cancel

You can also choose how to join the two tables together, a process that is also similar to JOIN statements in SQL. These join options include:

- **Left Outer** - Displays all rows from the first table and only the matching rows from the second.
- **Full Outer** - Displays all rows from both tables.
- **Inner** - Displays the matched rows between the two tables.

For this scenario, you'll choose to use a **Left Outer join**. Select **OK**, which will route you to a new window where you can view your merged query.

	1 ² orderid	orderdate	1 ² shipperid	1 ² OrderDetails.productid	1 ² OrderDetails.qty	1 ² OrderDetails.unitprice
1	1	4/23/2018	12	124	12	14
2	2	4/25/2018	24	134	55	11.2
3	3	6/12/2018	19	641	57	45
4	4	6/13/2018	13	98	5	112.5
5	5	7/23/2018	11	312	23	11.1
6	6	7/25/2018	33	124	78	11.2
7	7	8/1/2019	77	137	11	572.1
8	8	8/10/2019	11	124	36	1331.9
9	9	8/11/2019	81	789	85	898.1

Now, you can merge two queries or tables in different ways so that you can view your data in the most appropriate way for your business requirements.

For more information on this topic, see the [Shape and Combine Data in Power BI](#) documentation.

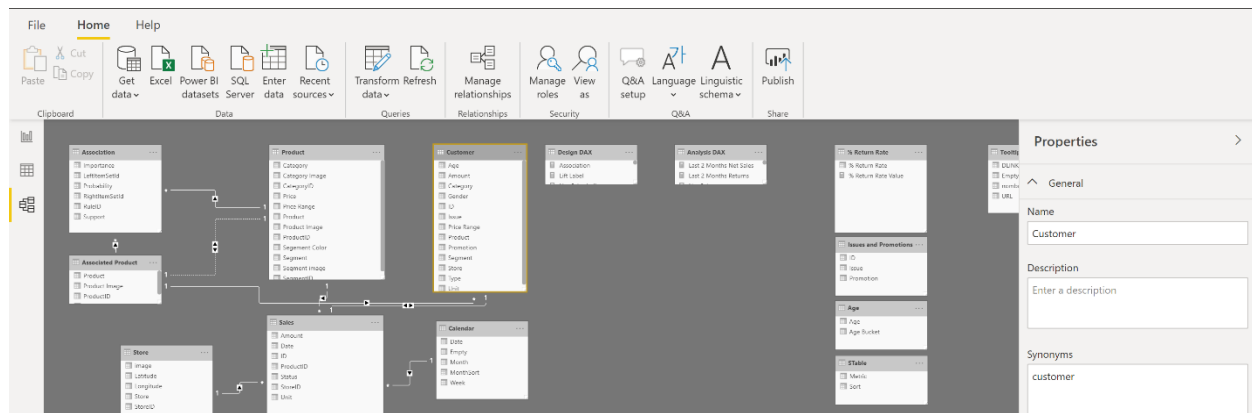
Profile data in Power BI

Profiling data is about studying the nuances of the data: determining anomalies, examining and developing the underlying data structures, and querying data statistics such as row counts, value distributions, minimum and maximum values, averages, and so on. This concept is important because it allows you to shape and organize the data so that interacting with the data and identifying the distribution of the data is uncomplicated, therefore helping to make your task of working with the data on the front end to develop report elements near effortless.

Assume that you are developing reports for the Sales team at your organization. You are uncertain how the data is structured and contained within the tables, so you want to profile the data behind the scenes before you begin developing the visuals. Power BI has inherent functionality that makes these tasks user-friendly and straightforward.

Examine data structures

Before you begin examining the data in Power Query Editor, you should first learn about the underlying data structures that data is organized in. You can view the current semantic model under the **Model** tab on Power BI Desktop.



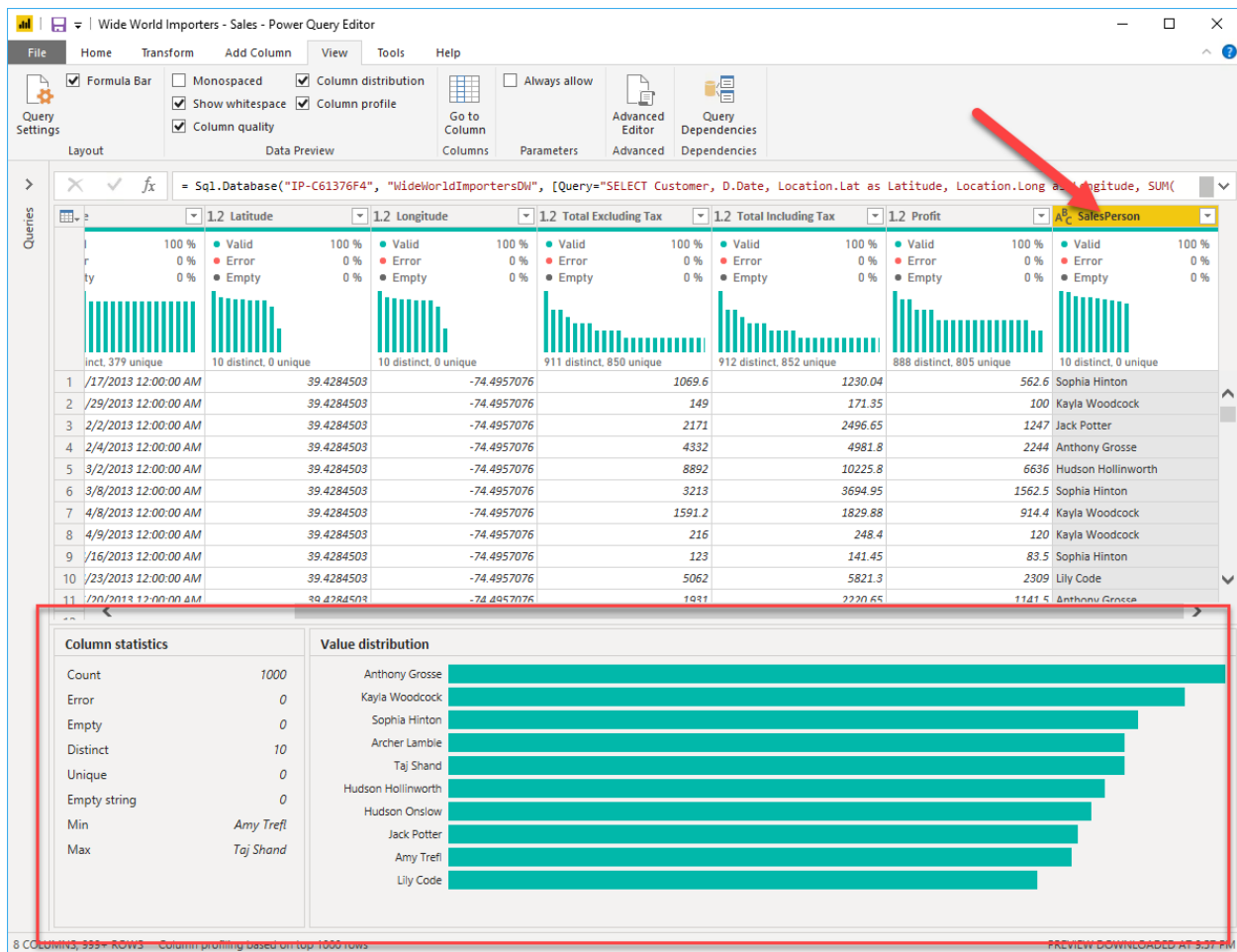
On the **Model** tab, you can edit specific column and table properties by selecting a table or columns, and you can transform the data by using the **Transform Data** button, which takes you to Power Query Editor. Additionally, you can manage, create, edit, and delete relationships between different tables by using **Manage Relationships**, which is located on the ribbon.

Find data anomalies and data statistics

After you have created a connection to a data source and have selected **Transform Data**, you are brought to Power Query Editor, where you can determine if anomalies exist within your data. Data anomalies are outliers within your data. Determining what those anomalies are can help you identify what the normal distribution of your data looks like and whether specific data points exist that you need to investigate further. Power Query Editor determines data anomalies by using the **Column Distribution** feature.

Select **View** on the ribbon, and under **Data Preview**, you can choose from a few options. To understand data anomalies and statistics, select the **Column Distribution**, **Column Quality**, and **Column Profile** options. The following figure shows the statistics that appear.

Column quality and **Column distribution** are shown in the graphs above the columns of data. **Column quality** shows you the percentages of data that is valid, in error, and empty. In an ideal situation, you want 100 percent of the data to be valid.



Note

By default, Power Query examines the first 1000 rows of your data set. To change this, select the profiling status in the status bar and select **Column profiling based on entire data set**.

Column distribution shows you the distribution of the data within the column and the counts of distinct and unique values, both of which can tell you details about the data counts. Distinct values are all the different values in a column, including duplicates and null values, while unique values do not include duplicates or nulls. Therefore, **distinct** in this table tells you the total count of how many values are present, while **unique** tells you how many of those values only appear once.

Column profile gives you a more in-depth look into the statistics within the columns for the first 1,000 rows of data. This column provides several different values, including the count of rows, which is important when verifying whether the importing of your data was successful. For example, if your original database had 100 rows, you could use this row count to verify that 100 rows were, in fact, imported correctly. Additionally, this row count will show how

many rows that Power BI has deemed as being outliers, empty rows and strings, and the min and max, which will tell you the smallest and largest value in a column, respectively. This distinction is particularly important in the case of numeric data because it will immediately notify you if you have a maximum value that is beyond what your business identifies as a "maximum." This value calls to your attention these values, which means that you can then focus your efforts when delving deeper into the data. In the case where data was in the text column, as seen in the previous image, the minimum value is the first value and the maximum value is the last value when in alphabetical order.

Additionally, the Value distribution graph tells you the counts for each distinct value in that specific column. When looking at the graph in the previous image, notice that the value distribution indicates that "Anthony Gross" appears the greatest number of times within the SalesPerson column and that "Lily Code" appears the least number of times. This information is particularly important because it identifies outliers. If a value appears far more than other values in a column, the Value distribution feature allows you to pinpoint a place to begin your investigation into why this is so.

On a numeric column, Column Statistics will also include how many zeroes and null values exist, along with the average value in the column, the standard deviation of the values in the column, and how many even and odd values are in the column. These statistics give you an idea of the distribution of data within the column and are important because they summarize the data in the column and serve as a starting point to determine what the outliers are.

For example, while looking through invoice data, you notice that the Value distribution graph shows that a few salespeople in the SalesPerson column appear the same number of times within the data. Additionally, you notice the same situation has occurred in the Profit column and in a few other tables as well. During your investigation, you discover that the data you were using was bad data and needed to be refreshed, so you immediately complete the refresh. Without viewing this graph, you might not have seen this error so quickly and, for this reason, value distribution is essential.

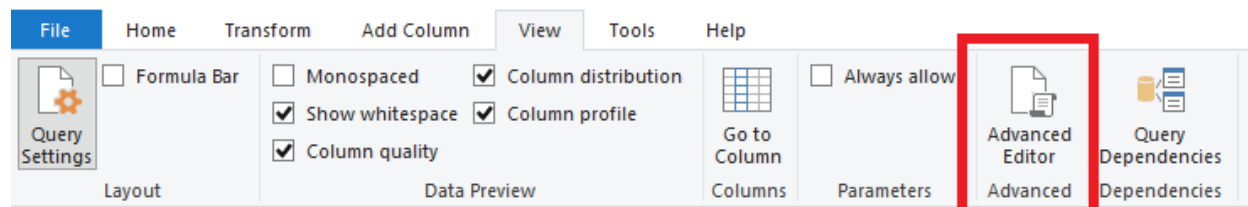
After you have completed your edits in Power Query Editor and are ready to begin building visuals, return to Home on the Power Query Editor ribbon. Select Close & Apply, which will return you to Power BI Desktop and any column edits/transformations will also be applied.

You have now determined the elements that make up profiling data in Power BI, which include loading data in Power BI, interrogating column properties to gain clarity about and make further edits to the type and format of data in columns, find data anomalies, and view data statistics in Power Query Editor. With this knowledge, you can include in your toolkit the ability to study your data in an efficient and effective manner.

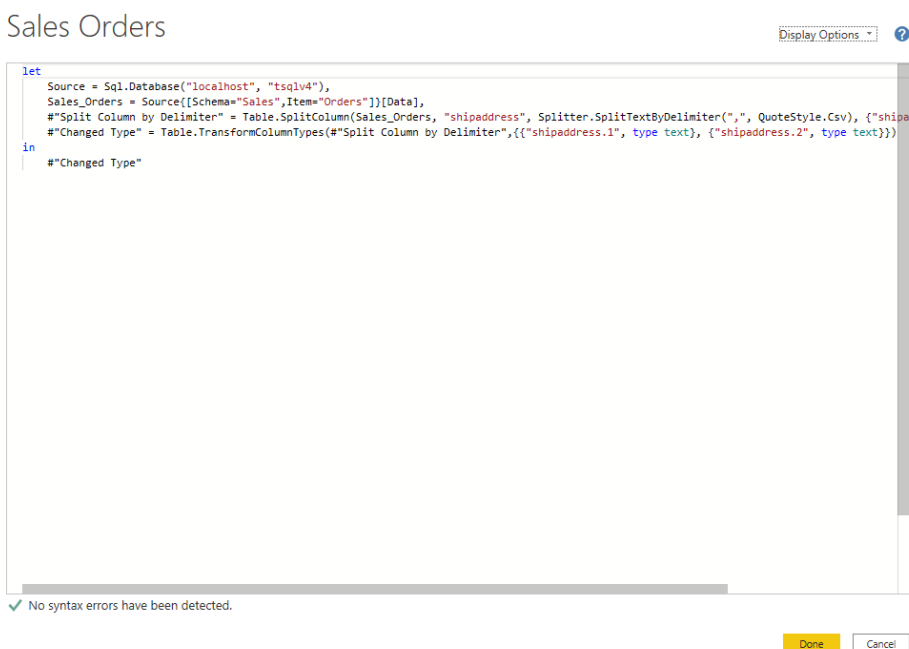
Use Advanced Editor to modify M code

Each time you shape data in Power Query, you create a step in the Power Query process. Those steps can be reordered, deleted, and modified where it makes sense. Each cleaning step that you made was likely created by using the graphical interface, but Power Query uses the M language behind the scenes. The combined steps are available to read by using the Power Query Advanced Editor. The M language is always available to be read and modified directly. It is not required that you use M code to take advantage of Power Query. You will rarely need to write M code, but it can still prove useful. Because each step in Power Query is written in M code, even if the UI created it for you, you can use those steps to learn M code and customize it to suit your needs.

After creating steps to clean data, select the **View** ribbon of Power Query and then select **Advanced Editor**.



The following screen should appear.



Each Power Query step will roughly align with one or two lines of M code. You don't have to be an expert in M code to be able to read it. You can even experiment with changing it. For

instance, if you need to [change the name of a database](#), you could [do it right in the code](#) and then select **Done**.

You might notice that **M code is written top-down**. [Later steps in the process can refer to previous steps by the variable name](#) to the left of the equal sign. Be [careful about reordering](#) these steps because [it could ruin the statement dependencies](#). [Write to a query formula step by using the in statement](#). Generally, [the last query step is used as the in final data set result](#).

Exercise - Load data in Power BI Desktop

This unit includes a lab to complete.

Use the free resources provided in the lab to complete the exercises in this unit. You will not be charged **for the lab environment**; however, you may need to bring your own subscription depending on the lab.

Microsoft provides this lab experience and related content for educational purposes. All presented information is owned by Microsoft and intended solely for learning about the covered products and services in this Microsoft Learn module.

Launch lab

Access your environment

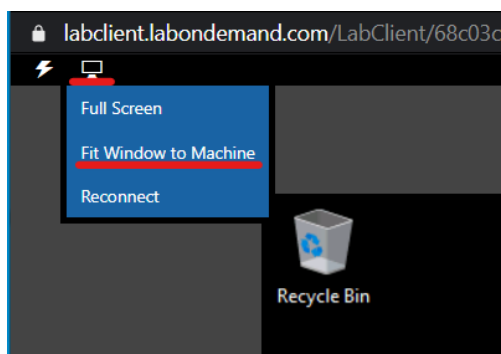
Before you start this lab (unless you are continuing from a previous lab), select **Launch lab** above.

You are automatically logged in to your lab environment as data-ai\student.

You can now begin your work on this lab.

Tip

To dock the lab environment so that it fills the window, select the PC icon at the top and then select **Fit Window to Machine**.



The estimated time to complete the lab is 45 minutes.

Note

A virtual machine containing the client tools you need is provided, along with the exercise instructions. Use the button above to launch the virtual machine.

A limited number of concurrent sessions are available - if the hosted environment is unavailable, try again later.

Alternatively, you can use these [setup instructions](#) to create your own lab environment, then follow these [exercise instructions](#).

Load Transformed Data in Power BI Desktop

Lab story

In this lab, you'll use data cleansing and transformation techniques to start shaping your data model. You'll then apply the queries to load each as a table to the data model.

In this lab you learn how to:

- Apply various transformations
- Load queries to the data model

This lab should take approximately 45 minutes.

Get started

To complete this exercise, first open a web browser and enter the following URL to download the zip folder:

<https://github.com/MicrosoftLearning/PL-300-Microsoft-Power-BI-Data-Analyst/raw/Main/Allfiles/Labs/02-load-data-with-power-query-in-power-bi-desktop/02-load-data.zip>

Extract the folder to the **C:\Users\Student\Downloads\02-load-data** folder.

Open the **02-Starter-Sales Analysis.pbix** file.

Note: You can dismiss the sign-in by selecting **Cancel**. Close any other informational windows. Select **Apply Later**, if prompted to apply changes.

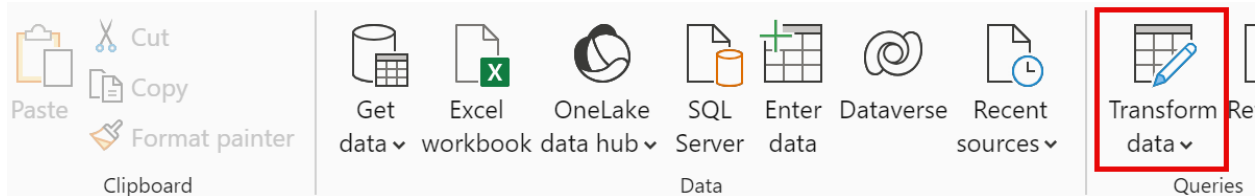
Configure the Salesperson query

In this task, you'll use Power Query Editor to configure the **Salesperson** query.

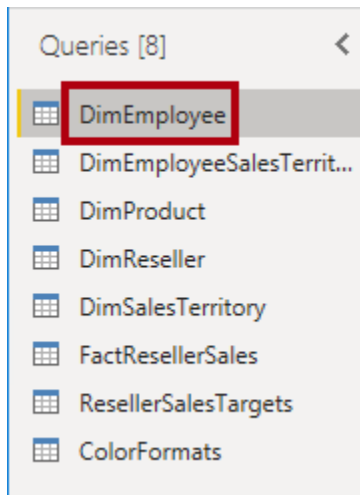
Important:

When instructed to rename columns, it's important that you rename them exactly as described.

1. To open the **Power Query Editor** window, on the **Home** ribbon tab, from inside the **Queries** group, select the **Transform Data** icon.



2. In the **Power Query Editor** window, in the **Queries** pane, select the **DimEmployee** query.



Note:

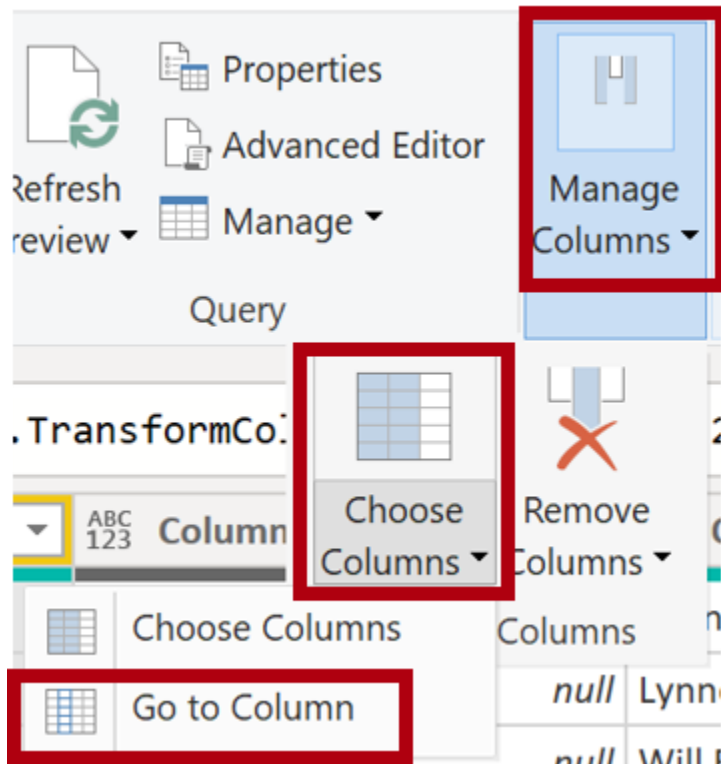
If you receive a warning message asking to specify how to connect, select **Edit Credentials**, connect using current credentials, and select **OK** to use an unencrypted connection.

3. To rename the query, in the **Query Settings** pane (located at the right), in the **Name** box, replace the text with **Salesperson**, and then press **Enter**. Then verify the name has been updated in **Queries** pane.

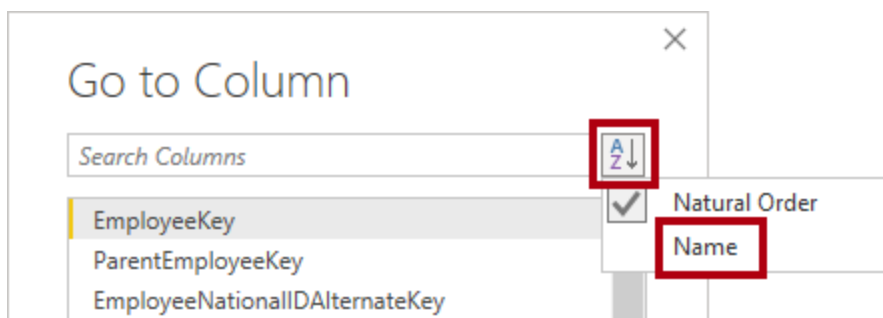
The query name determines the model table name. It's recommended to define concise and user-friendly names.

4. To locate a specific column, on the **Home** ribbon tab, select the **Manage Columns** down-arrow, select the **Choose Columns** down-arrow, and then select **Go to Column**.

Go to Column is a useful feature with many columns. Otherwise, you can horizontally scroll find columns.

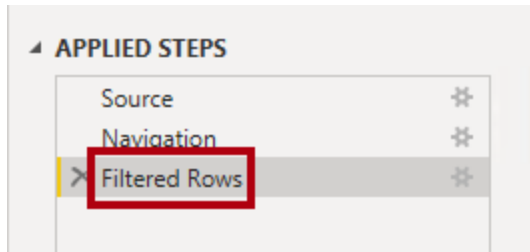


5. In the **Go to Column** window, to **order the list by column** name, select the **AZ sort** button, and then select **Name and SalesPersonFlag**. Click **OK**.

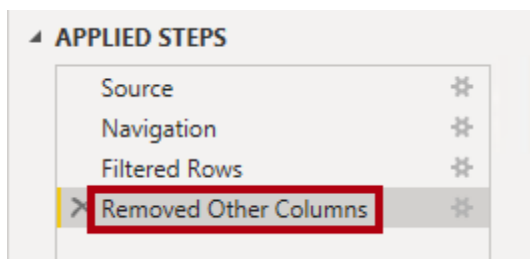


6. Locate the **SalesPersonFlag** column, then **filter the column** to select only Salespeople (that is, **TRUE**), and click **OK**.
7. In the **Query Settings** pane, in the **Applied Steps** list, notice the addition of the **Filtered Rows** step.

Each transformation you create results in another step logic. It's possible to edit or delete steps. It's also possible to select a step to preview the query results at that stage of the query transformation.



8. To remove columns, on the **Home** ribbon tab, select the **Manage Columns** group, select the **Choose Columns** icon.
9. In the **Choose Columns** window, to **uncheck all** columns, uncheck the **(Select All Columns)** item.
10. To **include columns**, check the following six columns:
 - EmployeeKey
 - EmployeeNationalIDAlternateKey
 - FirstName
 - LastName
 - Title
 - EmailAddress
11. In the **Applied Steps** list, notice the addition of another query step.



12. To create a single name column, first select the **FirstName** column header. While pressing the **Ctrl** key, select the **LastName** column.

123 EmployeeNationalIDAlternateKey	A ^B _C FirstName	A ^B _C LastName	A ^B _C Title
502097814	Stephen	Jiang	North American Sales Manager
112432117	Brian	Welcker	Director of Sales
841560125	Michael	Blythe	Sales Representative

13. **Right-click** either of the select column headers, and then in the context menu, select **Merge Columns**.

Many common transformations can be applied by right-clicking the column header, and then choosing them from the context menu. Note, however, more transformations are available in the ribbon.

14. In the **Merge Columns** window, in the **Separator** dropdown list, select **Space**.

15. In the **New Column Name** box, replace the text with **Salesperson**.

16. To rename the **EmployeeNationalIDAlternateKey** column, double-click the **EmployeeNationalIDAlternateKey** column header and replace the text with **EmployeeID**, and then press **Enter**.

17. Use the previous steps to rename the **EmailAddress** column to **UPN**.

UPN is an acronym for User Principal Name.

At the bottom-left, in the status bar, verify that the query has 5 columns and 18 rows.

Configure the SalespersonRegion query

In this task, you'll configure the **SalespersonRegion** query.

1. In the **Queries** pane, select the **DimEmployeeSalesTerritory** query.
2. In the **Query Settings** pane, rename the query to **SalespersonRegion**.
3. To remove the last two columns, first select the **DimEmployee** column header.
4. While pressing the **Ctrl** key, select the **DimSalesTerritory** column header.
5. Right-click either of the select column headers, and then in the context menu, select **Remove Columns**.

In the status bar, verify that the query has 2 columns and 39 rows.

Configure the Product query

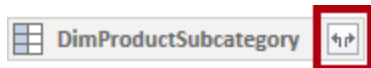
In this task, you'll configure the **Product** query.

Important:

When detailed instructions have already been provided, lab steps will provide more concise instructions. If you need the detailed instructions, you can refer back to the steps of previous tasks.

1. Select the **DimProduct** query and rename the query to **Product**.
2. Locate the **FinishedGoodsFlag** column, and then filter the column to retrieve products that are finished goods (that is, TRUE).

3. Remove all columns, **except** the following:
 - ProductKey
 - EnglishProductName
 - StandardCost
 - Color
 - DimProductSubcategory
4. Notice that the **DimProductSubcategory** column represents a related table (it contains **Value** links).
5. In the **DimProductSubcategory** column header, at the right of the column name, select the expand button.



6. See the full list of columns, then select the **Select All Columns** box to unselect all columns.
7. Select **EnglishProductSubcategoryName** and **DimProductCategory**, and uncheck the **Use Original Column Name as Prefix** checkbox before selecting **OK**.

*By selecting these two columns, a transformation will be applied to join to the **DimProductSubcategory** table, and then include these columns. The **DimProductCategory** column is, in fact, another related table in the data source.*

*Query column names must always be unique. If left checked, this checkbox would prefix each column with the expanded column name (in this case **DimProductSubcategory**). Because it's known that the selected column names don't collide with column names in the **Product** query, the option is deselected.*

8. Notice that the transformation resulted in the addition of two columns, and that the **DimProductSubcategory** column has been removed.
9. Expand the **DimProductCategory** column, and then introduce only the **EnglishProductCategoryName** column.
10. Rename the following four columns:
 - **EnglishProductName** to **Product**
 - **StandardCost** to **Standard Cost** (include a space)

- **EnglishProductSubcategoryName** to **Subcategory**
- **EnglishProductCategoryName** to **Category**

In the status bar, verify that the query has 6 columns and 397 rows.

Configure the Reseller query

In this task, you'll configure the **Reseller** query.

1. Select the **DimReseller** query and rename to **Reseller**.
2. Remove all columns, **except** the following:
 - ResellerKey
 - BusinessType
 - ResellerName
 - DimGeography
3. Expand the **DimGeography** column, to include **only** the following three columns:
 - City
 - StateProvinceName
 - EnglishCountryRegionName
4. On the **Business Type** column header, select the down-arrow, and then review the distinct column values, and notice both values **Warehouse** and **Ware House**.
5. Right-click the **Business Type** column header, and then select **Replace Values**.
6. In the **Replace Values** window, configure the following values:
 - In the **Value to Find** box, enter **Ware House**
 - In the **Replace With** box, enter **Warehouse**

Replace Values

Replace one value with another in the selected columns.

Value To Find	
Ware House	
Replace With	
Warehouse	

1. Rename the following four columns:
 - **BusinessType** to **Business Type** (include a space)
 - **ResellerName** to **Reseller**
 - **StateProvinceName** to **State-Province**
 - **EnglishCountryRegionName** to **Country-Region**

In the status bar, verify that the query has 6 columns and 701 rows.

Configure the Region query

In this task, you'll configure the **Region** query.

1. Select the **DimSalesTerritory** query and rename the query to **Region**.
2. Apply a filter to the **SalesTerritoryAlternateKey** column to remove the value 0 (zero).

This will remove one row.

3. Remove all columns, **except** the following:
 - SalesTerritoryKey
 - SalesTerritoryRegion
 - SalesTerritoryCountry
 - SalesTerritoryGroup
4. Rename the following three columns:
 - **SalesTerritoryRegion** to **Region**
 - **SalesTerritoryCountry** to **Country**
 - **SalesTerritoryGroup** to **Group**

In the status bar, verify that the query has 4 columns and 10 rows.

Configure the Sales query

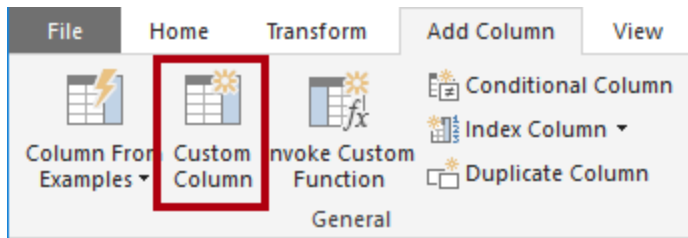
In this task, you'll configure the **Sales** query.

1. Select the **FactResellerSales** query and rename it to **Sales**.
2. Remove all columns, **except** the following:
 - SalesOrderNumber
 - OrderDate
 - ProductKey
 - ResellerKey
 - EmployeeKey
 - SalesTerritoryKey
 - OrderQuantity
 - UnitPrice
 - TotalProductCost
 - SalesAmount
 - DimProduct

Note:

*You may recall in the **Prepare Data in Power BI Desktop** lab that a small percentage of **FactResellerSales** rows had missing **TotalProductCost** values. The **DimProduct** column has been included to retrieve the product standard cost column to assist fixing the missing values.*

3. Expand the **DimProduct** column, uncheck all columns, and then include only the **StandardCost** column.
4. To create a custom column, on the **Add Column** ribbon tab, from inside the **General** group, select **Custom Column**.



5. In the **Custom Column** window, in the **New Column Name** box, replace the text with **Cost**.
6. In the **Custom Column Formula** box, enter the following expression (after the equals symbol), then save the new column:

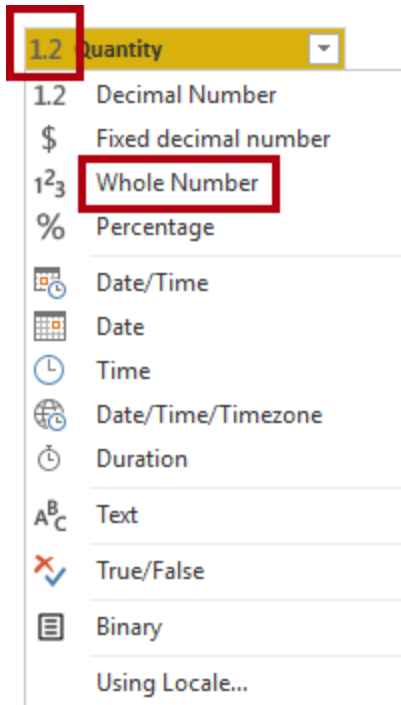
```
if [TotalProductCost] = null then [OrderQuantity] * [StandardCost] else [TotalProductCost]
```

Note: You can copy the expression from the **Snippets.txt** file in the 02-load-data folder.

This expression tests if the **TotalProductCost** value is missing. If missing, it produces a value by multiplying the **OrderQuantity** value by the **StandardCost** value; otherwise, it uses the existing **TotalProductCost** value.

7. Remove the following two columns:
 - TotalProductCost
 - StandardCost
8. Rename the following three columns:
 - **OrderQuantity** to **Quantity**
 - **UnitPrice** to **Unit Price** (include a space)
 - **SalesAmount** to **Sales**
9. To modify the column data type, in the **Quantity** column header, at the left of the column name, select the **1.2** icon, and then select **Whole Number**.

Configuring the correct data type is important. When the column contains numeric value, it's also important to choose the correct type if you expect to perform mathematic calculations.



10. Modify the following three column data types to **Fixed Decimal Number**.

The **fixed decimal number** data type allows for 19 digits, and allows for more precision to avoid rounding errors. It's important to use the fixed decimal number type for financial values, or rates (like exchange rates).

- Unit Price
- Sales
- Cost

In the status bar, verify that the query has **10 columns** and **999+ rows**. A maximum of 1000 rows will be loaded as preview data for each query.

Configure the Targets query

In this task, you'll configure the **Targets** query.

1. Select the **ResellerSalesTargets** query and rename to **Targets**.

Note: If you receive a warning message asking to specify how to connect, select **Edit Credentials**, and use anonymous access.

2. To unpivot the 12 month columns (**M01-M12**), first multi-select the **Year** and **EmployeeID** column headers.

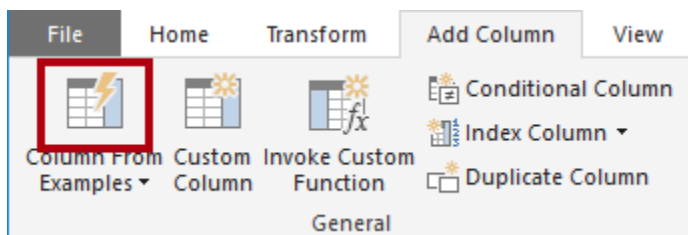
3. Right-click either of the select column headers, and then in the context menu, select **Unpivot Other Columns**.
4. Notice that the column names now appear in the **Attribute** column, and the values appear in the **Value** column.
5. Apply a filter to the **Value** column to remove hyphen (-) values.

You may recall that the hyphen character was used in the source CSV file to represent zero (0).

6. Rename the following two columns:
 - **Attribute** to **MonthNumber** (there's no space)
 - **Value** to **Target**
7. To **prepare the MonthNumber** column values, right-click the **MonthNumber** column header, and then select **Replace Values**.

*You'll now apply transformations to produce a date column. The date will be derived from the **Year** and **MonthNumber** columns. You'll create the column by using the **Columns From Examples** feature.*

8. In the **Replace Values** window, in the **Value To Find** box, enter **M** and leave the **Replace with** empty.
9. Modify the **MonthNumber** column data type to **Whole Number**.
10. On the **Add Column** ribbon tab, from inside the **General** group, select The **Column From Examples** icon.



11. Notice that the first row is for year **2017** and month number **7**.
12. In the **Column1** column, in the **first grid cell**, commence **entering 7/1/2017**, and then press **Enter**.

Note: *The virtual machine uses US regional settings, so this date is in fact July 1, 2017. Other regional settings may require a **0** before the date.*

13. Notice that the **grid cells update with predicted values**.

*The feature has accurately predicted that you're combining values from the **Year** and **MonthNumber** columns.*

14. Notice also the formula presented above the query grid.



Add Column From Examples

Enter sample values to create a new column (Ctrl+Enter to apply).

`Transform: Text.Combine({Text.From([MonthNumber], "en-US"), "/"}, Text.From([Year], "en-US"))`

15. To rename the new column, double-click the **Merged** column header and rename the column as **TargetMonth**.

16. Remove the following columns:

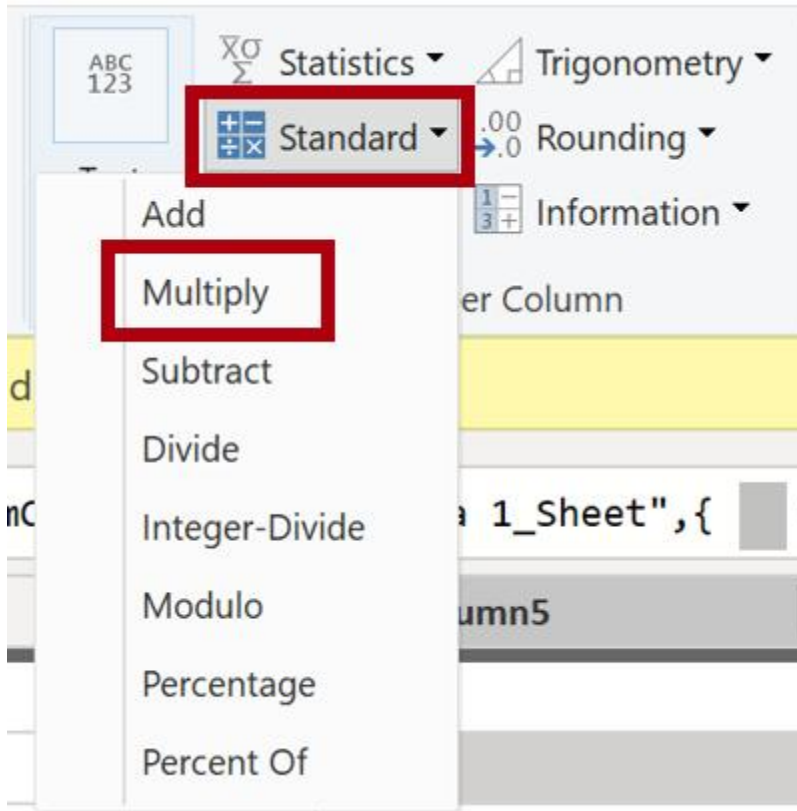
- Year
- MonthNumber

17. Modify the following column data types:

- **Target** as fixed decimal number
- **TargetMonth** as date

18. To **multiply the Target values by 1000**, select the **Target** column header, and then on the **Transform** ribbon tab, from inside the **Number Column group**, select **Standard**, and then select **Multiply**.

You may recall that the target values were stored as thousands.



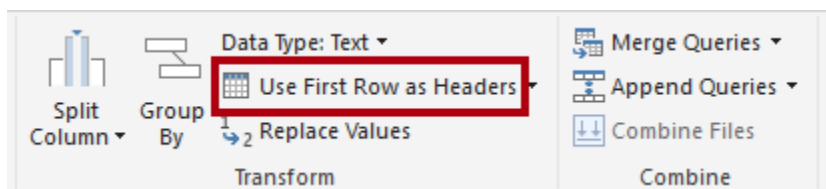
19. In the **Multiply** window, in the **Value** box, enter **1000**, and select **OK**.

In the status bar, verify that the query has 3 columns and 809 rows.

Configure the ColorFormats query

In this task, you'll configure the **ColorFormats** query.

1. Select the **ColorFormats** query and notice that the first row contains the column names.
2. On the **Home** ribbon tab, from inside the **Transform** group, select **Use First Row as Headers**.



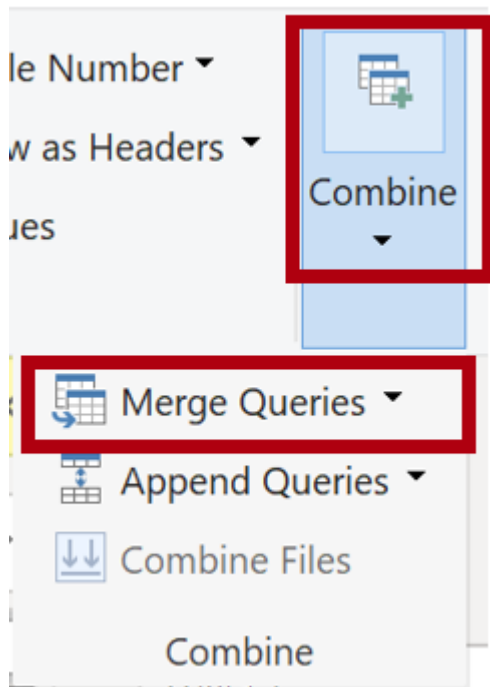
In the status bar, verify that the query has 3 columns and 10 rows.

Update the Product query

In this task, you'll update the **Product** query by merging the **ColorFormats** query.

1. Select the **Product** query.
2. To merge the **ColorFormats** query, on the **Home** ribbon tab, select the **Combine** down-arrow, then select **Merge Queries**.

Merging queries allows integrating data, in this case from different data sources (SQL Server and a CSV file).



3. In the **Merge** window, in the **Product** query grid, select the **Color** column header.

Merge

Select a table and matching columns to create a merged table.

Product

ProductKey	Product	Standard Cost	Color	Subcategory	Category
210	HL Road Frame - Black, 58	868.63	Black	Road Frames	Components
211	HL Road Frame - Red, 58	868.63	Red	Road Frames	Components
212	Sport-100 Helmet, Red	12.03	Red	Helmets	Accessories
213	Sport-100 Helmet, Red	13.88	Red	Helmets	Accessories
214	Sport-100 Helmet, Red	13.09	Red	Helmets	Accessories

4. Beneath the **Product** query grid, in the dropdown list, select the **ColorFormats** query.

Merge

Select a table and matching columns to create a merged table.

Product

ProductKey	Product	Standard Cost	Color	Subcategory	Category
210	HL Road Frame - Black, 58	868.63	Black	Road Frames	Components
211	HL Road Frame - Red, 58	868.63	Red	Road Frames	Components
212	Sport-100 Helmet, Red	12.03	Red	Helmets	Accessories
213	Sport-100 Helmet, Red	13.88	Red	Helmets	Accessories
214	Sport-100 Helmet, Red	13.09	Red	Helmets	Accessories

ColorFormats

5. In the **ColorFormats** query grid, select the **Color** column header.
6. When the **Privacy Levels** window opens, for each of the two data sources, in the corresponding dropdown list, select **Organizational**, then **Save**.

*Privacy levels can be configured for data source to determine whether data can be shared between sources. Setting each data source as **Organizational** allows them to share data, if necessary. Private data sources can never be shared with other data sources. It doesn't mean that Private data can't be shared; it means that the Power Query engine can't share data between the sources.*

Privacy levels

The privacy level is used to ensure data is combined without undesirable data transfer. Incorrect privacy levels may lead to sensitive data being leaked outside of a trusted scope. More information on privacy levels can be found [here](#).

☐ Ignore Privacy Levels checks for this file. Ignoring Privacy Levels could expose sensitive or confidential data to an unauthorized person.

	localhost		Organizational	
	https://raw.githubusercontent.com/		Organizational	

Save

Cancel

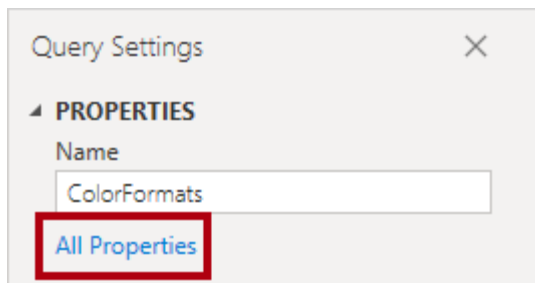
7. In the **Merge** window, use the default **Join Kind** - maintaining the selection of Left Outer and select **OK**.
8. Expand the **ColorFormats** column to include the following two columns:
 - Background Color Format
 - Font Color Format

In the status bar, verify that the query now has 8 columns and 397 rows.

Update the ColorFormats query

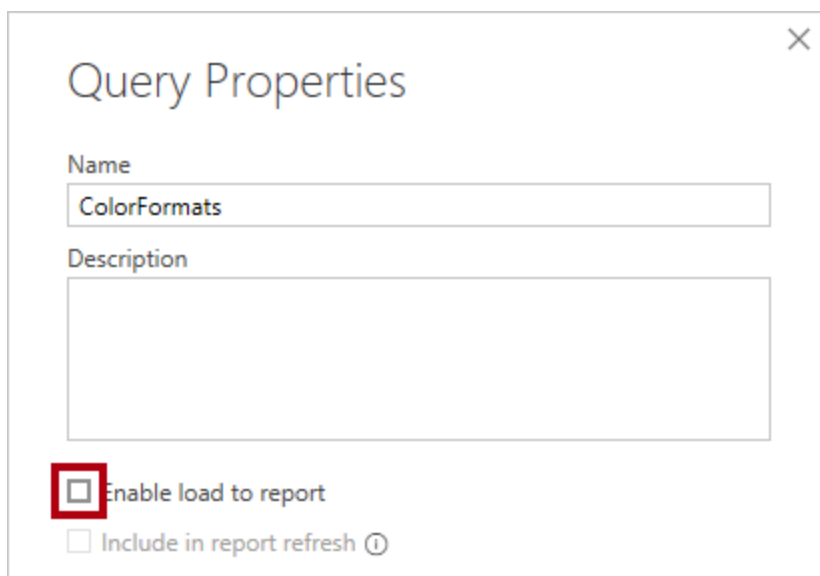
In this task, you'll update the **ColorFormats** to disable its load.

1. Select the **ColorFormats** query.
2. In the **Query Settings** pane, select the **All Properties** link.



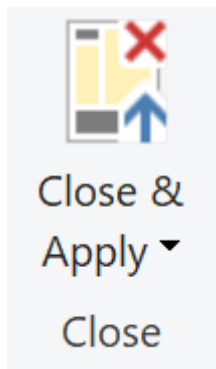
3. In the **Query Properties** window, **uncheck the Enable Load To Report** checkbox.

Disabling the load means it will not load as a table to the data model. This is done because the query was merged with the **Product** query, which is enabled to load to the data model.

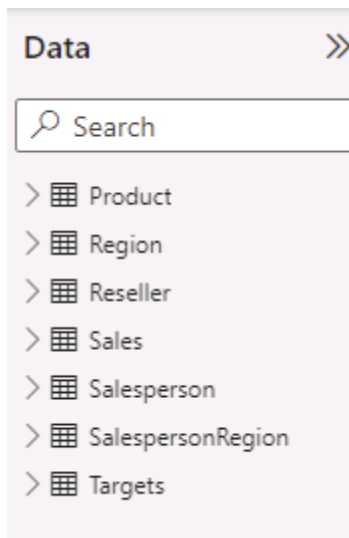


Review final product

1. In Power Query Editor, verify that you have **8 queries**, correctly named as follows:
 - Salesperson
 - SalespersonRegion
 - Product
 - Reseller
 - Region
 - Sales
 - Targets
 - ColorFormats (which won't load to the data model)
2. Select **Close & Apply** to load the data to the model, and close Power Query Editor window.



3. You can now see the canvas in Power BI Desktop, with Filters, Visualizations, and Data panes on the right. In the Data pane, notice the **7 tables** loaded to the data model.



Lab complete

Check your knowledge

Summary

This module explained how you can take data that is difficult to read, build calculations on, and discover and make it simpler for report authors and others to use. Additionally, you learned how to combine queries so that they were fewer in number, which makes data navigation more streamlined. You also replaced renamed columns into a human readable form and reviewed good naming conventions for objects in Power BI.

Choose a Power BI model framework

Introduction

For over [two decades](#), Microsoft [continues to make deep investments in](#) enterprise business intelligence (BI). Azure Analysis Services (AAS) and SQL Server Analysis Services (SSAS) are [based on mature BI data modeling technology](#) used by countless enterprises. The [same technology](#) is also at the heart of Power BI data models.

Power BI [offers you a choice when designing your model](#). You can [use Power BI Desktop](#) to develop your model, [and](#) you can develop it by [using different frameworks](#). These frameworks help to [deliver fast performance](#), [near real-time results](#), or both.

This module [introduces the frameworks](#), their [benefits and limitations](#), and [features](#) to help optimize your models. Lastly, it provides you with guidance [to help you choose the right framework and features](#) for your project.

Learning objectives

By the end of this module, you'll be able to:

- Describe Power BI model fundamentals.
- Determine **when** to develop an **import model**.
- Determine **when** to develop a **DirectQuery model**.
- Determine **when** to develop a **composite model**.
- **Choose an appropriate** Power BI model **framework**.

Describe Power BI model fundamentals

This unit **introduces Power BI model terms**. It's **important that you understand** these terms **in order to choose the appropriate** model framework for your project. This unit describes the following terms:

- Data model
- Power BI dataset
- Analytic query
- Tabular model
- Star schema design
- Table storage mode
- Model framework

Data model

A Power BI **data model** is a **query-able data resource that's optimized for analytics**. Reports **can query data models by using** one of **two analytic languages**: Data Analysis Expressions (**DAX**) or Multidimensional Expressions (**MDX**). **Power BI uses DAX**, while paginated reports can use either DAX or MDX. The **Analyze in Excel features uses MDX**.

Tip

A data model is **also described as semantic model**, especially in enterprise scenarios. Commonly, in the context of a data discussion and in **this module**, a **data model is simply referred to as a model**.

Power BI dataset

You develop a Power BI model in Power BI Desktop, and once published to a workspace in the Power BI service, it's then known as a dataset. A dataset is a Power BI artifact that's a source of data for visualizations in Power BI reports and dashboards.

Note

Not all datasets originate from models developed in Power BI Desktop. Some datasets represent connections to external-hosted models in AAS or SSAS. Others can represent real-time data structures, including push datasets, streaming datasets, or hybrid datasets. This module is concerned only with models developed in Power BI Desktop.

Analytic query

Power BI reports and dashboards must query a dataset. When Power BI visualizes dataset data, it prepares and sends an analytic query. An analytic query produces a query result from a model that's easy for a person to understand, especially when visualized.

An analytic query has three phases that are executed in this order:

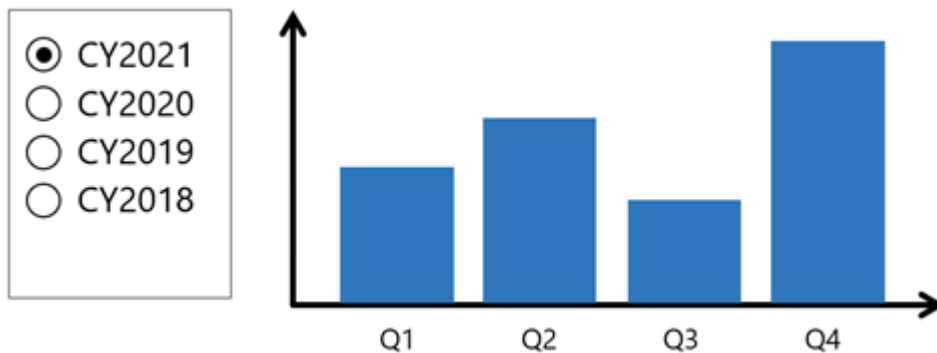
1. Filter
2. Group
3. Summarize

Filtering (sometimes known as slicing) narrows down on a subset of the model data. Filter values aren't visible in the query result. Most analytic queries apply filters because it's common to filter by a time period, and usually other attributes. Filtering happens in different ways. In a Power BI report, you can set filters at report, page, or visual level. Report layouts often include slicer visuals to filter visuals on the report page. When the model enforces row-level security (RLS), it applies filters to model tables to restrict access to specific data. Measures, which summarize model data, can also apply filters.

Grouping (sometimes known as dicing) divides query result into groups. Each group is also a filter, but unlike the filtering phase, filter values are visible in the query result. For example, grouping by customer filters each group by customer.

Summarization produces a single value result. Typically, a report visual summarizes a numeric field by using an aggregate function. Aggregate functions include sum, count, minimum, maximum, and others. You can achieve simple summarization by aggregating a column, or you can achieve complex summarization by creating a measure using a DAX formula.

Consider an example: A Power BI report page includes a slicer to filter by a single year. There's also a [column chart visual that shows quarterly sales for the filtered year](#).



In this example, the slicer [filters the visual by calendar year 2021](#). The column chart [groups by quarters](#) (of the filtered year). Each column is a group that represents a [visible filter](#). The [column heights represent the summarized sales values for each quarter of the filtered year](#).

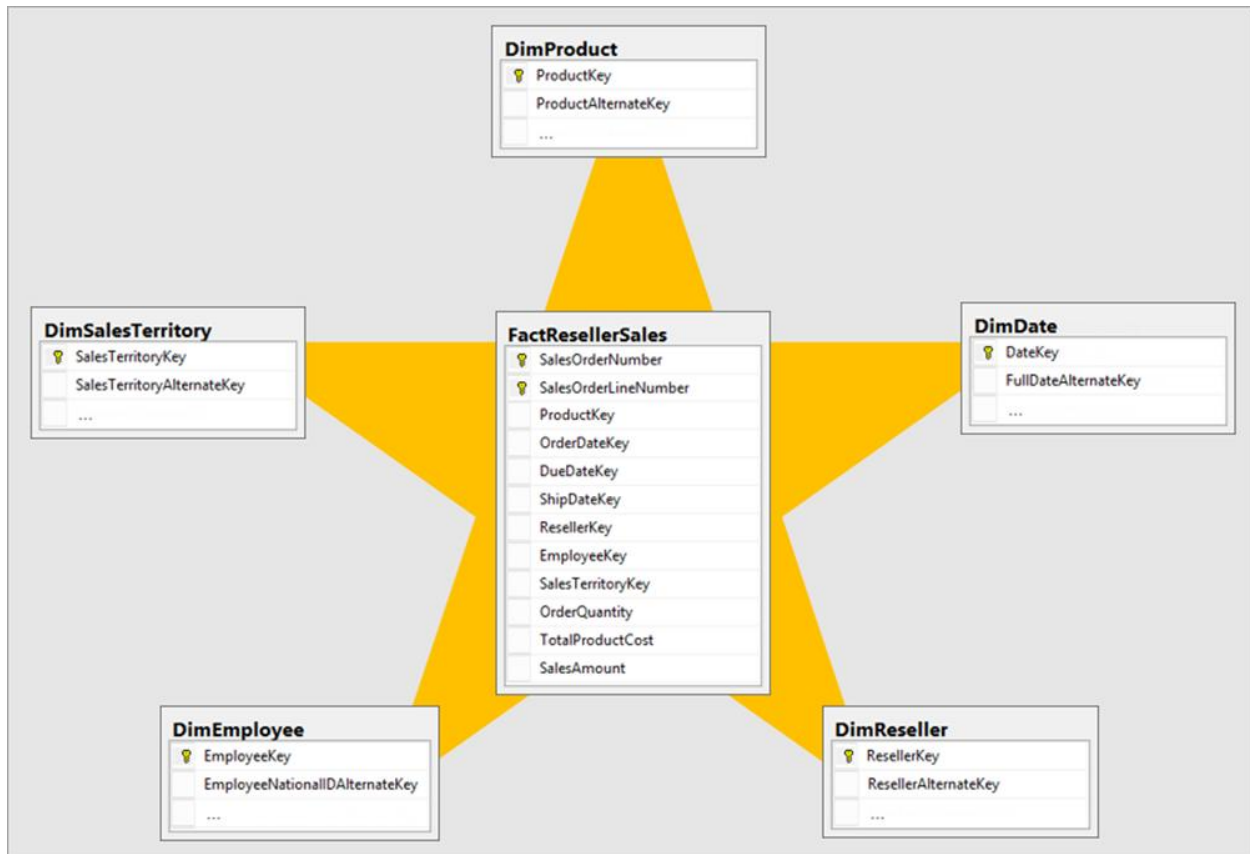
Tabular model

A Power BI model is a tabular model. A [tabular model](#) [comprises one or more tables of columns](#). It can also [include relationships, hierarchies, and calculations](#).

Star schema design

To produce an [optimized and easy-to-use tabular model](#), we [recommend you produce a star schema design](#). [Star schema design](#) is a [mature modeling approach widely adopted by relational data warehouses](#). It [requires you to classify model tables as either dimension or fact](#).

[Dimension tables](#) [describe business entities](#); the things you model. Entities can include [products, people, places, and concepts including time itself](#). [Fact tables](#) [store observations or events](#), and can be, for example, [sales orders, stock balances, exchange rates, or temperature readings](#). A [fact table contains dimension key columns that relate to dimension tables, and numeric measure columns](#). A [fact table forms the center of a star](#), and the related [dimension tables form the points of the star](#).



In an analytic query, dimensions table columns filter or group. Fact table columns are summarized.

For more information, see [Understand star schema and the importance for Power BI](#).

Table storage mode

Each Power BI model table (except calculated tables) has a storage mode property. The storage mode property can be either **Import**, **DirectQuery**, or **Dual**, and it determines whether table data is stored in the model.

- **Import** – Queries retrieve data that's stored, or cached, in the model.
- **DirectQuery** – Queries pass through to the data source.
- **Dual** – Queries retrieve stored data or pass through to the data source. Power BI determines the most efficient plan, striving to use cached data whenever possible.

Model framework

Table storage mode settings determine the model framework, which can be either import, DirectQuery, or composite. The following units in this module describe each of these frameworks and provides guidance on their use.

- An **import model** comprises tables that have their **storage mode** property set to **Import**.
- A **DirectQuery model** comprises tables that have their **storage mode** property set to **DirectQuery**, and they belong to the same source group. Source group is described later in this module.
- A **composite model** comprises more than one source group.

Determine when to develop an import model

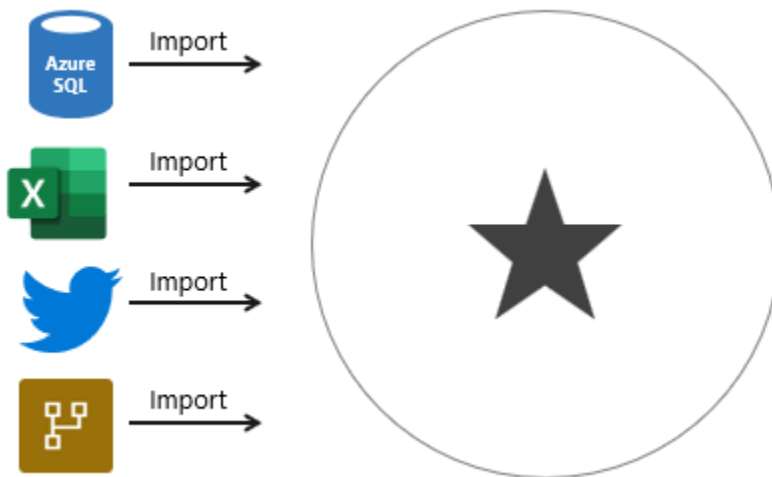
An **import model** comprises tables that have their **storage mode** property set to **Import**. It includes **calculated tables**, which you define with a DAX formula, too.

Import model benefits

Import models are the **most frequently developed model framework** because there are many benefits. Import models:

- **Support all Power BI data source types**, including **databases, files, feeds, web pages, dataflows**, and more.
- **Can integrate source data**. For example, **one table** sources its **data from a relational database while a related table** sources its data from a web page.
- **Support all DAX and Power Query (M) functionality**.
- **Support calculated tables**.
- Deliver the **best query performance**. That's because the **data cached in the model is optimized for analytic queries** (filter, group, and summarize) and the model is stored entirely in memory.

In short, import models **offer you the most options and design flexibility, and they deliver fast performance**. For this reason, Power BI Desktop defaults to use import storage mode when you “Get data.”



Import model limitations

Despite the many compelling benefits, [there are limitations](#) of import models that you must bear in mind. Limitations are [related to model size and data refresh](#).

Model size

Power BI imposes [dataset size restrictions](#), which limit the size of a model. [When you publish the model to a shared capacity](#), there's a [1-GB limit per dataset](#). When this size limit is [exceeded, the dataset will fail to refresh](#). When you publish the model to a dedicated capacity (also known as [Premium capacities](#)), it can grow [beyond 10-GB](#), [providing you enable the \[Large dataset storage format setting\]\(#\)](#) for the capacity.

You should always [strive to reduce the amount of data stored in tables](#). This strategy [helps to reduce the duration of model refreshes and speed up model queries](#). There are [numerous data reduction techniques](#) that you can apply, including:

- [Remove unnecessary columns](#)
- [Remove unnecessary rows](#)
- [Group by and summarize](#) to raise the [grain of fact tables](#)
- [Optimize column data types](#) with a [preference for numeric data](#)
- [Preference for custom columns in Power Query instead of calculated columns in the model](#)
- [Disable Power Query query load](#)
- [Disable auto date/time](#)

- **Use DirectQuery table** storage, as described in later units of this module.

For more information, see [Data reduction techniques for Import modeling](#).

Note

The **1-GB per dataset limit refers to the compressed size** of the Power BI model, **not the volume of data being collected** from the source system.

Data refresh

Imported data **must be periodically refreshed**. Dataset data is **only as current as the last successful data refresh**. To keep data current, you **set up scheduled data refresh**, or report consumers **can perform an on-demand refresh**.

Power BI **imposes limits on how often scheduled refresh operations can occur**. It's up to **eight times per day in a shared capacity, and up to 48 times per day in a dedicated capacity**.

You should determine whether this degree of latency is tolerable. It often **depends on the velocity** (or volatility) of the data, and the **urgency to keep users informed** about the current state of data. When **scheduled refresh limits aren't acceptable, consider using DirectQuery storage tables, or creating a hybrid table**. Or take a different approach, and create a real-time dataset instead.

Tip

Hybrid tables are described in [unit 4](#). For information about real-time datasets, work through the [Monitor data in real-time with Power BI](#) module.

You must also **consider refresh workload and duration**. By default, **to refresh a table, Power BI removes all data and reloads it again**. These operations can **place an unacceptable burden on source systems**, especially for large fact tables. **To reduce this burden**, you can set up the incremental refresh feature. **Incremental refresh automates the creation and management of time-period partitions and intelligently update only those partitions that require refresh.**

When your data source supports incremental refresh, it can **result in faster and more reliable refreshes, and reduced resource consumption** of Power BI and source systems.

Advanced data modelers can customize their own partitioning strategy. Automation scripts **can create, manage, and refresh table partitions**. For more information, see [Power BI usage scenarios: Advanced data model management](#). This usage scenario describes using the XMLA endpoint available with Power BI Premium.

Determine when to develop a DirectQuery model

A DirectQuery model comprises tables that have their storage mode property set to **DirectQuery**, and they belong to the same source group.

A **source group** is a **set of tables that connect to the same data source**. There are two types:

- **Import** – Represents all import storage mode tables including calculated tables. There can only be one import source group in a model.
- **DirectQuery** – Represents all DirectQuery storage mode tables that relate to a specific data source.

Note

An import model and a DirectQuery model only comprise a single source group. When there's more than one source group, the model framework is known as a **composite model**. Composite models are described in [Unit 5](#).

DirectQuery model benefits

There are **several benefits** to developing a DirectQuery model.

Model large or fast-changing data sources

A DirectQuery model is a great framework choice when your source data exhibits volume and/or velocity characteristics. Because **DirectQuery tables don't require refresh**, they're **well suited to large data stores**, like a **data warehouse**. It's impractical and inefficient, if not impossible, to import an entire data warehouse into a model. **When the source data changes rapidly and users need to see current data**, a DirectQuery model can deliver near real-time query results.

When a report queries a DirectQuery model, Power BI passes those queries to the underlying data source.



Enforce source RLS

DirectQuery is also useful when the source database enforces row-level security (RLS). Instead of replicating RLS rules in your Power BI model, the source data base can enforce its rules. This approach works only for some relational databases, and it involves setting up single sign-on for the dataset data source. For more information, see [Azure SQL Database with DirectQuery](#).

Data sovereignty restrictions

If your organization has security policies that restrict data leaving their premises, then it isn't possible to import data. A DirectQuery model that connects to an on-premises data source may be appropriate. (You can also consider installing Power BI Report Server for on-premises reporting.)

Create specialized datasets

Typically, DirectQuery mode supports relational database sources. That's because Power BI must translate analytic queries to native queries understood by the data source.

However, there's one powerful exception. You can connect to a Power BI dataset (or Azure Analysis Services model) and convert it to a DirectQuery local model. A local model is a relative term that describes a model's relationship to another model. In this case, the original dataset is a remote model, and the new dataset is the local model. These models are chained, which is term used to describe related models. You can chain up to three models in this way.

This capability to chain models supports the potential to personalize and/or extend a remote model. The simplest thing you can do is rename objects, like tables or columns, or add measures to the local model. You can also extend the model with calculated columns or

calculated tables, or add new import or DirectQuery tables. However, these extensions result in the creation of new source groups, which means the model becomes a composite model. That scenario is described in [Unit 5](#).

For more information, see [Using DirectQuery for Power BI datasets and Azure Analysis Services](#).

DirectQuery model limitations

There are many limitations related to DirectQuery models that you must bear in mind. Here are the main limitations:

- Not all data sources are supported. Typically, only major relational database systems are supported. Power BI datasets and Azure Analysis Services models are supported too.
- All Power Query (M) transformations are not possible, because these queries must translate to native queries that are understood by source systems. So, for example, it's not possible to use pivot or unpivot transformations.
- Analytic query performance can be slow, especially if source systems aren't optimized (with indexes or materialized views), or there are insufficient resources for the analytic workload.
- Analytic queries can impact on source system performance. It could result in a slower experience for all workloads, including OLTP operations.

Boost DirectQuery model performance

When there's a justification to develop a DirectQuery model, you can mitigate some limitations in two ways.

Data source optimizations

You can optimize the source database to ensure the expected analytic query workload performs well. Specifically, you can create indexes and materialized views, and ensure the database has sufficient resources for all workloads.

Tip

We recommend that you always work in collaboration with the database owner. It's important that they understand the additional workload a DirectQuery model can place on their database.

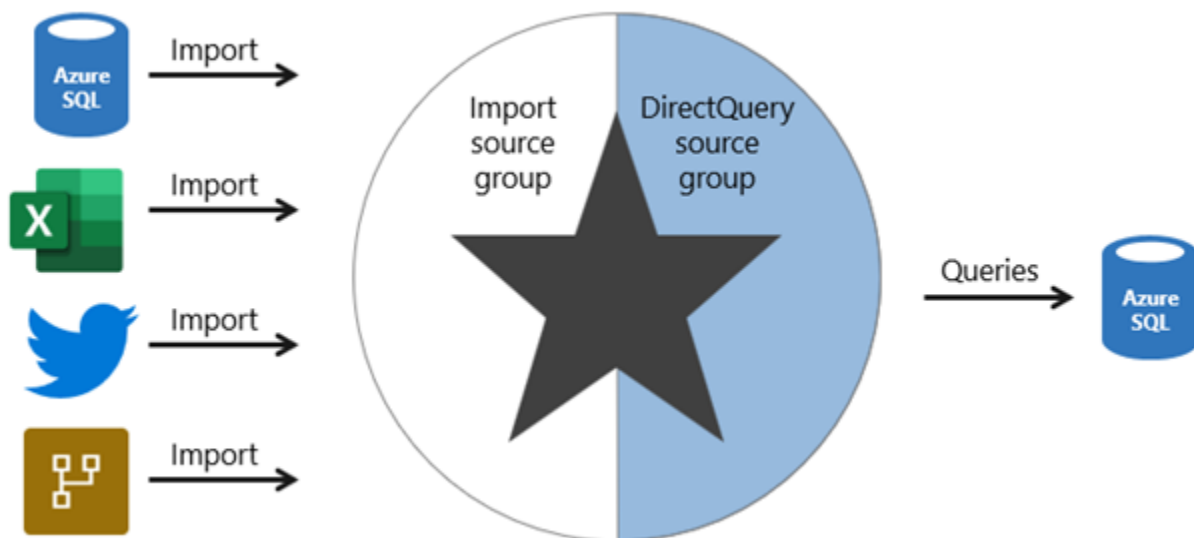
DirectQuery user-defined aggregation tables

You can add user-defined aggregation tables to a DirectQuery model. User-defined aggregation tables are special model tables that are hidden (from users, calculations, and RLS). They work best when they satisfy higher-grain analytic queries over large fact tables. When you set the aggregation table to use DirectQuery storage mode, it can query a materialized view in the data source. You can also set an aggregation table to use import storage mode or enable automatic aggregations, and these options are described in Unit 4.

For more information, see [DirectQuery model guidance in Power BI Desktop](#).

Determine when to develop a composite model

A composite model comprises more than one source group. Typically, there's always the import source group and a DirectQuery source group.



Note

Generally, the benefits and limitations associated with import and DirectQuery storage modes apply to composite models.

Composite model benefits

There are several benefits to developing a composite model.

Composite models provide you with design flexibility. You can choose to integrate data using different storage modes, striking the right balance between imported data and pass-through data. Commonly, enterprise models benefit from using DirectQuery tables on large data

sources and [by boosting query performance](#) with imported tables. Power BI features that support this scenario are described later in this unit.

Composite models [can also boost the performance of a DirectQuery model by providing Power BI with opportunity to satisfy some analytic queries from imported data](#). Querying [cached data almost always performs better](#) than pass-through queries.

Lastly, [when your model includes DirectQuery tables to a remote model](#), like a Power BI dataset, you [can extend your model with new calculated columns and tables](#). It [results in a specialized model](#) based on a core model. For more information, see [Power BI usage scenarios: Customizable managed self-service BI](#).

Composite model limitations

There are several limitations related to composite models.

- Import (or dual, as described later) storage mode tables [still require periodic refresh](#). [Imported data can become out of sync with DirectQuery sourced data](#), so it's important to refresh it periodically.
- When an analytic query must [combine imported and DirectQuery data](#), Power BI [must consolidate source group query results](#), which can [impact performance](#). To help [avoid this](#) situation for higher-grain queries, you [can add import aggregation tables to your model](#) (or enable automatic aggregations) and [set related dimension tables to use dual storage mode](#). This scenario is described later in this unit.
- When [chaining models](#) (DirectQuery to Power BI datasets), [modifications made to upstream models can break downstream models](#). Be sure to assess the impact of modifications by performing [dataset impact analysis](#) first.
- [Relationships between tables from different source groups](#) are known as [limited relationships](#). A model relationship is [considered limited](#) [when there's no guaranteed "one" side](#) (The "one" side means the column contains unique values; the "many" side means the column can contain duplicate values.) This can occur in [two situations](#). The [first](#) is when the relationship uses a [many-to-many cardinality type](#), even if one or both columns contain unique values. The [second](#) is when the [relationship spans different source groups](#), which [only happens in composite models](#). Limited relationships [may result in different evaluations of model queries and calculations](#). For more information, see [Relationship evaluation](#).

Boost DirectQuery model performance with import data

When there's a justification to develop a DirectQuery model, you can mitigate some limitations by using specific Power BI features that involve import tables.

Import aggregation tables

You can add import storage mode user-defined aggregation tables or enable automatic aggregations. This way, Power BI directs higher-grain fact queries to a cached aggregation. To boost query performance further, ensure that related dimension tables are set to use dual storage mode.

Automatic aggregations are a Premium feature. For more information, see [Automatic aggregations](#).

Dual storage mode

A dual storage mode table is set to use both import and DirectQuery storage modes. At query time, Power BI determines the most efficient mode to use. Whenever possible, Power BI attempts to satisfy analytic queries by using cached data.

Dual storage mode tables work well with import aggregation tables. They allow Power BI to satisfy higher-grain queries entirely from cached data.

Slicer visuals and filter card lists, which are often based on dimension table columns, render more quickly because they're queried from cached data.

Deliver real-time data from an import model

When you set up an import table with incremental refresh, you can enable the **Get the latest data in real-time with DirectQuery** option.

Incremental refresh and real-time data

Refresh large tables faster with incremental refresh. Plus, get the latest data in real time with DirectQuery (Premium only). [Learn more](#)

 These settings will apply when you publish the dataset to the Power BI service. Once you do that, you won't be able to download it back to Power BI Desktop. [Learn more](#)

1. Select table

FactInternetSales

2. Set import and refresh ranges

☒ Incrementally refresh this table

Archive data starting before refresh date

Data imported from 12/21/2016 to 12/18/2021.

Incrementally refresh data starting before refresh date

Data will be incrementally refreshed from 12/18/2021 to 12/21/2021.

3. Choose optional settings



☒ Get the latest data in real time with DirectQuery (Premium only) [Learn more](#)

By enabling this option, Power BI automatically creates a table partition that uses DirectQuery storage mode. In this case, the table becomes a hybrid table, meaning it has import partitions to store older data, and a single DirectQuery partition for current data.

When Power BI queries a hybrid table, the query uses the cache for older data, and passes through to the data source to retrieve current data.

This option is only available with a Premium license.

For more information, see [Configure incremental refresh and real-time data](#).

Choose a model framework

Here's some general guidance about choosing the appropriate model framework for your project. It especially applies to enterprise solutions, where data volumes are large, query throughput is high, and fast responsiveness is important.

Most importantly, choose the import model framework whenever possible. This framework offers you the most options, design flexibility, and delivers fast performance. Be sure to apply data reduction techniques to ensure that Power BI loads the least amount of data possible.

Choose the DirectQuery model framework when your data source stores large volumes of data and/or your report needs to deliver near real-time data.

Choose the composite model framework to:

- Boost the query performance of a DirectQuery model.
- Deliver near real-time query results from an import model.
- Extend a Power BI dataset (or AAS model) with additional data.

You can boost the query performance of a DirectQuery model by using aggregation tables, which can use import or DirectQuery storage mode. When using import aggregation tables, be sure to set related dimension tables to use dual storage mode. That way, Power BI can satisfy higher-grain queries entirely from cache.

You can deliver near real-time query results from in import model by creating a hybrid table. In this case, Power BI adds a DirectQuery partition for the current period.

Lastly, you can create specialized models by chaining to a core model by using DirectQuery. This type of development is typically done by a business analyst who extends core models, which IT delivers and supports.

Important

Plan carefully. In Power BI Desktop, it's always possible to convert a DirectQuery table to an import table. But it's not possible to convert an import table to a DirectQuery table.

Knowledge check

Summary

You can develop your Power BI model based on three different frameworks: import, DirectQuery, and composite. Each framework has its own benefits and limitations, and features to help you optimize your model.

Ultimately, you should strive to develop a model that efficiently delivers fast performance with low latency, even for high volume data sources.

Model data with Power BI

Configure a semantic model

Introduction

This module explains **how to design a semantic model**, which is the task you undertake as a data modeler once you apply your Power Query queries. At this point in your development, you have a model comprising one or possibly more tables—but there's more work to be done. Your goal is to develop a model that supports the reporting requirements, and that's **user-friendly and intuitive**. Essentially, you're developing a *semantic layer* over the data.

Your **first design efforts** should focus on **configuring the relationships** between model tables. You can **then move on to** enhance the model design by **setting table and column properties**, and by creating other model objects, like **hierarchies, measures, and parameters**.

Note

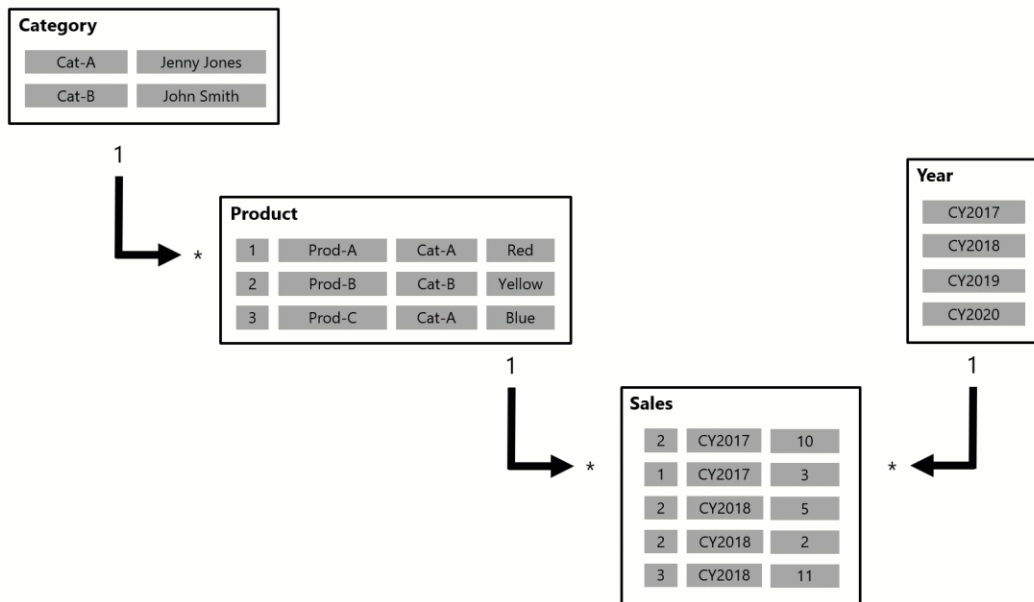
This module focuses on Import model designs. Therefore, it **doesn't cover** working with different model frameworks, like **DirectQuery and composite models**. Also, it **doesn't cover** user-defined aggregations, row-level security (RLS), or development tasks that involve writing Data Analysis Expressions (DAX) formulas.

Configure relationships

When your model **comprises more than one table**, you need to **ensure that appropriate relationships exist** between the tables. **Relationships propagate filters applied on one model table to a different model table. They continue to propagate so long as there's a relationship path to follow**, which can involve propagation to multiple tables.

For example, when a **visual filters the Year column of the Date table**, a relationship to the **Sales table automatically filters** that table so that rows representing sales for that year are summarized. For report authors, this is normal and expected behavior.

Here's an animated example that shows how relationships propagate filters to other tables.



Configure data load options

Before you apply your Power Query queries to load data to your model, you should first inspect the Power BI Desktop **Data load options** and adjust them when necessary.

Specifically, you might want to enable or disable relationship settings. When enabled, these settings can import relationships detected in source data, update or delete relationships when refreshing data, and autodetect new relationships.

Relationships

- ☒ Import relationships from data sources on first load ⓘ
- ☒ Update or delete relationships when refreshing data ⓘ
- ☒ Autodetect new relationships after data is loaded ⓘ

[Learn more](#)

When relationships aren't automatically created, you can create them in the **Manage relationships** window, or by switching to Model view.

Sometimes, a table doesn't need a relationship to another model table, which is known as a **disconnected table**. A disconnected table is useful when you want to support a what-if scenario or field parameters. Both scenarios are described later in this module.

Note

For a full explanation of model relationships and links to related guidance articles, see [Model relationships in Power BI Desktop](#).

Columns

Each relationship has a single "from" column and a single "to" column. It's important that the data types for these columns are the same (or equivalent) and that they contain matching values.

You should give consideration the relationships your model needs when defining the table structures in Power Query. To support the one-side of a relationship (described next), you must ensure that column contains unique values. Further, if your data source has a multi-column key, you need to transform the data to produce single-column keys for the related tables. If the related column data types don't match, you can adjust the data types with Power Query.

Understand cardinality

Every relationship has a cardinality type, which is either:

- One-to-many (1:*)
- Many-to-one (*:1)
- One-to-one (1:1)
- Many-to-many (*:*)

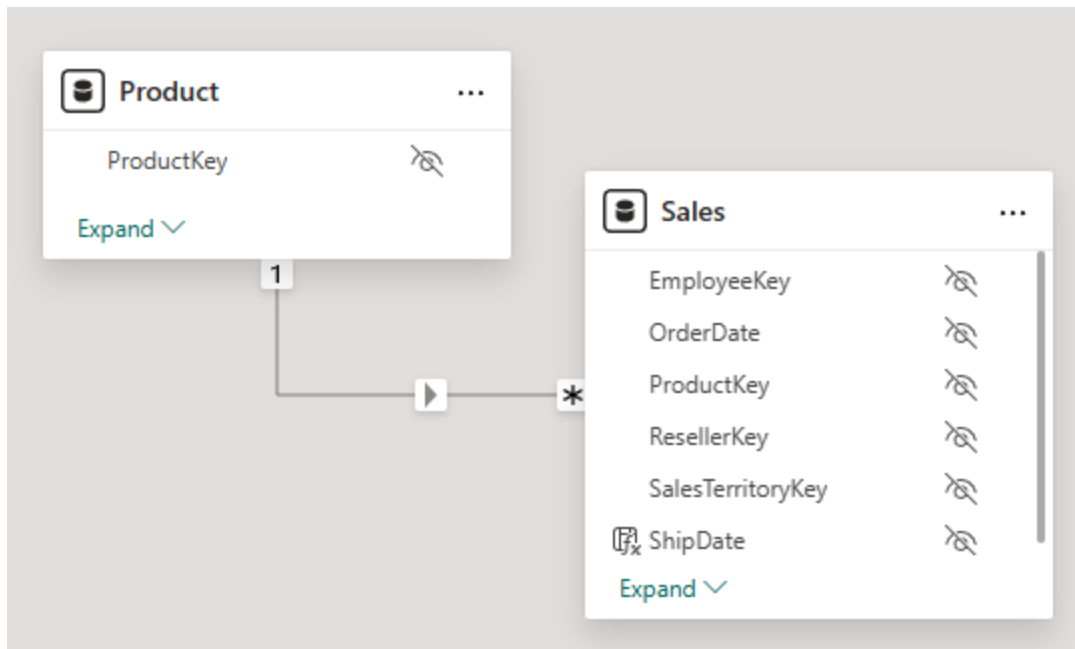
When Power BI Desktop automatically creates a relationship, it determines the cardinality based on the values already loaded into the columns. Sometimes, it might not set it correctly (because the table is yet to be loaded with rows of data), so you need to update the setting.

The One-to-many and Many-to-one relationships are essentially the same, just in different directions. They're the most commonly set cardinality types, typically supporting relationships between dimension tables (the "one" side) and fact tables (the "many" side) in a star schema design.

For example, the ProductKey column in the Product table has a one-to-many relationship with the ProductKey column in the Sales table.

Note

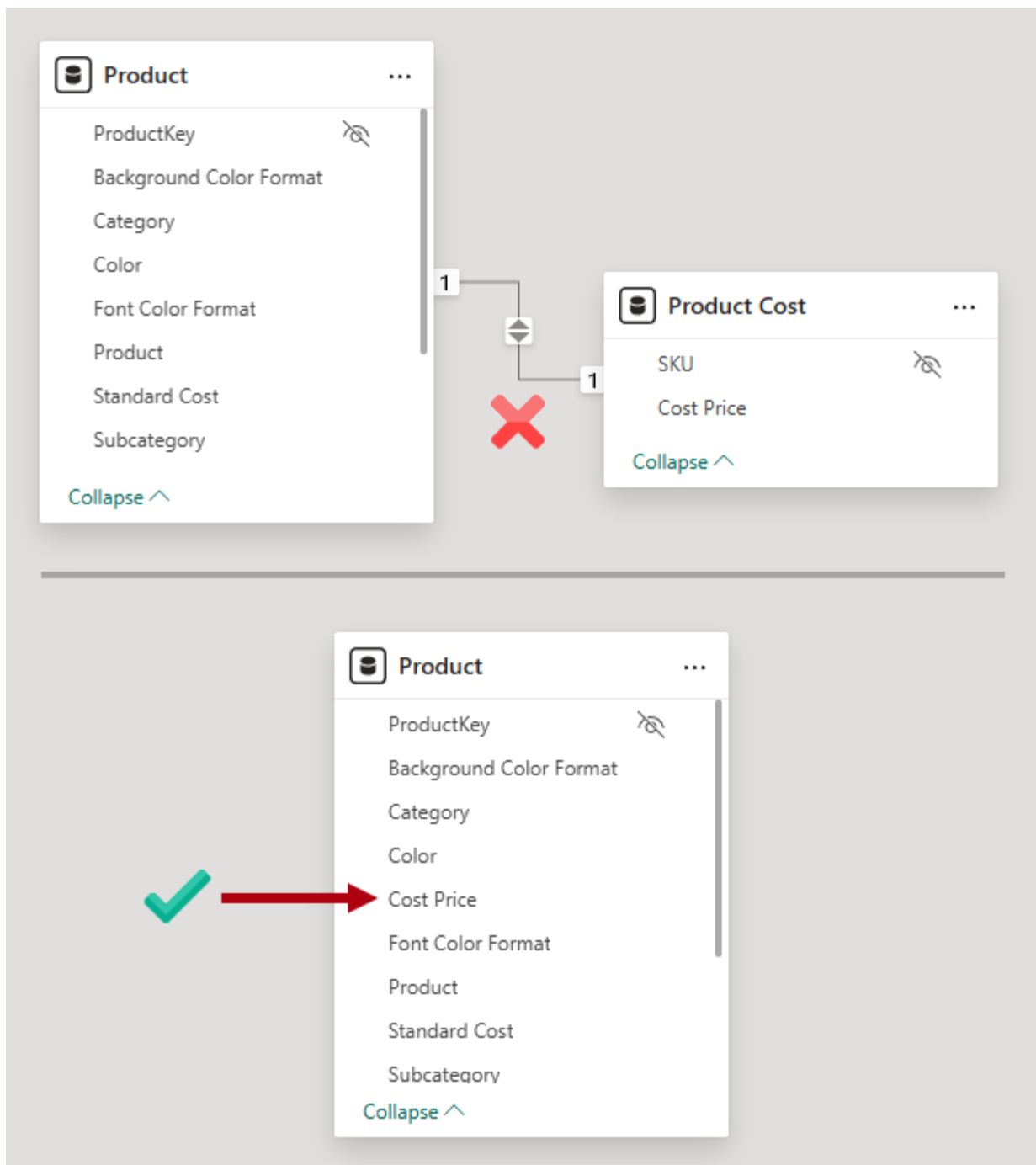
The screenshots of model diagrams in this unit only show columns that are used in relationships.



One-to-one cardinality allows you to relate two tables that each have a unique column. This type of relationship **isn't common** because it's considered a better practice to **use Power Query to merge queries to produce a single model table**. That way, you **reduce the number of model tables** and produce a more intuitive experience for report authors, who can find related fields in a single table.

Consider a scenario where there's a **Product Cost table** that contains the **Cost Price column**, which is **sourced from a supplementary data store**. Its **SKU column contains the product stock-keeping unit (SKU)**. When a **relationship is created** to the Product table on the ProductKey column, **a one-to-one relationship is established** because both columns contain unique values.

In a second scenario, the **modeler takes a different approach**. **They merge the Power Query queries together and add the Cost Price column to the Product query**. They then **disable the load of the Cost Price query**. It results in only one product table.



Many-to-many cardinality allows you to **configure complex relationships between model tables**. It's useful **when there isn't a column of unique values**. For example, the **Target** table stores facts at product category level, yet the **Product** table stores products at product SKU level. While the **ProductKey** column stores the SKU, the **Category** column stores the category name, of which there are many duplicates. When you **relate the Category column of the Product table to the Category column of the Target table**, the cardinality must be set

as *Many-to-many*. In this instance, it allows relating a dimension table to a fact table at a higher level or granularity.

Understand cross filter direction

A model relationship is defined with a **cross filter direction**. The **direction determines how filters propagate**. Where there's a "one" side, filters always propagate to the other side. Where there's a "many" side, it's possible to allow propagation to the other side too.

The possible cross filter options are dependent on the relationship cardinality type:

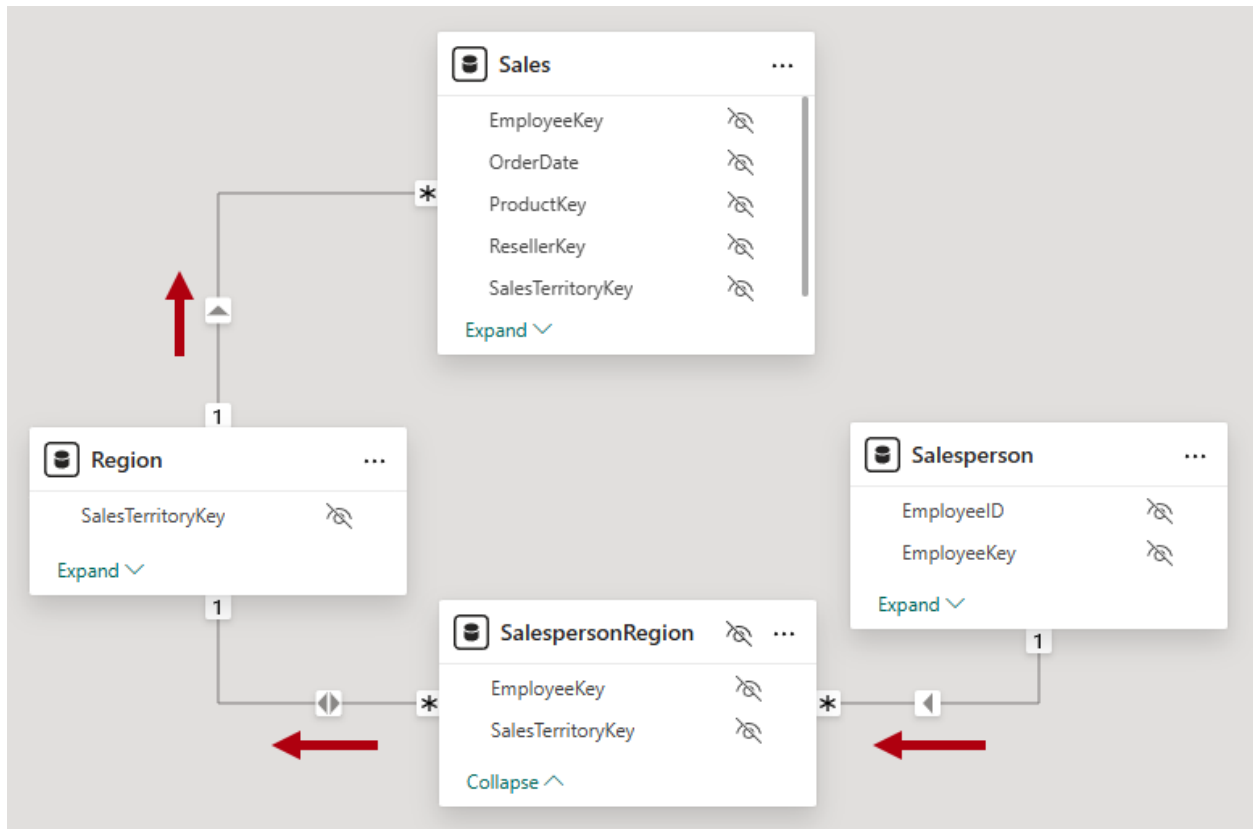
- **One-to-many (or Many-to-one):** Single or both
- **One-to-one:** Both
- **Many-to-many:** Single to either table, or both

Generally, it's a good practice to **avoid or minimize cross filters in both directions**. That's because **it results in better query performance** and usually produces an **intuitive experience for report consumers**.

A valid reason to allow cross filtering of a one-to-many relationship in both directions is to enable many-to-many analysis between two dimension tables. Consider an example where **salespeople are assigned to multiple regions, and conversely regions can have multiple salespeople assigned**. To implement this design, your model needs a bridging table that associates salespeople and regions.

The following screenshot shows a model diagram that relates the Salesperson table to Sales table. The SalespersonRegion table, which is a **bridging table**, contains the **EmployeeKey** and **SalesTerritoryKey** columns, of which **neither contains unique values**. Notice how the filter propagation works:

1. **Filters applied to the Salesperson table propagate to the SalespersonRegion table.**
2. **Filters applied to the SalespersonRegion table propagate to the Region table because cross filtering is supported in both directions.**
3. **Filters applied to the Region table propagate to the Sales table.**



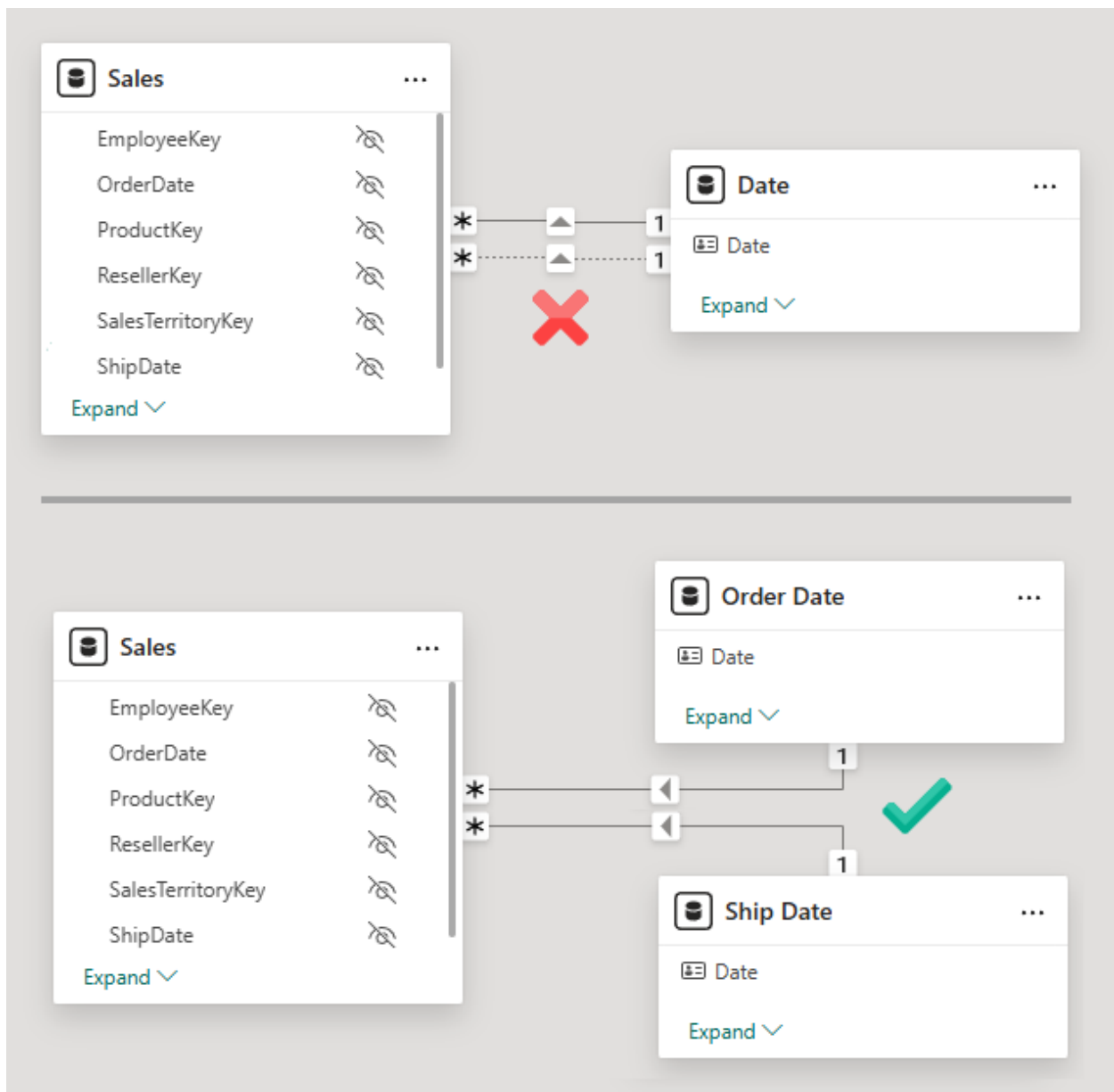
Active vs. inactive relationships

There can only be one active filter propagation path between two model tables. However, it's possible to introduce other relationship paths, though you must set these relationships as *inactive*. Inactive relationships can only be made active during the evaluation of a model calculation by using the USERELATIONSHIP DAX function.

It's common enough that multiple relationships to a dimension table exist, which is known as a *role-playing dimension*. For example, consider that the Sales table has two date columns: OrderDate and ShipDate. However, if you relate both columns to the Date table, only one relationship can be active.

To allow filtering by either order date, ship date, or both at the same time, you need two date tables. By creating Order Date and Ship Date tables, each can have active relationships to the Sales table.

The following screenshot compares two model design scenarios. The first scenario shows a single date table with an active and inactive relationship. The second scenario shows two dates tables, each with active relationships. (Active relationships are represented by solid lines; inactive relationships are represented by dotted lines.)



Work with the model diagram

In the model diagram, relationships are represented by the lines that connect tables.

An easy way to create a relationship is to drag and drop columns between tables in the model diagram. An easy way to edit a relationship is to simply double-click it.

You can interpret the properties of a relationship by viewing it in the model diagram:

- The **cardinality** of a relationship is described by the "one" (1) or "many" (*) icons located at the ends of the relationship line.

- The cross filter direction of a relationship is described by the arrows located in the middle of the relationship line.
- An active relationship is a solid line; an inactive relationship is a dotted line.

To determine which columns are related, you can hover the cursor over the relationship to highlight the related columns.

Configure tables

You're ready to enhance its design when your model contains the required tables and columns and the relationships are configured. You enhance the model by setting properties of model objects, like tables and columns. You can also create new model objects, like hierarchies, measures, and parameters.

Configure table properties

Power BI Desktop provides you with choice when creating model objects and setting their properties. You can use the ribbon, the **Data** pane, or the model diagram and its associated **Properties** pane. Typically, it's easier to work in Model view and use the **Properties** pane because it supports multi-select and bulk property updates.

Every model table has a **Name property**, which allows you to rename it. Its name is always inherited from the Power Query query name. So, if you rename a table in Power BI Desktop, the Power Query name is updated. Model table names must be unique in the model, and you should strive to set user-friendly names.

A model table also has a **Description property**, which is optional. It allows you to set a detailed definition of the table that appears when report authors hover their cursor over the table in the **Data** pane.

The **Synonyms property** allows you to set one or more alternative names for the table, so that Q&A or Copilot can better understand and resolve requests made to automatically generate visuals or DAX formulas.

A model table can be hidden, in which case it's not listed in the **Data** pane (unless the report author expressly wants hidden objects to be shown). You should hide model objects that shouldn't be used directly by report authors. For example, you should hide a bridging table used to support a many-to-many relationship between dimension tables, like the SalespersonRegion table described in the previous unit.

Mark date tables

By default, Power BI includes an **Auto date/time** feature to support time intelligence. When enabled, it **creates hidden date tables for each column that has a date or date/time data type.**

Time intelligence

☐ Auto date/time ⓘ [Learn more](#)

While this option might be useful for novice data modelers or experimentation, it's **better to disable it.** You can **use an existing source date table or create a calculated table with DAX.** Each date table in your model should **contain a column of dates and be *marked* as a date table.** That way, **DAX time intelligence functions return appropriate results.**

Marking a date table involves **selecting the column that contains date values.**

Mark as a date table

×

To enable the creation of date-related visuals, tables and quick measures using this table's

Keep in mind any built-in date tables that are already associated with this table will be ren referring to them may break. [Learn more](#)

Mark as a date table

☒ On

Choose a date column

Date ▼

✔ Validated successfully

Power BI Desktop performs **validations** to ensure that the **date column you select** contain unique values, no BLANKs, and comprises contiguous values from beginning to end.

Configure columns

There are many column properties that you can set. Like with tables, you can set the name, **description, synonyms, and "is hidden"** properties. It's **common to hide columns that are used by relationships**, especially when **they're based on aren't meaningful key values.**

Column names must be unique within the model table, and if the column is visible, you should set a user-friendly name. If you change the column name in Power BI Desktop, a new step is appended to the Power Query query to modify the column name there.

You can assign columns to a display folder, which helps organize the fields for a table. Consider using display folders when your table comprises many visible fields.

Configure column formatting

Every column has a Data type property, which is inherited from the Power Query query. It's concerned with how values are stored. If you change the data type in Power BI Desktop, a new step is appended to the query to modify the data type there.

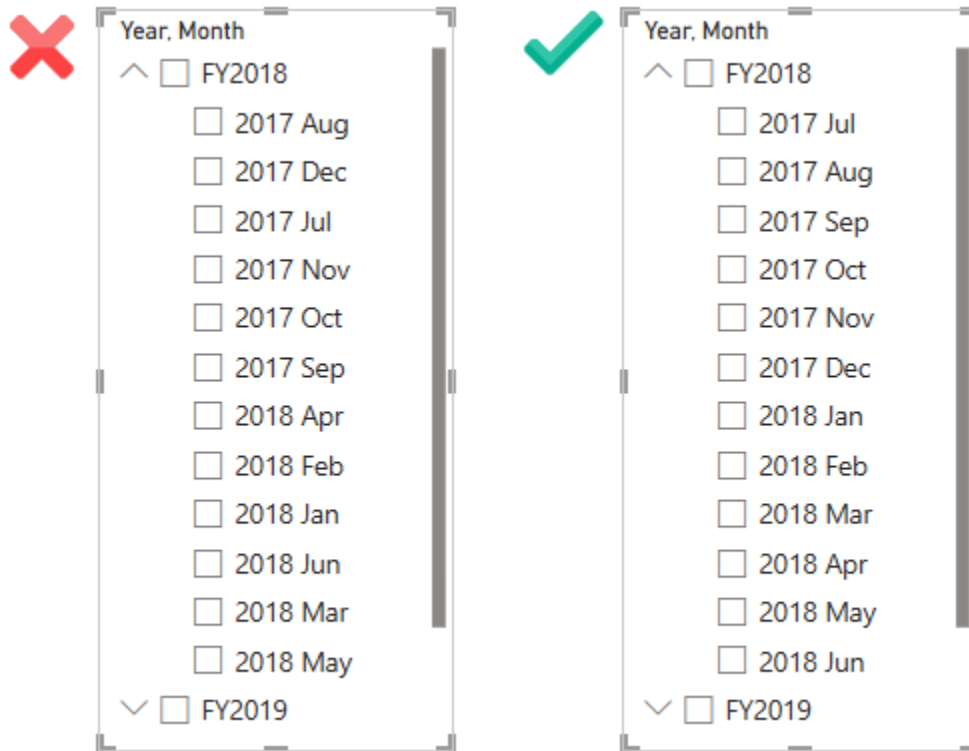
Related, though different, is the Format property. This property is concerned with how the values are presented in visuals. For example, a column with a fixed decimal number data type might be formatted as a currency.

Set sort order

Sometimes, the natural order of the column values doesn't meet your requirements. Text values naturally sort alphabetically, while numbers and dates naturally sort from smallest (or earlier) to largest (or later). When you need the values to sort differently, you can refer to a column in the same table that contains values suited to sorting the column.

For example, in the Date table, the Month column contains the English name of the months, like 2017 Aug (for August 2017). The natural sort order results in months listed alphabetically, not chronologically (in which case August is ordered before all other months). In this case, you can add the MonthKey column to the table, which stores a numeric value comprising the year and month number.

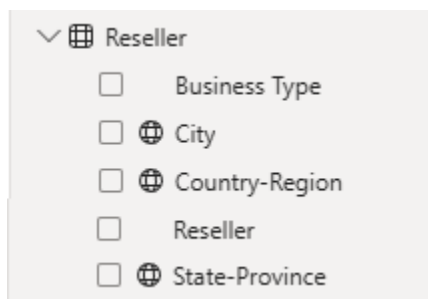
You can then set the Sort by Column property of the Month column to use the MonthKey column. You should then hide the MonthKey column because it shouldn't be used directly by report authors.



Categorize data

You can set the **Data category** property to [describe the content of a column](#).

Setting this property is especially [useful when a column stores a spatial value](#), either in [text](#) or as a [decimal latitude or longitude value](#). Columns that are categorized with a spatial value appear in the **Data** pane with a [spatial icon](#), and Power BI can more [accurately geocode](#) and visualize the data [when used in map visuals](#).



You can also categorize columns that contain a URL.

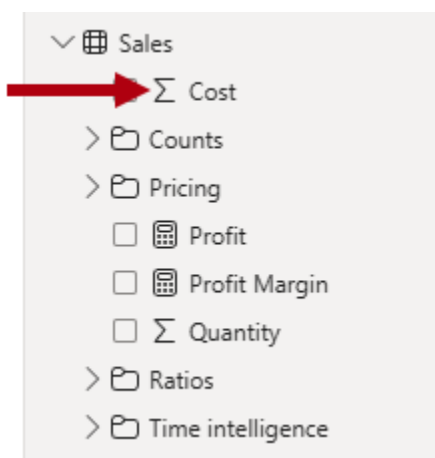
When the URL contains a general web link, set the **Data category** property to [Web URL](#). When used in a table or matrix visual, the report author can configure the format options to [condense the URL](#) into a compact and [easily recognizable link icon](#).

When the URL contains a link to an image, set the property to *Image URL*. When used in a table, matrix, slicer, or multi-row card visual, the report author can configure the format options to display the images.

Set summarization

You can set the **Summarize by** property for numeric columns to determine how they summarize (or not) by default. Options include Sum, Min, Max, Average, Count, and Discount Count (or None).

Summarizable columns appear in the **Data** pane with a sigma symbol (Σ), and report authors can determine how they're summarized when used by a visual.



Tip

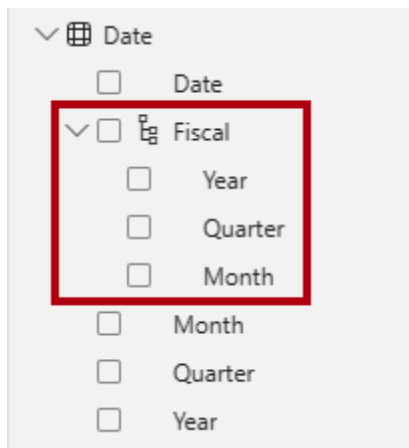
When you want to control how report authors (or Q&A) summarize a numeric column, create a measure (described in a later unit) and hide the column.

Configure hierarchies

Hierarchies are optional. By creating them, you provide clues to report authors about relationships between columns in a table. If you don't create the hierarchy, report authors can still achieve the same result by adding multiple columns to a visual, but it involves more knowledge and effort.

You can add one or more hierarchies to any model table. The hierarchy levels must be based on columns from that table, and they provide a navigation path across those columns.

For example, you can add a hierarchy named *Fiscal* to the Date table with levels based on the Year, Quarter, and Month columns.



The following screenshot shows a [matrix visual that has the Fiscal hierarchy](#) on its rows.

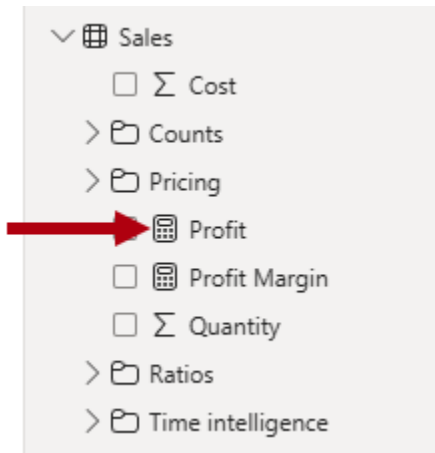
Year	Sales
FY2018	\$16,429,043
FY2018 Q1	\$3,195,733
2017 Jul	\$489,328
2017 Aug	\$1,540,072
2017 Sep	\$1,166,332
FY2018 Q2	\$4,874,023
FY2018 Q3	\$4,069,304
FY2018 Q4	\$4,289,983
FY2019	\$27,979,780
FY2020	\$33,139,748
Total	\$77,548,570

Like model columns, you can set description, synonym, display folder, and "is hidden" properties for hierarchies.

Configure measures

While report authors can summarize any column in Power BI visuals, they can't achieve complex summarization requirements, like a [year-to-date \(YTD\) calculation](#). In this case, you need to create a measure.

A **measure** is a **named DAX formula that's added to a model table**. It achieves summarization, and it appears in the **Data** pane with a **calculator icon**. Measure names **must be unique within the model**. Measures can be simple, such as [SUM](#) or [AVERAGE](#). Measures can also be [complex and calculate the Sales Amount minus Product Cost](#) to find the **Profit**.



These calculations extend the data in your semantic model by providing the essential calculations required for effective data visualization, enabling you to uncover deeper insights and trends within your data.

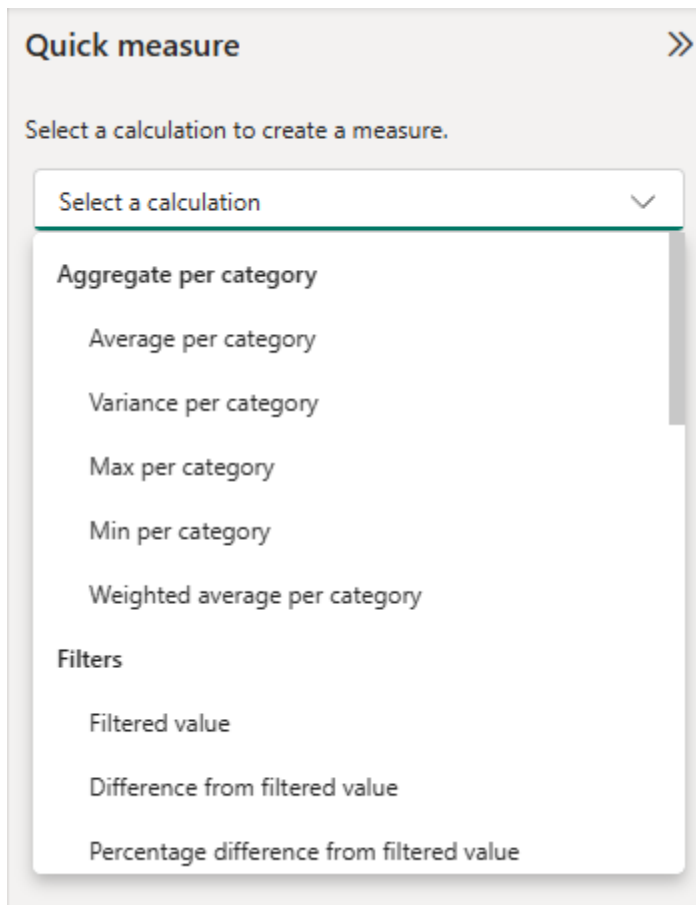
Note

If the model is queried by using Multi-Dimensional Expressions (MDX), which is the case when you use [Analyze in Excel](#), then you must also create measures.

If you don't have DAX skills, you can [use Copilot for Power BI to generate a measure](#) based on a prompt. You can also use the *Quick measures* feature.

Use Quick measures

Quick measures allow you to create a measure by selecting a calculation template and dragging fields to configure it. [Power BI Desktop then creates a measure](#) based on your configuration.



Like model columns, you can set description, synonym, display folder, and "is hidden" properties for measures. You can also set formatting properties. The **Home table** property allows assigning the measure to any model table.

Tip

You can also use Copilot to autogenerate a description value for the measure, which it does by inspecting the DAX formula. For more information, see the [Copilot for Power BI documentation](#).

Configure parameters

A **parameter** in Power BI lets users change report settings, like filters or calculations, without changing the original data. You can add two types of parameters to your model: **Numeric range** and **Fields**.

Create a Numeric range parameter

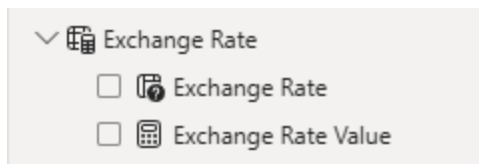
You define a **Numeric range** parameter by setting a numeric data type, minimum and maximum values, an increment value, and a default value. Power BI Desktop then creates a

calculated table by using DAX. It also creates a measure that represents the value that's used to filter the table. The table doesn't have (or need) a relationship to any other model table, creating a *disconnected table*.

A numeric range parameter can support a what-if scenario, where the report consumer uses a slicer to filter the table, and measures do something relevant with the selected value. For example, a report could allow the report consumer to set a hypothetical currency exchange rate. A report visual could then display a measure that applies the exchange rate in a meaningful way.

Parameters appear in the **Data** pane with a question mark symbol (?).

In the following screenshot, the Exchange Rate table is the calculated table, and the Exchange Rate column contains the numeric range values. The Exchange Rate Value is a measure that returns the select value in the range.



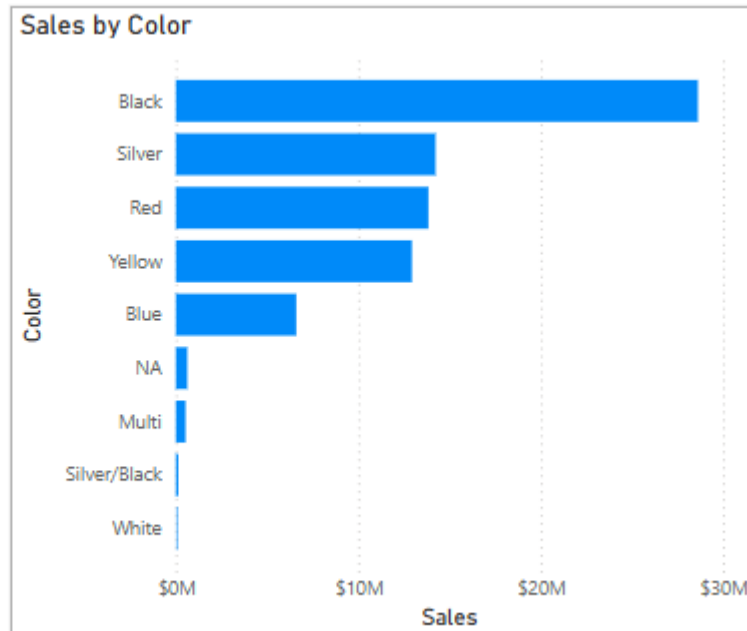
Create a Fields parameter

You can define a Fields parameter by creating a group of different fields. The parameter field can then be used in visuals and also to set up a slicer. Slicer selection allows report consumers to choose which field to visualize.

Consider an example where a fields parameter named Product Grouping comprises four columns from the Product table: Category, Subcategory, Product, and Color. When the parameter is added to the report page, the report consumer can select which field. The corresponding bar chart places the selected field on its Y-axis.

Product Grouping ▾

- ☐ Category
- ☐ Subcategory
- ☐ Product
- ☒ Color



Exercise - Configure a semantic model in Power BI

Lab story

In this lab, you'll commence developing the data model. It will involve creating relationships between tables and then configuring table and column properties to improve the friendliness and usability of the data model. You'll also create hierarchies and quick measures.

In this lab, you learn how to:

- Create model relationships.
- Configure table and column properties.
- Create hierarchies.
- Create quick measures.
- Configure a many-to-many relationship.

This lab should take approximately 45 minutes.

Get started

To complete this exercise, first open a web browser and enter the following URL to download the zip file:

<https://github.com/MicrosoftLearning/PL-300-Microsoft-Power-BI-Data-Analyst/raw/Main/Allfiles/Labs/03-configure-semantic-model/03-model-data.zip>

Extract the file to the **C:\Users\Student\Downloads\03-model-data** folder.

Open the **03-Starter-Sales Analysis.pbix** file.

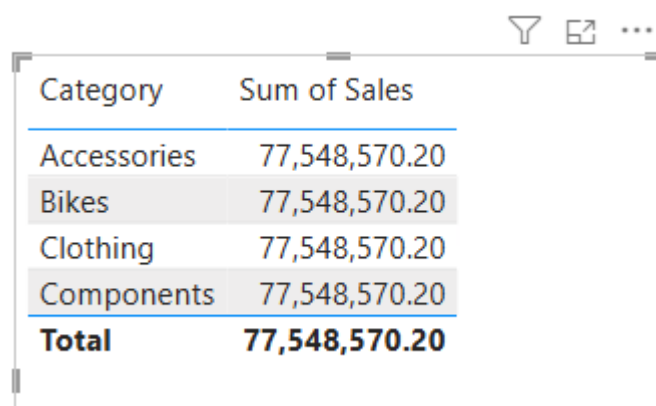
Note: You can dismiss the sign-in by selecting **Cancel**. Close any other informational windows. Select **Apply Later**, if prompted to apply changes.

Create model relationships

In this task, you'll create model relationships. The file was configured to not identify relationships between tables, which isn't the default setting, but is recommended to prevent extra work creating the correct relationships for your model.

The labs use a shorthand notation to reference a field. It will look like this: Product | Category. In this example, Product is the table name and Category is the field name.

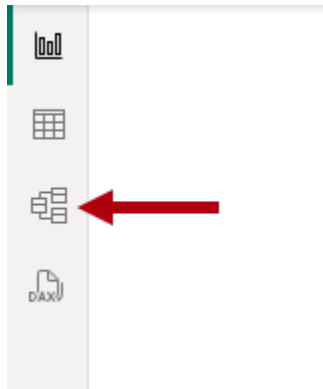
1. In Power BI Desktop, to view all table fields, in the **Data** pane, right-click an empty area, and then select **Expand All**.
2. To create a table visual, in the **Data** pane, from inside the Product table, check the Category field.
3. To add another column to the table, in the **Data** pane, check the Sales | Sales field.
4. Notice that the table visual lists four product categories, and that the sales value is the same for each, and the same for the total.



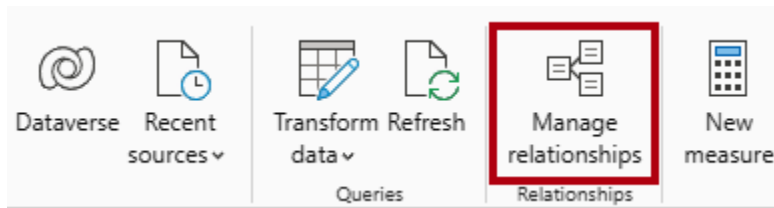
Category	Sum of Sales
Accessories	77,548,570.20
Bikes	77,548,570.20
Clothing	77,548,570.20
Components	77,548,570.20
Total	77,548,570.20

The issue is that the table is based on fields from different tables. The expectation is that each product category displays the sales for that category. However, because there isn't a model relationship between these tables, the Sales table isn't filtered. You'll now add a relationship to propagate filters between the tables.

5. To switch to the model designer, at the left select the Model view icon.



6. On the **Home** ribbon, select **Manage Relationships**.



7. In the **Manage Relationships** window, notice that no relationships are yet defined.
8. To create a relationship, select **+ New relationship**.
9. To configure a relationship from Product table to Sales table, in the **From table** dropdown list, select the Product table, and in the **To table** dropdown list, select the Sales table.

←

New relationship

×

Select tables and columns that are related.

From table

1

Product

Background C...	Category	Color	Font Color Fo...	Product	ProductKey	Subcatego
#000000	Components	Black	#FFFFFF	HL Road Fra...	210	Road Fra
#000000	Accessories	Black	#FFFFFF	Sport-100 Hel...	215	Helmets
#000000	Accessories	Black	#FFFFFF	Sport-100 Hel...	216	Helmets

To table

2

Sales

EmployeeKey	OrderDate	ProductKey	Quantity	ResellerKey	SalesOrderNu...	SalesTerrit
282	Friday, 25 Au...	235	2	312	SO43897	4
282	Friday, 25 Au...	351	2	312	SO43897	4
282	Friday, 25 Au...	348	2	312	SO43897	4

10. Notice the following properties were automatically configured:

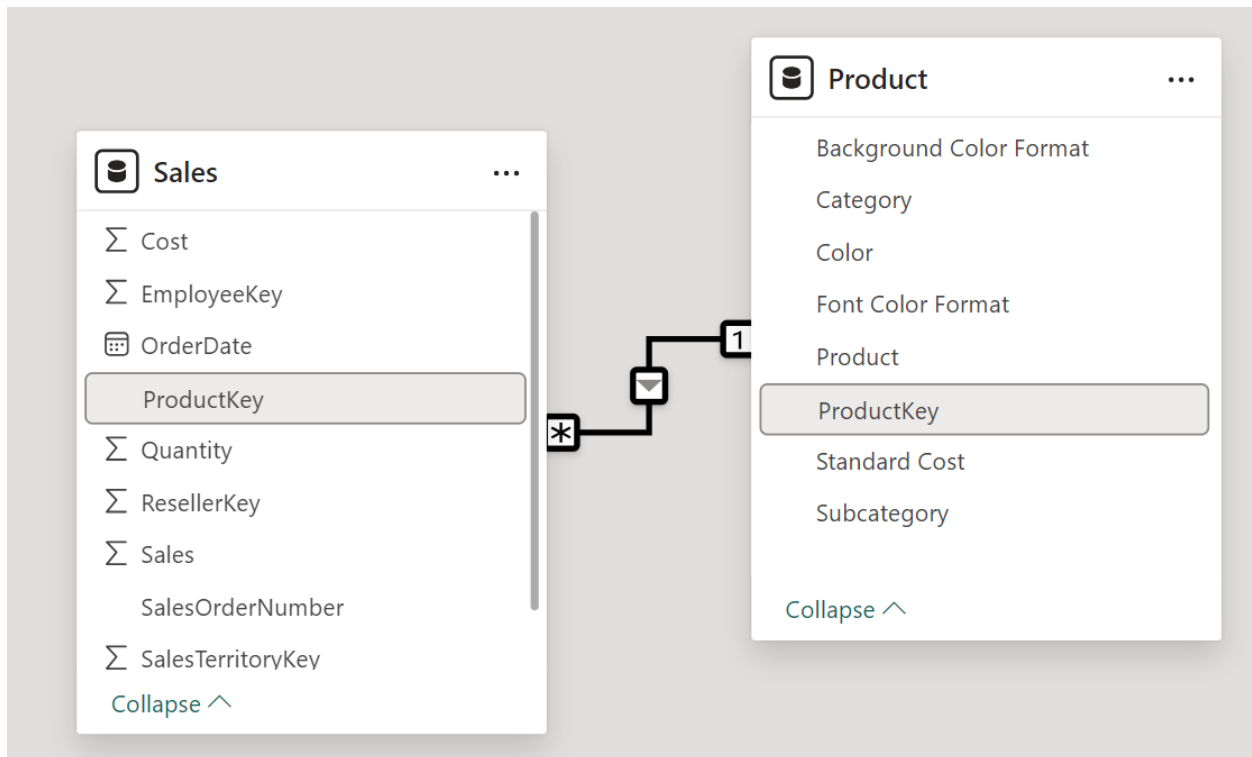
- **ProductKey columns in each table are selected.** *The columns were selected because they share the same name and data type. You may need to find matching columns with different names in real data.*
- **Cardinality type is One To Many (1:*)**. *The cardinality was automatically detected, because Power BI understands that the ProductKey column from the Product table contains unique values. One-to-many relationships are the most common cardinality, and all relationship you create in this lab will be this type.*
- **Cross Filter Direction type is Single**. *Single filter direction means that filters propagate from the “one side” to the “many side”. In this case, it means filters applied to the Product table will propagate to the Sales table, but not in the opposite direction.*
- **Make This Relationship Active is checked.** *Active relationships propagate filters. It’s possible to mark a relationship as inactive so filters don’t propagate. Inactive relationships can exist when there are multiple relationship paths*

between tables. In this case, model calculations can use special functions to activate them.

Cardinality	Cross-filter direction
One to many (1:*)	Single
☒ Make this relationship active	☐ Apply security filter in both directions
☐ Assume referential integrity	

11. Select **Save**, notice in the **Manage Relationships** window that the new relationship is listed, and then select **Close**.

12. In the model diagram, notice there's now a connector between the two tables (you might choose to reposition the tables to see the relationship more clearly).



You can interpret many of the relationship properties by looking at the relationship line in the diagram:

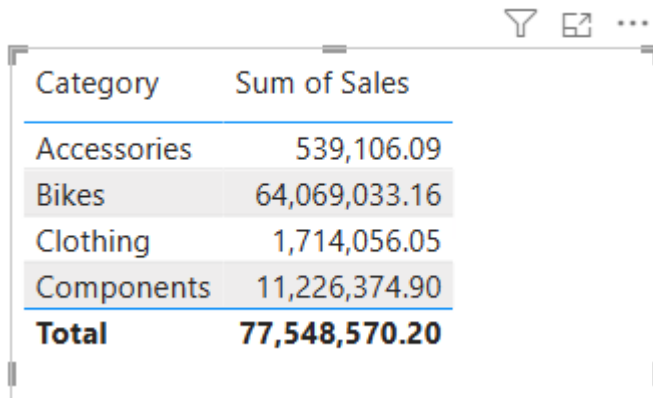
- Cardinality is represented by the **1** and **(*)** indicators.
- Filter direction is represented by the arrow head.

- A solid line represents an active relationship; a dotted line represents an inactive relationship.

Tip: If you hover the cursor over the relationship to highlight the related columns._

13. Switch to Report view, and then notice that the table visual updated to display different values for each product category.

Filters applied to the Product table now propagate to the Sales table.



The screenshot shows a table visual with a filter icon, a refresh icon, and a menu icon in the top right corner. The table has two columns: 'Category' and 'Sum of Sales'. The rows are: Accessories (539,106.09), Bikes (64,069,033.16), Clothing (1,714,056.05), Components (11,226,374.90), and a Total row (77,548,570.20). The 'Bikes' row is highlighted with a grey background.

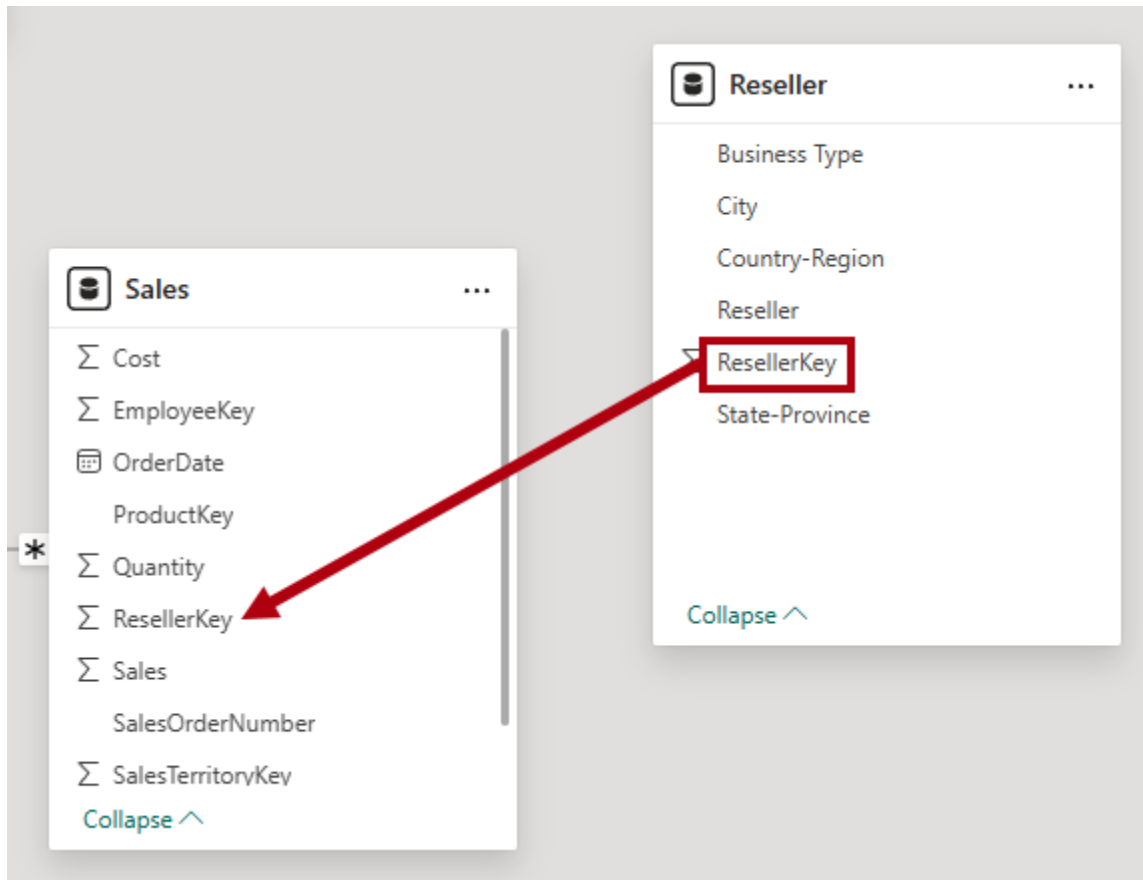
Category	Sum of Sales
Accessories	539,106.09
Bikes	64,069,033.16
Clothing	1,714,056.05
Components	11,226,374.90
Total	77,548,570.20

Create additional relationships

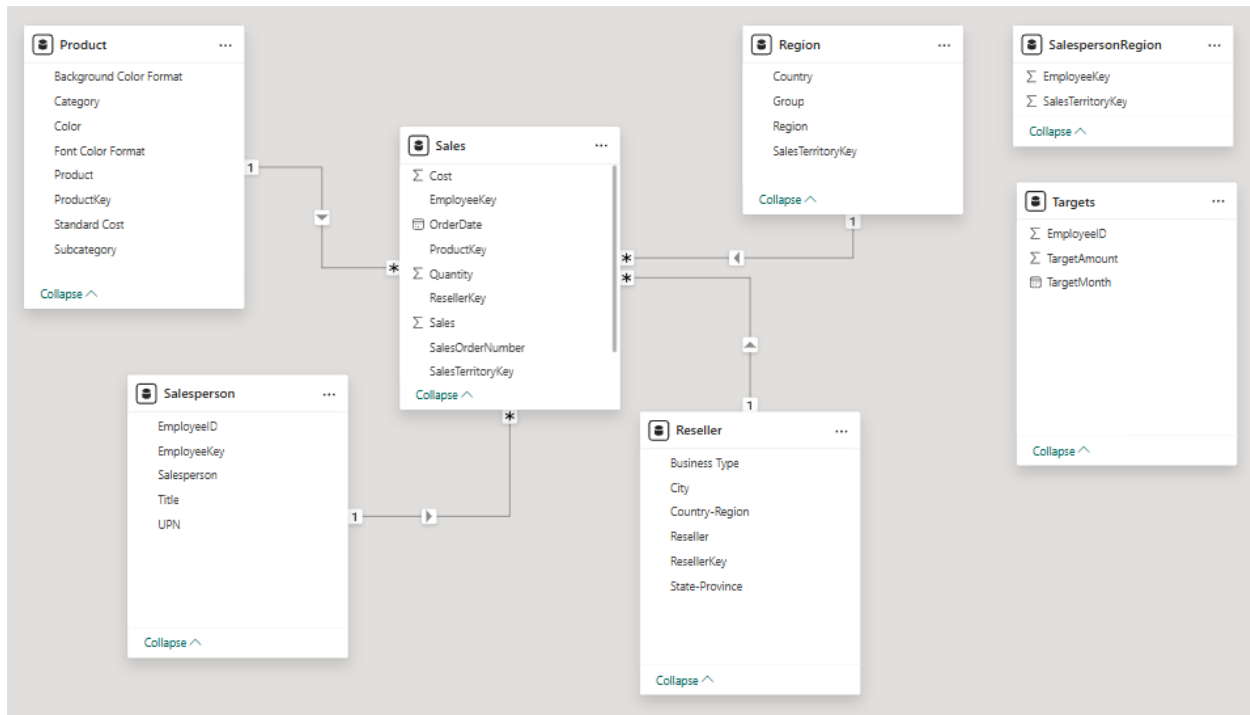
There's an easier way to create a relationship. In the model diagram, you can drag and drop columns to create a new relationship.

1. To create a new relationship using a different technique, switch to Model view.
2. From the Reseller table, drag the ResellerKey column onto the ResellerKey column of the Sales table.

Important: Sometime a column doesn't want to be dragged. If this situation arises, select a different column, and then select the column you intend to drag again, and then try again. Ensure that you see the new relationship added to the diagram.



3. In the **New relationship** window, review the configuration, and then select **Save**.
4. Use the new technique to create the following two model relationships:
 - Region | SalesTerritoryKey to Sales | SalesTerritoryKey
 - Salesperson | EmployeeKey to Sales | EmployeeKey
5. In the diagram, arrange the tables so that the Sales table is positioned in the center of the diagram, and the related tables are arranged about it. Position the disconnected tables to the side.

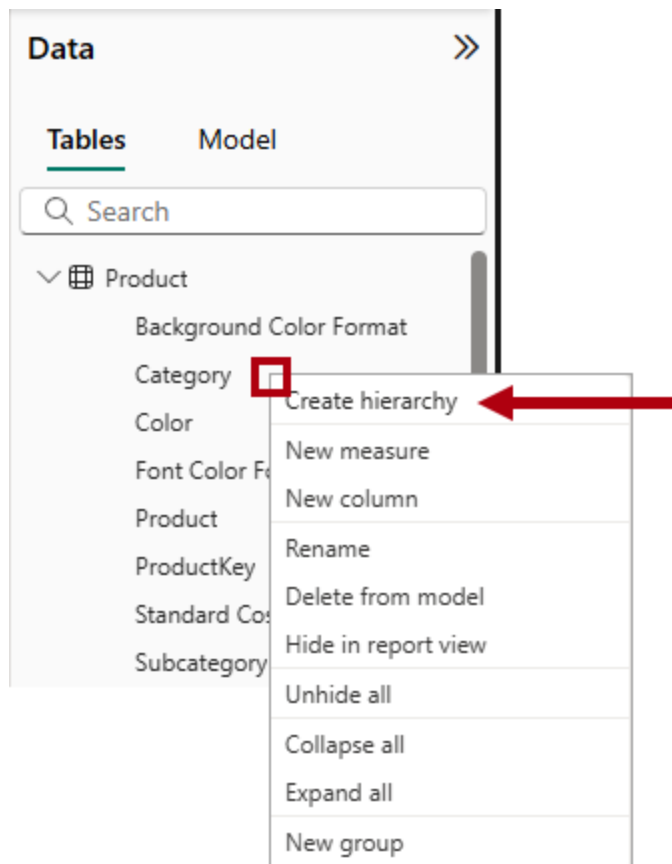


6. Save the Power BI Desktop file.

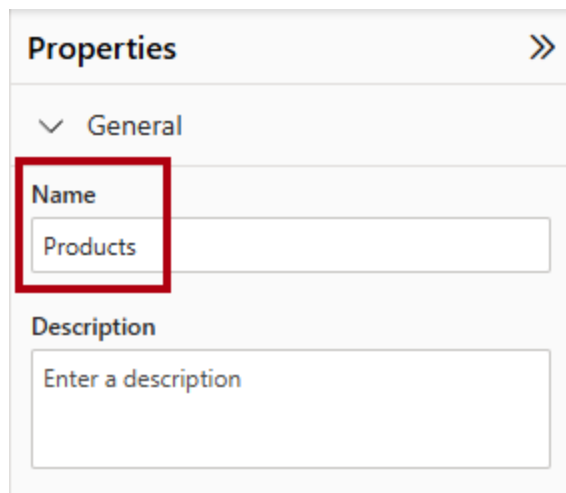
Configure the Product table

In this task, you'll configure the Product table with a hierarchy and display folder.

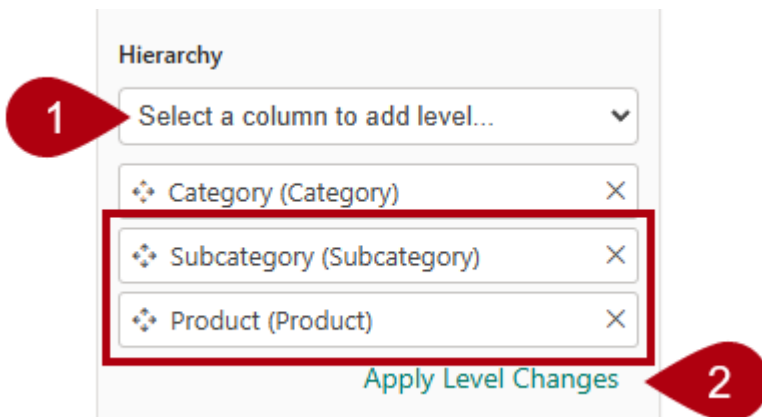
1. Switch to Model view.
2. In the **Data** pane, if necessary, expand the Product table to reveal all fields.
3. In the Product table, right-click the Category column, and then select **Create hierarchy**.



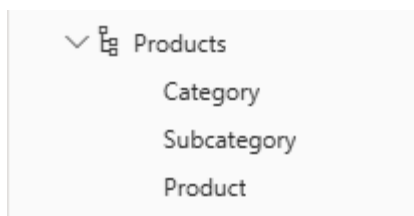
4. In the **Properties** pane, in the **Name** box, replace the value with *Products*.



5. Two add levels to the hierarchy, in the **Hierarchy** dropdown list, select **Subcategory** and then select **Product**, and then select **Apply Level Changes**.



6. In the **Data** pane, notice the Products hierarchy. To reveal the hierarchy levels, expand it.

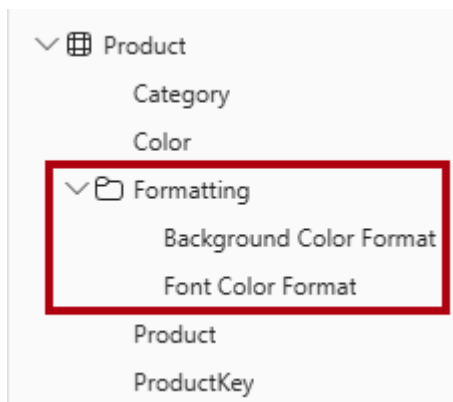


7. To organize columns into a display folder, in the **Data** pane, first select the Background Color Format column.
8. While pressing the **Ctrl** key, select the Font Color Format column.
9. In the **Properties** pane, in the **Display Folder** box, enter *Formatting*.

Display folder



10. In the **Data** pane, notice that the two columns are now inside a folder.

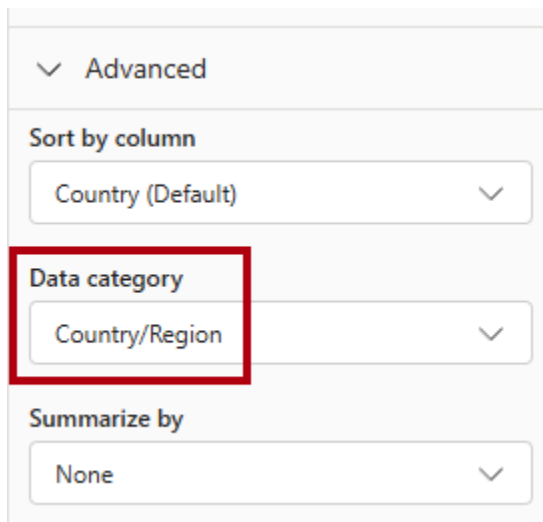


Display folders are a great way to organize tables, especially for tables that comprise many fields. They're logical presentation only.

Configure the Region table

In this task, you'll configure the Region table with a hierarchy and updated categories.

1. In the Region table, create a hierarchy named *Regions*, with the following three levels:
 - Group
 - Country
 - Region
2. Select the Country column (not the Country hierarchy level).
3. In the **Properties** pane, expand the **Advanced** section (located at the bottom of the pane), and then in the **Data Category** dropdown list, select **Country/Region**.



The screenshot shows the 'Advanced' section of a properties pane. It contains three dropdown menus: 'Sort by column' set to 'Country (Default)', 'Data category' set to 'Country/Region' (highlighted with a red box), and 'Summarize by' set to 'None'.

Data categorization can provide hints to the report designer. In this case, categorizing the column as country or region provides more accurate information to Power BI when it renders a map visualization.

Configure the Reseller table

In this task, you'll configure the Reseller table to add a hierarchy and update data categories.

1. In the Reseller table, create a hierarchy named *Resellers*, with the following two levels:
 - Business Type

- Reseller
- 2. Create a second hierarchy named *Geography*, with the following four levels:
 - Country-Region
 - State-Province
 - City
 - Reseller
- 3. Set the data category for the following columns (not within the hierarchy):
 - Country-Region to **Country/Region**
 - State-Province to **State or Province**
 - City to **City**

Configure the Sales table

In this task, you'll configure the Sales table with updated descriptions, formatting, and summarization.

1. In the Sales table, select the Cost column.
2. In the **Properties** pane, in the **Description** box, enter: *Based on standard cost*

*Descriptions can be applied to tables, columns, hierarchies, or measures. In the **Data** pane, description text is revealed in a tooltip when a report author hovers their cursor over the field.*

3. Select the Quantity column.
4. In the **Properties** pane, from inside the **Formatting** section, set the **Thousands Separator** property to Yes.
5. Select the Unit Price column.
6. In the **Properties** pane, from inside the **Formatting** section, set the **Decimal Places** property to 2.
7. In the **Advanced** group (you may need to scroll down to locate it), in the **Summarize By** dropdown list, select **Average**.

By default, numeric columns will summarize by summing values together. This default behavior isn't suitable for a column like Unit Price, which represents a rate. Setting the default summarization to average will produce a meaningful result.

Bulk update properties

In this task, you'll update multiple columns using single bulk updates. You'll use this approach to hide columns, and format column values.

1. In the **Data** pane (or model diagram), select the Product | ProductKey column.
2. While pressing the **Ctrl** key, select the following 13 columns (spanning multiple tables):
 - Region | SalesTerritoryKey
 - Reseller | ResellerKey
 - Sales | EmployeeKey
 - Sales | ProductKey
 - Sales | ResellerKey
 - Sales | SalesOrderNumber
 - Sales | SalesTerritoryKey
 - Salesperson | EmployeeID
 - Salesperson | EmployeeKey
 - Salesperson | UPN
 - SalespersonRegion | EmployeeKey
 - SalespersonRegion | SalesTerritoryKey
 - Targets | EmployeeID
3. In the **Properties** pane, set the **Is Hidden** property to Yes.

The columns were hidden because they're either used by relationships or will be used in row-level security configuration or calculation logic.

*You'll use the SalesOrderNumber column in a calculation in the **Create DAX Calculations in Power BI Desktop** lab.*

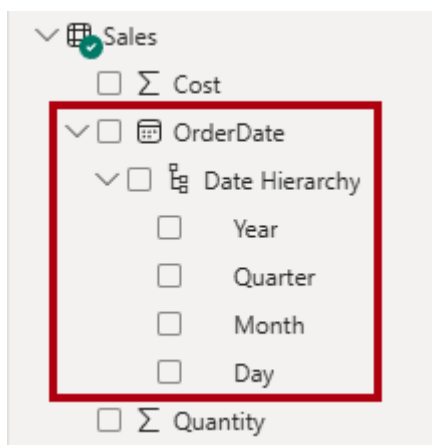
4. Multi-select the following three columns:
 - Product | Standard Cost
 - Sales | Cost

- Sales | Sales
- 5. In the **Properties** pane, from inside the **Formatting** section, set the **Decimal Places** property to 0 (zero).

Explore the model interface

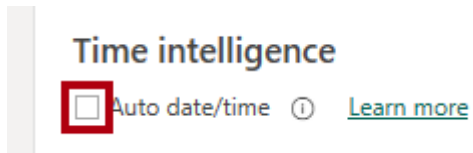
In this task you'll switch to Report view, review the data model interface, and configure the auto date/time setting.

1. Switch to Report view.
2. In the **Data** pane, notice the following:
 - Columns, hierarchies and their levels are fields, which can be used to configure report visuals.
 - Only fields relevant to report authoring are visible.
 - The SalespersonRegion table isn't visible—because all of its fields are hidden.
 - Spatial fields in the Region and Reseller table are adorned with a spatial icon.
 - Fields adorned with the sigma symbol (Σ) will summarize, by default.
 - A tooltip appears when hovering the cursor over the Sales | Cost field.
3. Expand the Sales | OrderDate field, and then notice that it reveals a Date Hierarchy. The Targets | TargetMonth field delivers a similar hierarchy.



Important: These hierarchies weren't created by you. They were created automatically as a default setting. There's a problem, however. The Adventure Works financial year commences on July 1 of each year. But, in these automatically created date hierarchies, the date hierarchy year commences on January 1 of each year.

4. To turn off the auto date/time setting, navigate to **File > Options and Settings > Options**.
5. In the **Options** window, on the **Current File** section, navigate to **Data Load > Time Intelligence**, and uncheck **Auto Date/Time**.

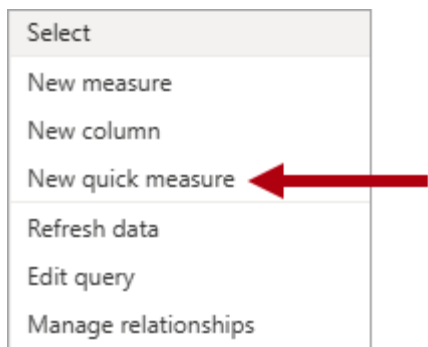


6. In the **Data** pane, notice that the date hierarchies are no longer available.

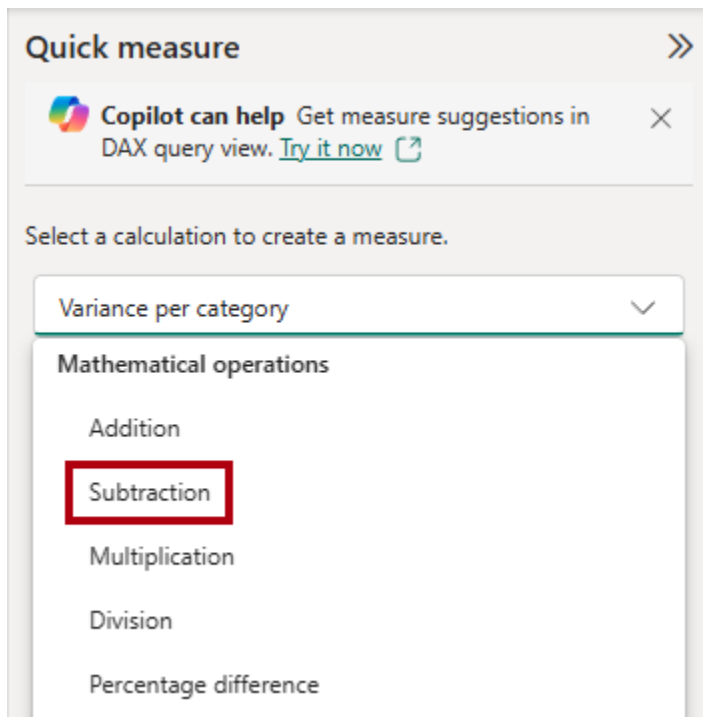
Create quick measures

In this task, you'll create two quick measures to calculate profit and profit margin. A quick measure creates the calculation formula for you. They're easy and fast to create for simple and common calculations.

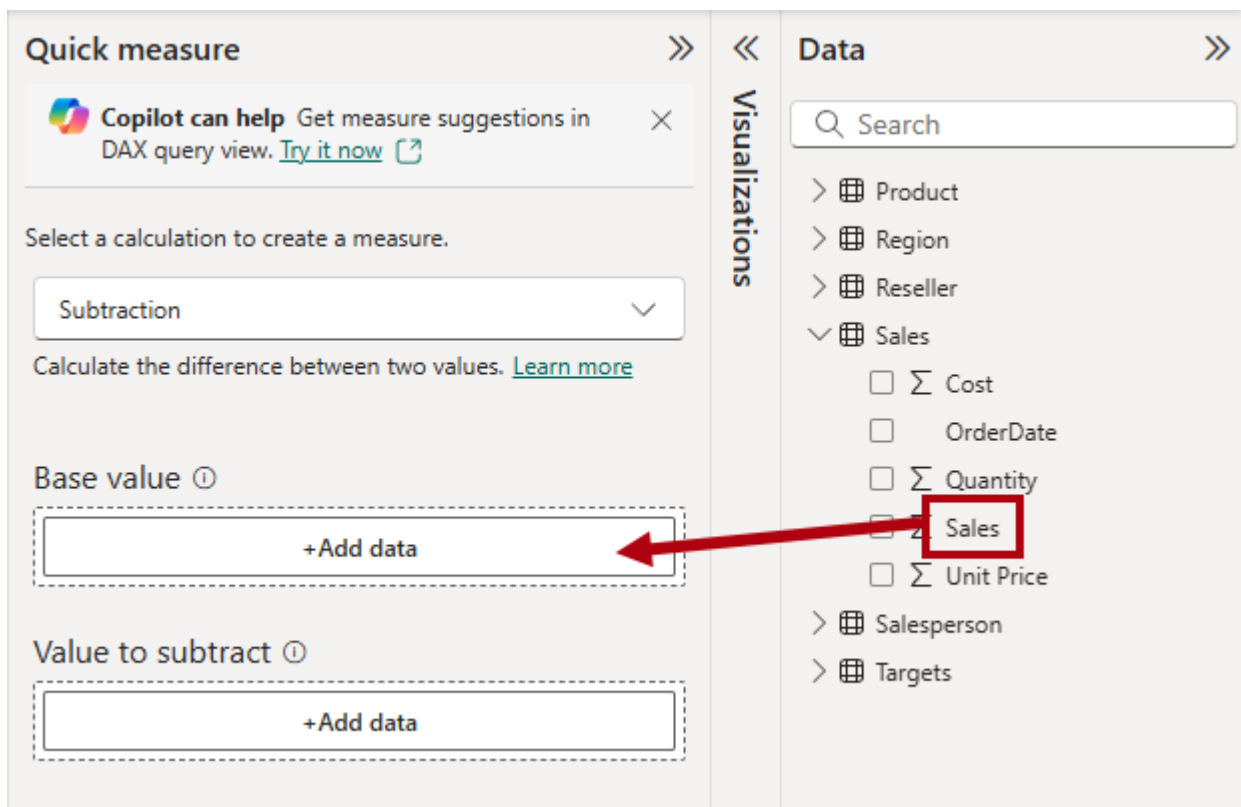
1. In the **Data** pane, right-click the Sales table, and then select **New Quick Measure**.



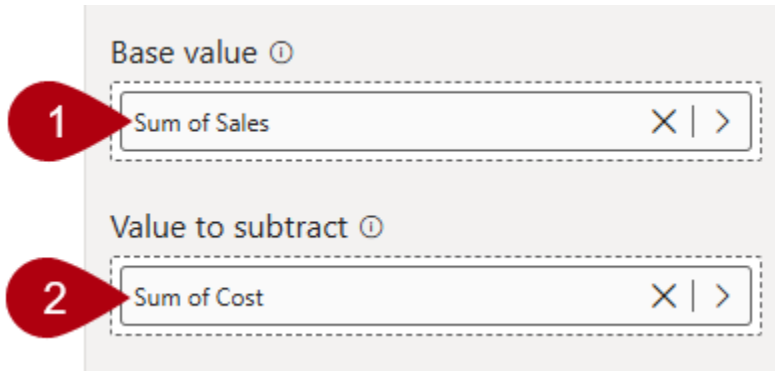
2. In the **Quick Measure** pane, in the **Select a Calculation** dropdown list, from inside the **Mathematical Operations** group, select **Subtraction**.



3. From the **Data** pane, drag the Sales | Sales field into the **Base Value** well.



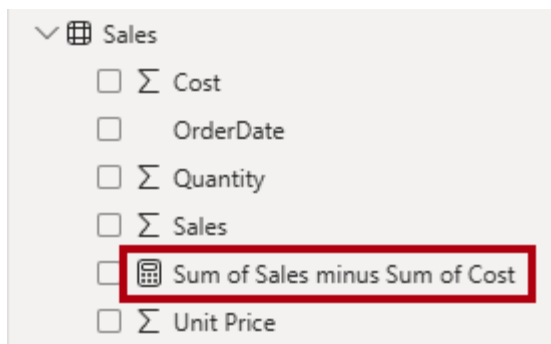
4. Drag the Sales | Cost field into the **Value to Subtract** box.



5. Select **Add**.

6. In the **Data** pane, inside the Sales table, notice that new measure.

Measures are indicated by the calculator icon.



7. To rename the measure, right-click it, select **Rename**, then rename as *Profit*.

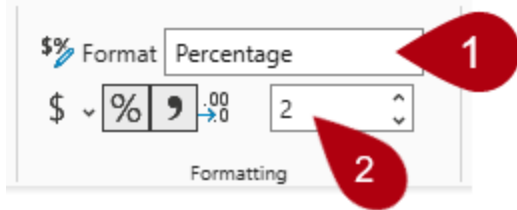
*Tip: To rename a field, you can also double-click it, or select it and press **F2**.*

8. In the Sales table, add a second quick measure, based on the following requirements:

_Important: If the option to create a quick measure doesn't appear in the context menu, use the command located on the **Home** ribbon, from inside the **Calculations** group.

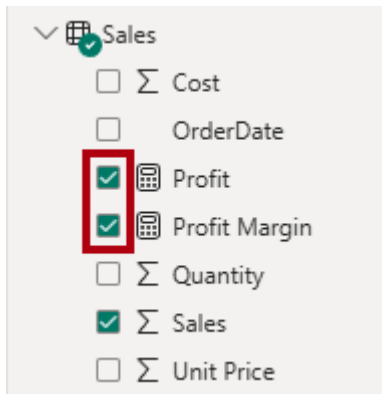
- Use the **Division** mathematical operation.
- Set the **Numerator** to the Sales | Profit field.
- Set the **Denominator** to Sales | Sales field.
- Rename the measure as *Profit Margin*.

9. Ensure the Profit Margin measure is selected, and then on the **Measure Tools** contextual ribbon, set the format to **Percentage**, with two decimal places.



10. To test the two measures, first select the existing table visual on the page.

11. In the **Data** pane, check the Profit and Profit Margin measures to add them to the table.



12. Select and drag the right guide to widen the table visual.

Category	Sales	Profit	Prc
Accessories	\$539,106	\$188,081	
Bikes	\$64,069,033	(\$435,679)	
Clothing	\$1,714,056	\$245,857	
Components	\$11,226,375	\$1,001,235	
Total	\$77,548,570	\$999,495	

13. Verify that the measures produce reasonable results that are correctly formatted.

Category	Sales	Profit	Profit Margin
Accessories	\$539,106	\$188,081	34.89 %
Bikes	\$64,069,033	(\$435,679)	-0.68 %
Clothing	\$1,714,056	\$245,857	14.34 %
Components	\$11,226,375	\$1,001,235	8.92 %
Total	\$77,548,570	\$999,495	1.29 %

Create a many-to-many relationship

In this task, you'll create a many-to-many relationship between the Salesperson table and the Sales table.

1. In Report view, select a blank area of the report page.
2. To create a new table visual, in the **Data** pane, check the following two fields:
 - Salesperson | Salesperson
 - Sales | Sales

Salesperson	Sum of Sales
Amy Alberts	\$737,568
David Campbell	\$3,614,761
Garrett Vargas	\$3,486,102
Jae Pak	\$8,099,817
Jillian Carson	\$9,755,992
José Saraiva	\$5,536,439
Linda Mitchell	\$10,158,635
Lynn Tsoflias	\$1,216,917
Michael Blythe	\$8,952,751
Pamela Ansman-Wolfe	\$3,187,720
Rachel Valdez	\$1,681,208
Ranjit Varkey Chudukatil	\$4,429,368
Shu Ito	\$6,283,193
Stephen Jiang	\$1,073,651
Syed Abbas	\$174,108
Tete Mensa-Annan	\$2,184,211
Tsvi Reiter	\$6,976,128
Total	\$77,548,570

The table visual displays sales made by each salesperson. However, there's another relationship between salespeople and sales. Some salespeople belong to one, two, or

possibly more sales regions. In addition, sales regions can have multiple salespeople assigned to them.

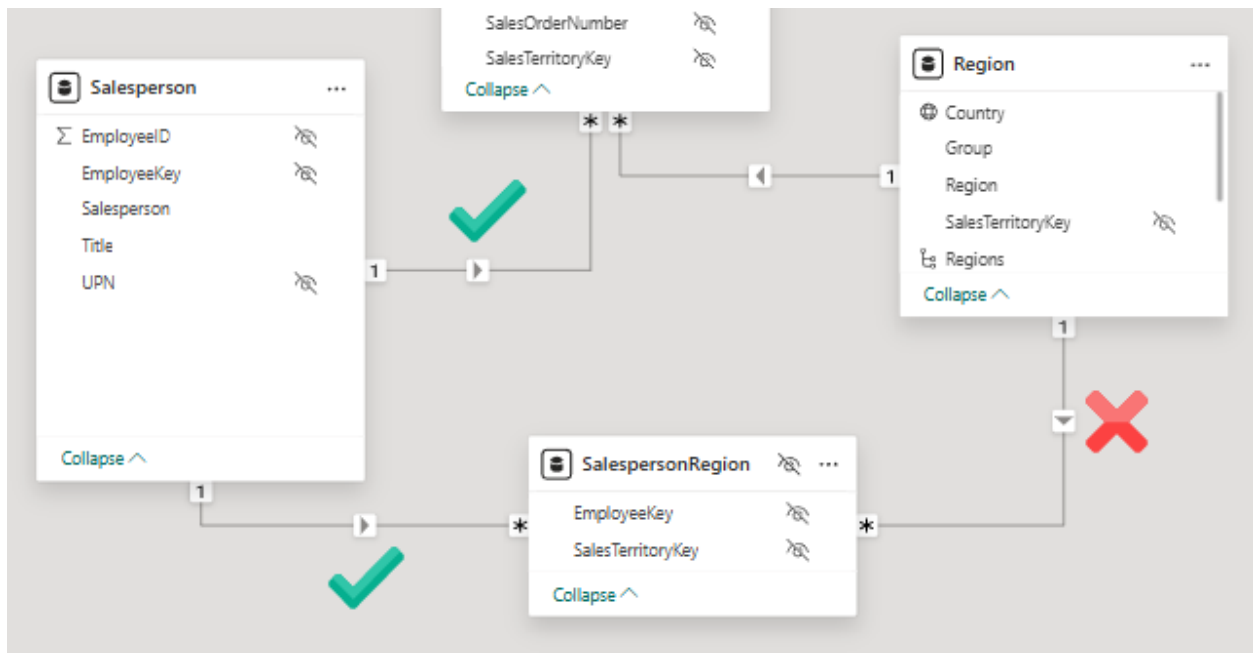
From a performance management perspective, a salesperson's sales (based on their assigned territories) need to be analyzed and compared with sales targets. You'll create relationships to support this analysis in the next exercise.

3. Notice that *Michael Blythe* has generated almost 9 million dollars of sales.
4. Switch to Model view, then drag the SalespersonRegion table to position it between the Region and Salesperson tables.
5. Use the drag-and-drop technique to create the following two model relationships:
 - Salesperson | EmployeeKey to SalespersonRegion | EmployeeKey
 - Region | SalesTerritoryKey to SalespersonRegion | SalesTerritoryKey

The SalespersonRegion table can be considered to be a bridging table.

6. Switch to Report view, and then notice that the visual hasn't updated—the sales result for Michael Blythe hasn't changed.
7. Switch back to Model view, and then follow the relationship filter directions (arrowhead) from the Salesperson table.

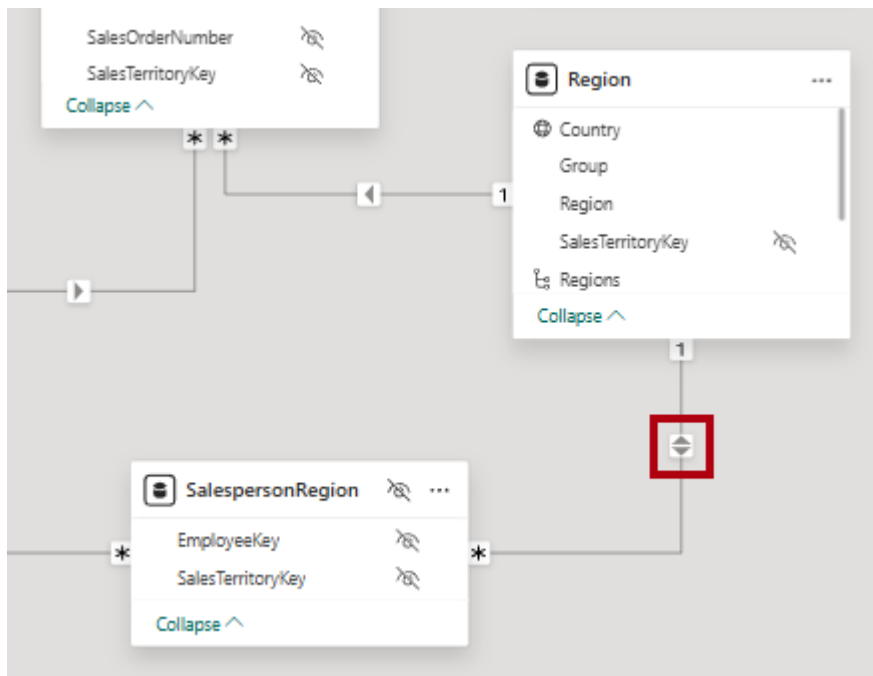
Consider that the Salesperson table filters the Sales table. It also filters the SalespersonRegion table, but it doesn't continue by propagating filters to the Region table (the arrowhead is pointing the wrong direction).



8. To edit the relationship between the Region and SalespersonRegion tables, double-click the relationship.
9. In the **Edit Relationship** window, in the **Cross Filter Direction** dropdown list, select *Both*.
10. Check the **Apply Security Filter in Both Directions** checkbox.

The screenshot shows the 'Edit Relationship' window. A red callout bubble with the number 1 points to the 'Cross-filter direction' dropdown menu, which is set to 'Both'. Another red callout bubble with the number 2 points to the 'Apply security filter in both directions' checkbox, which is checked.

11. Select **Save**.
12. Notice that the relationship has a double arrowhead now.



13. Switch to Report view, and then notice that the sales values have still not changed.

*The issue now relates to the fact that there are two possible filter propagation paths between the Salesperson and Sales tables. This ambiguity is internally resolved, based on a “least number of tables” assessment. To be clear, you shouldn’t design models with this type of ambiguity—the issue will be addressed in part later in this lab, and by the completion of the **Create DAX Calculations in Power BI Desktop** lab.*

14. Switch to Model view.

15. To force filter propagation via the bridging table, edit (double-click) the relationship between the Salesperson and Sales tables.

16. In the **Edit Relationship** window, uncheck the **Make This Relationship Active** checkbox.

Cardinality

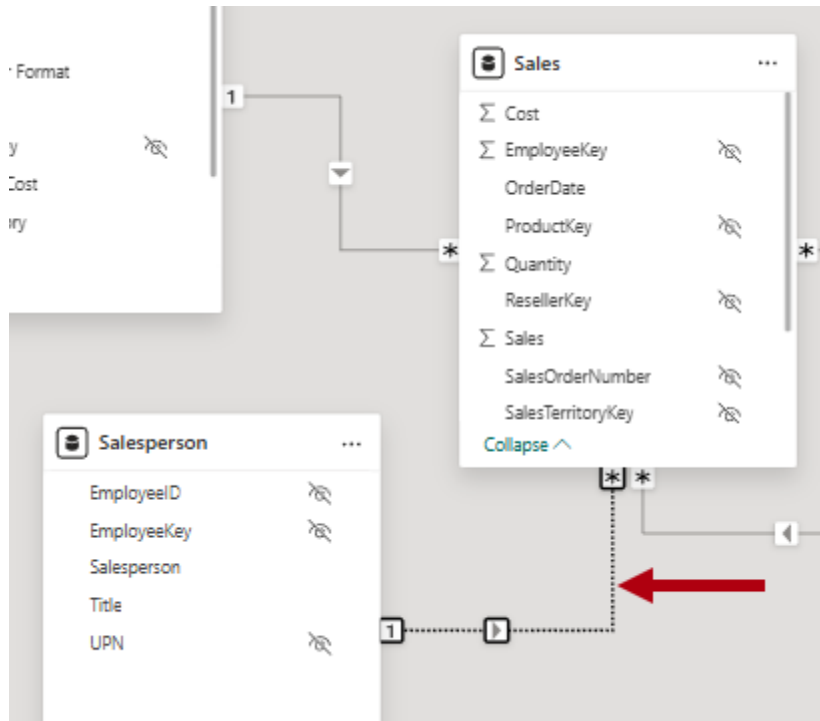
Many to one (*:1)

☐ Make this relationship active

17. Select **Save**.

Filter propagation will now follow the only active path.

18. In the model diagram, notice that the inactive relationship is represented by a dotted line.



19. Switch to Report view, and then notice that the sales for Michael Blythe are now nearly 22 million dollars.

20. Notice also, that the sales for each salesperson—if added—would exceed the table total.

It's a common observation of a many-to-many relationship due to the double, triple, etc. counting of regional sales results. Consider Brian Welcker, the second salesperson listed. His sales amount equals the total sales amount. It's the correct result due to the fact that he's the Director of Sales; his sales are measured by the sales of all regions.

*While the many-to-many relationship is now working, it's now not possible to analyze sales made by a salesperson (because the relationship is inactive). You'll be able to reactivate the relationship when you introduce a calculated table that will allow analyzing sales made in the sales region(s) assigned to the salesperson (for performance analysis) in the **Create DAX Calculations in Power BI Desktop** lab.*

21. Switch to Model view, and then in the model diagram, select the Salesperson table.

22. In the **Properties** pane, in the **Name** box, replace the text with *Salesperson (Performance)*.

The renamed table now reflects its purpose: it's used to report and analyze the performance of salespeople based on the sales of their assigned sales regions.

Relate the Targets table

In this task, you'll create a relationship to the Targets table.

1. Create a relationship from the Salesperson (Performance) | EmployeeID column and the Targets | EmployeeID column.
2. In Report view, add the Targets | Target field to the table visual.
3. Resize the table visual so all columns are visible.

Salesperson	Sum of Sales	Sum of Target
Amy Alberts	\$10,288,626	\$19,450,000
Brian Welcker	\$77,548,570	\$221,700,000
David Campbell	\$12,004,822	\$19,625,000
Garrett Vargas	\$13,875,633	\$23,675,000
Jae Pak	\$8,410,883	\$13,575,000
Jillian Carson	\$7,633,387	\$13,675,000
José Saraiva	\$13,875,633	\$18,875,000
Linda Mitchell	\$25,634,503	\$40,850,000
Lynn Tsofilas	\$1,391,025	\$3,210,000
Michael Blythe	\$21,987,348	\$31,150,000
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000
Rachel Valdez	\$1,877,743	\$4,125,000
Ranjit Varkey Chudukatil	\$4,527,840	\$9,050,000
Shu Ito	\$18,001,116	\$59,850,000
Stephen Jiang	\$65,868,919	\$110,150,000
Syed Abbas	\$1,391,025	\$3,050,000
Tete Mensa-Annan	\$12,004,822	\$17,100,000
Tsvi Reiter	\$7,638,607	\$13,250,000
Total	\$77,548,570	\$676,210,000

It's now possible to visualize sales and targets—but for now take care for two reasons. First, there's no filter on a time period, and so targets also include future target amounts. Second, targets aren't additive, and so the total shouldn't be displayed. They can either be disabled by formatting the visual or removed by using calculation logic.

1. Save the Power BI Desktop file.

Lab complete

Knowledge check

Summary

In this module, you learned how to develop a semantic layer over your data.

First, you learned about the importance of model relationships and their role to automatically propagate filters to other model tables.

Next, you learned how to create and configure model objects, including tables, columns, hierarchies, measures, and parameters. The end result is a model that delivers your reporting requirements, and that's user-friendly and intuitive.

Write DAX formulas for semantic models

Introduction

DAX (Data Analysis Expressions) lets you create powerful calculations in Power BI, helping you analyze and visualize your data in new ways.

For example, imagine you need to quickly find year-over-year sales growth or count unique customers in your reports. DAX makes these tasks simple and efficient, even when your data is complex or contains blanks.

In this module, you learn how to:

- Write DAX formulas for calculated tables, columns, and measures.
- Work with different data types and handle blank values.
- Use various DAX functions, including those similar to Excel.
- Apply operators and understand how they work together.
- Improve your formulas with variables for better performance and readability.

After finishing this module, you'll know how to build and optimize DAX calculations in Power BI, making your reports more dynamic and insightful.

Understand DAX calculation types

By using Data Analysis Expressions (DAX), you can add three types of calculations to your semantic model:

- [Calculated tables](#)

- Calculated columns
- Measures

Note

DAX can also be used to define row-level security (RLS) rules, which are expressions that enforce filters over model tables. However, rules aren't considered to be model calculations so they're out of scope for this module. For more information, see [Row-level security \(RLS\) with Power BI](#).

Calculated tables

You can write a DAX formula to add a calculated table to your model. The formula can duplicate or transform existing model data, or create a series of data, to produce a new table. Calculated table data is always imported into your model, so it increases the model storage size and can prolong data refresh time.

Note

A calculated table can't connect to external data; you need to use Power Query to accomplish that task.

Calculated tables can be useful in various scenarios:

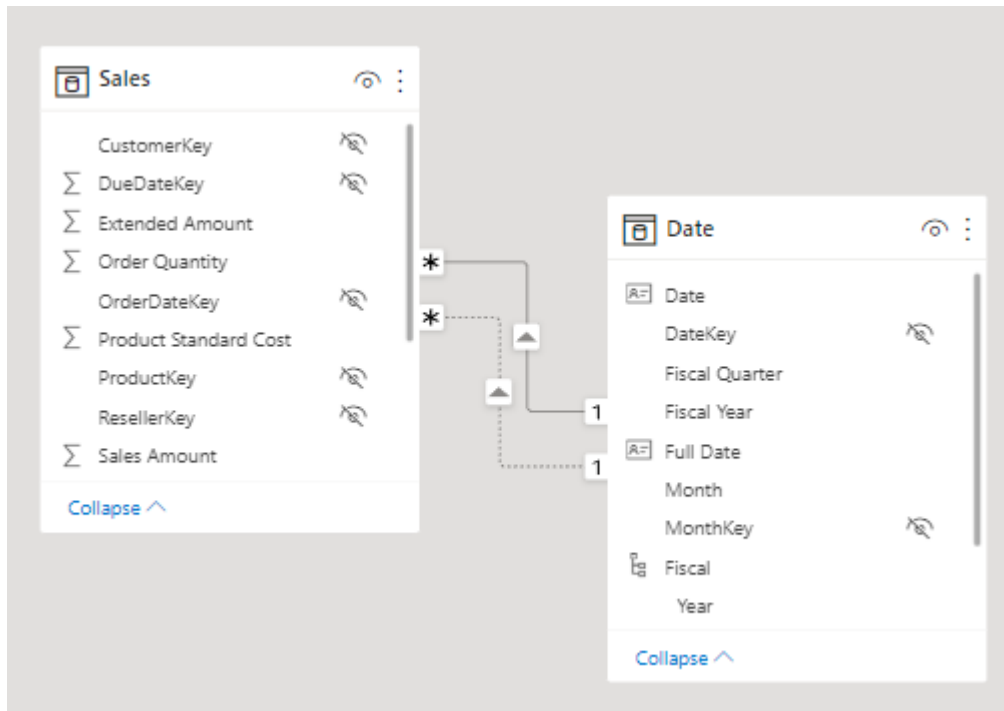
- Date tables
- Role-playing dimensions
- What-if analysis

Date tables

Date tables are required to apply special time filters known as *time intelligence*. DAX time intelligence functions only work correctly when a date table is set up. When your source data doesn't include a date table, you can create one as calculated tables by using the CALENDAR or CALENDARAUTO DAX functions.

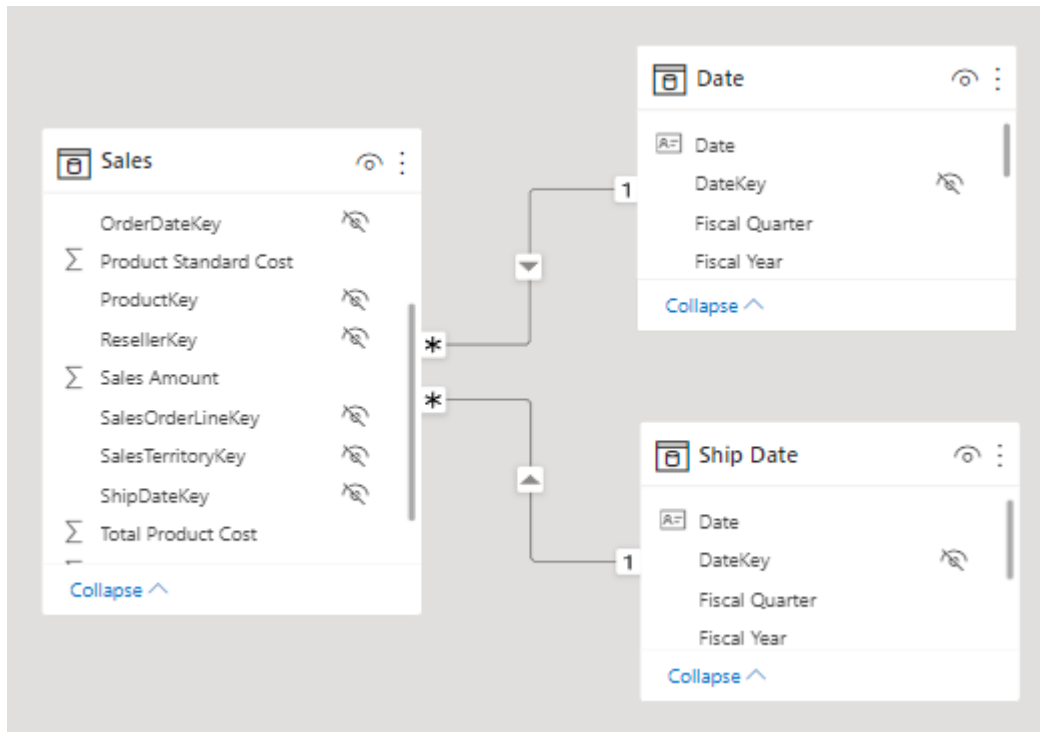
Role-playing dimensions

When two model tables have multiple relationships, it could be because your model has a role-playing dimension. For example, if you have a table named Sales that includes two date columns, OrderDateKey and ShipDateKey, both columns are related to the Date column in the Date table. In this case, the Date table is described as a role-playing dimension because it could play the role of order date or ship date.



Microsoft Power BI models only allow one active relationship between tables, which in the model diagram is indicated as a solid line. The active relationship is used by default to propagate filters, which in this case would be from the Date table to the OrderDateKey column in the Sales table. Any remaining relationships between the two tables are inactive. In a model diagram, the relationships are represented as dashed lines. Inactive relationships are only used when they're expressly requested in a calculated formula by using the USERELATIONSHIP DAX function.

Perhaps a better model design could have two date tables, each with an active relationship to the Sales table. Thus, report users can filter by order date or ship date, or both at the same time. A calculated table can duplicate the Date table data to create the Ship Date table.



What-if analysis

Power BI Desktop supports a feature called [What-if parameters](#). When you create a what-if parameter, a calculated table is automatically added to your model.

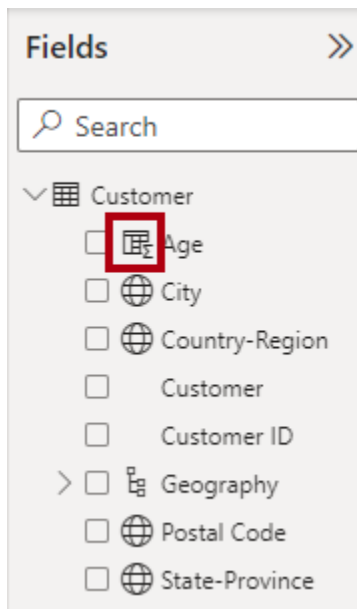
What-if parameters allow report users to select or filter by values that are stored in the calculated table. Measure formulas can use selected value(s) in a meaningful way. For example, a **what-if** parameter could allow the report user to select a hypothetical currency exchange rate, and a measure could divide revenue values (in a local currency) by the selected rate.

Notably, what-if calculated tables aren't related to other model tables because they're not used to propagate filters. For this reason, they're sometimes called *disconnected tables*.

Calculated columns

You can write a DAX formula to add a calculated column to any table in your model. The formula is evaluated for each table row and it returns a single value. When added to an Import storage mode table, the formula is evaluated when the semantic model is refreshed, and it increases the storage size of your model. When added to a DirectQuery storage mode table, the formula is evaluated by the underlying source database when the table is queried.

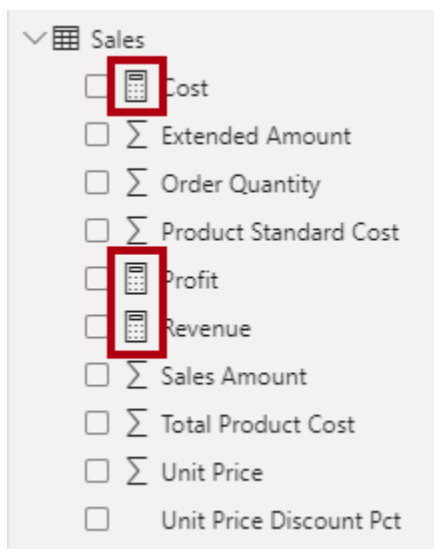
In the **Fields** pane, calculated columns are enhanced with a special icon. The following example shows a single calculated column in the **Customer** table called **Age**.



Measures

You can write a DAX formula to add a measure to any table in your model. The formula is concerned with achieving summarization over model data. Similar to a calculated column, the formula must return a single value. Unlike calculated columns, which are evaluated at data refresh time, measures are evaluated at query time. Their results are never stored in the model.

In the **Fields** pane, measures are shown with the calculator icon. The following example shows three measures in the **Sales** table: **Cost**, **Profit**, and **Revenue**.



Occasionally, measures can be described as **explicit measures**. To be clear, explicit measures are model calculations that are written in DAX and are commonly referred to as

simply *measures*. Yet, the concept of *implicit measures* exists, too. Implicit measures are columns that can be summarized by visuals in simplistic ways, like `count`, `sum`, `minimum`, `maximum`, and so on. You can identify implicit measures in the **Fields** pane because they're shown with the sigma symbol (Σ).

Note

Any column can be summarized when added to a visual. Therefore, whether they're shown with the sigma symbol or not, when they're added to a visual, they can be set up as implicit measures.

Additionally, no such concept as a *calculated measure* exists in tabular modeling. The word *calculated* is used to describe calculated tables and calculated columns, which distinguishes them from tables and columns that originate from Power Query. Power Query doesn't have the concept of an explicit measure.

Write DAX formulas

Each model calculation type, calculated table, calculated column, or measure is defined by its name, followed by the equals symbol (=), which is then followed by a DAX formula. Use the following template to create a model calculation:

DAX

```
<Calculation name> = <DAX formula>
```

For example, the definition of the **Ship Date** calculated table that duplicates the **Date** table data is:

DAX

```
Ship Date = 'Date'
```

A DAX formula consists of expressions that return a result. The result is either a table object or a scalar value. Calculated table formulas must return a table object; calculated column and measure formulas must return a scalar value (single value).

Formulas are assembled by using:

- DAX functions
- DAX operators
- References to model objects
- Constant values, like the number 24 or the literal text "FY" (abbreviation for fiscal year)

- DAX variables
- Whitespace

Tip

When entering DAX formulas in Power BI Desktop, you have the benefit of **IntelliSense**. IntelliSense is a code-completion aid that lists functions and model resources. When you select a DAX function, it also provides you with a definition and description. We recommend that you use IntelliSense to help you quickly build accurate formulas.

DAX functions

Similar to Microsoft Excel, DAX is a functional language meaning that formulas rely on functions to accomplish specific goals. Typically, DAX functions have arguments that allow passing in variables. Formulas can use many function calls and will often nest functions within other functions.

In a formula, function names must be followed by parentheses. Within the parentheses, variables are passed in.

Note

Some functions don't take arguments, or arguments might be optional.

Working with DAX functions is described later in this module.

DAX operators

Formulas also rely on operators, which can perform arithmetic calculations, compare values, work with strings, or test conditions.

DAX operators are described in more detail later in this module.

References to model objects

Formulas can only refer to three types of model objects: tables, columns, or measures. A formula can't refer to a hierarchy or a hierarchy level. (Recall that a hierarchy level is based on a column, so your formula can refer to a hierarchy level's column.)

Table references

When you reference a table in a formula, officially, the table name is enclosed within single quotation marks. In the following calculated table definition, notice that the **Date** table is enclosed within single quotation marks.

DAX

```
Ship Date = 'Date'
```

However, single quotation marks can be omitted when both of the following conditions are true:

1. The table name does not include embedded spaces.
2. The table name isn't a reserved word that's used by DAX. All DAX function names and operators are reserved words. *Date* is a DAX function name, which explains why, when you are referencing a table named **Date**, that you must enclose it within single quotation marks.

In the following calculated table definition, it's possible to omit the single quotation marks when referencing the **Airport** table:

DAX

```
Arrival Airport = Airport
```

Column references

When you reference a column in a formula, the column name must be enclosed within square brackets. Optionally, it can be preceded by its table name. For example, the following measure definition refers to the **Sales Amount** column.

DAX

```
Revenue = SUM([Sales Amount])
```

Because column names are unique within a table but not necessarily unique within the model, you can disambiguate the column reference by preceding it with its table name. This disambiguated column is known as a *fully qualified column*. Some DAX functions require passing in fully qualified columns.

Tip

To improve the readability of your formulas, we recommend that you always precede a column reference with its table name.

The previous example measure definition can be rewritten as:

DAX

```
Revenue = SUM(Sales[Sales Amount])
```

Measure references

When you reference a measure in a formula, like column name references, the measure name **must be enclosed within square brackets**. For example, the following measure definition **refers to the Revenue and Cost** measures.

DAX

```
Profit = [Revenue] - [Cost]
```

If you're a DAX beginner, the fact that **column and measure references are always enclosed within square brackets can cause confusion** when you're trying to read a formula. However, as you **become proficient with DAX fundamentals, you'll be able to determine which type** of object it is because, in DAX formulas, columns, and measures are used in different ways.

Tip

It's possible to precede a measure reference with its table name. However, **measures are a model-level object**. While **they're assigned to a home table, it's only a cosmetic relationship to logically organize measures in the Fields pane**.

Therefore, while we **recommend that you always precede a column reference with its table name, the inverse is true for measures**: We recommend that you never precede a measure reference with its table name.

For more information, see [Column and measure references](#).

DAX variables

Formulas **can declare DAX variables to store results**.

How and when to use DAX variables is described later in this module.

Whitespace

Whitespace refers to **characters that you can use to format your formulas** in a way that's **quick and simple to understand**. Whitespace characters include:

- Spaces
- Tabs
- Carriage returns

Whitespace is **optional and it doesn't modify your formula logic or negatively impact performance**. We strongly recommend that you adopt a format style and apply it consistently, and consider the following recommendations:

- Use **spaces between operators**.
- Use **tabs to indent nested function** calls.
- Use **carriage returns to separate function arguments**, especially when it's too long to fit on a single line. Formatting in this way makes it simpler to troubleshoot, especially when the formula is missing a parenthesis.
- **Err on the side of too much whitespace** than too little.

Tip

In the formula bar, **to enter a carriage return, press Shift+Enter**. Pressing **Enter** alone will commit your formula.

Consider the following measure definition that's **written in a single line** and that includes five DAX function calls:

DAX

```
Revenue YoY % = DIVIDE([Revenue] - CALCULATE([Revenue],
SAMEPERIODLASTYEAR('Date'[Date])),
CALCULATE([Revenue],
SAMEPERIODLASTYEAR('Date'[Date])))
```

The following example is the **same measure definition but now formatted**, which helps make it easier to read and understand:

DAX

```
Revenue YoY % =
DIVIDE(
    [Revenue]
    - CALCULATE(
        [Revenue],
        SAMEPERIODLASTYEAR('Date'[Date])
    ),
    CALCULATE(
        [Revenue],
        SAMEPERIODLASTYEAR('Date'[Date])
    )
)
```

Try formatting the measure on your own. Open the **Adventure Works DW 2020 M02.pbix** Power BI Desktop file and then, in the **Fields** pane, expand the **Sales** table and then select the **Revenue YoY %** measure. In the formula bar, use tab and carriage return characters to produce the same result as the previous example. When you add a carriage return, remember to press **Shift+Enter**.

This measure definition can be further improved for readability and performance, which will be explained later in this module.

Tip

An excellent formatting tool from another source that can help you format your calculations is **DAX Formatter**. This tool allows you to paste in your calculation and format it. You can then copy the formatted calculation to the clipboard and paste it back into Power BI Desktop.

DAX data types

Semantic model columns have a set data type, which ensures that all column values conform to that data type. Column data types are defined in Power Query, or in the case of calculated columns, it's inferred from the formula. Measure data types, similar to calculated column data types, are inferred from the formula.

Model data types aren't the same as DAX data types, though a direct relationship exists between them. The following table lists the model data types and DAX data types. Notice the supported range of values for each data type.

Model data type	DAX data type	Description
Whole number	64-bit integer	-2^{63} through $2^{63}-1$
Decimal number	64-bit real	Negative: -1.79×10^{308} through -2.23×10^{-308} - zero (0) - positive: 2.23×10^{-308} through 1.79×10^{308} - Limited to 17 decimal digits
Boolean	Boolean	TRUE or FALSE
Text	String	Unicode character string
Date	Date/time	Valid dates are all dates after January 1, 1900

Currency	Currency	-9.22 x 10 ¹⁴ through 9.22 x 10 ¹⁴ - limited to four decimal digits of fixed precision
N/A	BLANK	In some cases, it's the equivalent of a database (SQL) NULL

BLANK data type

The BLANK data type [deserves a special mention](#). DAX uses BLANK for both database NULL and for blank cells in Excel. BLANK doesn't mean zero. Perhaps it might be simpler to think of it as the *absence of a value*.

Two DAX functions are related to the BLANK data type: the **BLANK** DAX function returns BLANK, while the **ISBLANK** DAX function tests whether an expression evaluates to BLANK.

Work with DAX functions

The DAX function library [consists of hundreds of functions](#), each designed to accomplish a specific goal.

Because DAX originated with the Power Pivot add-in for Microsoft Excel 2010, [over 80 functions](#) are available that [can also be found in Excel](#). It was a deliberate design strategy by Microsoft to ensure that [Excel users can quickly become productive with DAX](#).

However, [many functions exist that you won't find in Excel because they're specific to data modeling](#):

- Relationship navigation functions
- Filter context modification functions
- Iterator functions
- Time intelligence functions
- Path functions

Tip

[To search for documentation](#) that is related to a DAX function, in a web search, [enter the keyword DAX followed by the function name](#).

For more information, see the [DAX function reference](#).

Functions that originate from Excel

The following sections consider several useful functions that [you might already be familiar with because they exist in Excel](#).

The **IF** DAX function tests whether a condition that's provided as the first argument is met. It returns one value if the condition is TRUE and returns the other value if the condition is FALSE. The function's syntax is:

DAX

```
IF(<logical_test>, <value_if_true>[, <value_if_false>])
```

Tip

A function argument is optional when documentation shows it enclosed within square brackets.

If **logical_test** evaluates to FALSE and **value_if_false** isn't provided, the function will return **BLANK**.

Many Excel summarization functions are available, including SUM, COUNT, AVERAGE, MIN, MAX, and many others. The only difference is that in DAX, you pass in a column reference, whereas in Excel, you pass in a range of cells.

Many Excel mathematic, text, date and time, information, and logical functions are available as well. For example, a small sample of Excel functions that are available in DAX include ABS, ROUND, SQRT, LEN, LEFT, RIGHT, UPPER, DATE, YEAR, MONTH, NOW, ISNUMBER, TRUE, FALSE, AND, OR, NOT, and IFERROR.

Functions that don't originate from Excel

Two useful DAX functions that aren't specific to modeling and that don't originate from Excel are DISTINCTCOUNT and DIVIDE.

DISTINCTCOUNT function

You can use the **DISTINCTCOUNT** DAX function to count the number of distinct values in a column. This function is especially powerful in an analytics solution. Consider that the count of customers is different from the count of *distinct* customers. The latter doesn't count repeat customers, so the difference is "How many customers" compared with "How many *different* customers."

DIVIDE function

You can use the **DIVIDE** DAX function to achieve division. You must pass in numerator and denominator expressions. Optionally, you can pass in a value that represents an alternate result. The DIVIDE function's syntax is:

DAX

```
DIVIDE(<numerator>, <denominator>[, <alternate_result>])
```

The DIVIDE function automatically handles division by zero cases. If an alternate result isn't passed in, and the denominator is zero or BLANK, the function returns BLANK. When an alternate result is passed in, it's returned instead of BLANK.

This function is convenient because it saves your expression from having to first test the denominator value. The function is also better optimized for testing the denominator value than the IF function. The performance gain is significant because checking for division by zero is expensive. What's more, using the DIVIDE function results in a more concise and elegant expression.

Tip

We recommend that you use the DIVIDE function whenever the denominator is an expression that could return zero or BLANK. In the case that the denominator is a constant value, we recommend that you use the divide operator (/), which is introduced later in this module. In this case, the division is guaranteed to succeed, and your expression will perform better because it will avoid unnecessary testing.

Use DAX operators

Your DAX formulas can use operators to create expressions that perform arithmetic calculations, compare values, work with strings, or test conditions.

Tip

Many DAX operators and precedence order are the same as those found in Excel.

Arithmetic operators

The following table lists the arithmetic operators.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Exponentiation

Remember, when you are dividing two expressions, and when the denominator could return zero or BLANK, it's more efficient and safer to use the DIVIDE DAX function.

Comparison operators

The following table lists the **comparison operators**, which are used to compare two values. The result is either TRUE or FALSE.

Operator	Description
=	Equal to
==	Strict equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<>	Not equal to

All comparison operators, except **strict equal to (==)**, treat BLANK as equal to the number zero, an empty string (""), the date December 30, 1899, or FALSE. It means that the expression `[Revenue] = 0` will be TRUE when the value of [Revenue] is either zero or BLANK. In contrast, `[Revenue] == 0` is TRUE only when the value of [Revenue] is zero.

Text concatenation operator

Use the **ampersand (&) character to connect, or concatenate, two text values** to produce one continuous text value. For example, consider the following calculated column definition:

DAX

```
Model Color = 'Product'[Model] & "-" & 'Product'[Color]
```

Logical operators

Use logical operators **to combine expressions that produce a single result**. The following

Operator	Description
&&	Creates an AND condition between two expressions where each has a Boolean result. If both expressions return TRUE, the combination of the expressions also returns TRUE; otherwise the combination returns FALSE.
(double pipe)	Creates an OR condition between two logical expressions. If either expression returns TRUE, the result is TRUE; only when both expressions are FALSE is the result FALSE.

IN	Creates a logical OR condition between each row that is being compared to a table. Note: The table constructor syntax uses braces.
NOT	Inverts the state of a Boolean expression (FALSE to TRUE, and vice versa).

An example that uses the **IN** logical operator is the **ANZ Revenue** measure definition, which uses the **CALCULATE** DAX function to enforce a specific filter of two countries/regions: Australia and New Zealand.

Note

You'll be introduced to the powerful CALCULATE function when you learn how to modify the filter context.

DAX

```
ANZ Revenue =
CALCULATE(
    [Revenue],
    Customer[Country-Region] IN {
        "Australia",
        "New Zealand"
    }
)
```

Operator precedence

When your DAX formula includes multiple operators, DAX uses rules to determine the evaluation order, which is known as an operator precedence. Operations are ordered according to the following table.

Operator	Description
^	Exponentiation
-	Sign (as in -1)
* and /	Multiplication and division
NOT	NOT
+ and -	Addition and subtraction

&	Concatenation of two strings of text
=,==,<,>,<=,>=,<>	Comparison

When the operators have equal precedence value, they're ordered from left to right.

In general, operator precedence is the same as what's found in Excel. If you need to override the evaluation order, then group operations within parentheses.

For example, consider the following calculated column definition:

DAX

```
Extended Amount = Sales[Order Quantity] * Sales[Unit Price] * 1 - [Unit Price Discount Pct]
```

This sample calculated column definition produces an incorrect result because multiplication happens before the subtraction. The following correct calculated column definition uses parentheses to ensure that the subtractions happen before the multiplications.

DAX

```
Extended Amount = Sales[Order Quantity] * Sales[Unit Price] * (1 - [Unit Price Discount Pct])
```

Tip

Remembering operator precedence rules can be challenging, especially for DAX beginners. Consequently, we recommend that you test your formulas thoroughly. When the formulas don't produce the correct result due to an incorrect evaluation order, you can experiment by adding parentheses to adjust the evaluation order. You can also add parentheses to improve the readability of your formulas.

For more information about DAX operators and precedence order, see [DAX operators](#).

Implicit conversion

When writing a DAX formula that uses operators to combine different data types, you don't need to explicitly convert types. Usually, DAX automatically identifies the data types of referenced model objects and performs implicit conversions where necessary to complete the specified operation.

However, some limitations might exist on the values that can be successfully converted. If a value or a column has a data type that's incompatible with the current operation, DAX returns an error. For example, the attempt to multiply a date value will create an error because it isn't logical.

BLANK is handled differently, depending on the operator that is used. It's handled similar to how Excel treats BLANK, but differently to how databases (SQL) treat NULL. BLANK is treated as zero when acted on by arithmetic operators and as an empty string when concatenated to a string.

Tip

Remembering how BLANK is handled can be challenging, especially for DAX beginners. Consequently, we recommend that you test your formulas thoroughly. When BLANKs create unexpected results, consider using the IF and ISBLANK DAX functions to test for BLANK, and then respond in an appropriate way.

Use DAX variables

You can declare DAX variables in your formula expressions. When you declare at least one variable, a RETURN clause is used to define the expression, which then refers to the variables.

We recommend that you use variables because they offer several benefits:

- Improving the readability and maintenance of your formulas.
- Improving performance because variables are evaluated once and only when or if they're needed.
- Allowing (at design time) straightforward testing of a complex formula by returning the variable of interest.

The following example shows a formula that declares a variable. The Revenue YoY % measure definition is rewritten to declare a variable that's assigned the value of the prior year's revenue.

DAX

```
Revenue YoY % =  
VAR RevenuePriorYear =  
    CALCULATE(  
        [Revenue],  
        SAMEPERIODLASTYEAR('Date'[Date])  
    )  
RETURN  
    DIVIDE(
```

```
[Revenue] - RevenuePriorYear,  
RevenuePriorYear  
)
```

Notice that the **RETURN** clause refers to the variable twice. This improved measure definition formula will **run in at least half the time** because it **doesn't need to evaluate** the prior year's revenue **twice**.

In the [Adventure Works DW 2020 M02.pbix](#) Power BI Desktop file, refactor the **Revenue YoY %** measure to [produce the same result as the previous example](#).

For more information on using DAX variables, see [Use variables to improve your formulas](#).

Knowledge check

Summary

In this module, you learned how to use DAX to add calculations to your model. You explored three types of calculations:

- Calculated tables
- Calculated columns
- Measures

You also learned the basics of DAX, including:

- Writing formulas
- Using different data types
- Applying functions, operators, and variables
- Referencing model objects and constants

Create DAX calculations in semantic models

Introduction

When you first create a semantic model by applying Power Query queries, the model consists of tables and columns. At this point, the model probably isn't ready for use. That's because you might need to create or adjust model relationships, create other tables or columns, add hierarchies, or set model properties. You might also identify the need for calculations to summarize the model data, especially when the requirement can't be

achieved by summarizing a single column. For example, you might want to calculate year-to-date (YTD) sales revenue, which requires special time filters.

You can use Data Analysis Expressions (DAX) to add three types of calculations to your semantic model to complete your model design:

- **Calculated tables**, which can be useful to create date tables or role-playing dimensions, or to enable what-if analysis.
- **Calculated columns**, which can be added to tables in your model.
- **Measures**, which achieve summarization over model data.

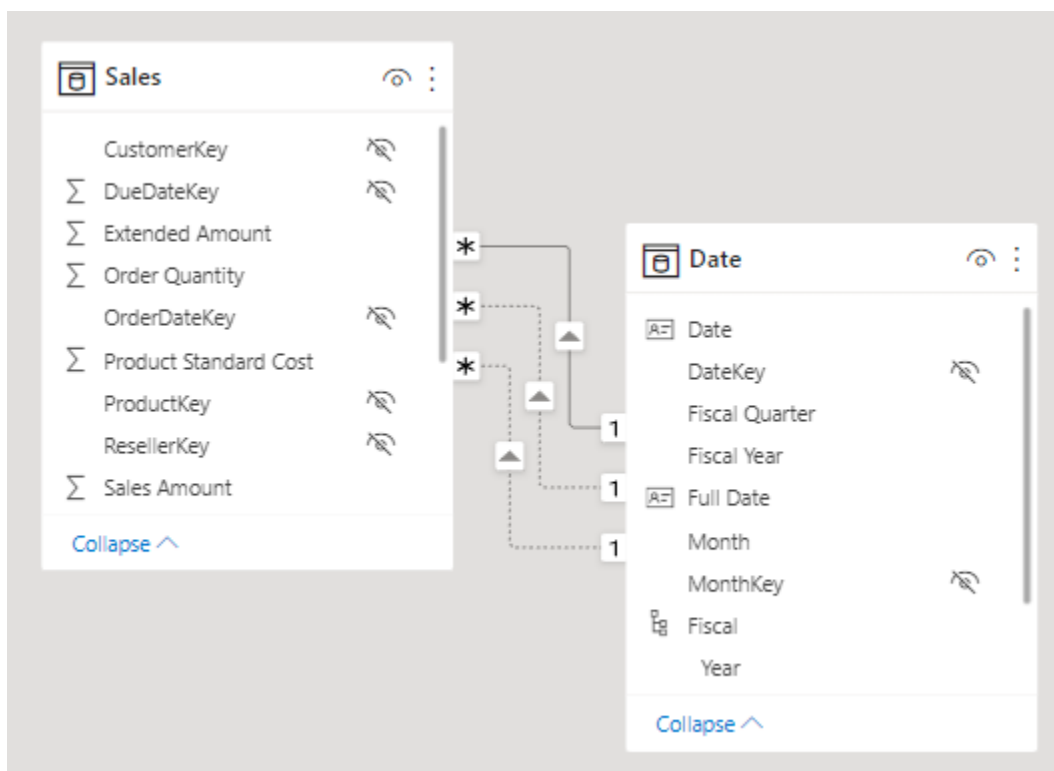
In this module, you learn why and how to create each of these calculation types.

Create calculated tables

A calculated table is created with a DAX formula that returns a table object, allowing you to duplicate or transform existing model data to produce a new table.

Duplicate a table

A common design challenge in data modeling is handling multiple relationships between tables. For example, the Sales table might have three relationships to the Date table, as shown in the following diagram:



The Sales table stores sales data by order date, ship date, and due date, resulting in three relationships to the Date table. However, only one relationship can be active at a time, as indicated by a solid line in the diagram. The other relationships are inactive and shown as dashed lines.

In our example, the active relationship filters the OrderDateKey column in the Sales table, so filters applied to the Date table affect sales by order date only.

To enable filtering sales by ship date, a new table can be created by duplicating the Date table with the following formula:

DAX

```
Ship Date = 'Date'
```

This formula creates a new table named Ship Date with the same columns and rows as the original Date table. When the Date table is refreshed, the Ship Date table is also recalculated to remain synchronized. Now you can create an active relationship between Ship Date and Sales tables to allow filtering by ship dates too.

Configure duplicate tables

When you create a calculated table, you need to apply any custom configurations to the new duplicate table you want carried over. For instance, it's a good idea to rename columns so that they better describe their purpose.

In our example, the Fiscal Year column in the Ship Date table can be renamed as Ship Fiscal Year. The Ship Full Date column should be sorted by the Ship Date column and the MonthKey column can be hidden to improve sorting and reporting.

Calculated tables are useful to work in scenarios when multiple relationships between two tables exist, as previously described. However, calculated tables increase the model's storage size and can prolong data refresh times, especially when they depend on other tables.

Note

While duplicate tables solve this challenge, there are other more performant solutions. We cover this concept again when discussing measures later in this module.

Create a date table

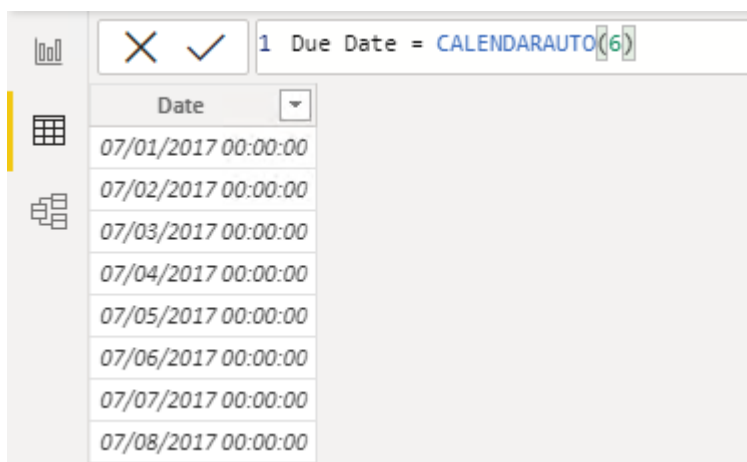
A good use case for calculated tables is to add a date table to your model. Date tables are required to apply special time filters known as *time intelligence*.

You can create a calculated date table using the [CALENDAR](#) function. The CALENDAR function scans all date and date/time columns in the model to determine the earliest and latest dates, then generates a complete set of dates that span all years in the data. The argument specifies the last month of the financial year; for example, passing 6 sets June as the year end.

DAX

```
Due Date = CALENDAR(AUTO(6))
```

The resulting Due Date table contains a single column of dates. The following image shows the Due Date table in data view, with dates sorted from earliest to latest:



Date
07/01/2017 00:00:00
07/02/2017 00:00:00
07/03/2017 00:00:00
07/04/2017 00:00:00
07/05/2017 00:00:00
07/06/2017 00:00:00
07/07/2017 00:00:00
07/08/2017 00:00:00

Tip

The [CALENDAR](#) function can also be used to create a date table by specifying a start and end date, either as static values or as expressions based on model data.

If the earliest date in the model is October 15, 2021, and the latest is June 15, 2022, the function returns dates from July 1, 2021, to June 30, 2022. This ensures the table includes complete years, which is required for marking a date table.

Mark as a date table

Once you create a date table, you need to *Mark it as a date table* in Power BI Desktop. This setting allows you to use time intelligence functions in DAX calculations. When you specify your own date table, Power BI Desktop performs the following validations of that column and its data, to ensure that the data contains:

- Unique values.
- No null values.
- Contiguous date values (from beginning to end).

- Same timestamp across each value for Date/Time data types.

This setting applies to any date tables, either imported or created in Power Query or calculated tables.

Note

You must either use a custom date table or use the built-in auto/date time feature in Power BI to use time intelligence. The auto/date time feature has limited values and can't be customized, which is one reason to consider using a custom date table.

Create calculated columns

Occasionally, you need more columns than exist in your data. Ideally, you add these columns directly to the data source, which supports easier maintenance and allows the column logic to be reused in other models or reports. However, if you need to add the column once connected to the data in Power BI, you can either create a custom column in Power Query Editor or add a calculated column to the semantic model. No matter how you add the column, the result is the same from a report user's perspective.

In general, custom columns in Power Query are preferred because they're loaded into the model in a more compact and optimal way. Calculated columns are recommended only when adding columns to a calculated table or when the formula:

- Depends on summarized model data.
- Requires specialized modeling functions available only in DAX, such as RELATED, RELATEDTABLE, or functions for parent-child hierarchies.

Create calculated columns

A DAX formula can be used to add a *calculated column* to any table in the model. The formula must return a single value (scalar) for each row.

Calculated columns in import models increase storage size and can prolong data refresh times, especially when they depend on other tables that are refreshed. Therefore, be cautious when using too many calculated columns and consider if a measure can be used instead.

In the following examples, several calculated columns are created to support fiscal analysis to demonstrate the versatility of calculated columns.

Fiscal Year

This column determines the fiscal year for each date. The fiscal year starts in July, so dates from July to December are assigned to the next calendar year. The formula concatenates "FY" with the year, incrementing the year by one for dates in the second half of the year.

DAX

```
Due Fiscal Year =  
"FY"  
& YEAR('Due Date'[Due Date])  
+ IF(  
    MONTH('Due Date'[Due Date]) > 6,  
    1  
)
```

The following steps describe how Microsoft Power BI evaluates the calculated column formula:

1. The addition operator (+) is evaluated before the text concatenation operator (&).
2. The [YEAR](#) function returns the whole number value of the due date year.
3. The [IF](#) function returns the value when the due date month number is 7-12 (July to December); otherwise, it returns BLANK. (For example, because the Adventure Works financial year is July-June, the last six months of the calendar year will use the next calendar year as their financial year.)
4. The year value is added to the value that is returned by the IF function, which is the value one or BLANK. If the value is BLANK, it's implicitly converted to zero (0) to allow the addition to produce the fiscal year value.
5. The literal text value "FY" concatenated with the fiscal year value, which is implicitly converted to text.

Fiscal Quarter

This column assigns a fiscal quarter to each date, based on the fiscal year structure where Quarter 1 is July–September. The formula appends Q and the quarter number to the fiscal year label.

DAX

```
Due Fiscal Quarter =
```

```

'Due Date'[Due Fiscal Year] & " Q"
    & IF(
        MONTH('Due Date'[Due Date]) <= 3,
        3,
        IF(
            MONTH('Due Date'[Due Date]) <= 6,
            4,
            IF(
                MONTH('Due Date'[Due Date]) <= 9,
                1,
                2
            )
        )
    )
)

```

Month Key

The MonthKey formula multiplies the due date year by the value 100 and then adds the month number of the due date. It produces a numeric value that can be used to sort the Due Month text values in chronological order.

DAX

```

MonthKey =
(YEAR('Due Date'[Due Date]) * 100) + MONTH('Due Date'[Due Date])

```

Full Date Label

The [FORMAT](#) function converts the Due Date column value to text by using a format string. In this case, the format string produces a label that describes the year, abbreviated month name, and day:

DAX

```

Due Full Date =
FORMAT('Due Date'[Due Date], "yyyy mmm, dd")

```

Note

Many user-defined date/time formats exist. For more information, see [Custom date and time formats for the FORMAT function](#).

After these additions, the Due Date table contains six columns: the original date column and five calculated columns. These columns support both time intelligence functions and readability for report consumers.

 		1 MonthKey = 2 (YEAR('Due Date'[Due Date]) * 100) + MONTH('Due Date'[Due Date])				
	Due Date	Due Fiscal Year	Due Fiscal Quarter	Due Month	Due Full Date	MonthKey
	07/01/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 01	201707
	07/02/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 02	201707
	07/03/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 03	201707
	07/04/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 04	201707
	07/05/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 05	201707
	07/06/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 06	201707
	07/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 07	201707

Understand row context

Power BI evaluates a calculated column's formula for each row in the table, which is called row context. Row context just means "the current row." For example, here's the Due Fiscal Year calculated column:

DAX

```
Due Fiscal Year =  
"FY"  
& YEAR('Due Date'[Due Date])  
+ IF(  
    MONTH('Due Date'[Due Date]) <= 6,  
    1  
)
```

When Power BI runs this formula, 'Due Date'[Due Date] gives the value from the current row. If you've used Excel tables, this idea might feel familiar.

Row context only applies to the current table. If you need values from another table, you have two main options:

- If a relationship exists between the two tables, use the [RELATED](#) or [RELATEDTABLE](#) function. RELATED gets a value from the one-side of a relationship. RELATEDTABLE gets a table of values from the many-side.
- If there's no relationship, use the [LOOKUPVALUE](#) function.

Try to use RELATED when you can. It usually works faster than LOOKUPVALUE because of how Power BI stores and indexes data.

Here's an example. This formula adds a Discount Amount column to the Sales table:

DAX

```
Discount Amount =
(
    Sales[Order Quantity]
    * RELATED('Product'[List Price])
) - Sales[Sales Amount]
```

Power BI calculates this formula for each row in the Sales table. It gets Order Quantity and Sales Amount from the current row. To get List Price from the Product table, it uses the RELATED function to find the right value for each sale.

Row context always applies when Power BI evaluates calculated column formulas. It also comes into play with iterator functions, which let you create more advanced summaries. You learn about iterator functions later in this module.

Understand implicit measures

Measures in Power BI models are either *implicit* or *explicit*. Implicit measures are automatic behaviors that let visuals summarize column data. Explicit measures, often called *measures*, are calculations you add to your model. This section explains how implicit measures work and how you can use them.

Identify implicit measures

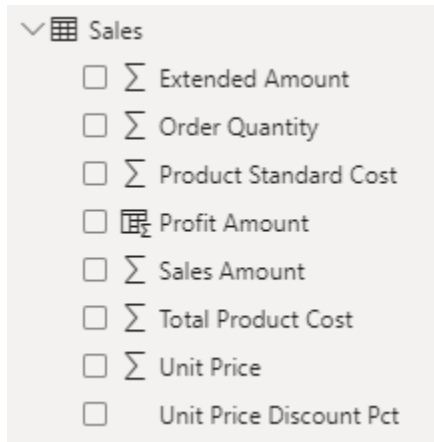
As a data modeler, you control how a column summarizes by setting the **Summarization** property. You can choose **Don't summarize** or select a specific aggregation function. If you set a column to **Don't summarize**, the sigma symbol disappears in the **Data** pane.

In the **Data** pane, a column with the sigma symbol (Σ) shows two facts:

- The column is numeric.

- Values are summarized when used in a visual that supports summarization.

The following image shows the Sales table with implicit measures, a calculated column, and one column that can't be summarized.



Notice in the example with the Sales table, if you add the Sales Amount field from the Sales table to a matrix visual that groups by fiscal year and month, Power BI summarizes the values implicitly. The **Sum** aggregation function is selected by default.

Fiscal Year	Sales Amount
FY2018	\$23,860,891.17
2017 Jul	\$1,423,357.32
2017 Aug	\$2,057,902.45
2017 Sep	\$2,523,947.55
2017 Oct	\$561,681.48
2017 Nov	\$4,764,920.16
2017 Dec	\$596,746.56

If you add the Unit Price field to the matrix visual, Power BI uses **Average** as the default summarization, because summing unit prices doesn't make sense since they're rates, not totals.

Fiscal Year	Sales Amount	Unit Price
FY2018	\$23,860,891.17	\$1,273.63
2017 Jul	\$1,423,357.32	\$1,817.15
2017 Aug	\$2,057,902.45	\$1,181.42
2017 Sep	\$2,523,947.55	\$1,030.56
2017 Oct	\$561,681.48	\$3,228.05
2017 Nov	\$4,764,920.16	\$1,009.71
2017 Dec	\$596,746.56	\$3,174.18

The default summarization is now set to **Average** (the modeler knows that it's inappropriate to sum unit price values together because they're rates, which are non-additive).

Implicit measures allow the report author to start with a default summarization technique and lets them modify it to suit their visual requirements. Numeric columns support the widest range of aggregation functions to choose from, including:

- Sum
- Average
- Minimum
- Maximum
- Count (Distinct)
- Count
- Standard deviation
- Variance
- Median

Summarize non-numeric columns

Non-numeric columns like text, dates, and boolean (true/false) values can also be summarized in your visuals. While the sigma symbol (Σ) only appears next to numeric fields in the Data pane, these columns are still being aggregated.

- Text columns: First, Last, Count, Count (Distinct)
- Date columns: Earliest, Latest, Count, Count

This flexibility is useful when you want to answer questions such as:

- "How many unique products are there?" (Count distinct on a text column)
- "What was the earliest order date?" (Earliest on a date column)
- "How many orders were marked as complete?" (Count on a boolean column)

You can choose the aggregation option that best fits your analysis when you add a non-numeric field to your visual.

Considerations for implicit measures

Implicit measures are easy to use and flexible. They let report authors start with a default summarization and change it to fit their needs. Implicit measures let report authors quickly visualize data without needing to write calculations. As a data modeler, you spend less time creating explicit measures.

However, even if you set a default summarization, report authors can change it to something that might not make sense. For example, they could set Unit Price to **Sum**, which produces misleading results, as shown in the following image. The Unit Price values are large because they're the sum of unit prices instead of the static unit price per product.

Fiscal Year	Sales Amount	Sum of Unit Price
FY2018	\$23,860,891.17	\$13,905,519.77
2017 Jul	\$1,423,357.32	\$1,164,795.52
2017 Aug	\$2,057,902.45	\$1,115,258.56
2017 Sep	\$2,523,947.55	\$1,291,296.17
2017 Oct	\$561,681.48	\$561,681.48
2017 Nov	\$4,764,920.16	\$2,159,760.38
2017 Dec	\$596,746.56	\$596,746.56

The biggest limitation is that implicit measures only work for simple scenarios. They can summarize column values using a single aggregation function, but they can't handle more complex calculations. For example, if you need to calculate the ratio of each month's sales amount over the yearly sales amount, you must create an explicit measure with a DAX formula.

Create explicit measures

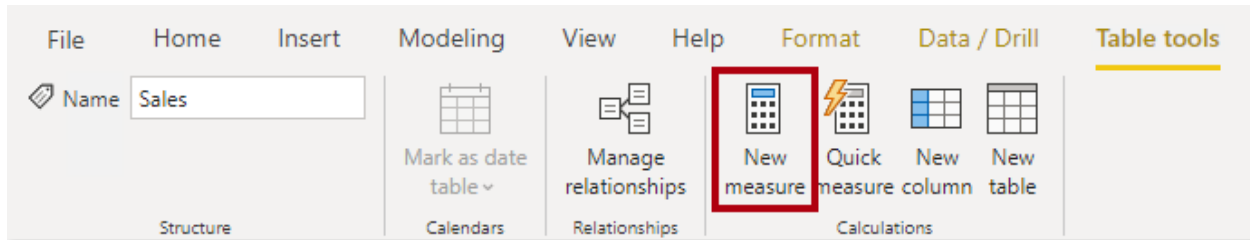
You can add a measure to any table in your model by writing a DAX formula. A measure formula must return a single value. This section explains how to create explicit measures.

Measures don't store values in the model. Instead, Power BI calculates them at query time to summarize model data. Measures can't reference a table or column directly, so you must use a function to summarize the data.

Simple measures

A *simple* measure aggregates the values of a single column, just like an implicit measure.

For example, you can add a measure to the Sales table. In the **Data** pane, select the Sales table. On the **Table Tools** ribbon, select **New measure**.



The following formula creates a measure called *Revenue*. This measure uses the [SUM](#) function to total the values in the Sales Amount column. If you add this measure to a table alongside the Sales Amount implicit measure, the results are the same.

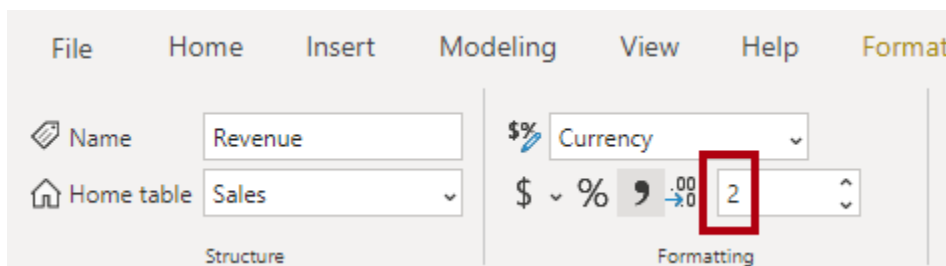
DAX

```
Revenue =
SUM(Sales[Sales Amount])
```

Tip

Adding measures and hiding columns helps report authors use the explicit measures instead.

In the **Measure tools** contextual ribbon, you can format the measure, set data type, and change the home table. The home table refers to where the measure shows when looking at the data pane.



Tip

It's a good practice to set the formatting options right after you create a measure. This ensures your values look consistent in all report visuals.

Consider you might need a measure to calculate Profit as shown in the following code:

DAX

```
Profit =
SUM(Sales[Profit Amount])
```

In this example, the Profit Amount column is a calculated column. This approach isn't optimal because you don't need that column. In the next section, you see how to create a measure that calculates profit directly, which reduces model size and improves refresh times.

The following code creates two different measures to return **Order Line Count** and **Order Count**. The [COUNT](#) function counts non-BLANK values in a column. The [DISTINCTCOUNT](#) function counts unique values. Since an order can have multiple order lines, the Sales Order column has duplicates. Using DISTINCTCOUNT gives you the correct order count.

DAX

```
Order Line Count =
COUNT(Sales[SalesOrderLineKey])

Order Count =
DISTINCTCOUNT('Sales Order'[Sales Order])
```

You can also write the Order Line Count measure using [COUNTROWS](#), which counts the number of rows in a table:

DAX

```
Order Line Count =
COUNTROWS(Sales)
```

All the measures referenced are considered simple measures because they aggregate a single column or single table.

Fiscal Year	Order Count	Order Line Count	Cost	Minimum Price	Average Price	Maximum Price	Profit	Quantity	Revenue
FY2018	3,198	10,918	\$20,824,958	\$4.75	\$1,274	\$3,578.27	\$3,035,934	24,349	\$23,860,891
FY2019	4,738	26,050	\$30,362,108	\$2.29	\$567	\$2,443.35	\$3,708,000	83,339	\$34,070,109
FY2020	23,519	84,285	\$46,070,842	\$1.33	\$329	\$2,443.35	\$5,807,433	167,088	\$51,878,275
Total	31,455	121,253	\$97,257,908	\$1.33	\$465	\$3,578.27	\$12,551,366	274,776	\$109,809,274

Create compound measures

A *compound measure* references one or more other measures. For example, you can redefine the Profit measure by referencing other measures. This measure can be used instead of the calculated column previously referenced.

DAX

```
Profit =  
[Revenue] - [Cost]
```

This change to the model presents an important lesson: By removing the calculated column, you optimize the semantic model because it results in a decreased semantic model size and shorter data refresh times. The Profit Amount calculated column wasn't required after all because the Profit measure can directly produce the required result by using existing measures.

Note

Sometimes, it makes sense to define measures that depend on other measures. Always test changes carefully, because updates can affect all dependent measures.

Use Quick measures

Quick measures let you perform common calculations without writing DAX yourself. Power BI generates the DAX expression for you, which helps you learn and build your DAX skills.

For example, you can use a Quick measure to create a Profit Margin measure through the following steps:

1. Select Quick measure in the Table tools ribbon.
2. Choose Mathematical operations > Division.
3. Add the Profit measure into **Numerator** and Revenue into **Denominator**.

Quick measure

Select a calculation to create a measure.

Division

Calculate the ratio of a value to another one. [Learn more](#)

Numerator ⓘ

Profit

Denominator ⓘ

Revenue

The new measure appears in the **Data** pane, and you can review its DAX formula:

DAX

Profit Margin =
DIVIDE([Profit], [Revenue])

Compare calculated columns and measures

Many DAX beginners find calculated columns and measures confusing at first. Both are created in the semantic model using DAX formulas, however, calculated columns and measures behave differently.

	Calculated columns	Measures
Purpose	Adds a new column to a table	Defines how to summarize model data
Evaluation	Evaluated using row context at data refresh time	Evaluated using filter context at query time
Storage	Stores a value for each row in the table (Import mode)	Never stores values in the model
Use in visuals	Can filter, group, or summarize data (as implicit measures)	Designed specifically to summarize data

Performance impact	Can increase memory usage and model size	More efficient; better performance in large models
Ideal for	New fields for slicing or relationships	Dynamic calculations based on filters

Recognizing these differences helps you choose the right approach for your modeling and reporting needs.

Use iterator functions

Iterator functions evaluate an expression for each row in a table. They give you flexibility and control over how your model summarizes data.

Single-column summarization functions, such as SUM, COUNT, MIN, and MAX, have equivalent iterator functions with an "X" suffix, like SUMX, COUNTX, MINX, and MAXX. Specialized iterator functions also exist for filtering, ranking, and semi-additive calculations over time.

Every iterator function requires a table and an expression. The table can be a model table or any expression that returns a table. The expression must return a single value for each row.

Single-column summarization functions, like SUM, act as shorthand. Power BI internally converts SUM to SUMX. For example, both of the following measures return the same result and have the same performance:

DAX

```
Revenue = SUM(Sales[Sales Amount])
```

DAX

```
Revenue =
SUMX(
    Sales,
    Sales[Sales Amount]
)
```

Iterator functions evaluate the expression for each row in a table, using row context—meaning they process one row at a time to compute the final result. Then the table is evaluated in filter context. For example, if a report visual filters by fiscal year FY2020, the Sales table contains only sales rows from that year.

Important

Using iterator functions with large tables and complex expressions can slow performance. Functions like [SEARCH](#) and [LOOKUPVALUE](#) can be expensive. When possible, use [RELATED](#) for better performance.

Iterator functions for complex summarization

Iterator functions let you aggregate more than a single column. For example, a revenue measure can multiply order quantity, unit price, and a discount factor for each row, then sum the results.

DAX

```
Revenue =  
SUMX(  
    Sales,  
    Sales[Order Quantity] * Sales[Unit Price] * (1 - Sales[Unit Price Discount  
Pct])  
)
```

Iterator functions can also reference related tables. The discount measure can use the [RELATED](#) function to access the list price from the product table:

DAX

```
Discount =  
SUMX(  
    Sales,  
    Sales[Order Quantity]  
    * (  
        RELATED('Product'[List Price]) - Sales[Unit Price]  
    )  
)
```

The following image shows a table visual with the Month, Revenue, and Discount columns. Revenue and Discount are the measures previously created.

Month	Revenue	Discount
2017 Jul	\$1,423,357.32	\$326,219.06
2017 Aug	\$2,057,902.45	\$1,031,506.86
2017 Sep	\$2,523,947.55	\$1,342,355.67
2017 Oct	\$561,681.48	\$0.00
2017 Nov	\$4,764,920.16	\$2,692,205.61
2017 Dec	\$596,746.56	\$0.00
2018 Jan	\$1,327,674.63	\$475,813.07
2018 Feb	\$3,936,463.31	\$2,237,412.77
2018 Mar	\$700,873.18	\$0.00
2018 Apr	\$1,519,275.24	\$588,995.26
2018 May	\$2,960,378.09	\$1,514,891.49
2018 Jun	\$1,487,671.19	\$1,659,893.12

Iterator functions for higher grain summarization

Iterator functions can also summarize data at different levels of detail (grain). For example, you might want to calculate an average at the line item level or at the sales order level.

In this example, the Sales table contains one row for each line item in a sales order. Each row includes details such as the sales order number, product, quantity sold, unit price, and discount. Multiple rows can have the same sales order number, representing different items within the same order.

To calculate the average revenue per order line (line item), you can use the AVERAGEX function to iterate over each row in the Sales table. The formula calculates revenue for each line item, then averages the result across all line items in the current filter context:

DAX

```
Revenue Avg Order Line =
AVERAGEX(
    Sales,
    Sales[Order Quantity] * Sales[Unit Price] * (1 - Sales[Unit Price Discount Pct])
)
```

Month	Revenue	Discount	Revenue Avg
2017 Jul	\$1,423,357.32	\$326,219.06	\$2,220.53
2017 Aug	\$2,057,902.45	\$1,031,506.86	\$2,179.98
2017 Sep	\$2,523,947.55	\$1,342,355.67	\$2,014.32
2017 Oct	\$561,681.48	\$0.00	\$3,228.05
2017 Nov	\$4,764,920.16	\$2,692,205.61	\$2,227.64
2017 Dec	\$596,746.56	\$0.00	\$3,174.18
2018 Jan	\$1,327,674.63	\$475,813.07	\$2,329.25
2018 Feb	\$3,936,463.31	\$2,237,412.77	\$2,321.03
2018 Mar	\$700,873.18	\$0.00	\$3,200.33
2018 Apr	\$1,519,275.24	\$588,995.26	\$2,182.87
2018 May	\$2,960,378.09	\$1,514,891.49	\$2,219.17
2018 Jun	\$1,487,671.19	\$1,659,893.12	\$1,398.19

If you want to calculate the average revenue per sales order (rather than per line item), you can use the VALUES function to get a list of unique sales order numbers first. Then, AVERAGEX iterates over each sales order and averages the total revenue for each order:

DAX

```
Revenue Avg Order =
AVERAGEX(
    VALUES('Sales Order'[Sales Order]),
    [Revenue]
)
```

The VALUES function returns the unique sales orders based on the current filter context, so AVERAGEX iterates over each sales order for each month.

Month	Revenue	Discount	Revenue Avg Order Line	Revenue Avg Order
2017 Jul	\$1,423,357.32	\$326,219.06	\$2,220.53	\$4,352.77
2017 Aug	\$2,057,902.45	\$1,031,506.86	\$2,179.98	\$8,794.45
2017 Sep	\$2,523,947.55	\$1,342,355.67	\$2,014.32	\$9,670.30
2017 Oct	\$561,681.48	\$0.00	\$3,228.05	\$3,228.05
2017 Nov	\$4,764,920.16	\$2,692,205.61	\$2,227.64	\$12,441.04
2017 Dec	\$596,746.56	\$0.00	\$3,174.18	\$3,174.18
2018 Jan	\$1,327,674.63	\$475,813.07	\$2,329.25	\$5,698.17
2018 Feb	\$3,936,463.31	\$2,237,412.77	\$2,321.03	\$12,301.45
2018 Mar	\$700,873.18	\$0.00	\$3,200.33	\$3,200.33
2018 Apr	\$1,519,275.24	\$588,995.26	\$2,182.87	\$6,356.80
2018 May	\$2,960,378.09	\$1,514,891.49	\$2,219.17	\$9,642.93
2018 Jun	\$1,487,671.19	\$1,659,893.12	\$1,398.19	\$4,752.94

Ranking with iterator functions

The [RANKX](#) function calculates ranks by iterating over a table and evaluating an expression for each row.

Order direction can be ascending or descending. Ranking revenue usually uses descending order, so the highest value ranks first. Ranking something like complaints might use ascending order, so the lowest value ranks first. By default, RANKX uses descending order and skips ranks for ties.

For example, a product quantity rank measure can use RANKX and the ALL function to rank products by quantity:

DAX

```
Product Quantity Rank =
RANKX(
    ALL('Product'[Product]),
    [Quantity]
)
```

The ALL function removes filters, so RANKX ranks all products. In the following image, two products tie for tenth place, so the next product is ranked twelfth and rank 11 is skipped.

Bike Sales				
Product	Quantity	Product	Quantity	Rank
Mountain-200 Black, 38	2,977			1
Mountain-200 Black, 42	2,664			2
Mountain-200 Silver, 38	2,394			3
Road-650 Black, 52	2,265			4
Road-650 Red, 44	2,244			5
Mountain-200 Silver, 42	2,234			6
Road-650 Red, 60	2,221			7
Mountain-200 Silver, 46	2,216			8
Mountain-200 Black, 46	2,111			9
Road-650 Red, 48	1,886			10
Road-650 Red, 62	1,886			10
Road-650 Black, 58	1,865			12
Road-550-W Yellow, 48	1,763			13
Road-550-W Yellow, 38	1,744			14
Road-250 Black, 44	1,642			15

You can also use dense ranking, which assigns the next rank after a tie without skipping numbers. To use dense ranking, the measure can include the DENSE argument:

DAX

```
Product Quantity Rank =
RANKX(
    ALL('Product'[Product]),
    [Quantity],
    ,
    ,
    DENSE
)
```

Now, after two products tie for tenth place, the next product is ranked eleventh and numbering continues sequentially without skipping rank 11.

Product	Quantity	Product Quantity Rank
Mountain-200 Black, 38	2,977	1
Mountain-200 Black, 42	2,664	2
Mountain-200 Silver, 38	2,394	3
Road-650 Black, 52	2,265	4
Road-650 Red, 44	2,244	5
Mountain-200 Silver, 42	2,234	6
Road-650 Red, 60	2,221	7
Mountain-200 Silver, 46	2,216	8
Mountain-200 Black, 46	2,111	9
Road-650 Red, 48	1,886	10
Road-650 Red, 62	1,886	10
Road-650 Black, 58	1,865	11
Road-550-W Yellow, 48	1,763	12
Road-550-W Yellow, 38	1,744	13
Road-250 Black, 44	1,642	14

In this visual, the total row for the Product Quantity Rank measure shows one, because the total for all products is also ranked and there's only one value.

Road-350-W Yellow, 40	1,477	19
Road-750 Black, 52	1,338	20
Total	90,220	1

To avoid ranking the total, the measure can use the [HASONEVALUE](#) function to return BLANK unless a single product is filtered:

DAX

```
Product Quantity Rank =
IF(
    HASONEVALUE('Product'[Product]),
    RANKX(
        ALL('Product'[Product]),
        [Quantity],
        ,
        ,
    )
)
```

DENSE

)

)

Now, the total for Product Quantity Rank is blank.

Road-350-W Yellow, 40	1,477	19
Road-750 Black, 52	1,338	20
Total	90,220	

The HASONEVALUE function checks if the product column has a single value in filter context. This is true for each product group, but not for the total, which represents all products.

Iterator functions provide powerful ways to summarize, aggregate, and rank data in Power BI models. They support complex calculations and let you control the level of detail in your reports.

Exercise - Create DAX calculations in semantic models

Lab story

In this lab, you'll create calculated tables, calculated columns, and simple measures by using Data Analysis Expressions (DAX).

In this lab, you learn how to:

- Create calculated tables.
- Create calculated columns.
- Create measures.

This lab should take approximately 45 minutes.

Get started

To complete this exercise, first open a web browser and enter the following URL to download the zip file:

<https://github.com/MicrosoftLearning/PL-300-Microsoft-Power-BI-Data-Analyst/raw/Main/Allfiles/Labs/04-create-dax-calculations\04-dax-calculations.zip>

Extract the file to the **C:\Users\Student\Downloads\04-dax-calculations** folder.

Open the **04-Starter-Sales Analysis.pbix** file.

Note: You can dismiss the sign-in by selecting **Cancel**. Close any other informational windows. Select **Apply Later**, if prompted to apply changes.

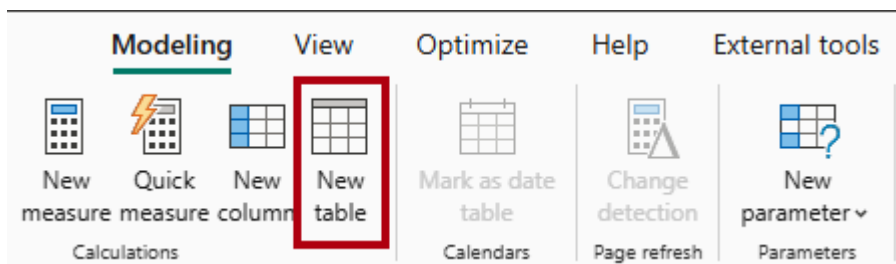
Create the Salesperson calculated table

In this task, you'll create the Salesperson calculated table (that will have a direct relationship to the Sales table).

A calculated table is created by first entering the table name, followed by the equals symbol (=), followed by a DAX formula that returns a table. The table name can't already exist in the data model.

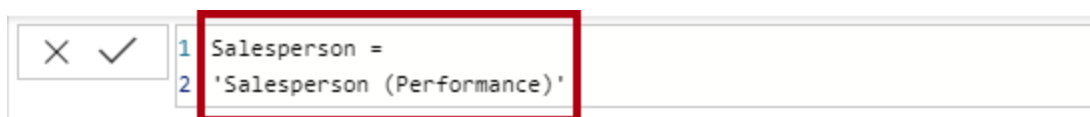
You enter a valid DAX formula in the *formula bar*. The formula bar includes features like auto-complete, Intellisense and color-coding, which allow you to quickly and accurately enter the formula.

1. In Power BI Desktop, in Report view, on the **Modeling** ribbon, from inside the **Calculations** group, select **New Table**.



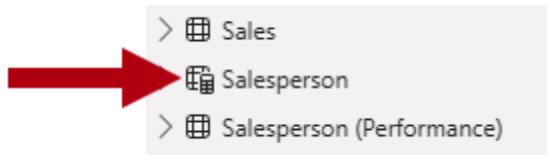
2. In the formula bar (which opens directly beneath the ribbon when you create or edit calculations), type Salesperson =, press **Shift+Enter**, type 'Salesperson (Performance)', and then press **Enter**.

Note: For your convenience, all DAX definitions in this lab can be copied from the snippets file, located in the **04-dax-calculations\Snippets.txt** file.



This table definition creates a copy of the Salesperson (Performance) table. It copies the data only, however model properties like visibility, formatting, and others aren't copied.

3. In the **Data** pane, notice that the icon for the new table has an additional calculator in front of it (denoting a calculated table).



Note: Calculated tables are defined by using a DAX formula that returns a table. It's important to understand that calculated tables increase the size of the data model because they materialize and store values. Also, they're recomputed whenever formula dependencies are refreshed, as will be the case for this data model when new (future) date values are loaded into tables.

Unlike Power Query-sourced tables, calculated tables can't be used to load data from external data sources. They can only transform data based on what has already been loaded into the data model.

4. Switch to Model view, and notice that the Salesperson table is available.
5. Create a relationship from the Salesperson | EmployeeKey column to the Sales | EmployeeKey column.

The labs use a shorthand notation to reference a field. It will look like this: Salesperson | EmployeeKey. In this example, Salesperson is the table name and EmployeeKey is the column name.

6. Right-click the inactive relationship (dotted line) between the Salesperson (Performance) and Sales tables, and then select **Delete**. When prompted to confirm the deletion, select **Yes**.
7. In the Salesperson table, multi-select the following columns, and then hide them (set the **Is Hidden** property to Yes):
 - EmployeeID
 - EmployeeKey
 - UPN
8. In the model diagram, select the Salesperson table.
9. In the **Properties** pane, in the **Description** box, enter: *Salesperson related to sales*

You may recall that descriptions appear as tooltips in the **Data** pane whenever the user hovers their cursor over a table or field.

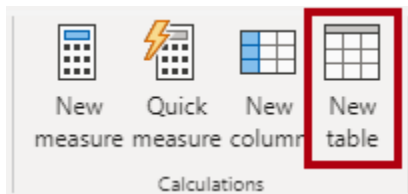
10. For the Salesperson (Performance) table, set the description to: *Salesperson related to region(s)*

The data model now provides two alternatives when analyzing salespeople. The Salesperson table allows analyzing sales made by a salesperson, while the Salesperson (Performance) table allows analyzing sales made in the sales region(s) assigned to the salesperson.

Create the Date table

In this task, you'll create the Date table.

1. Switch to Table view. On the **Home** ribbon tab, from inside the **Calculations** group, select **New Table**.



2. In the formula bar, enter the following DAX:

codeCopy

Date =

CALENDARAUTO(6)



The CALENDARAUTO function returns a single-column table comprising date values. The “auto” behavior scans all data model date columns to determine the earliest and latest date values stored in the data model. It then creates one row for each date within this range, extending the range in either direction to ensure full years of data is stored.

This function can take a single optional argument that is the last month number of a year. When omitted, the value is 12, meaning that December is the last month of the year. In this case, 6 is entered, meaning that June is the last month of the year.

3. Notice the column of date values which might be formatted using US regional settings (that is, mm/dd/yyyy).

Date
07/01/2017 00:00:00
07/02/2017 00:00:00
07/03/2017 00:00:00
07/04/2017 00:00:00
07/05/2017 00:00:00
07/06/2017 00:00:00

- At the bottom-left corner, in the status bar, notice the table statistics, confirming that 1826 rows of data have been generated, which represents five full years' data.

TABLE: Date (1,826 rows)

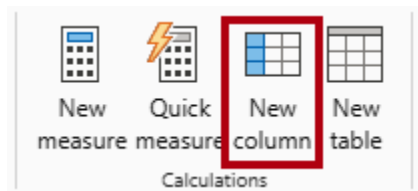
Create calculated columns

In this task, you'll add more columns to enable filtering and grouping by different time periods. You'll also create a calculated column to control the sort order of other columns.

Note: For your convenience, all DAX definitions in this lab can be copied from the **Snippets.txt** file.

- On the **Table Tools** contextual ribbon, from inside the **Calculations** group, select **New Column**.

A calculated column is created by first entering the column name, followed by the equals symbol (=), followed by a DAX formula that returns a single-value result. The column name can't already exist in the table.



- In the formula bar, type the following (or copy from the snippets file), and then press **Enter**:

The formula uses the date's year value but adds one to the year value when the month is after June. That's how fiscal years at Adventure Works are calculated.

codeCopy

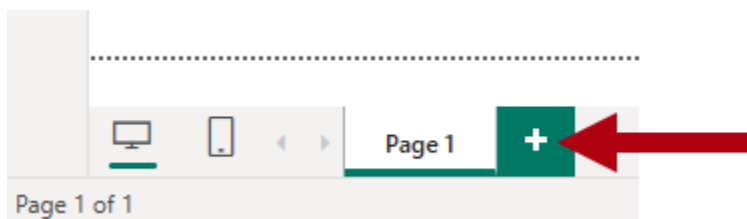
Year =

```
"FY" & YEAR('Date'[Date]) + IF(MONTH('Date'[Date]) > 6, 1)
```

3. Use the snippets file definitions to create the following two calculated columns for the Date table:
 - Quarter
 - Month
4. Verify the new columns have been added.

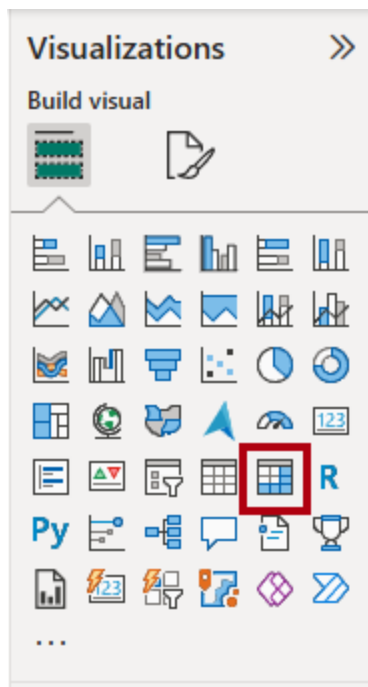
Date	Year	Quarter	Month
01/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
02/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
03/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
04/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
05/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
06/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
07/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul

5. To validate the calculations, switch to Report view.
6. To create a new report page, select the plus icon next to **Page 1**.

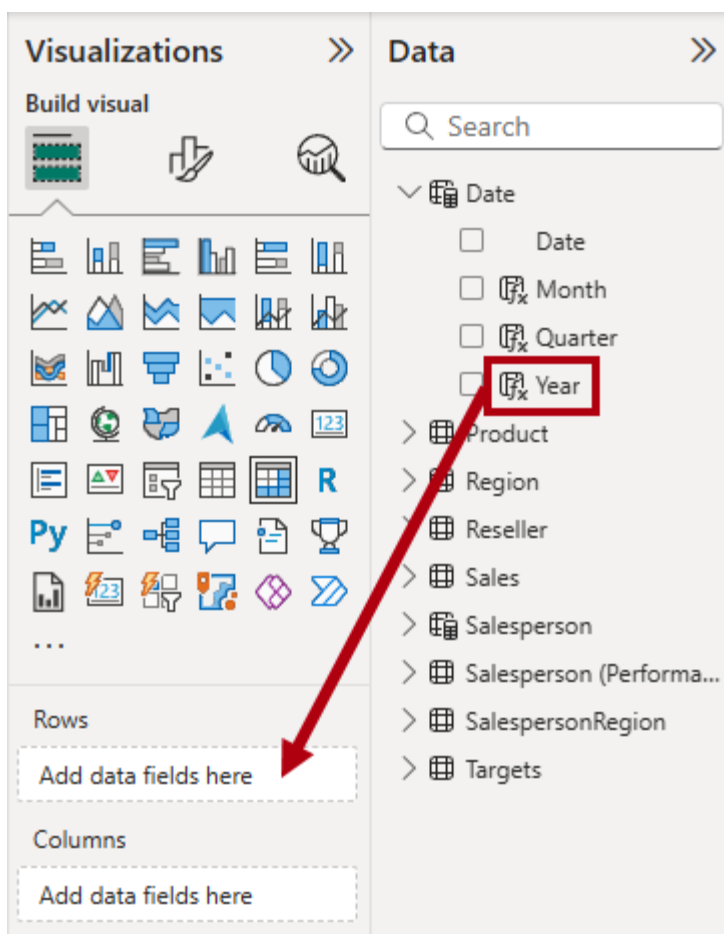


7. To add a matrix visual to the new report page, in the **Visualizations** pane, select the matrix visual type.

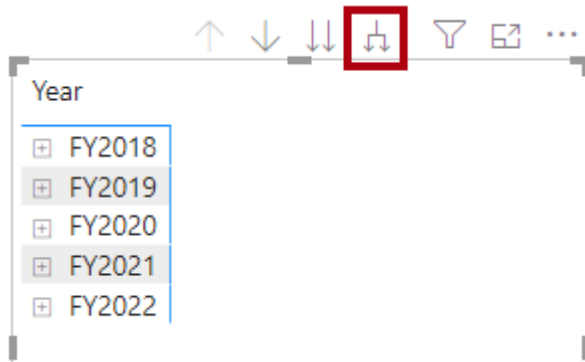
Tip: You can hover the cursor over each icon to reveal a tooltip describing the visual type.



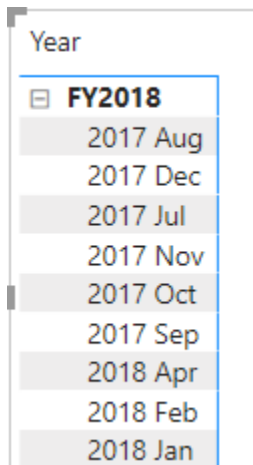
8. In the **Data** pane, from inside the Date table, drag the Year field into the **Rows** well.



9. Drag the Month field into the **Rows** well, directly beneath the Year field.
10. At the top-right of the matrix visual (or bottom, depending on the location of the visual), select the forked-double arrow icon (which will expand all years down one level).



11. Notice that the years expand to months, and that the months are sorted alphabetically rather than chronologically.



By default, text values sort alphabetically, numbers sort from smallest to largest, and dates sort from earliest to latest.

12. To customize the Month field sort order, switch to Table view.
13. Add the MonthKey column to the Date table.

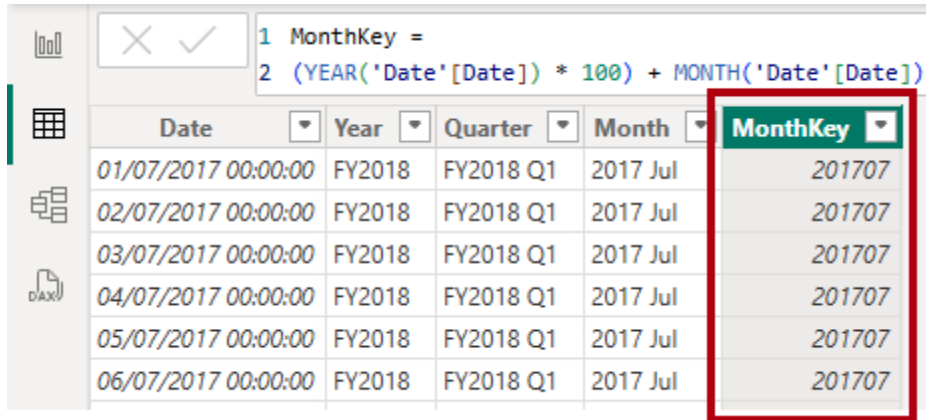
codeCopy

MonthKey =

```
(YEAR('Date'[Date]) * 100) + MONTH('Date'[Date])
```

This formula computes a numeric value for each year/month combination.

14. In Table view, verify that the new column contains numeric values (for example, 201707 for July 2017, and so on).



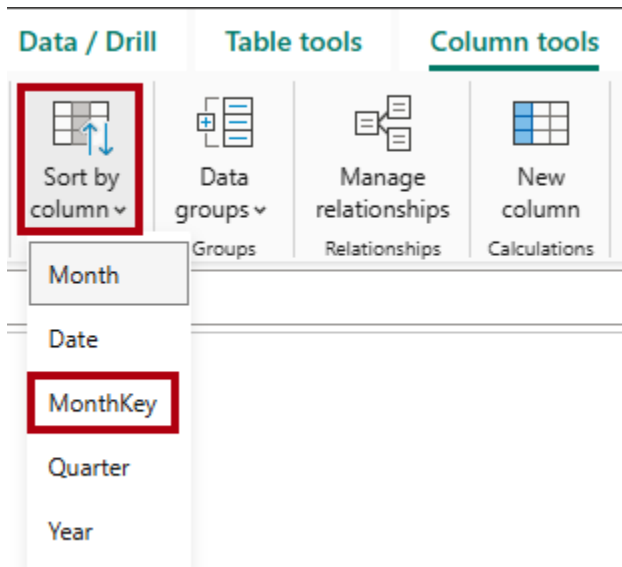
1 MonthKey =
2 (YEAR('Date'[Date]) * 100) + MONTH('Date'[Date])

Date	Year	Quarter	Month	MonthKey
01/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
02/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
03/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
04/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
05/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
06/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707

15. Switch back to Report view.

16. In the **Data** pane and select the Month field.

17. On the **Column Tools** contextual ribbon, from inside the **Sort** group, select **Sort by Column**, and then select **MonthKey**.



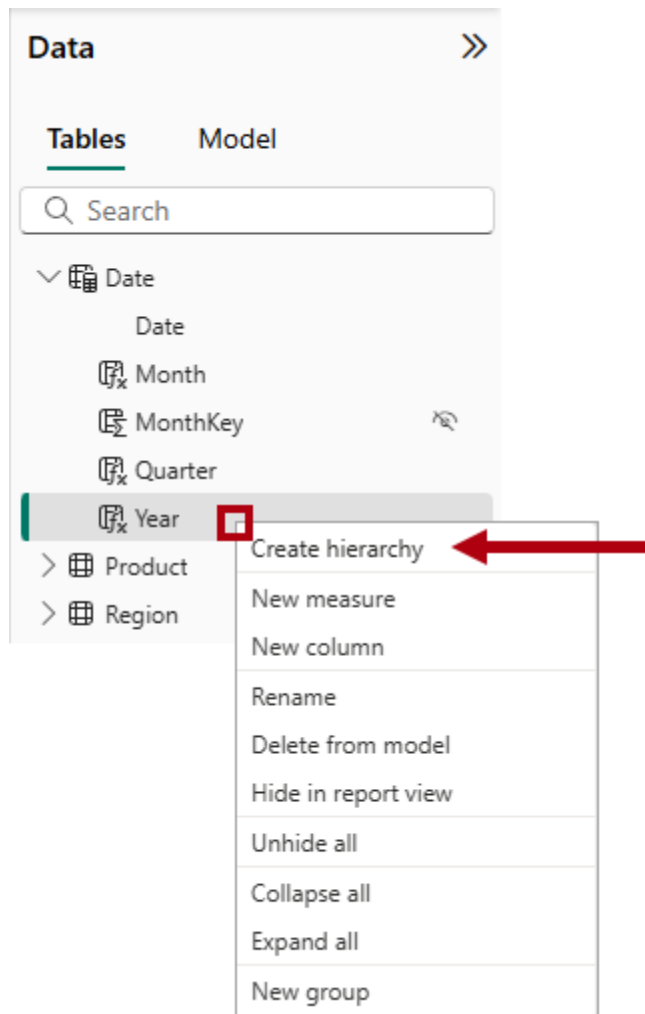
18. In the matrix visual, notice that the months are now chronologically sorted.

Year	
☐	FY2018
	2017 Jul
	2017 Aug
	2017 Sep
	2017 Oct
	2017 Nov
	2017 Dec
	2018 Jan
	2018 Feb
	2018 Mar

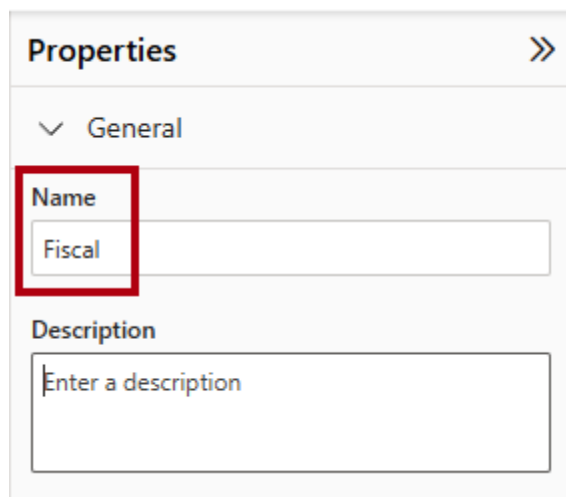
Complete the Date table

In this task, you'll complete the design of the Date table by hiding a column and creating a hierarchy. You'll then create relationships to the Sales and Targets tables.

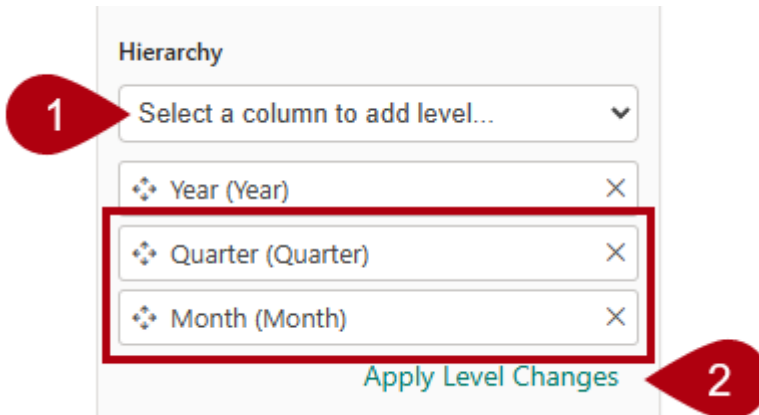
1. Switch to Model view.
2. In the Date table, hide the MonthKey column (set **Is Hidden** to Yes).
3. In the **Data** pane, select the Date table, right-click the Year column, and then select **Create hierarchy**.



4. In the **Properties** pane, in the **Name** box, replace the value with *Fiscal*.



- Two add levels to the hierarchy, in the **Hierarchy** dropdown list, select **Quarter** and then select **Month**, and then select **Apply Level Changes**.

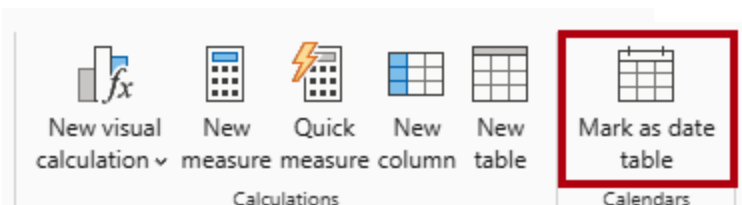


- Create the following two model relationships:
 - Date | Date to Sales | OrderDate
 - Date | Date to Targets | TargetMonth
- Hide the following two columns:
 - Sales | OrderDate
 - Targets | TargetMonth

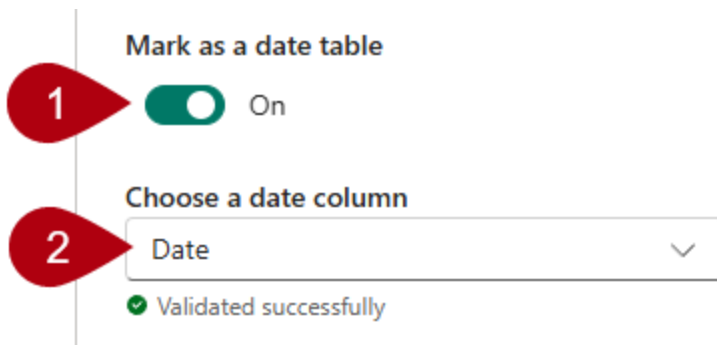
Mark the Date table

In this task, you'll mark the Date table as a date table.

- Switch to Report view.
- In the **Data** pane, select the Date table (not the Date field).
- On the **Table Tools** contextual ribbon, from inside the **Calendars** group, select **Mark as Date Table**.



- In the **Mark as a Date Table** window, slide the **Mark as a Date Table** property to Yes.
- In the **Choose a date column** dropdown list, select **Date**.



6. Select **Save**.

7. Save the Power BI Desktop file.

Power BI Desktop now understands that this table defines date (time).

This design approach for a date table is suitable when you don't have a date table in your data source. If you have a data warehouse, it would be appropriate to load date data from its date dimension table rather than "redefining" date logic in your data model.

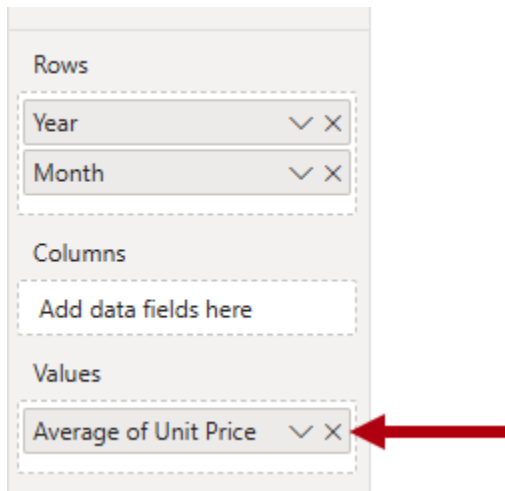
Create simple measures

In this task, you'll create simple measures. Simple measures aggregate values in a single column or count rows of a table.

1. In Report view, on **Page 2**, from the **Data** pane, drag the Sales | Unit Price field into the matrix visual.

Year	Average of Unit Price
FY2018	\$748.68
2017 Aug	\$758.93
2017 Dec	\$734.58
2017 Jul	\$655.59
2017 Nov	\$752.31
2017 Oct	\$677.45

2. In the visual fields pane (located in the **Visualizations** pane), in the **Values** well, notice that Unit Price field is configured as **Average of Unit Price**.



3. Select the down-arrow for **Average of Unit Price**, and then notice the available menu options.

Visible numeric columns allow report authors at report design time to decide how column values will summarize (or not). However, it can result in inappropriate reporting.

Some data modelers don't like leaving things to chance, so they choose to hide these columns and instead expose aggregation logic defined in measures. It's the approach you'll now take in this lab.

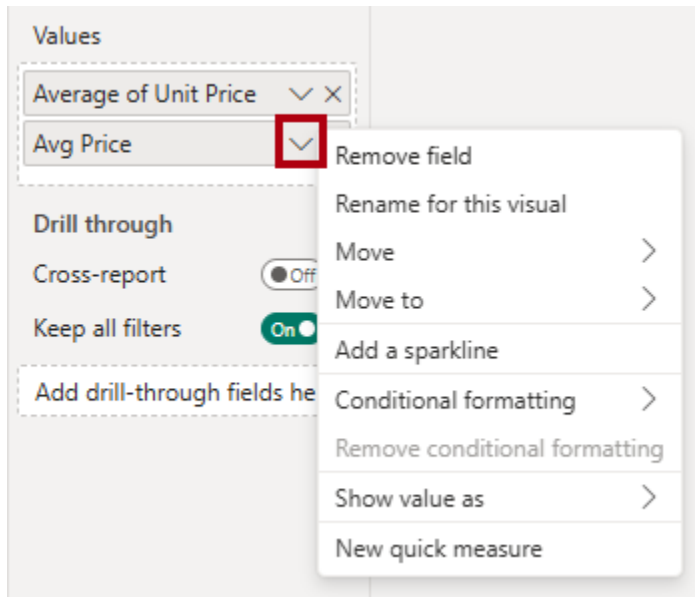
4. To create a measure, in the **Data** pane, right-click the Sales table, and then select **New Measure**.
5. In the formula bar, add the following measure definition:

codeCopy

Avg Price =

AVERAGE(Sales[Unit Price])

6. Add the Avg Price measure to the matrix visual, and notice that it produces the same result as the Unit Price column (but with different formatting).
7. In the **Values** well, open the context menu for the Avg Price field, and notice that it isn't possible to change the aggregation technique.



It's not possible to modify the aggregation behavior of a measure.

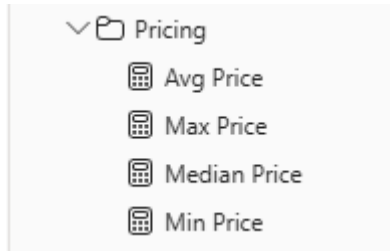
8. Use the snippets file definitions to create the following five measures for the Sales table:
 - Median Price
 - Min Price
 - Max Price
 - Orders
 - Order Lines

The DISTINCTCOUNT function used in the Orders measure counts orders only once (ignoring duplicates). The COUNTROWS function used in the Order Lines measure operates over a table.

In this case, the number of orders is calculated by counting the distinct SalesOrderNumber column values, while the number of order lines is simply the number of table rows (each row is a line of an order).

9. Switch to Model view, and then multi-select the four price measures: Avg Price, Max Price, Median Price, and Min Price.
10. For the multi-selection of measures, configure the following requirements:
 - Set the format to two decimal places.

- Assign to a display folder named *Pricing* (use the **Display folder** property in the **Properties** pane).

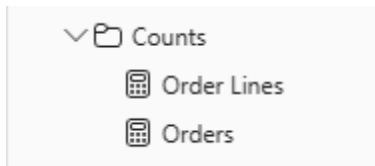


11. Hide the Unit Price column.

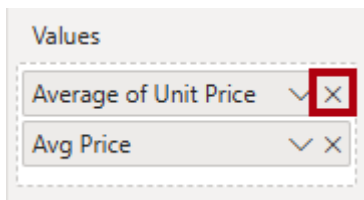
The Unit Price column is no longer available to report authors. They must use the pricing measures you've added to the model. This design approach ensures that report authors won't inappropriately aggregate prices, for example, by summing them.

12. Multi-select the Order Lines and Orders measures, and then configure the following requirements:

- Set the format use the thousands separator.
- Assign to a display folder named *Counts*.



13. In Report view, in the **Values** well of the matrix visual, for **Average of Unit Price**, select **X** to remove it.



14. Increase the size of the matrix visual to fill the page width and height.

15. Add the following five measures to the matrix visual:

- Median Price
- Min Price
- Max Price

- Orders
- Order Lines

16. Verify that the results look sensible and are correctly formatted.

Year	Avg Price	Median Price	Min Price	Max Price	Orders	Order Lines
 FY2018	\$748.68	\$419.46	\$4.75	\$2,146.96	739	8,459
2017 Jul	\$655.59	\$419.46	\$5.19	\$2,146.96	38	352
2017 Aug	\$758.93	\$419.46	\$4.75	\$2,146.96	75	785
2017 Sep	\$741.85	\$419.46	\$5.19	\$2,146.96	60	593
2017 Oct	\$677.45	\$419.46	\$5.19	\$2,146.96	40	499
2017 Nov	\$752.31	\$419.46	\$5.01	\$2,146.96	90	1,106
2017 Dec	\$734.58	\$419.46	\$5.01	\$2,146.96	63	803

Create additional measures

In this task, you'll create more measures that use more complex formulas.

1. In Report view, select **Page 1** and review the table visual of salespeople (on the right), noticing the total for the **Sum of Target** column.

Salesperson	Sum of Sales	Sum of Target
Amy Alberts	\$10,288,626	\$19,450,000
Brian Welcker	\$77,548,570	\$221,700,000
David Campbell	\$12,004,822	\$19,625,000
Garrett Vargas	\$13,875,633	\$23,675,000
Jae Pak	\$8,410,883	\$13,575,000
Jillian Carson	\$7,633,387	\$13,675,000
José Saraiva	\$13,875,633	\$18,875,000
Linda Mitchell	\$25,634,503	\$40,850,000
Lynn Tsoflias	\$1,391,025	\$3,210,000
Michael Blythe	\$21,987,348	\$31,150,000
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000
Rachel Valdez	\$1,877,743	\$4,125,000
Ranjit Varkey Chudukatil	\$4,527,840	\$9,050,000
Shu Ito	\$18,001,116	\$59,850,000
Stephen Jiang	\$65,868,919	\$110,150,000
Syed Abbas	\$1,391,025	\$3,050,000
Tete Mensa-Annan	\$12,004,822	\$17,100,000
Tsvi Reiter	\$7,638,607	\$13,250,000
Total	\$77,548,570	\$676,210,000

2. Select the table visual, and then in the **Visualizations** pane, remove **Sum of Target**.

3. Rename the Targets | Target column as *TargetAmount*.

*Tip: There are several ways to rename the column in Report view: In the **Data** pane, you can right-click the column, and then select **Rename**—or, double-click the column, or press **F2**.*

4. Create the following measure on the Targets table:

codeCopy

Target =

IF(

HASONEVALUE('Salesperson (Performance)'[Salesperson]),

SUM(Targets[TargetAmount])

)

The **HASONEVALUE** function tests whether a single value in the Salesperson column is filtered. When true, the expression returns the sum of target amounts (for just that salesperson). When false, **BLANK** is returned.

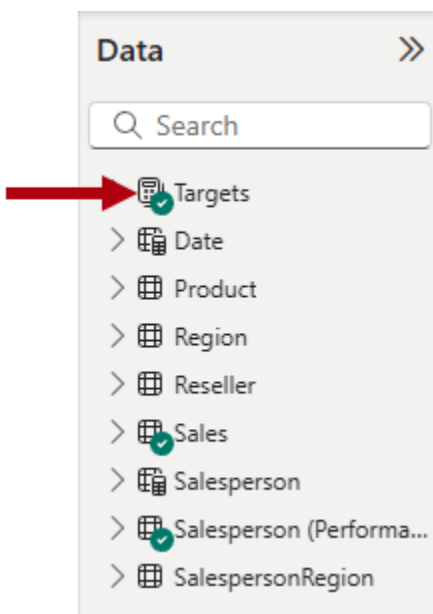
5. Format the Target measure for zero decimal places.

*Tip: You can use the **Measure Tools** contextual ribbon.*

6. Hide the TargetAmount column.

*Tip: You can right-click the column in the **Data** pane, and then select **Hide**.*

7. Notice that the Targets table now appears at the top of the list.



Tables that comprise only visible measures are automatically listed at the top of the list.

8. Add the Target measure to the table visual.
9. Notice that the **Target** column total is now **BLANK**.

Salesperson	Sum of Sales	Target
Amy Alberts	\$10,288,626	\$19,450,000
Brian Welcker	\$77,548,570	\$221,700,000
David Campbell	\$12,004,822	\$19,625,000
Garrett Vargas	\$13,875,633	\$23,675,000
Jae Pak	\$8,410,883	\$13,575,000
Jillian Carson	\$7,633,387	\$13,675,000
José Saraiva	\$13,875,633	\$18,875,000
Linda Mitchell	\$25,634,503	\$40,850,000
Lynn Tsofilas	\$1,391,025	\$3,210,000
Michael Blythe	\$21,987,348	\$31,150,000
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000
Rachel Valdez	\$1,877,743	\$4,125,000
Ranjit Varkey Chudukatil	\$4,527,840	\$9,050,000
Shu Ito	\$18,001,116	\$59,850,000
Stephen Jiang	\$65,868,919	\$110,150,000
Syed Abbas	\$1,391,025	\$3,050,000
Tete Mensa-Annan	\$12,004,822	\$17,100,000
Tsvi Reiter	\$7,638,607	\$13,250,000
Total	\$77,548,570	

10. Use the snippets file definitions to create the following two measures for the Targets table:
 - Variance
 - Variance Margin
11. Format the Variance measure for zero decimal places.
12. Format the Variance Margin measure as percentage with two decimal places.
13. Add the Variance and Variance Margin measures to the table visual.
14. Resize the table visual so all columns and rows can be seen.

Salesperson	Sum of Sales	Target	Variance	Variance Margin
Amy Alberts	\$10,288,626	\$19,450,000	-\$9,161,374	-47.10%
Brian Welcker	\$77,548,570	\$221,700,000	-\$144,151,430	-65.02%
David Campbell	\$12,004,822	\$19,625,000	-\$7,620,178	-38.83%
Garrett Vargas	\$13,875,633	\$23,675,000	-\$9,799,367	-41.39%
Jae Pak	\$8,410,883	\$13,575,000	-\$5,164,117	-38.04%
Jillian Carson	\$7,633,387	\$13,675,000	-\$6,041,613	-44.18%
José Saraiva	\$13,875,633	\$18,875,000	-\$4,999,367	-26.49%
Linda Mitchell	\$25,634,503	\$40,850,000	-\$15,215,497	-37.25%
Lynn Tsoflias	\$1,391,025	\$3,210,000	-\$1,818,975	-56.67%
Michael Blythe	\$21,987,348	\$31,150,000	-\$9,162,652	-29.41%
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000	-\$23,844,061	-44.28%
Rachel Valdez	\$1,877,743	\$4,125,000	-\$2,247,257	-54.48%
Ranjit Varkey Chudukatil	\$4,527,840	\$9,050,000	-\$4,522,160	-49.97%
Shu Ito	\$18,001,116	\$59,850,000	-\$41,848,884	-69.92%
Stephen Jiang	\$65,868,919	\$110,150,000	-\$44,281,081	-40.20%
Syed Abbas	\$1,391,025	\$3,050,000	-\$1,658,975	-54.39%
Tete Mensa-Annan	\$12,004,822	\$17,100,000	-\$5,095,178	-29.80%
Tsvi Reiter	\$7,638,607	\$13,250,000	-\$5,611,393	-42.35%
Total	\$77,548,570			

While it appears all salespeople aren't meeting target, remember that the table visual isn't yet filtered by a specific time period. You'll produce sales performance reports that filter by a user-selected time period in the **Design a Report in Power BI Desktop** lab.

15. Save the Power BI Desktop file.

Lab complete

Knowledge check

Summary

This module covered using DAX calculations to extend your semantic model. You learned the differences between calculated columns and measures, including when and how Power BI evaluates them and how they store data. Explicit measures are important because they allow you to create complex DAX formulas to achieve the precise calculations that your report visuals need.

You also learned how calculated columns are evaluated within row context, and iterator functions are available in measures for advanced row-by-row calculations.

Understanding how to create and use DAX calculations is fundamental for building effective, flexible, and maintainable semantic models in Power BI. These concepts help you design reports that deliver accurate insights and support a wide range of analytical requirements.

For more information, see [Introduction to DAX in Power BI](#).

Use DAX time intelligence functions in semantic models

Introduction

Time intelligence relates to calculations over time. Specifically, it relates to calculations over dates, months, quarters, or years, and possibly time. Rarely would you need to calculate over time in the sense of hours, minutes, or seconds.

In Data Analysis Expressions (DAX) calculations, time intelligence refers to *modifying the filter context for date filters*.

For example, at the Adventure Works company, their financial year begins on July 1 and ends on June 30 of the following year. They produce a table visual that displays monthly revenue and year-to-date (YTD) revenue.

Year	Revenue	Revenue YTD
☐ FY2018	\$23,860,891.17	\$23,860,891.17
2017 Jul	\$1,423,357.32	\$1,423,357.32
2017 Aug	\$2,057,902.45	\$3,481,259.78
2017 Sep	\$2,523,947.55	\$6,005,207.32
2017 Oct	\$561,681.48	\$6,566,888.80
2017 Nov	\$4,764,920.16	\$11,331,808.96
2017 Dec	\$596,746.56	\$11,928,555.52
2018 Jan	\$1,327,674.63	\$13,256,230.15
2018 Feb	\$3,936,463.31	\$17,192,693.45
2018 Mar	\$700,873.18	\$17,893,566.64
2018 Apr	\$1,519,275.24	\$19,412,841.88
2018 May	\$2,960,378.09	\$22,373,219.97
2018 Jun	\$1,487,671.19	\$23,860,891.17
☐ FY2019	\$34,070,108.50	\$34,070,108.50
2018 Jul	\$2,939,691.00	\$2,939,691.00
2018 Aug	\$3,964,801.20	\$6,904,492.20
2018 Sep	\$3,287,605.93	\$10,192,098.13

The filter context for *2017 August* contains each of the 31 dates of August, which are stored in the Date table. However, the calculated year-to-date revenue for *2017 August* applies a different filter context. It's the first date of the year through to the last date in filter context. In this example, that's July 1, 2017 through to August 31, 2017.

Time intelligence calculations modify date filter contexts. They can help you answer these time-related questions:

- What's the accumulation of revenue for the year, quarter, or month?
- What revenue was produced for the same period last year?
- What growth in revenue has been achieved over the same period last year?
- How many new customers made their first order in each month?
- What's the inventory stock on-hand value for the company's products?

This module describes how to create time intelligence measures to answer these questions.

Note

Many units in this module present inline activities to follow along and create calculations. These activities are optional. If you want to complete the activities, you can download the Power BI Desktop file by using the links provided in the units.

At the end of this module, you have the opportunity to complete an exercise in a virtual machine to apply the lessons on how to create calculated tables, calculated columns, and simple measures using DAX.

Use DAX time intelligence functions

DAX includes several time intelligence functions to simplify the task of modifying date filter context. You can write many of these intelligence formulas by using a CALCULATE function that modifies date filters, but that creates more work.

Note

Many DAX time intelligence functions work with standard date periods such as years, quarters, and months. If you have irregular time periods, like financial months that do not match the calendar, or if you need to work with weeks or shorter time periods, the built-in DAX functions will not help. In these cases, use the CALCULATE function with custom date or time filters.

Date table requirement

To work with time intelligence DAX functions, you need to meet the prerequisite model requirement of having at least one *date table* in your model. A date table is a table that is marked as a Date table. That table must have a column of data type Date (or date/time), known as the *date column*. The date column must:

- Contain unique values.
- Span full years.
- Not contain BLANKs.
- Not have any missing dates.

For more information, see [Create date tables in Power BI Desktop](#).

Summarizations over time

One group of DAX time intelligence functions is concerned with summarizations over time:

- DATESYTD - Returns a single-column table that contains dates for the year-to-date (YTD) in the current filter context. This group also includes the DATESMTD and DATESQTD functions for month-to-date (MTD) and quarter-to-date (QTD). You can pass these functions as filters into the CALCULATE function.
- TOTALYTD - Evaluates an expression for YTD in the current filter context. The equivalent QTD and MTD DAX functions of TOTALQTD and TOTALMTD are also included.
- DATESBETWEEN - Returns a table that contains a column of dates that begins with a given start date and continues until a given end date.
- DATESINPERIOD - Returns a table that contains a column of dates that begins with a given start date and continues for the specified number of intervals.

Note

While the TOTALYTD function is simple to use, you're limited to passing in one filter expression. If you need to apply multiple filter expressions, use the CALCULATE function and then pass the DATESYTD function in as one of the filter expressions.

In the following example, you'll create your first time intelligence calculation that uses the TOTALYTD function. The syntax is as follows:

DAX

```
TOTALYTD(<expression>, <dates>, [, <filter>][, <year_end_date>])
```

The function requires an expression and, as is common to all time intelligence functions, a reference to the date column of a marked date table. Optionally, a single filter expression or the year-end date can be passed in (required only when the year doesn't finish on December 31).

Download and open the [Adventure Works DW 2020 M07.pbix](#) file. Then, add the following measure definition to the Sales table that calculates YTD revenue. Format the measure as currency with two decimal places.

DAX

```
Revenue YTD =  
TOTALYTD(  
    [Revenue],  
    'Date' [Date],  
    "6-30"  
)
```

The year-end date value of "6-30" represents June 30.

On **Page 1** of the report, add the Revenue YTD measure to the matrix visual. Notice that it produces a summarization of the revenue amounts from the beginning of the year through to the filtered month.

Year	Revenue	Revenue YTD
 FY2018	\$23,860,891.17	\$23,860,891.17
2017 Jul	\$1,423,357.32	\$1,423,357.32
2017 Aug	\$2,057,902.45	\$3,481,259.78
2017 Sep	\$2,523,947.55	\$6,005,207.32
2017 Oct	\$561,681.48	\$6,566,888.80
2017 Nov	\$4,764,920.16	\$11,331,808.96
2017 Dec	\$596,746.56	\$11,928,555.52
2018 Jan	\$1,327,674.63	\$13,256,230.15
2018 Feb	\$3,936,463.31	\$17,192,693.45
2018 Mar	\$700,873.18	\$17,893,566.64
2018 Apr	\$1,519,275.24	\$19,412,841.88
2018 May	\$2,960,378.09	\$22,373,219.97
2018 Jun	\$1,487,671.19	\$23,860,891.17

Comparisons over time

Another group of DAX time intelligence functions is concerned with shifting time periods:

- **DATEADD** - Returns a table that contains a column of dates, shifted either forward or backward in time by the specified number of intervals from the dates in the current filter context.
- **PARALLELPERIOD** - Returns a table that contains a column of dates that represents a period that is parallel to the dates in the specified dates column, in the current filter context, with the dates shifted a number of intervals either forward in time or back in time.
- **SAMEPERIODLASTYEAR** - Returns a table that contains a column of dates that are shifted one year back in time from the dates in the specified dates column, in the current filter context.
- Many helper DAX functions for navigating backward or forward for specific time periods, all of which returns a table of dates. These helper functions include **NEXTDAY**, **NEXTMONTH**, **NEXTQUARTER**, **NEXTYEAR**, and **PREVIOUSDAY**, **PREVIOUSMONTH**, **PREVIOUSQUARTER**, and **PREVIOUSYEAR**.

Now, you'll add a measure to the Sales table that calculates revenue for the prior year by using the **SAMEPERIODLASTYEAR** function. Format the measure as currency with two decimal places.

DAX

Revenue PY =

```
VAR RevenuePriorYear = CALCULATE(
    [Revenue],
    SAMEPERIODLASTYEAR('Date'[Date])
)
RETURN
    RevenuePriorYear
```

Add the Revenue PY measure to the matrix visual. Notice that it produces results that are similar to the previous year's revenue amounts.

Year	Revenue	Revenue YTD	Revenue PY
FY2018	\$23,860,891.17	\$23,860,891.17	
2017 Jul	\$1,423,357.32	\$1,423,357.32	
2017 Aug	\$2,057,902.45	\$3,481,259.78	
2017 Sep	\$2,523,947.55	\$6,005,207.32	
2017 Oct	\$561,681.48	\$6,566,888.80	
2017 Nov	\$4,764,920.16	\$11,331,808.96	
2017 Dec	\$596,746.56	\$11,928,555.52	
2018 Jan	\$1,327,674.63	\$13,256,230.15	
2018 Feb	\$3,936,463.31	\$17,192,693.45	
2018 Mar	\$700,873.18	\$17,893,566.64	
2018 Apr	\$1,519,275.24	\$19,412,841.88	
2018 May	\$2,960,378.09	\$22,373,219.97	
2018 Jun	\$1,487,671.19	\$23,860,891.17	
FY2019	\$34,070,108.50	\$34,070,108.50	\$23,860,891.17
2018 Jul	\$2,939,691.00	\$2,939,691.00	\$1,423,357.32
2018 Aug	\$3,964,801.20	\$6,904,492.20	\$2,057,902.45
2018 Sep	\$3,287,605.93	\$10,192,098.13	\$2,523,947.55
2018 Oct	\$2,157,287.40	\$12,349,385.53	\$561,681.48
2018 Nov	\$3,611,092.23	\$15,960,477.76	\$4,764,920.16
2018 Dec	\$2,624,078.39	\$18,584,556.15	\$596,746.56
2019 Jan	\$1,847,691.91	\$20,432,248.06	\$1,327,674.63
2019 Feb	\$2,829,361.64	\$23,261,609.70	\$3,936,463.31
2019 Mar	\$2,092,434.35	\$25,354,044.05	\$700,873.18
2019 Apr	\$2,405,970.99	\$27,760,015.05	\$1,519,275.24
2019 May	\$3,459,444.04	\$31,219,459.08	\$2,960,378.09
2019 Jun	\$2,850,649.42	\$34,070,108.50	\$1,487,671.19

Next, you'll modify the measure by renaming it to Revenue YoY % and then updating the RETURN clause to calculate the change ratio. Be sure to change the format to a percentage with two decimals places.

DAX

```
Revenue YoY % =  
VAR RevenuePriorYear = CALCULATE(  
    [Revenue],  
    SAMEPERIODLASTYEAR('Date'[Date])  
)  
RETURN  
    DIVIDE(  
        [Revenue] - RevenuePriorYear,  
        RevenuePriorYear  
    )
```

Notice that the Revenue YoY % measure produces a ratio of change factor over the previous year's monthly revenue. For example, July 2018 represents a 106.53 percent *increase* over the previous year's monthly revenue, and November 2018 represents a 24.22 percent *decrease* over the previous year's monthly revenue.

Year	Revenue	Revenue YTD	Revenue YoY %
FY2018	\$23,860,891.17	\$23,860,891.17	
2017 Jul	\$1,423,357.32	\$1,423,357.32	
2017 Aug	\$2,057,902.45	\$3,481,259.78	
2017 Sep	\$2,523,947.55	\$6,005,207.32	
2017 Oct	\$561,681.48	\$6,566,888.80	
2017 Nov	\$4,764,920.16	\$11,331,808.96	
2017 Dec	\$596,746.56	\$11,928,555.52	
2018 Jan	\$1,327,674.63	\$13,256,230.15	
2018 Feb	\$3,936,463.31	\$17,192,693.45	
2018 Mar	\$700,873.18	\$17,893,566.64	
2018 Apr	\$1,519,275.24	\$19,412,841.88	
2018 May	\$2,960,378.09	\$22,373,219.97	
2018 Jun	\$1,487,671.19	\$23,860,891.17	
FY2019	\$34,070,108.50	\$34,070,108.50	42.79%
2018 Jul	\$2,939,691.00	\$2,939,691.00	106.53%
2018 Aug	\$3,964,801.20	\$6,904,492.20	92.66%
2018 Sep	\$3,287,605.93	\$10,192,098.13	30.26%
2018 Oct	\$2,157,287.40	\$12,349,385.53	284.08%
2018 Nov	\$3,611,092.23	\$15,960,477.76	-24.22%
2018 Dec	\$2,624,078.39	\$18,584,556.15	339.73%
2019 Jan	\$1,847,691.91	\$20,432,248.06	39.17%
2019 Feb	\$2,829,361.64	\$23,261,609.70	-28.12%
2019 Mar	\$2,092,434.35	\$25,354,044.05	198.55%
2019 Apr	\$2,405,970.99	\$27,760,015.05	58.36%
2019 May	\$3,459,444.04	\$31,219,459.08	16.86%
2019 Jun	\$2,850,649.42	\$34,070,108.50	91.62%

Note

The Revenue YoY % measure demonstrates a good use of DAX variables. The measure improves the readability of the formula and allows you to unit test part of the measure logic (by returning the RevenuePriorYear variable value). Additionally, the measure is an optimal formula because it doesn't need to retrieve the prior year's revenue value twice. Having stored it once in a variable, the RETURN clause uses to the variable value twice.

Additional time intelligence calculations

Other DAX time intelligence functions exist that are concerned with returning a single date. In this unit, you learn about these functions by applying them in two different scenarios.

The [FIRSTDATE](#) and the [LASTDATE](#) functions return the first and last date in the current filter context for the specified column of dates.

Calculate new occurrences

Another use of time intelligence functions is to count new occurrences. The following example shows how you can calculate the number of new customers for a time period. A new customer is counted in the time period in which they made their first purchase.

Your first task is to add the following measure to the Sales table that counts the number of distinct customers *life-to-date* (LTD). Life-to-date means from the beginning of time until the last date in filter context. Format the measure as a whole number by using the thousands separator.

DAX

```
Customers LTD =  
VAR CustomersLTD =  
    CALCULATE(  
        DISTINCTCOUNT(Sales[CustomerKey]),  
        DATESBETWEEN(  
            'Date'[Date],  
            BLANK(),  
            MAX('Date'[Date])  
        ),  
        'Sales Order'[Channel] = "Internet"  
    )  
RETURN  
    CustomersLTD
```

Add the Customers LTD measure to the matrix visual. Notice that it produces a result of distinct customers LTD until the end of each month.

Year	Revenue	Revenue YTD	Revenue YoY %	Customers LTD
☐ FY2018	\$23,860,891.17	\$23,860,891.17		2,459
2017 Jul	\$1,423,357.32	\$1,423,357.32		289
2017 Aug	\$2,057,902.45	\$3,481,259.78		448
2017 Sep	\$2,523,947.55	\$6,005,207.32		609
2017 Oct	\$561,681.48	\$6,566,888.80		783
2017 Nov	\$4,764,920.16	\$11,331,808.96		1,013
2017 Dec	\$596,746.56	\$11,928,555.52		1,201
2018 Jan	\$1,327,674.63	\$13,256,230.15		1,394
2018 Feb	\$3,936,463.31	\$17,192,693.45		1,571
2018 Mar	\$700,873.18	\$17,893,566.64		1,790
2018 Apr	\$1,519,275.24	\$19,412,841.88		1,992
2018 May	\$2,960,378.09	\$22,373,219.97		2,214
2018 Jun	\$1,487,671.19	\$23,860,891.17		2,459

The DATESBETWEEN function returns a table that contains a column of dates that begins with a given start date and continues until a given end date. When the start date is BLANK, it uses the first date in the date column. (Conversely, when the end date is BLANK, it uses the last date in the date column.) In this case, the end date is determined by the MAX function, which returns the last date in filter context. Therefore, if the month of August 2017 is in filter context, then the MAX function will return August 31, 2017 and the DATESBETWEEN function will return all dates through to August 31, 2017.

Next, you modify the measure by renaming it to New Customers and by adding a second variable to store the count of distinct customers *before* the time period in filter context. The RETURN clause now subtracts this value from LTD customers to produce a result, which is the number of new customers in the time period.

DAX

```
New Customers =
VAR CustomersLTD =
    CALCULATE(
        DISTINCTCOUNT(Sales[CustomerKey]),
        DATESBETWEEN(
            'Date'[Date],
            BLANK(),
            MAX('Date'[Date])
        )
    )
```

```

    ),
    'Sales Order'[Channel] = "Internet"
)
VAR CustomersPrior =
    CALCULATE(
        DISTINCTCOUNT(Sales[CustomerKey]),
        DATESBETWEEN(
            'Date'[Date],
            BLANK(),
            MIN('Date'[Date]) - 1
        ),
        'Sales Order'[Channel] = "Internet"
    )
RETURN
    CustomersLTD - CustomersPrior

```

Year	Revenue	Revenue YTD	Revenue YoY %	New Customers
 FY2018	\$23,860,891.17	\$23,860,891.17		2,459
2017 Jul	\$1,423,357.32	\$1,423,357.32		289
2017 Aug	\$2,057,902.45	\$3,481,259.78		159
2017 Sep	\$2,523,947.55	\$6,005,207.32		161
2017 Oct	\$561,681.48	\$6,566,888.80		174
2017 Nov	\$4,764,920.16	\$11,331,808.96		230
2017 Dec	\$596,746.56	\$11,928,555.52		188
2018 Jan	\$1,327,674.63	\$13,256,230.15		193
2018 Feb	\$3,936,463.31	\$17,192,693.45		177
2018 Mar	\$700,873.18	\$17,893,566.64		219
2018 Apr	\$1,519,275.24	\$19,412,841.88		202
2018 May	\$2,960,378.09	\$22,373,219.97		222
2018 Jun	\$1,487,671.19	\$23,860,891.17		245

For the CustomersPrior variable, notice that the DATESBETWEEN function includes dates until the first date in filter context *minus* one. Because Power BI internally stores dates as numbers, you can add or subtract numbers to shift a date.

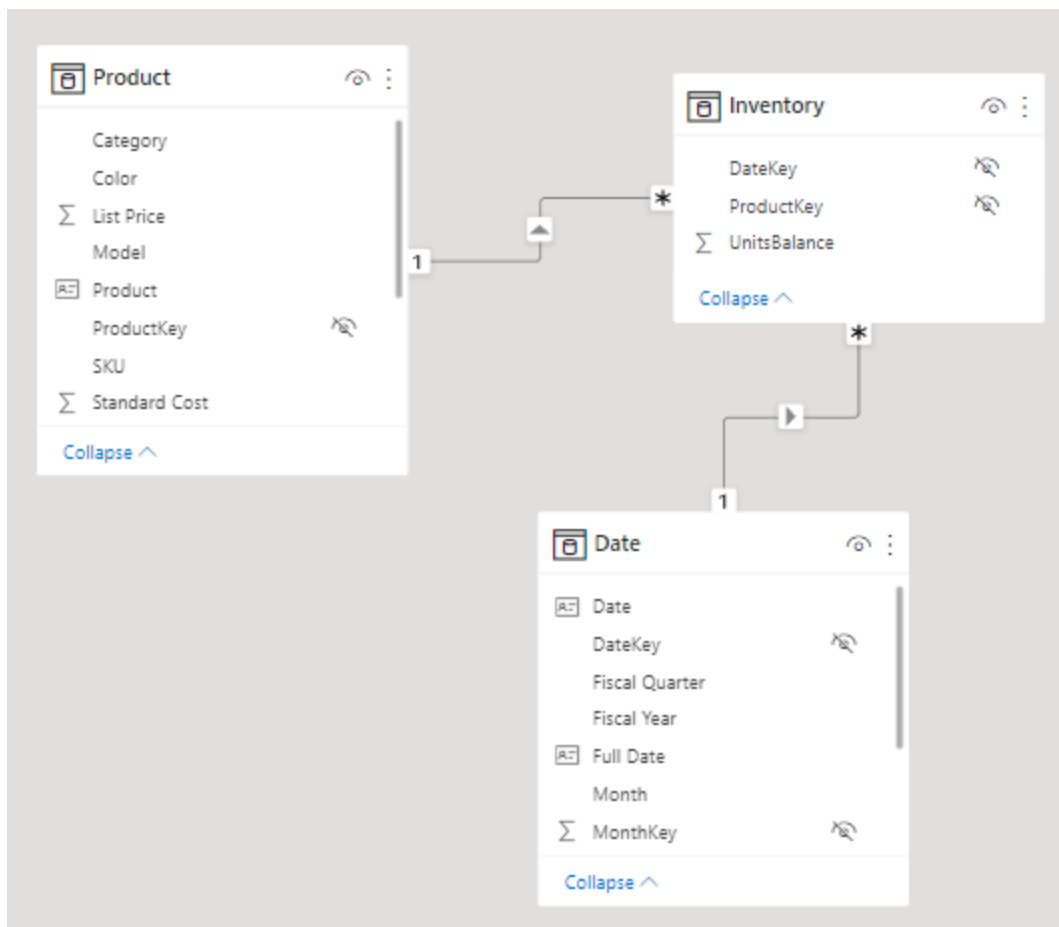
Snapshot calculations

Occasionally, fact data is stored as snapshots in time. Common examples include inventory stock levels or account balances. A snapshot of values is loaded into the table on a periodic basis.

When summarizing snapshot values (like inventory stock levels), you can summarize values across any dimension except date. Adding stock level counts across product categories produces a meaningful summary, but adding stock level counts across dates doesn't. Adding yesterday's stock level to today's stock level isn't a useful operation to perform (unless you want to average that result).

When you're summarizing snapshot tables, measure formulas can rely on DAX time intelligence functions to enforce a single date filter.

In the following example, you explore a scenario for the Adventure Works company. Switch to model view and select the **Inventory** model diagram.



Notice that the diagram shows three tables: Product, Date, and Inventory. The Inventory table stores snapshots of unit balances for each date and product.

Importantly, the table contains no missing dates and no duplicate entries for any product on the same date. Also, the last snapshot record is stored for the date of June 15, 2020.

Now, switch to report view and select **Page 2** of the report. Add the UnitsBalance column of the Inventory table to the matrix visual. Its default summarization is set to sum values.

FY2020 Mountain-200 Bike Stock													
Product	2019 Jul	2019 Aug	2019 Sep	2019 Oct	2019 Nov	2019 Dec	2020 Jan	2020 Feb	2020 Mar	2020 Apr	2020 May	2020 Jun	Total
Mountain-200 Black, 38	4,884	4,649	4,784	4,873	4,605	4,943	5,208	4,872	5,208	5,040	5,208	2,520	56,794
Mountain-200 Black, 42	5,248	5,175	5,246	5,382	5,200	5,415	5,177	4,843	5,177	5,010	5,177	2,505	59,555
Mountain-200 Black, 46	5,523	5,617	5,287	5,419	5,273	5,663	5,890	5,510	5,890	5,700	5,890	2,850	64,512
Mountain-200 Silver, 38	5,412	5,190	5,114	5,245	5,220	5,406	5,890	5,510	5,890	5,700	5,890	2,850	63,317
Mountain-200 Silver, 42	5,252	5,262	5,043	5,266	5,072	5,208	5,022	4,698	5,022	4,860	5,022	2,430	58,157
Mountain-200 Silver, 46	5,004	5,019	4,960	5,012	4,847	5,231	5,053	4,727	5,053	4,890	5,053	2,445	57,294
Total	31,323	30,912	30,434	31,197	30,217	31,866	32,240	30,160	32,240	31,200	32,240	15,600	359,629

This visual configuration is an example of how not to summarize a snapshot value. Adding daily snapshot balances together doesn't produce a meaningful result. Therefore, remove the UnitsBalance field from the matrix visual.

Now, you add a measure to the Inventory table that sums the UnitsBalance value *for a single date*. The date is the last date of each time period. It's achieved by using the LASTDATE function. Format the measure as a whole number with the thousands separator.

DAX

```

Stock on Hand =
CALCULATE(
    SUM(Inventory[UnitsBalance]),
    LASTDATE('Date'[Date])
)

```

Note

Notice that the measure formula uses the SUM function. An aggregate function must be used (measures don't allow direct references to columns), but given that only one row exists for each product for each date, the SUM function only operates over a single row.

Add the Stock on Hand measure to the matrix visual. The value for each product is now based on the last recorded units balance for each month.

FY2020 Mountain-200 Bike Stock													
Product	2019 Jul	2019 Aug	2019 Sep	2019 Oct	2019 Nov	2019 Dec	2020 Jan	2020 Feb	2020 Mar	2020 Apr	2020 May	2020 Jun	Total
Mountain-200 Black, 38	151	171	99	172	30	168	168	168	168	168	168		
Mountain-200 Black, 42	165	186	116	176	76	167	167	167	167	167	167		
Mountain-200 Black, 46	182	184	131	172	111	190	190	190	190	190	190		
Mountain-200 Silver, 38	171	173	129	190	85	190	190	190	190	190	190		
Mountain-200 Silver, 42	177	169	109	170	120	162	162	162	162	162	162		
Mountain-200 Silver, 46	181	158	126	178	88	163	163	163	163	163	163		
Total	1,027	1,041	710	1,058	510	1,040	1,040	1,040	1,040	1,040	1,040		

The measure returns BLANKs for June 2020 because no record exists for the last date in June. According to the data, it hasn't happened yet.

Filtering by the last date in filter context has inherent problems: A recorded date might not exist because it hasn't yet happened, or perhaps because stock balances aren't recorded on weekends.

Your next step is to adjust the measure formula to determine the last date *that has a non-BLANK result* and then filter by that date. You can achieve this task by using the [LASTNONBLANK](#) function.

Use the following measure definition to modify the Stock on Hand measure.

DAX

```

Stock on Hand =
CALCULATE(
    SUM(Inventory[UnitsBalance]),
    LASTNONBLANK(
        'Date'[Date],
        CALCULATE(SUM(Inventory[UnitsBalance]))
    )
)

```

In the matrix visual, notice the values for June 2020 and the total (representing the entire year).

2020 Apr	2020 May	2020 Jun	Total
168	168	168	168
167	167	167	167
190	190	190	190
190	190	190	190
162	162	162	162
163	163	163	163
1,040	1,040	1,040	1,040

The LASTNONBLANK function is an iterator function. It returns the last date that produces a non-BLANK result. It achieves this result by iterating through all dates in filter context *in descending chronological order*. (Conversely, the FIRSTNONBLANK iterates in ascending chronological order.) For each date, it evaluates the passed in expression. When it encounters a non-BLANK result, the function returns the date. That date is then used to filter the CALCULATE function.

Note

The LASTNONBLANK function evaluates its expression in row context. The CALCULATE function must be used to transition the row context to filter context to correctly evaluate the expression.

You should now hide the UnitsBalance column in the Inventory table. It prevents report authors from inappropriately summarizing snapshot unit balances.

Exercise - Use time intelligence functions

DAX time intelligence functions in Power BI

Lab story

In this lab, you'll create measures with DAX expressions that involve time intelligence.

In this lab, learn how to:

- Use various time intelligence functions to manipulate filter context that specific concerns dates.

This lab should take approximately 15 minutes.

Get started

To complete this exercise, first open a web browser and enter the following URL to download the zip file:

<https://github.com/MicrosoftLearning/PL-300-Microsoft-Power-BI-Data-Analyst/raw/Main/Allfiles/Labs/06-use-dax-time-intelligence/06-time-intelligence.zip>

Extract the file to the **C:\Users\Student\Downloads\06-time-intelligence** folder.

Open the **06-Starter-Sales Analysis.pbix** file.

Note: You can dismiss the sign-in by selecting **Cancel**. Close any other informational windows. Select **Apply Later**, if prompted to apply changes.

Create a YTD measure

In this task, you'll create a sales year-to-date (YTD) measure by using time intelligence functions.

1. In Power BI Desktop, in Report view, on **Page 2**, notice the matrix visual that displays various measures with years and months grouped on the rows.
2. Add a measure to the Sales table, based on the following expression, and formatted to zero decimal places:

codeCopy

Sales YTD =

TOTALYTD(

 SUM(Sales[Sales]),

 'Date'[Date],

 "6-30"

)

The TOTALYTD function evaluates an expression—in this case the sum of the Sales column—over a given date column. The date column must belong to a date table marked as a date table.

The function can also take a third optional argument representing the last date of a year. The absence of this date means that December 31 is the last date of the year. For Adventure Works, June is in the last month of their year, and so “6-30” is used.

3. Add the Sales field and the Sales YTD measure to the matrix visual.
4. Notice the accumulation of sales values within the year.

Sum of Sales	Sales YTD
\$16,429,043	\$16,429,043
\$489,328	\$489,328
\$1,540,072	\$2,029,400
\$1,166,332	\$3,195,733
\$844,833	\$4,040,566
\$2,325,755	\$6,366,320
\$1,703,435	\$8,069,756
\$713,230	\$8,782,985
\$1,900,794	\$10,683,780
\$1,455,280	\$12,139,060
\$883,011	\$13,022,071
\$2,269,720	\$15,291,791
\$1,137,252	\$16,429,043
\$27,979,780	\$27,979,780

The *TOTALYTD* function performs filter manipulation, specifically time filter manipulation. For example, to compute YTD sales for September 2017 (the third month of the fiscal year), all filters on the Date table are removed and replaced with a new filter of dates commencing at the beginning of the year (July 1, 2017) and extending through to the last date of the in-context date period (September 30, 2017).

Many [time intelligence functions](#) are available in DAX to support common time filter manipulations.

Create a YoY growth measure

In this task, you'll create a sales YoY growth measure by using a variable.

Variables help you simplify the formula and are more efficient if using the logic multiple times within a formula. Variables are declared with a unique name, and the measure expression must then be output after the RETURN keyword. Unlike some other coding language variables, DAX variables can only be used within the single formula._

1. Add another measure to the Sales table, based on the following expression:

codeCopy

Sales YoY Growth =

```

VAR SalesPriorYear =
    CALCULATE(
        SUM(Sales[Sales]),
        PARALLELPERIOD(
            'Date'[Date],
            -12,
            MONTH
        )
    )
RETURN
    SalesPriorYear

```

The SalesPriorYear variable is assigned an expression that calculates the sum of the Sales column in a modified context. That context uses the PARALLELPERIOD function to shift 12 months back from each date in filter context.

2. Add the Sales YoY Growth measure to the matrix visual.
3. Notice that the new measure returns BLANK for the first 12 months (because there were no sales recorded before fiscal year 2017).
4. Notice that the Sales YoY Growth measure value for 2018 Jul is the sales value for 2017 Jul.

Sum of Sales	Sales YTD	Sales YoY Growth
\$16,429,043	\$16,429,043	
\$489,328	\$489,328	
\$1,540,072	\$2,029,400	
\$1,166,333	\$3,195,733	
\$844,833	\$4,040,566	
\$2,325,755	\$6,366,320	
\$1,703,435	\$8,069,756	
\$713,230	\$8,782,985	
\$1,900,794	\$10,683,780	
\$1,455,280	\$12,139,060	
\$883,011	\$13,022,071	
\$2,269,720	\$15,291,791	
\$1,137,252	\$16,429,043	
\$27,979,780	\$27,979,780	\$16,429,042.6
\$2,411,559	\$2,411,559	\$489,328.4
\$3,615,914	\$6,027,473	\$1,540,072.02

Now that the “difficult part” of the formula has been tested, you can overwrite the measure with the final formula that computes the growth result.

- To complete the measure, overwrite the Sales YoY Growth measure with this formula, formatting it as a percentage with two decimal places:

codeCopy

Sales YoY Growth =

VAR SalesPriorYear =

CALCULATE(

SUM(Sales[Sales]),

PARALLELPERIOD(

'Date'[Date],

-12,

MONTH

)

)

RETURN

DIVIDE(

(SUM(Sales[Sales]) - SalesPriorYear),

SalesPriorYear

)

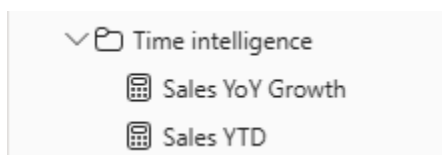
6. In the formula, in the RETURN clause, notice that the variable is referenced twice.

7. Verify that the YoY growth for 2018 Jul is 392.83 percent.

Sum of Sales	Sales YTD	Sales YoY Growth
\$16,429,043	\$16,429,043	
\$489,328	\$489,328	
\$1,540,072	\$2,029,400	
\$1,166,332	\$3,195,733	
\$844,833	\$4,040,566	
\$2,325,755	\$6,366,320	
\$1,703,435	\$8,069,756	
\$713,230	\$8,782,985	
\$1,900,794	\$10,683,780	
\$1,455,280	\$12,139,060	
\$883,011	\$13,022,071	
\$2,269,720	\$15,291,791	
\$1,137,252	\$16,429,043	
\$27,979,780	\$27,979,780	70.31%
\$2,411,559	\$2,411,559	392.83%
\$3,615,914	\$6,027,473	134.79%

The YoY growth measure identifies almost 400 percent (or 4x) increase of sales during the same period of the previous year.

8. In Model view, place the two new measures into a display folder named *Time intelligence*.



9. Save the Power BI Desktop file.

Lab complete

Knowledge check

Summary

In this module, you learned how time intelligence calculations work by changing the filter context for date filters. You learned about several DAX time intelligence functions. These functions help you create calculations like year-to-date (YTD) and year-over-year (YoY).

You also learned about life-to-date (LTD) calculations. LTD calculations help you count new occurrences in your fact data. Additionally, you saw how to filter snapshot data to ensure that only a single snapshot value is returned.

Model data with Power BI – Archived

Design a semantic model in Power BI

Introduction

Creating a great semantic model is one of the most important tasks that a data analyst can perform in Microsoft Power BI. By [doing this job well](#), you help make it easier for people to understand your data, which will [make building valuable Power BI reports easier for them and for you](#).

The pages in this module are instructional only, no data files are provided. You have a chance to work with real data in the labs.

A good semantic model offers the following [benefits](#):

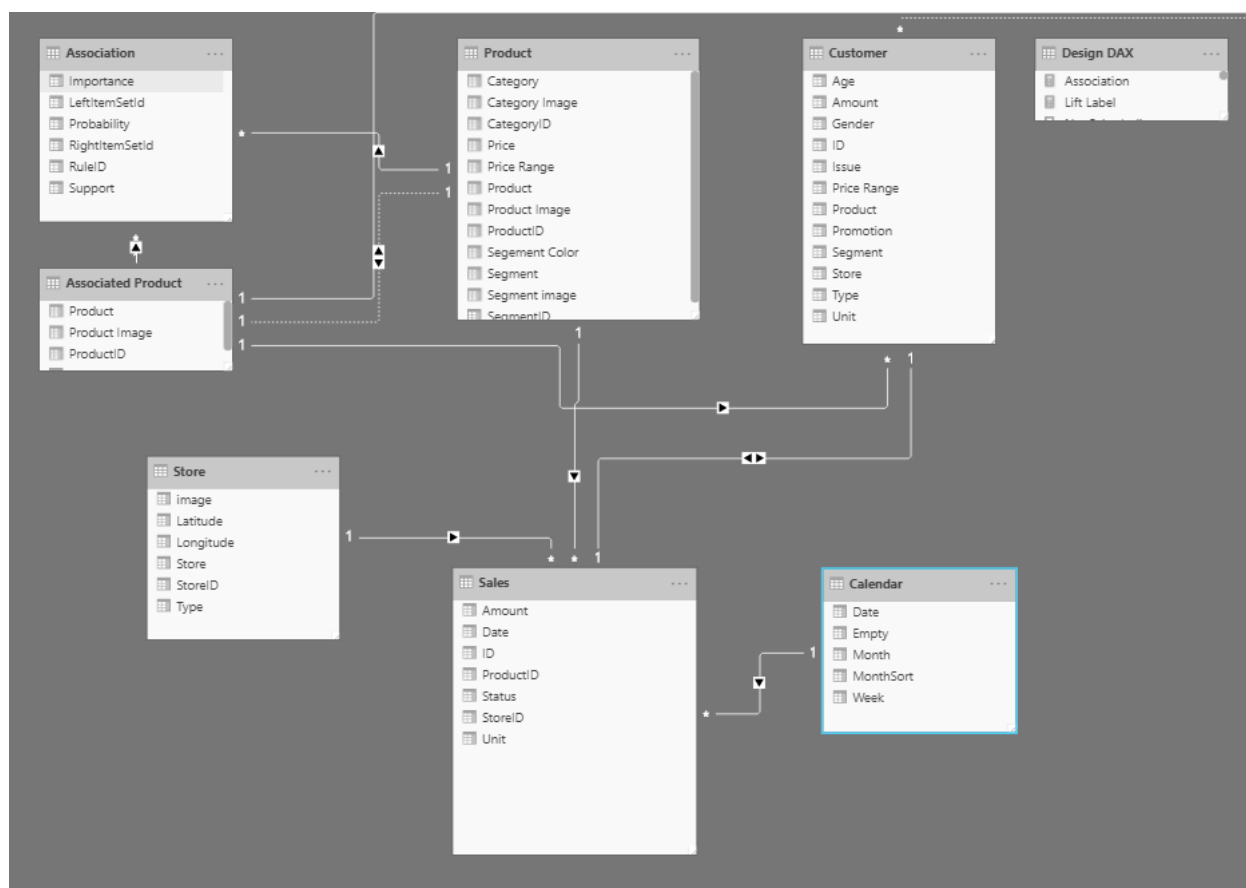
- Data [exploration is faster](#).
- [Aggregations are simpler](#) to build.
- Reports are [more accurate](#).
- Writing [reports takes less time](#).
- Reports are [easier to maintain](#) in the future.

Providing set rules for what makes a [good semantic model is difficult because all data is different](#), and the usage of that data varies. Generally, a [smaller semantic model is better](#)

because it performs faster and will be simpler to use. However, defining what a smaller semantic model entails is equally as problematic because it's a heuristic and subjective concept.

Typically, a smaller semantic model is composed of fewer tables and fewer columns in each table that the user can see. If you import all necessary tables from a sales database, but the total table count is 30 tables, the user will not find that intuitive. Collapsing those tables into five tables make the semantic model more intuitive to the user, whereas if the user opens a table and finds 100 columns, they might find it overwhelming. Removing unneeded columns to provide a more manageable number increases the likelihood that the user reads all column names. To summarize, you should aim for simplicity when designing your semantic models.

The following image is an example semantic model. The boxes contain tables of data, where each line item within the box is a column. The lines that connect the boxes represent relationships between the tables. These relationships can be complex, even in such a simplistic model. The semantic model can become easily disorganized, and the total table count in the model can gradually increase. Keeping your semantic model simple, comprehensive, and accurate requires constant effort.



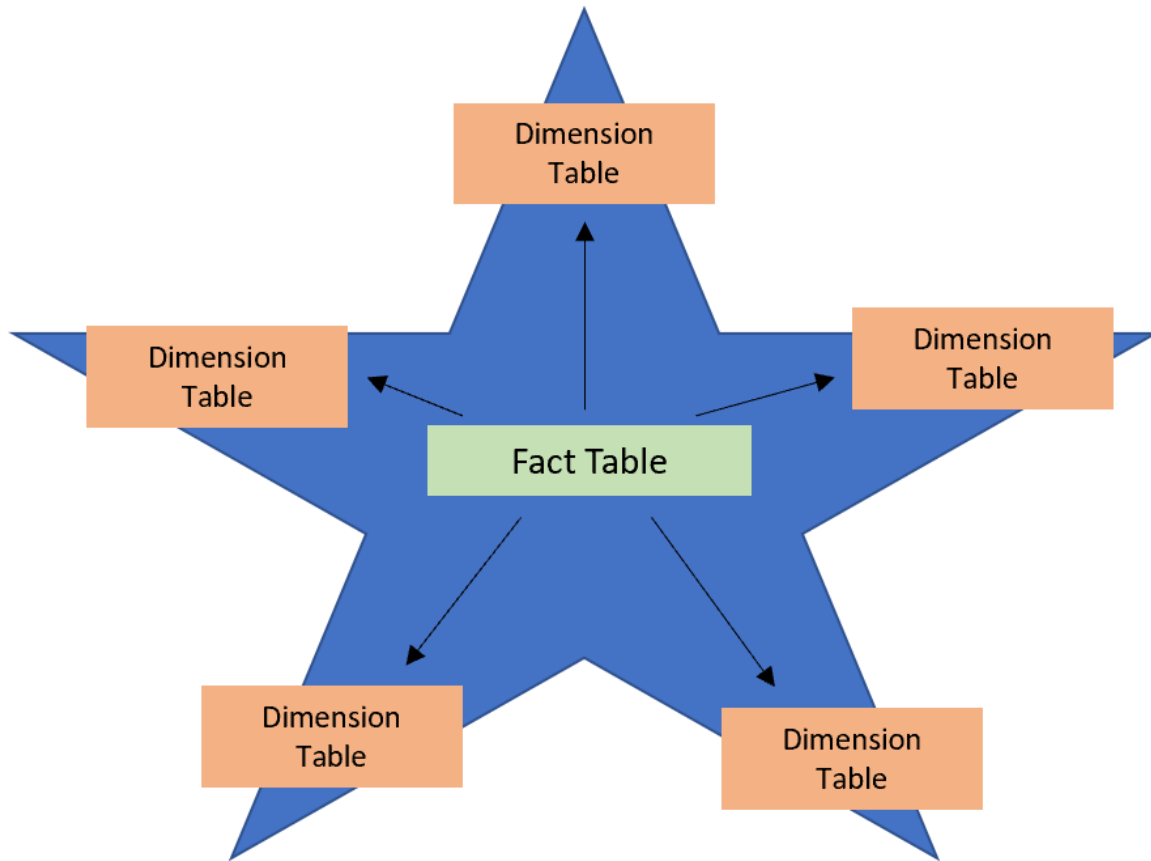
Relationships are defined between tables through primary and foreign keys. Primary keys are column(s) that identify each unique, non-null data row. For instance, if you have a Customers table, you could have an index that identifies each unique customer. The first row has an ID of 1, the second row an ID of 2, and so on. Each row is assigned a unique value, which can be referred to by this simple value: the primary key. This process becomes important when you are referencing rows in a different table, which is what foreign keys do. Relationships between tables are formed when you have primary and foreign keys in common between different tables.

Power BI allows relationships to be built from tables with different data sources, a powerful function that enables you to pull one table from Microsoft Excel and another from a relational database. You would then create the relationship between those two tables and treat them as a unified semantic model.

Now that you have learned about the relationships that make up the data schema, you are able to explore a specific type of schema design, the star schema, which is optimized for high performance and usability.

Star schemas

You can design a star schema to simplify your data. It's not the only way to simplify your data, but it is a popular method; therefore, every Power BI data analyst should understand it. In a star schema, each table within your semantic model is defined as a dimension or a fact table, as shown in the following visual.



Fact tables contain observational or event data values: sales orders, product counts, prices, transactional dates and times, and quantities. Fact tables can contain several repeated values. For example, one product can appear multiple times in multiple rows, for different customers on different dates. These values can be aggregated to create visuals. For instance, a visual of the total sales orders is an aggregation of all sales orders in the fact table. With fact tables, it is common to see columns that are filled with numbers and dates. The numbers can be units of measurement, such as sale amount, or they can be keys, such as a customer ID. The dates represent time that is being recorded, like order date or shipped date.

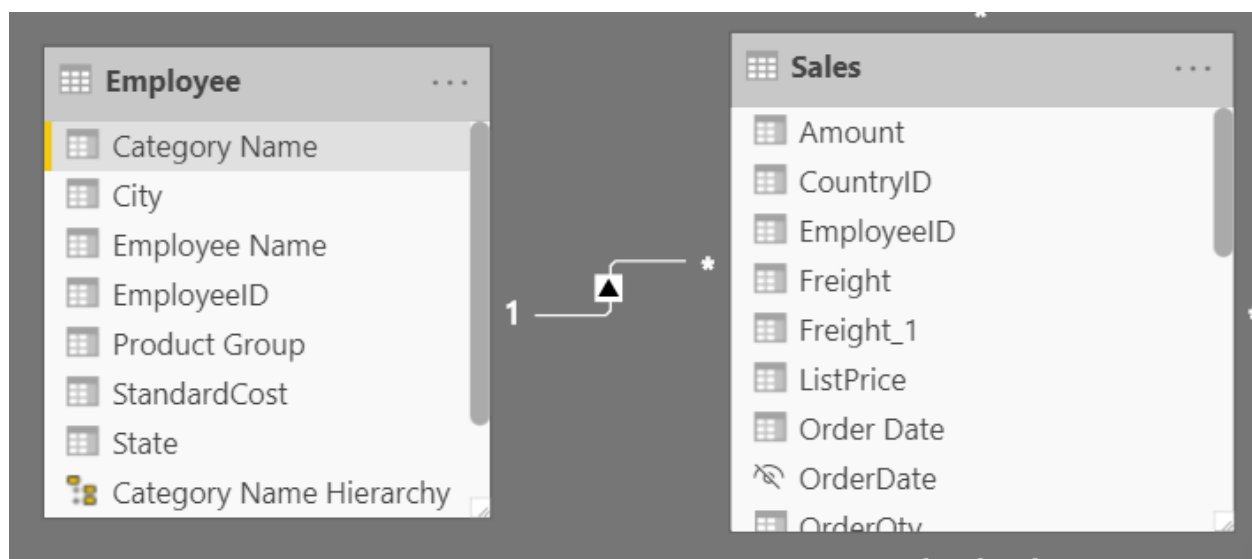
Dimension tables contain the details about the data in fact tables: products, locations, employees, and order types. These tables are connected to the fact table through key columns. Dimension tables are used to filter and group the data in fact tables. The fact tables, on the other hand, contain the measurable data, such as sales and revenue, and

each row represents a unique combination of values from the dimension tables. For the total sales orders visual, you could group the data so that you see total sales orders by product, in which product is data in the dimension table.

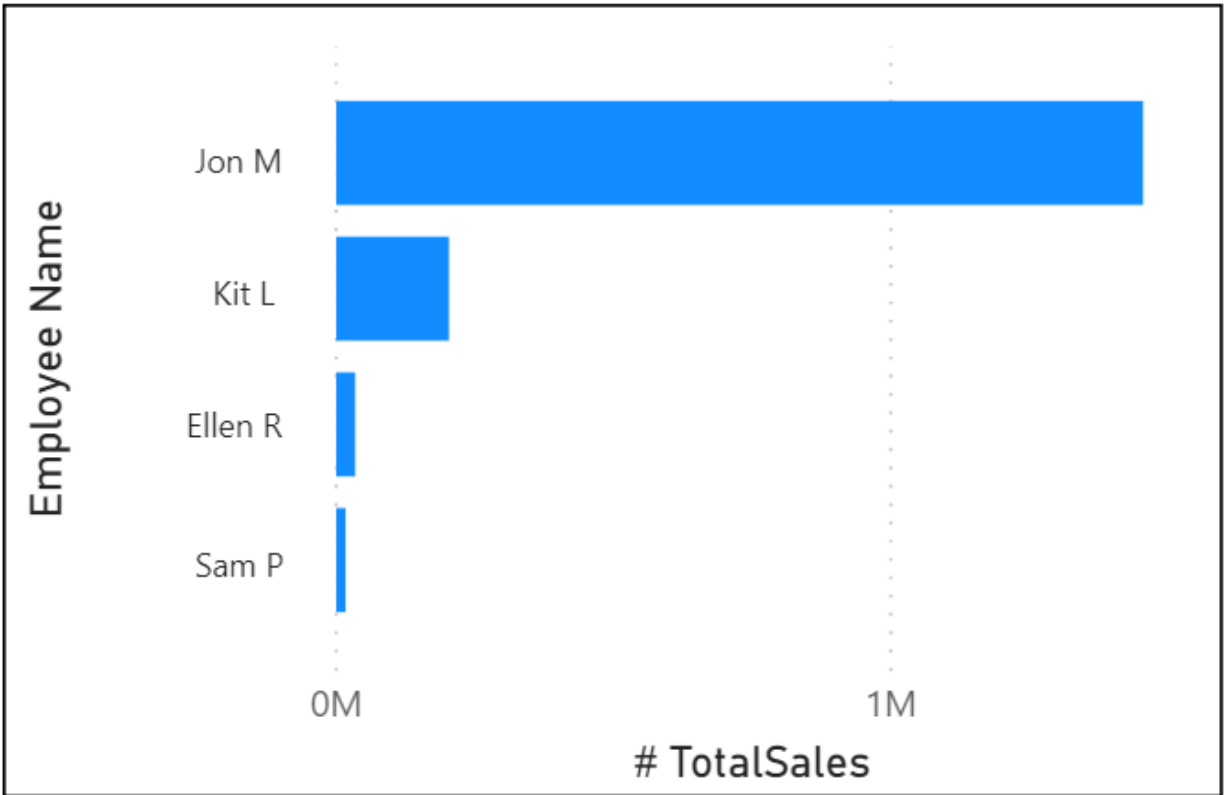
Fact tables are much larger than dimension tables because numerous events occur in fact tables, such as individual sales. Dimension tables are typically smaller because you are limited to the number of items that you can filter and group on. For instance, a year contains only so many months, and the United States are composed of only a certain number of states.

Considering this information about fact tables and dimension tables, you might wonder how you can build this visual in Power BI.

The pertinent data resides in two tables, Employee and Sales, as shown in the following semantic model. Because the Sales table contains the sales order values, which can be aggregated, it is considered a fact table. The Employee table contains the specific employee name, which filters the sales orders, so it would be a dimension table. The common column between the two tables, which is the primary key in the Employee table, is EmployeeID, so you can establish a relationship between the two tables based on this column.



When creating this relationship, you can build the visual according to the requirements, as shown in the following figure. If you did not establish this relationship, while keeping in mind the commonality between the two tables, you would have had more difficulty building your visual.



Star schemas and the underlying semantic model are the foundation of organized reports; the more time you spend creating these connections and design, the easier it will be to create and maintain reports.

Work with tables

When users see fewer tables, they will enjoy using your semantic model considerably more. For example, suppose you've imported dozens of tables from many data sources and now the visual appears disorderly. In this case, you need to ensure that, before you begin working on building reports, your semantic model and table structure are simplified.

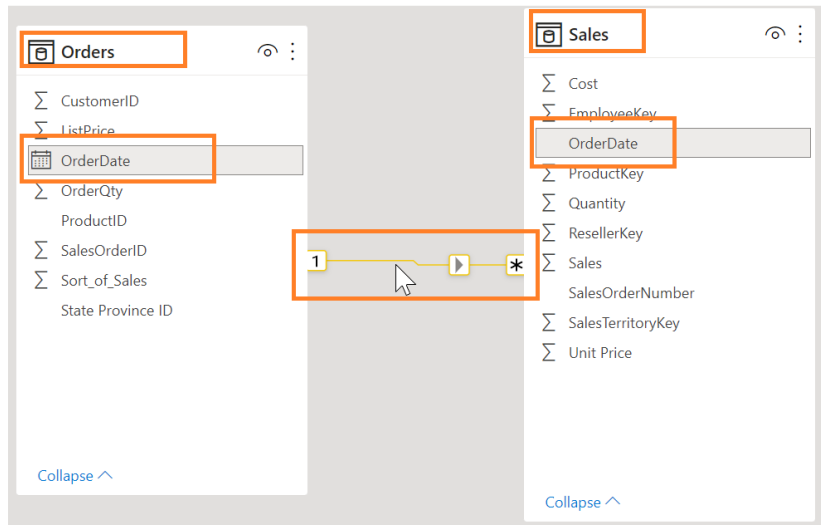
A simple table structure will:

- Be simple to navigate because of column and table properties that are specific and user-friendly.
- Have merged or appended tables to simplify the tables within your data structure.
- Have good-quality relationships between tables that make sense.

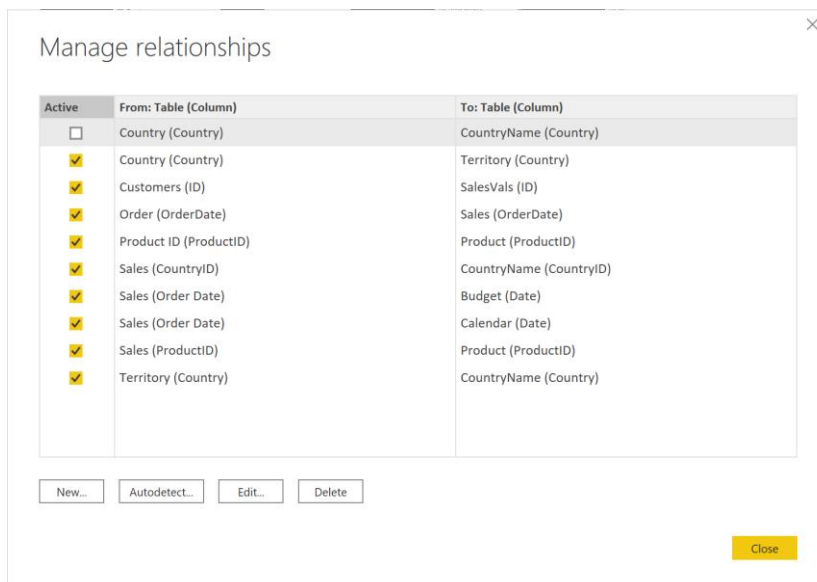
The following sections further explain how you might work with your tables to ensure a simple and readable table structure.

Configure semantic model and build relationships between tables

Assuming that you've already retrieved your data and cleaned it in Power Query, you can then go to the **Model** tab, where the **semantic model is located**. The following image shows how the **relationship** between the **Order** and **Sales** tables can be seen through the **OrderDate** column.



To manage these relationships, go to **Manage Relationships** on the ribbon, where the following window will appear.



In this view, you can create, edit, and delete relationships between tables and also autodetect relationships that already exist. When you load your data into Power BI, the **Autodetect** feature will help you establish relationships between columns that are

named similarly. Relationships can be inactive or active. Only one active relationship can exist between tables, which is discussed in a future module.

While the **Manage Relationships** feature allows you to configure relationships between tables, you can also configure table and column properties to ensure organization in your table structure.

Configure table and column properties

The **Model** view in Power BI desktop provides many options within the column properties that you can view or update. A simple method to get to this menu to update the tables and fields is by Ctrl+clicking or Shift+clicking items on this page.

The screenshot shows the 'Properties' pane in Power BI, specifically the 'General' tab for a column named 'Date'. The pane is divided into three sections: General, Formatting, and Advanced. The General section includes fields for Name (Date), Description (Enter a description), Synonyms (date), Display folder (Enter the display folder), and Is hidden (No, with a toggle switch). The Formatting section includes Data type (Date) and Date time format (Wednesday, March 14, 2001 (dddd, mmmr)). The Advanced section includes Sort by column (Date (Default)).

Properties >

General

Name
Date

Description
Enter a description

Synonyms
date

Display folder
Enter the display folder

Is hidden
No ☐

Formatting

Data type
Date ▼

Date time format
Wednesday, March 14, 2001 (dddd, mmmr ▼)

Advanced

Sort by column
Date (Default) ▼

Under the **General** tab, you can:

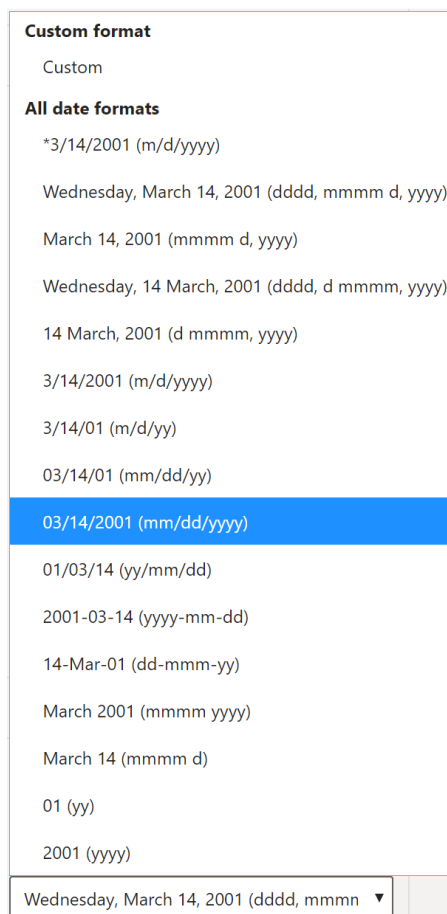
- Edit the name and description of the column.

- Add synonyms that can be used to identify the column when you are using the Q&A feature.
- Add a column into a folder to further organize the table structure.
- Hide or show the column.

Under the **Formatting** tab, you can:

- Change the data type.
- Format the date.

For instance, suppose that the dates in your column are formatted, as seen in the [previous screenshot](#), in the form of "Wednesday, March 14, 2001". If you want to change the format so that the date was in the "mm/dd/yyyy" format, you would select the drop-down menu under **All date time formats** and then choose the appropriate date format, as shown in the following figure.



After selecting the appropriate date format, return to the **Date** column, where you should see that the format has indeed changed, as shown in the following figure.

Date
01/01/2019
01/06/2019
01/13/2019
01/20/2019
01/27/2019
02/03/2019
02/10/2019

Under the **Advanced** tab, you can:

- **Sort** by a specific column.
- **Assign a specific category** to the data.
- **Summarize** the data.
- **Determine** if the column or table **contains null values**.

Additionally, Power BI has a new functionality to update these properties on many tables and fields by Ctrl+clicking or Shift+clicking items.

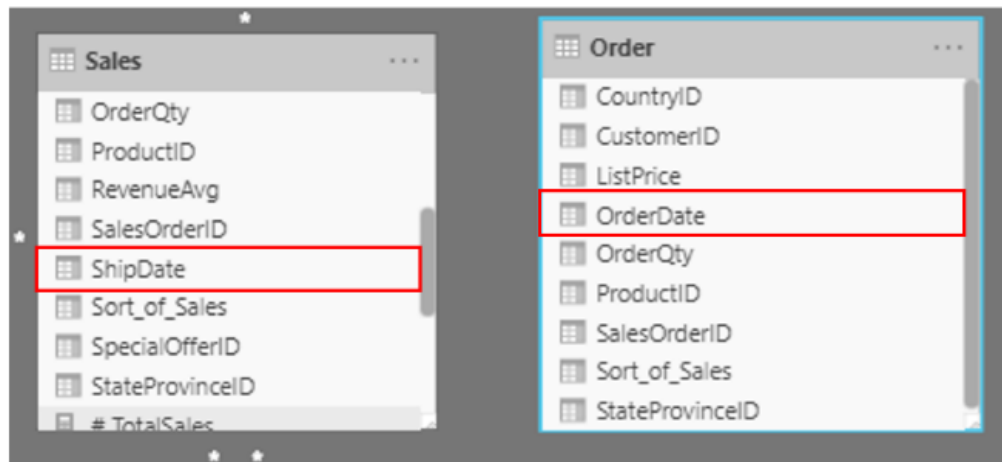
These **examples are only some of the many types of transformations** that you can make to simplify the table structure. **This step is important to take before you begin making your visuals** so that you **don't have to go back and forth** when making formatting changes. This process of formatting and configuring tables **can also be done in Power Query**.

Create a date table

During report creation in Power BI, a **common business requirement is to make calculations based on date and time**. Organizations want to know **how their business is doing over months, quarters, fiscal years, and so on**. For this reason, it is **crucial that these time-oriented values are formatted correctly**. **Power BI autodetects for date columns and tables**; however, **situations can occur where you will need to take extra steps** to get the dates in the format that your organization requires.

For example, suppose that **you are developing reports for the Sales team** at your organization. The database contains **tables for sales, orders, products, and more**. You notice that **many of these tables**, including Sales and Orders, **contain their own date columns**, as shown by the **ShipDate** and **OrderDate** columns in the Sales and Orders tables. **You are**

tasked with developing a table of the total sales and orders by year and month. How can you build a visual with multiple tables, each referencing their own date columns?



To solve this problem, you can create a common date table that can be used by multiple tables. The following section explains how you can accomplish this task in Power BI.

Create a common date table

Ways that you can build a common date table are:

- Source data
- DAX
- Power Query

Source data

Occasionally, source databases and data warehouses already have their own date tables. If the administrator who designed the database did a thorough job, these tables can be used to perform the following tasks:

- Identify company holidays
- Separate calendar and fiscal year
- Identify weekends versus weekdays

Source data tables are mature and ready for immediate use. If you have a table as such, bring it into your semantic model and don't use any other methods that are outlined in this section. We recommend that you use a source date table because it is likely shared with other tools that you might be using in addition to Power BI.

If you do not have a source data table, you can use other ways to build a common date table.

DAX

You can use the Data Analysis Expression (DAX) functions `CALENDARAUTO()` or `CALENDAR()` to build your common date table. The `CALENDAR()` function returns a contiguous range of dates based on a start and end date that are entered as arguments in the function. Alternatively, the `CALENDARAUTO()` function returns a contiguous, complete range of dates that are automatically determined from your semantic model. The starting date is chosen as the earliest date that exists in your semantic model, and the ending date is the latest date that exists in your semantic model plus data that has been populated to the fiscal month that you can choose to include as an argument in the `CALENDARAUTO()` function. For the purposes of this example, the `CALENDAR()` function is used because you only want to see the data from May 31, 2011 (the first day that Sales began its tracking of this data) and forward for the next 10 years.

In Power BI Desktop, select **New Table**, and then enter in the following DAX formula:

DAX

```
Dates = CALENDAR(DATE(2011, 5, 31), DATE(2022, 12, 31))
```

```
1 Dates = CALENDAR(DATE(2011, 5, 31), DATE(2022, 12, 31))
```

Now, you have a column of dates that you can use. However, this column is slightly sparse. You also want to see columns for just the year, the month number, the week of the year, and the day of the week. You can accomplish this task by selecting **New Column** on the ribbon and entering the following DAX equation, which will retrieve the year from your Date table.

DAX

```
Year = YEAR(Dates[Date])
```

1 Year = YEAR(Dates[Date])	
Date	Year
Tuesday, May 31, 2011	2011
Wednesday, June 1, 2011	2011
Thursday, June 2, 2011	2011
Friday, June 3, 2011	2011
Saturday, June 4, 2011	2011

You can perform the same process to retrieve the month number, week number, and day of the week:

DAX

```
MonthNum = MONTH(Dates[Date])
```

DAX

```
WeekNum = WEEKNUM(Dates[Date])
```

DAX

```
DayoftheWeek = FORMAT(Dates[Date], "DDDD")
```

When you have finished, your table will contain the columns that are shown in the following figure.

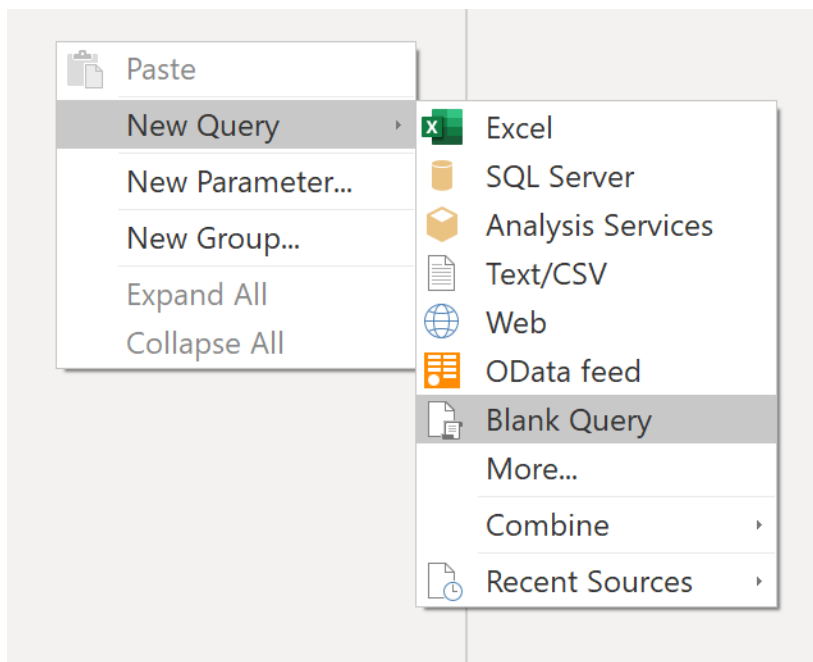
Date	Year	MonthNum	WeekNum	DayoftheWeek
Tuesday, May 31, 2011	2011	5	23	Tuesday
Wednesday, June 1, 2011	2011	6	23	Sunday
Thursday, June 2, 2011	2011	6	23	Monday
Friday, June 3, 2011	2011	6	23	Tuesday

You have now created a common date table by using DAX. This process only adds your new table to the semantic model; you will still need to establish relationships between your date table and the Sales and Order tables, and then mark your table as the official date table of your semantic model. However, before you complete those tasks, make sure that you consider another way of building a common date table: by using Power Query.

Power Query

You can use M-language, the development language that is used to build queries in Power Query, to define a common date table.

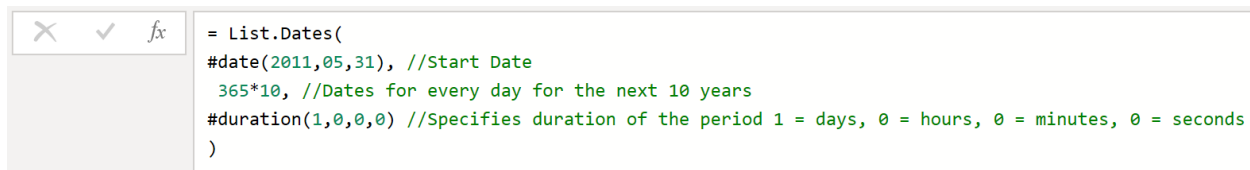
Select **Transform Data** in Power BI Desktop, which will direct you to Power Query. In the blank space of the left **Queries** pane, right-click to open the following drop-down menu, where you will select **New Query > Blank Query**.



In the resulting **New Query** view, enter the following M-formula to build a calendar table:

M

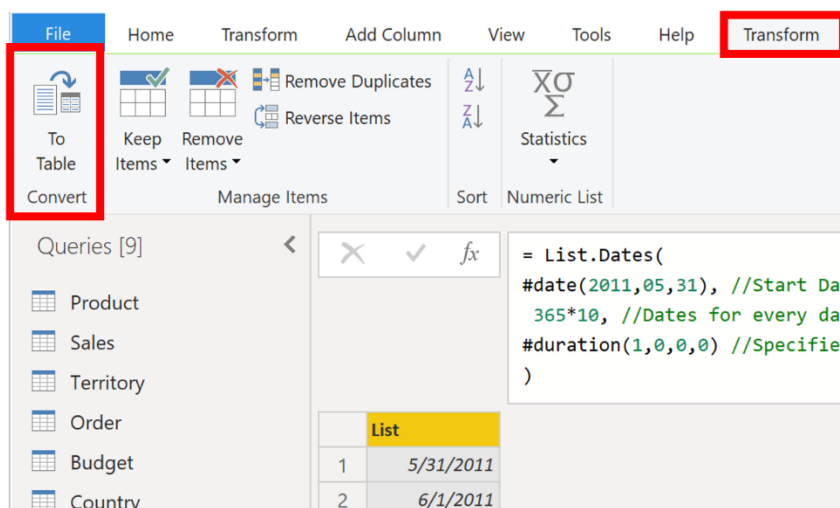
```
= List.Dates(#date(2011,05,31), 365*10, #duration(1,0,0,0))
```



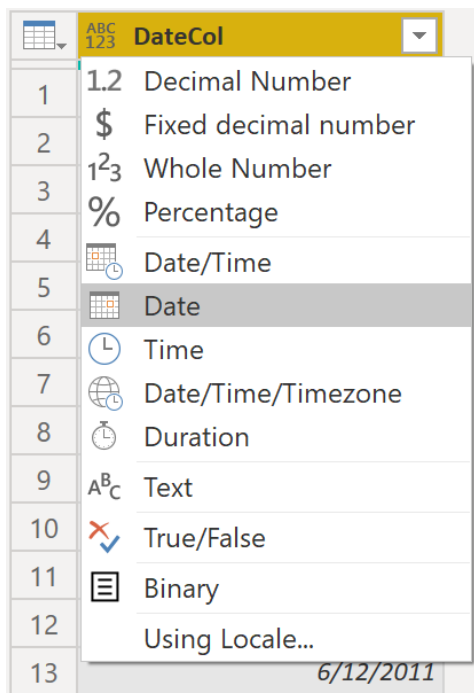
For your sales data, you want the **start date to reflect the earliest date** that you have in your data: **May 31, 2011**. Additionally, you want to **see dates for the next 10 years**, including dates in the future. This **approach ensures that, as new sales data flows in, you won't have to re-create this table**. You can **also change duration**. In this case, you want a data point for every **day**, but you can also increment by **hours, minutes, and seconds**. The following figure shows the result.

	List
1	5/31/2011
2	6/1/2011
3	6/2/2011
4	6/3/2011
5	6/4/2011
6	6/5/2011
7	6/6/2011
8	6/7/2011
9	6/8/2011

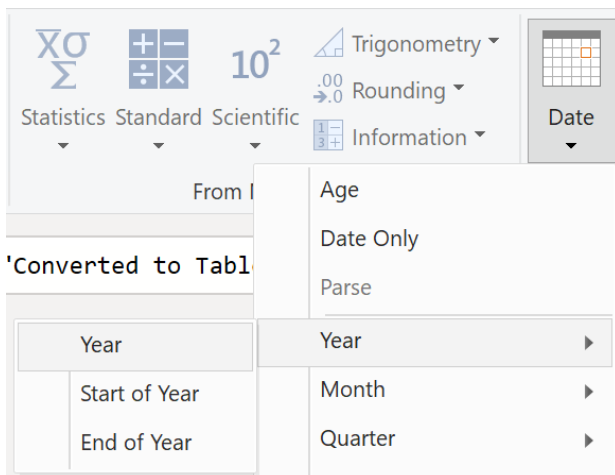
After you have realized success in the process, you notice that you have a list of dates instead of a table of dates. To correct this error, go to the **Transform** tab on the ribbon and select **Convert > To Table**. As the name suggests, this feature will convert your list into a table. You can also rename the column to **DateCol**.



Next, you want to add columns to your new table to see dates in terms of year, month, week, and day so that you can build a hierarchy in your visual. Your first task is to change the column type by selecting the icon next to the name of the column and, in the resulting drop-down menu, selecting the **Date** type.



After you have finished selecting the **Date** type, you can add columns for year, months, weeks, and days. Go to **Add Column**, select the drop-down menu under **Date**, and then select **Year**, as shown in the following figure.



Notice that Power BI has added a column of all years that are pulled from **DateCol**.

DateCol	Year
5/31/2011	2011
6/1/2011	2011
6/2/2011	2011

Complete the **same process for months, weeks, and days**. After you have finished this process, the table will contain the columns that are shown in the following figure.

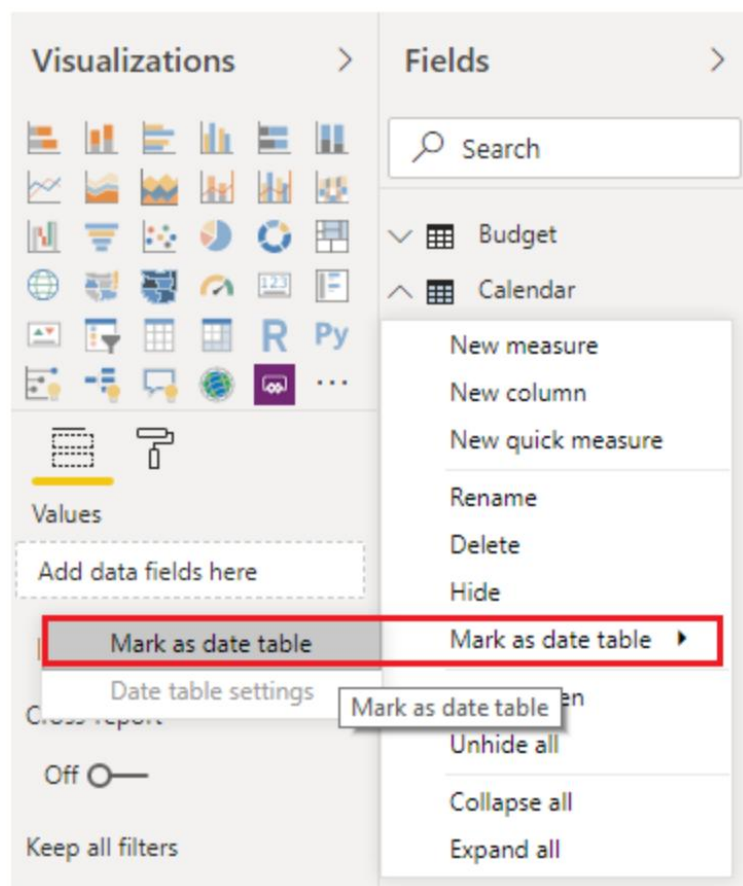
	DateCol	¹ / ₂ / ₃ Year	¹ / ₂ / ₃ Month	¹ / ₂ / ₃ Week of Year	^A / _B / _C Day Name
1	5/31/2011	2011		5	23 Tuesday
2	6/1/2011	2011		6	23 Wednesday
3	6/2/2011	2011		6	23 Thursday

You have **now successfully used Power Query to build a common date table**.

The **previous steps show how to get the table into the semantic model**. Now, you need to **mark your table as the official date table so that Power BI can recognize it for all future values and ensure that formatting is correct**.

Mark as the official date table

Your first task in marking your table as the official date table is to find the **new table on the Fields pane**. **Right-click** the name of the table and then select **Mark as date table**, as shown in the following figure.



By marking your table as a date table, Power BI performs validations to ensure that the data contains zero null values, is unique, and contains continuous date values over a period. You

can also choose specific columns in your table to mark as the date, which can be useful when you have many columns within your table. Right-click the table, select **Mark as date table**, and then select **Date table settings**. The following window will appear, where you can choose which column should be marked as **Date**.

Mark as date table

Select a column to be used for the date. The column must be of the data type 'date' and must contain only unique values. [Learn more](#)

Date column

Date ▼

Validated successfully

ⓘ

When you mark this as a date table, the built-in date tables that were associated with this table are removed. Visuals or DAX expressions referring to them may break.
[Learn how to fix visuals and DAX expressions](#)

OK

Cancel

Selecting **Mark as date table** will remove autogenerated hierarchies from the **Date** field in the table that you marked as a date table. For other date fields, the auto hierarchy will still be present until you establish a relationship between that field and the date table or until you turn off the **Auto Date/Time** feature. You can manually add a hierarchy to your common date table by right-clicking the year, month, week, or day columns in the **Fields pane** and then selecting **New hierarchy**. This process is further discussed later in this module.

Build your visual

To build your visual between the Sales and Orders tables, you will need to establish a relationship between this new common date table and the Sales and Orders tables. As a result, you will be able to build visuals by using the new date table. To complete this task, go to **Model tab > Manage Relationships**, where you can create relationships between the common date table and the Orders and Sales tables by using the **OrderDate** column. The following screenshot shows an example of one such relationship.

Create relationship

Select tables and columns that are related.

Sales

SalesOrderID	OrderDate	Sort_of_Sales	Freight	Freight_1	ProductID	OrderQty	SpecialOfferID
52242	Sunday, July 7, 2013	Internet	0.1248	0.1248	870	1	
52592	Sunday, July 14, 2013	Internet	0.1248	0.1248	870	1	
52694	Tuesday, July 16, 2013	Internet	0.1248	0.1248	870	1	

DatesTable

DateCol	Year	Month	Week of Year	Day Name
Tuesday, May 31, 2011	2011	5	23	Tuesday
Tuesday, June 7, 2011	2011	6	24	Tuesday
Tuesday, June 14, 2011	2011	6	25	Tuesday

Cardinality: Many to one (*:1)

Cross filter direction: Both

☒ Make this relationship active

☐ Assume referential integrity

☐ Apply security filter in both directions

OK Cancel

After you have built the relationships, you can build your **Total Sales and Order Quantity by Time** visual with your common date table that you developed by using the DAX or Power Query method.

To determine the total sales, you need to add all sales because the **Amount** column in the Sales table only looks at the revenue for each sale, not the total sales revenue. You can complete this task by using the following measure calculation, which will be explained in later discussions. The calculation that you will use when building this measure is as follows:

DAX

```
#Total Sales = SUM(Sales[Amount])
```

After you have finished, you can create a table by returning to the **Visualizations** tab and selecting the **Table** visual. You want to see the total orders and sales by year and month, so you only want to include the **Year** and **Month** columns from your date table, the **OrderQty** column, and the **#TotalSales** measure. When you learn about hierarchies, you can also build a hierarchy that will allow you drill down from years to months. For this example, you can view them side-by-side. You have now successfully created a visual with a common date table.

Year	Month	OrderQty	# TotalSales
2011	5	825	853,422
2011	6	141	460,085
2011	7	2209	3,130,880
2011	8	2904	3,917,345
2011	9	157	503,668
2011	10	5382	7,426,033
2011	11	230	740,105
2011	12	1040	1,815,966
2012	1	3967	6,319,337
2012	2	1442	2,106,429
2012	3	3184	4,575,015
Total		274914	170,964,700

Work with dimensions

When building a star schema, you will have dimension and fact tables. Fact tables contain information about events such as sales orders, shipping dates, resellers, and suppliers. Dimension tables store details about business entities, such as products or time, and are connected back to fact tables through a relationship.

You can use hierarchies as one source to help you find detail in dimension tables. These hierarchies form through natural segments in your data. For instance, you can have a hierarchy of dates in which your dates can be segmented into years, months, weeks, and days. Hierarchies are useful because they allow you to drill down into the specifics of your data instead of only seeing the data at a high level.

Hierarchies

When you are building visuals, Power BI automatically enters values of the date type as a hierarchy (if the table has not been marked as a date table).

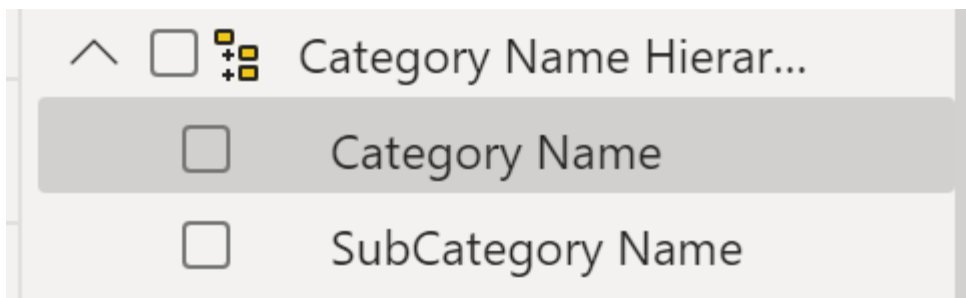
Date	∨ ×
Year	×
Quarter	×
Month	×
Day	×

In the preceding **Date** column, the date is shown in increasingly finer detail through year, quarters, months, and days. You can also manually create hierarchies.

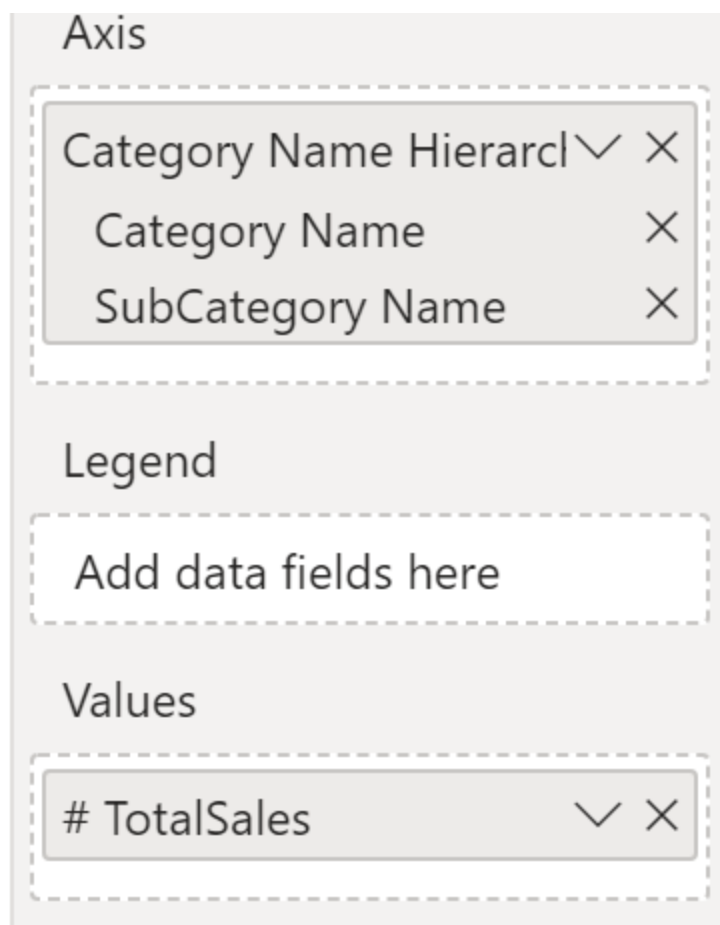
For example, consider a situation where you want to create a stacked bar chart of **Total Sales by Category and Subcategory**. You can accomplish this task by creating a hierarchy in the **Product** table for categories and subcategories. To create a hierarchy, go to the **Fields** pane on Power BI and then right-click the column that you want the hierarchy for. Select **New hierarchy**, as shown in the following figure.



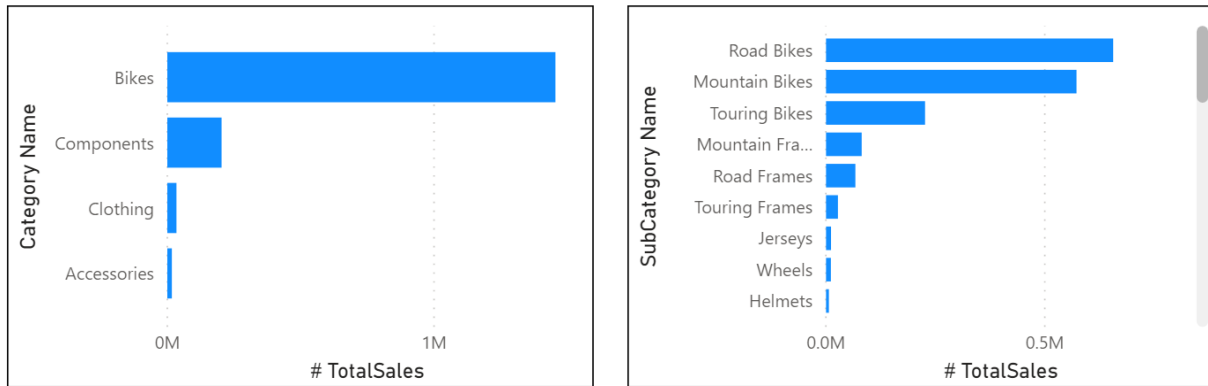
Next, drag and drop the subcategory column into this new hierarchy that you've created. This column will be added as a sublevel on the hierarchy.



Now, you can build the visual by selecting a stacked bar chart in the **Visualizations** pane. Add your **Category Name Hierarchy** in the **Axis** field and **Total Sales** in the **Values** field.



You can drill down on the visual to view both **Category** and **Subcategory**, depending on what you want to see. Hierarchies allow you to view increasing levels of data on a single view.



Now that you have learned about hierarchies, you can take a step further and examine parent-child hierarchies and their role in multiple relationships between fact tables and dimension tables.

Parent-child hierarchy

In the following example, you have an Employee table within the database that tells you important information about the employees, their managers, and their IDs. When looking at this table, you notice that **Roy F** has been repeated multiple times in the **Manager** column. As the image shows, multiple employees can have the same manager, which indicates a hierarchy between managers and employees.

	Employee ID	Employee	Manager ID	Manager
1	1010	Roy F	null	
2	1011	Pam H	1010	Roy F
3	1012	Guy L	1010	Roy F
4	1013	Roger M	1011	Pam H
5	1014	Kaylie S	1011	Pam H
6	1015	Mike O	1012	Guy L
7	1016	Rudy Q	1012	Guy L

The **Manager** column determines the hierarchy and is therefore the **parent**, while the "children" are the **employees**. For this example, you want to be able to see all levels of this hierarchy. Power BI does not default to showing you all levels of the hierarchy, so it is your responsibility to ensure that you see all levels of this hierarchy or "flatten" it so that you can see more data granularity.

Flatten parent-child hierarchy

The process of viewing multiple child levels based on a top-level parent is known as **flattening the hierarchy**. In this process, you are creating multiple columns in a table to show the hierarchical path of the parent to the child in the same record. You will use **PATH()**,

a simple DAX function that returns a text version of the managerial path for each employee, and PATHITEM() to separate this path into each level of managerial hierarchy.

Important

DAX has not been covered yet; however, it will be in another module. This function is included in this section because it's explaining hierarchies. If use of DAX in this capacity is confusing, refer to the DAX module and then return to this section afterward.

While on the table, go to the Modeling tab and select New Column. In the resulting formula bar, enter the following function, which creates the text path between the employee and manager. This action creates a calculated column in DAX.

DAX

```
Path = PATH(Employee[Employee ID], Employee[Manager ID])
```

1 Path = PATH(Employee[Employee ID], Employee[Manager ID])

PATH(ID_ColumnName, Parent_ColumnName)

Returns a string which contains a delimited list of IDs, starting with the top/root of a hierarchy and ending with the specified ID.

The completed path between the employee and the manager appears in the new column, as shown in the following screenshot.

Employee ID	Manager ID	Employee	Manager	Path
1010		Roy F		1010
1011	1010	Pam H	Roy F	1010 1011
1012	1010	Guy L	Roy F	1010 1012
1013	1011	Roger M	Pam H	1010 1011 1013
1014	1011	Kaylie S	Pam H	1010 1011 1014
1015	1012	Mike O	Guy L	1010 1012 1015
1016	1012	Rudy Q	Guy L	1010 1012 1016

If you look at Roger M, the path of IDs is 1010 | 1011 | 1013, which means that one level above Roger M (ID 1013) is his manager, Pam H (ID 1011), and one level above Pam H is her manager Roy F (ID 1010). In this row, Roger M is on the bottom of the hierarchy, at the child level, and Roy F is at the top of the hierarchy and is at the parent level. This path is created for every

employee. To flatten the hierarchy, you can separate each level by using the PATHITEM function.

To view all three levels of the hierarchy separately, you can create four columns in the same way that you did previously, by entering the following equations. You will use the PATHITEM function to retrieve the value that resides in the corresponding level of your hierarchy.

Level 1 = PATHITEM(Employee[Path],1)

Level 2 = PATHITEM(Employee[Path],2)

Level 3 = PATHITEM(Employee[Path],3)

1

Level 1 = PATHITEM(Employee[Path], 1)

▼

Manager ID

PATHITEM(Path, Position, [Type])
Returns the nth item in the delimited list produced by the Path function.

After you have finished, notice that you now have each level of hierarchy within your table. Roy F is at the top of the hierarchy and, as you go through Levels 2-3, notice that the managers and employees map with each other.

Employee ID	Name	Manager	Manager ID	Path	Level 1	Level 2	Level 3	Level 4
1000	Quincy Howard			1000	1000			
1001	Mallory Yang	Quincy Howard	1000	1000 1001	1000	1001		
1002	Donovan Maynard	Quincy Howard	1000	1000 1002	1000	1002		
1003	Giselle McClain	Mallory Yang	1001	1000 1001 1003	1000	1001	1003	
1004	Melvin Marsh	Mallory Yang	1001	1000 1001 1004	1000	1001	1004	
1005	Ria Snow	Giselle McClain	1003	1000 1001 1003 1005	1000	1001	1003	1005
1006	Callie Savage	Giselle McClain	1003	1000 1001 1003 1006	1000	1001	1003	1006

Now, you can create a hierarchy on the Fields pane, as you did previously. Right-click Level 1, because this is the first hierarchy level, and then select New Hierarchy. Then, drag and drop Level 2 and Level 3 into this hierarchy.

□

fx

Level 1

Check

New hierarchy

New measure

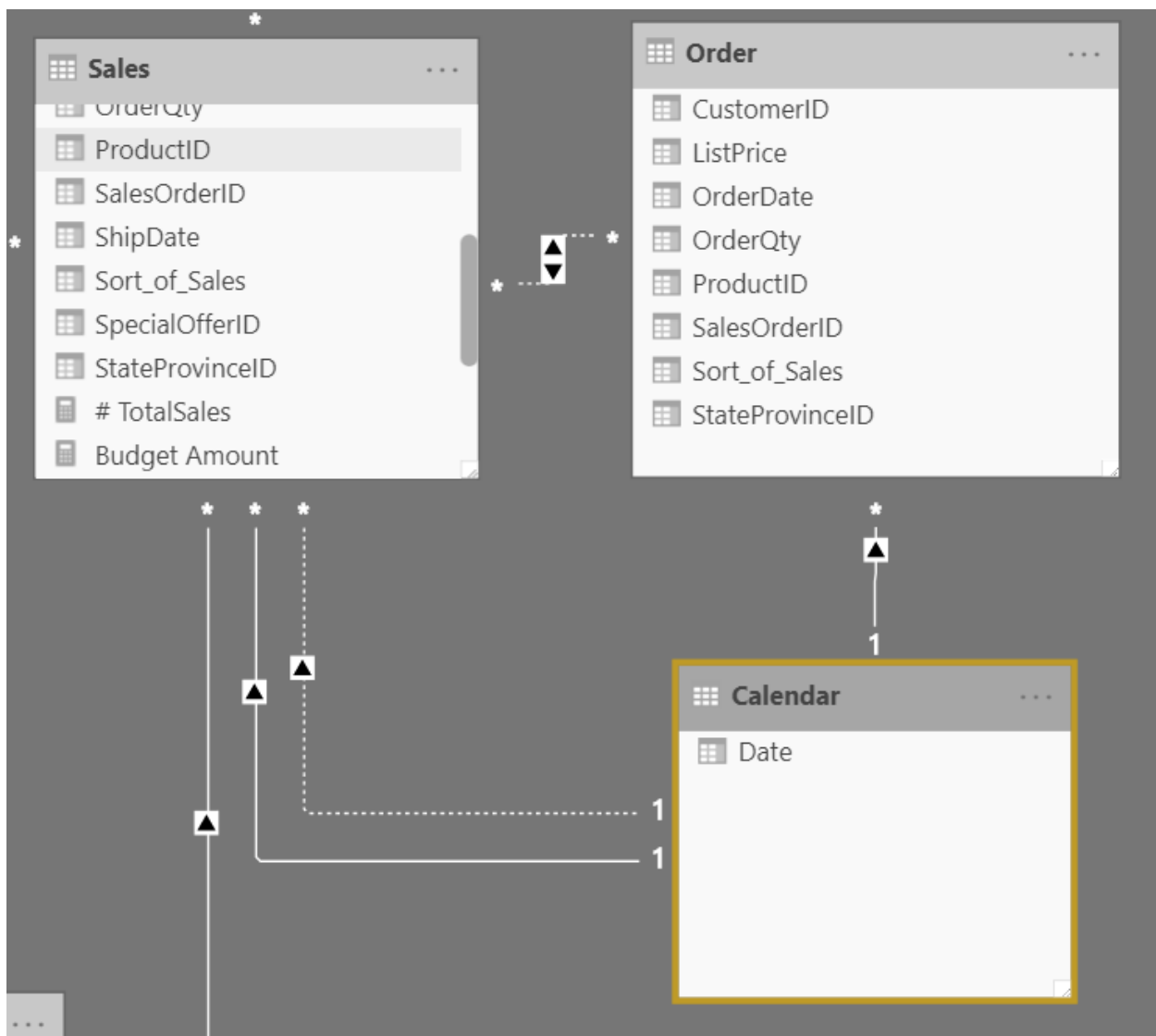
New hierarchy

You have now successfully flattened a hierarchy so that you can view individual levels.

Previously, you've considered dimensions that have only one relationship with a fact table. However, situations do occur where your dimension table will have multiple relationships with a fact table.

Role-playing dimensions

Role-playing dimensions have multiple valid relationships with fact tables, meaning that the same dimension can be used to filter multiple columns or tables of data. As a result, you can filter data differently depending on what information you need to retrieve. This topic is complex, so it is only introduced in this section. Working with role-playing dimensions requires complex DAX functions that will be discussed in later sections.



The preceding visual shows the Calendar, Sales, and Order tables. Calendar is the dimension table, while Sales and Order are fact tables. The dimension table has two relationships: one with Sales and one with Order. This example is of a role-playing dimension because the Calendar table can be used to group data in both Sales and Order. If you wanted to build a visual in which the Calendar table references the Order and the Sales tables, the Calendar table would act as a role-playing dimension.

Define data granularity

Data granularity is the detail that is represented within your data, meaning that the more granularity your data has, the greater the level of detail within your data.

Data granularity is an important topic for all data analysts, regardless of the Power BI tools that you're using. Defining the correct data granularity can have a big impact on the performance and usability of your Power BI reports and visuals.

Data granularity defined

Consider a scenario where your company manages 1,000 refrigerated semi-trucks. Every few minutes, each truck uses a Microsoft Azure IoT application to record its current temperature. This temperature is important to your organization because, if the refrigeration were to malfunction, it could spoil the entire load, costing thousands of dollars. With so many trucks and so many sensors, extensive data is generated every day. Your report users don't want to sift through numerous records to find the ones that they are particularly interested in.

How can you change the granularity of the data to make the semantic model more usable?

In this scenario, you might want to import the data by using a daily average for each truck. That approach would reduce the records in the database to one record for each truck for each day. If you decide that the approach was acceptable enough for tracking costs and errors, then you could use that data granularity. Alternatively, you could select the last recorded temperature, or you could only import records that are above or below a normal range of temperatures. Any of these methods will reduce the total records that you import, while still bringing in data that is comprehensive and valuable.

For different scenarios, you could settle on data granularity that is defined weekly, monthly, or quarterly. Generally, the fewer the records that you are working with, the faster your reports and visuals will function. This approach translates to a faster refresh rate for the entire semantic model, which might mean that you can refresh more frequently.

However, that approach has a drawback. If your users want to drill into every single transaction, summarizing the granularity will prevent them from doing that, which can have

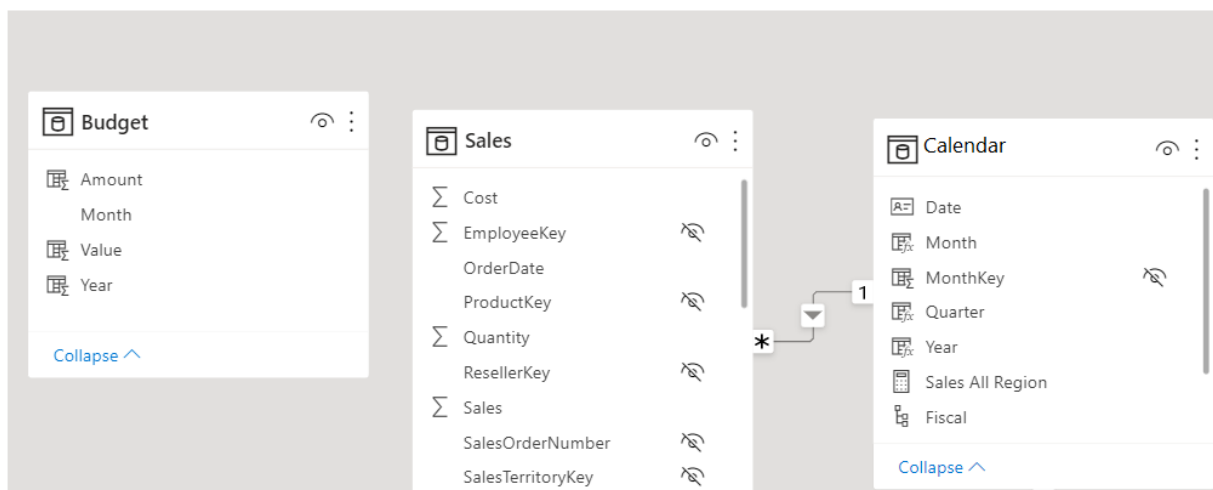
a negative impact on the user experience. It is important to negotiate the level of data granularity with report users so they understand the implications of these choices.

Change data granularity to build a relationship between two tables

Data granularity can also have an impact when you're building relationships between tables in Power BI.

For example, consider that you're building reports for the Sales team at Tailwind Traders. You've been asked to build a matrix of total sales and budget over time by using the Calendar, Sales, and Budget tables. You notice that the lowest level of time-based detail that the Sales table goes into is by day, for instance 5/1/2020, 6/7/2020, and 6/18/2020. The Budget table only goes to the monthly level, for instance, the budget data is 5/2020 and 6/2020. These tables have different granularities that need to be reconciled before you can build a relationship between tables.

The following figure shows your current semantic model.

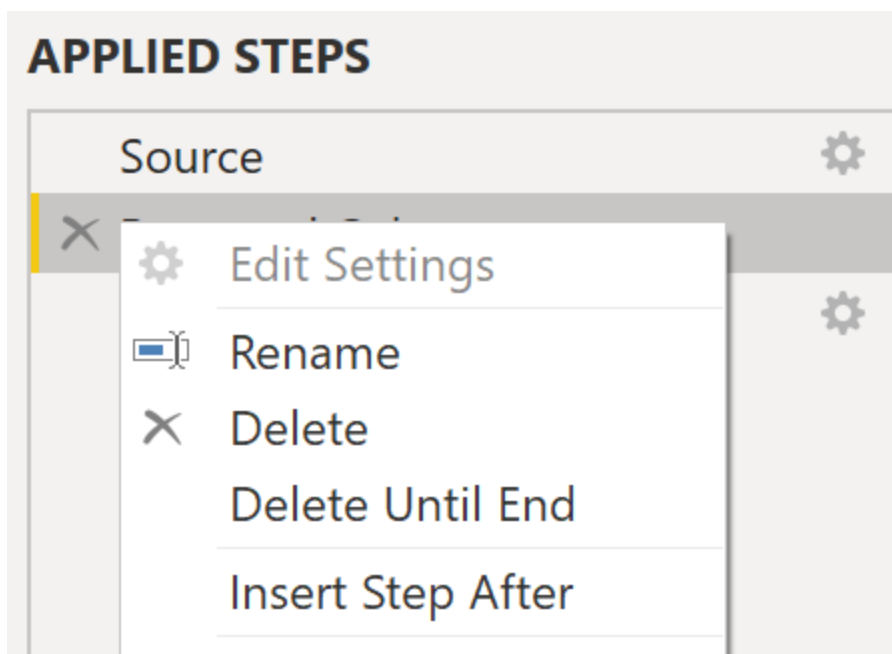


As shown in the preceding figure, a relationship between Budget and Calendar is missing. Therefore, you need to create this relationship before you can build your visual. Notice that if you transform the Year and Month columns in the Calendar table into a new column, and do the same transformation in the Budget table, you can match the format of the Date column in the Calendar table. Then, you can establish a relationship between the two columns. To complete this task, you'll concatenate the Year and Month columns and then change the format.

Year ▼	Month ▼	Date ▼
2013	1	01/01/2011
2013	2	01/02/2011
2013	3	01/03/2011
2013	4	01/04/2011
2013	5	01/05/2011
2013	6	01/06/2011
2013	7	01/07/2011
2013	8	01/07/2011

Budget Calendar

Select **Transform Data** on the ribbon. On **Applied Steps**, on the right pane, right-click the last step and then select **Insert Step After**.



Under **Add Column** on the Home ribbon, select **Custom Column**. Enter the following equation, which will concatenate the **Year** and **Month** columns, and then add a dash in between the column names.

M

```
Column = Table.AddColumn("#Renamed Columns", "Custom", each [Year] & "-" & [Month])
```

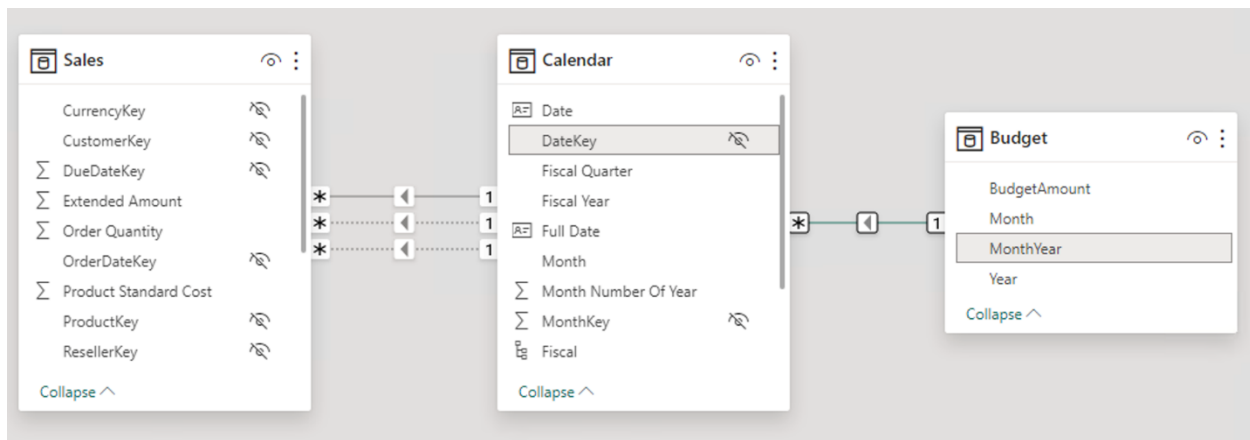
Change the data type to **Date** and then **rename** the column. Your Budget table should resemble the following figure.

Year	Month	BudgetAmount	MonthYear
2011	1	9493	2011-1
2011	2	2490	2011-2
2011	3	16585	2011-3
2011	4	5575	2011-4
2011	5	11611	2011-5
2011	6	14017	2011-6
2011	7	11919	2011-7
2011	8	3832	2011-8
2011	9	7094	2011-9
2011	10	6216	2011-10
2011	11	7769	2011-11
2011	12	12552	2011-12
2012	1	12781	2012-1
2012	2	7889	2012-2
2012	3	10329	2012-3
2012	4	5350	2012-4
2012	5	19216	2012-5
2012	6	5849	2012-6
2012	7	19269	2012-7
2012	8	3080	2012-8
2012	9	2381	2012-9

Now, you can create a relationship between the Budget and the Calendar tables.

Create a relationship between tables

Power BI automatically detects relationships, but you can also go **to Manage Relationships > New** and create the relationship on the **Date** column. The relationship should resemble the following figure.



By completing this task, you've ensured that the granularity is the same between your different tables. Now, you need to create DAX measures to calculate **Total Sales** and **BudgetAmount**. Go to the **Data pane** on Power BI Desktop, select **New Measure**, and then create two measures with the following equations:

DAX

```
TotalSales = SUM(Sales[Total Sales])
```

DAX

```
BudgetAmount = SUM (Budget[BudgetAmount])
```

Select the table visual on the **Visualizations** pane, and then enter these measures and the **Date** into the **Values** field. You've now accomplished the goal of building a matrix of the total sales and budgets over time.

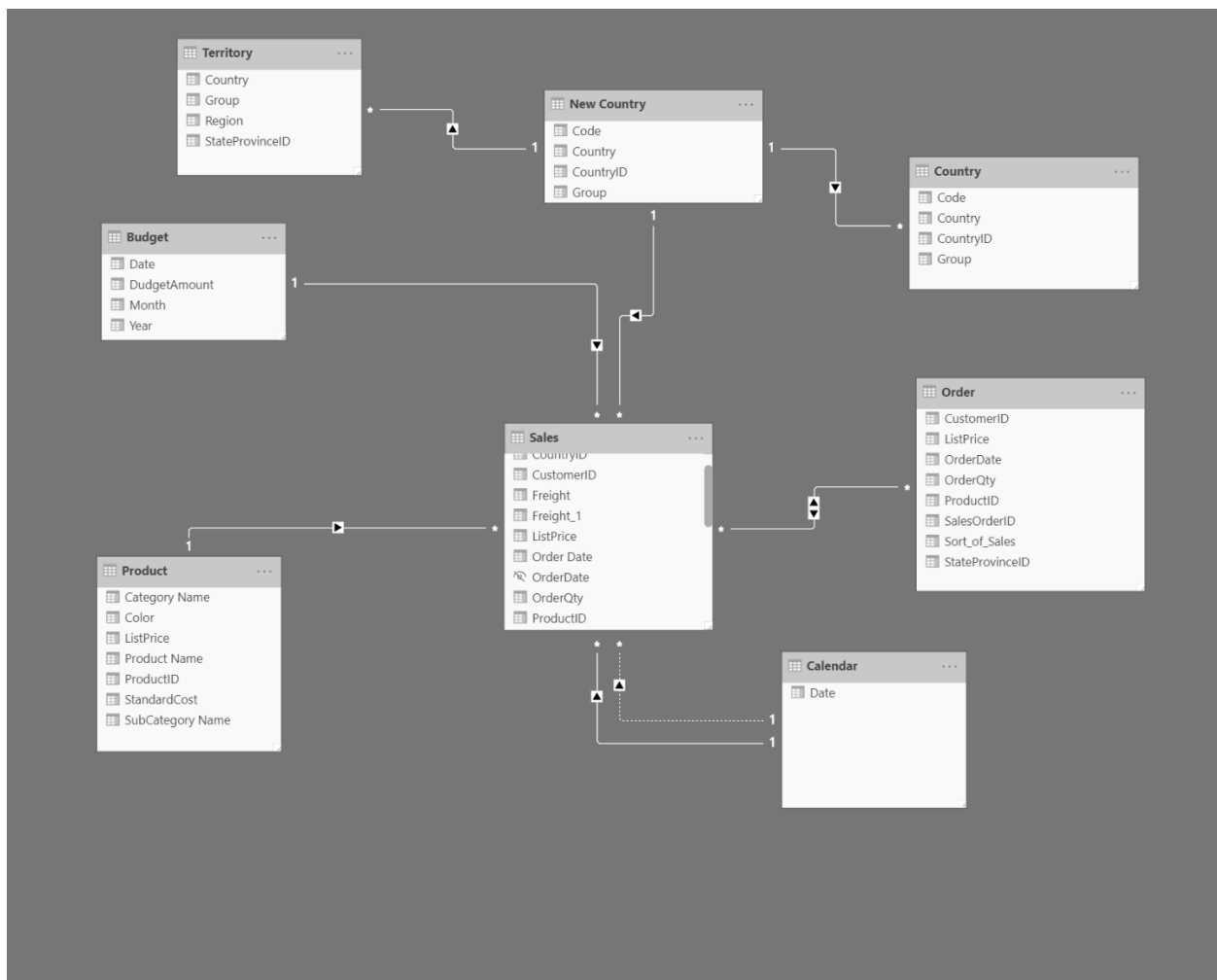
The screenshot shows the Power BI Desktop interface. On the left, a matrix visual displays data for TotalSales and Budget Amount, categorized by Year, Quarter, and Month. The matrix is filtered by Year and Quarter. The Fields pane on the right shows the Date field in the Values section, with the measures TotalSales and Budget Amount also listed in the Values section.

Year	Quarter	Month	TotalSales	Budget Amount
2014	Qtr 1	January	61,274	6,500
2014	Qtr 1	February	13,396	6,000
2014	Qtr 1	March	110,484	5,000
2014	Qtr 2	April	17,980	7,000
2014	Qtr 2	May	77,477	9,000
2014	Qtr 2	June	490	11,000
2013	Qtr 1	January	33,438	5,000
2013	Qtr 1	February	38,068	4,000
Total			1,709,647	120,500

Work with relationships and cardinality

Unlike other database management systems, Power BI has the concept of *directionality* to a relationship. This directionality [plays an important role in filtering data between multiple tables](#). When you load data, Power BI [automatically looks for relationships that exist within the data by matching column names](#). You can also [use Manage Relationships](#) to edit these options manually.

For example, you've [retrieved many tables](#) from the Sales database, and the following image is an example of your semantic model. Power BI has [autodetected several relationships](#), but you [can't discern what they mean](#). You want to make sure that the relationships accurately reflect those that exist in your data.



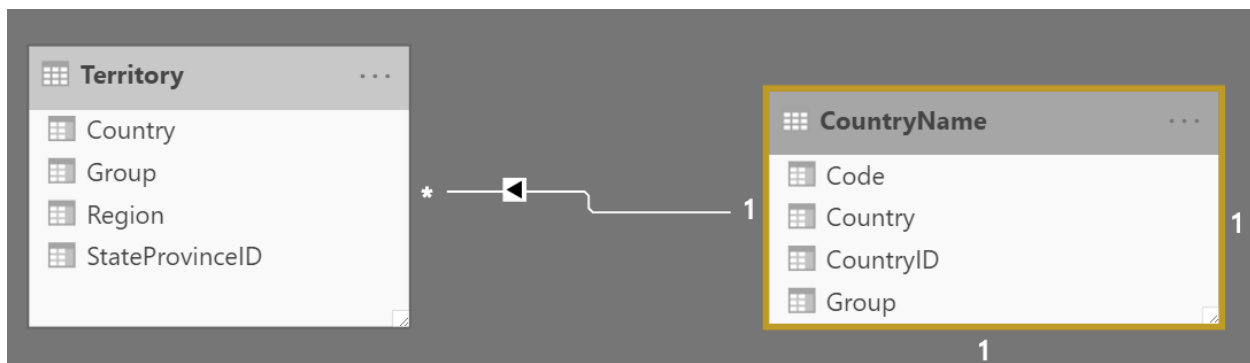
Relationships

The following are different types of relationships that you'll find in Power BI.

Many-to-one (*:1) or **one-to-many** (1:*) relationship

- Describes a relationship in which you have many instances of a value in one column that are related to only one unique corresponding instance in another column.
- Describes the directionality between fact and dimension tables.
- Is the most common type of directionality and is the Power BI default when you are automatically creating relationships.

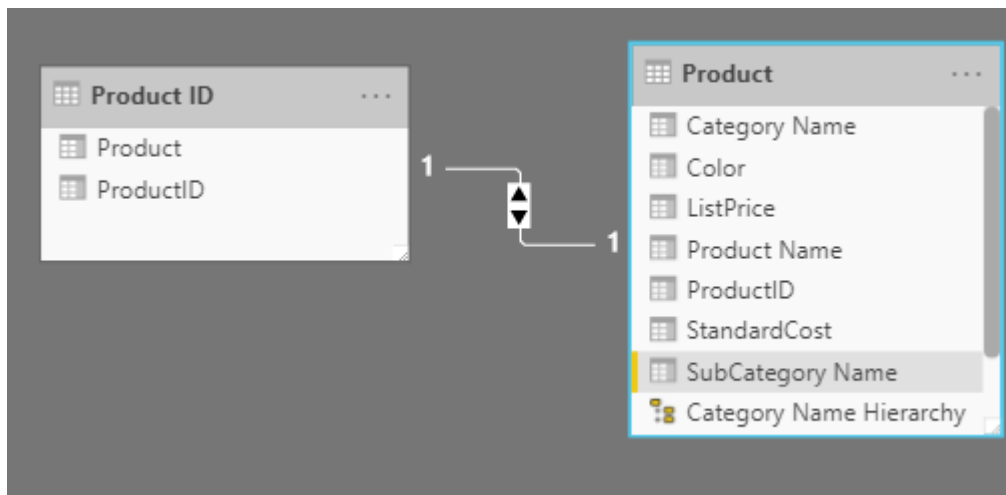
An example of a one-to-many relationship would be between the CountryName and Territory tables, where you can have many territories that are associated with one unique country/region.



One-to-one (1:1) relationship:

- Describes a relationship in which only one instance of a value is common between two tables.
- Requires unique values in both tables.
- Is not recommended because this relationship stores redundant information and suggests that the model is not designed correctly. It is better practice to combine the tables.

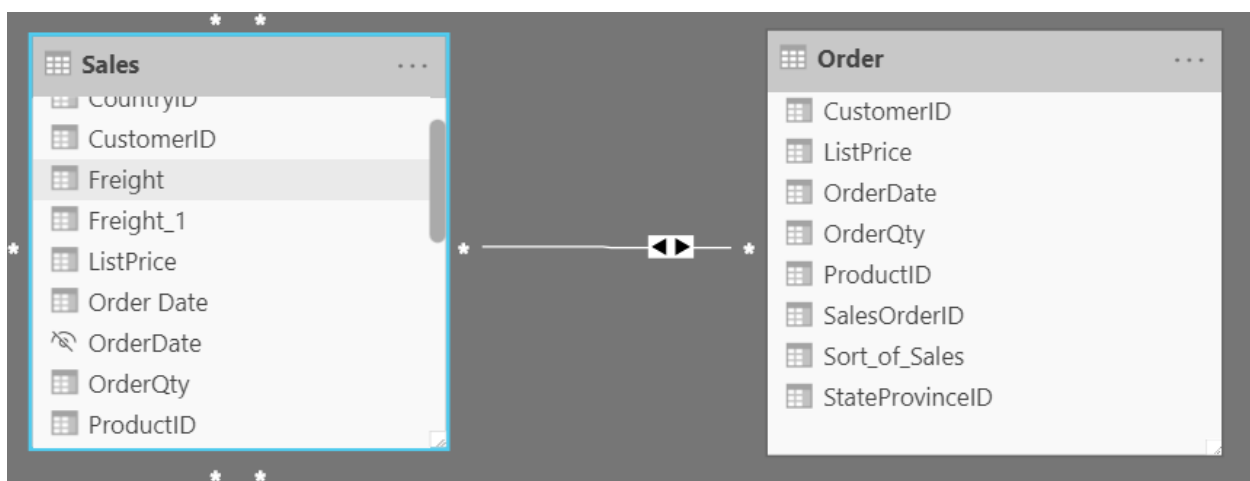
An example of a one-to-one relationship would be if you had products and product IDs in two different tables. Creating a one-to-one relationship is redundant and these two tables should be combined.



Many-to-many (.) relationship:

- Describes a relationship where **many values are in common between two tables**.
- **Does not require unique values** in either table in a relationship.
- Is **not recommended**; a lack of unique values **introduces ambiguity** and **your users might not know which column of values is referring** to what.

For instance, the following figure shows a many-to-many relationship between the Sales and Order tables on the **OrderDate** column because multiple sales can have multiple orders associated with them. Ambiguity is introduced because both tables can have the same order date.



Cross-filter direction

Data can be filtered on one or both sides of a relationship.

With a **single cross-filter direction**:

- Only one table in a relationship can be used to filter the data. For instance, Table 1 can be filtered by Table 2, but Table 2 cannot be filtered by Table 1.

Tip

Follow the direction of the arrow on the relationship between your tables to know which direction the filter will flow. You typically want these arrows to point to your fact table.

- For a one-to-many or many-to-one relationship, the cross-filter direction will be from the "one" side, meaning that the filtering will occur in the table that has many values.

With both cross-filter directions or bi-directional cross-filtering:

- One table in a relationship can be used to filter the other. For instance, a dimension table can be filtered through the fact table, and the fact tables can be filtered through the dimension table.
- You might have lower performance when using bi-directional cross-filtering with many-to-many relationships.

A word of caution regarding bi-directional cross-filtering: You should not enable bi-directional cross-filtering relationships unless you fully understand the ramifications of doing so. Enabling it can lead to ambiguity, over-sampling, unexpected results, and potential performance degradation.

Cardinality and cross-filter direction

For one-to-one relationships, the only option that is available is bi-directional cross-filtering. Data can be filtered on either side of this relationship and result in one distinct, unambiguous value. For instance, you can filter on one Product ID and be returned a single Product, and you can filter on a Product and be returned a single Product ID.

For many-to-many relationships, you can choose to filter in a single direction or in both directions by using bi-directional cross-filtering. The ambiguity that is associated with bi-directional cross-filtering is amplified in a many-to-many relationship because multiple paths will exist between different tables. If you create a measure, calculation, or filter, unintended consequences can occur where your data is being filtered and, depending on which relationship that the Power BI engine chooses when applying the filter, the final result might be different. This situation is also true for bi-directional relationships and why you should be cautious when using them.

For this reason, many-to-many relationships and/or bi-directional relationships are complicated. Unless you are certain what your data looks like when aggregated, these types

of open-ended relationships with multiple filtering directions can introduce multiple paths through the data.

Create many-to-many relationships

Consider the scenario where you are tasked with building a visual that examines budgets for customers and accounts. You can have multiple customers on the same account and multiple accounts with the same customer, so you know that you need to create a many-to-many relationship.

CustID	CustName
1022	Roy M
1023	Bob K
1024	Ellen L
1025	Mitch W
1026	Regan Q
1027	Lulu S
1028	Aliya R

CustomerTable

CustomerID	AccountID	AccountName
1022	12	BHP
1023	12	BHP
1024	13	RogerInc
1024	14	MyShip
1026	15	Holdings Unl.
1025	16	Key Biz Insiders
1028	17	Ty Inc
1022	17	Ty Inc

AccountTable

To create this relationship, go to **Manage Relationships > New**. In the resulting window, create a relationship between the **Customer ID** column in **CustomerTable** and **AccountTable**. The relationship is set to **many-to-many**, and the filter type is in both directions. Immediately, you will be warned that you should only use this type of relationship if it is expected that neither column will have unique values because you might get unexpected values. Because you want to filter in both directions, choose **bi-directional cross-filtering**.

Select **OK**. You have now successfully created a many-to-many relationship.

×

Create relationship

Select tables and columns that are related.

AccountTable

CustomerID	AccountID	AccountName
1022	12	BHP
1023	12	BHP
1024	13	RogerInc

CustomerTable

CustID	CustName
1022	Roy M
1023	Bob K
1024	Ellen L

Cardinality

Cross filter direction

Many to Many (*:*)

Both

☒ Make this relationship active
 ☐ Apply security filter in both directions

☐ Assume referential integrity


 This relationship has cardinality Many-Many. This should only be used if it is expected that neither column (CustomerID and CustID) contains unique values, and that the significantly different behavior of Many-many relationships is understood. [Learn more](#)

OK

Cancel

Resolve modeling challenges

Modeling data is about establishing and maintaining relationships so that you can effectively visualize the data in the form that your business requires. When you are creating these relationships, a common pitfall that you might encounter are circular relationships.

For example, you are developing reports for the Sales team and are examining the relationships between tables. In a poorly designed semantic model, Table 1 has a many-to-one relationship with a column in Table 2, but Table 2 has a one-to-many relationship with Table 3 that has its own relationship with Table 1. This web of relationships is difficult to manage and becomes a daunting task to build visuals because it is no longer clear what

relationships exist. Therefore, it is important that you are able to identify circular relationships so that your data is usable.

Relationship dependencies

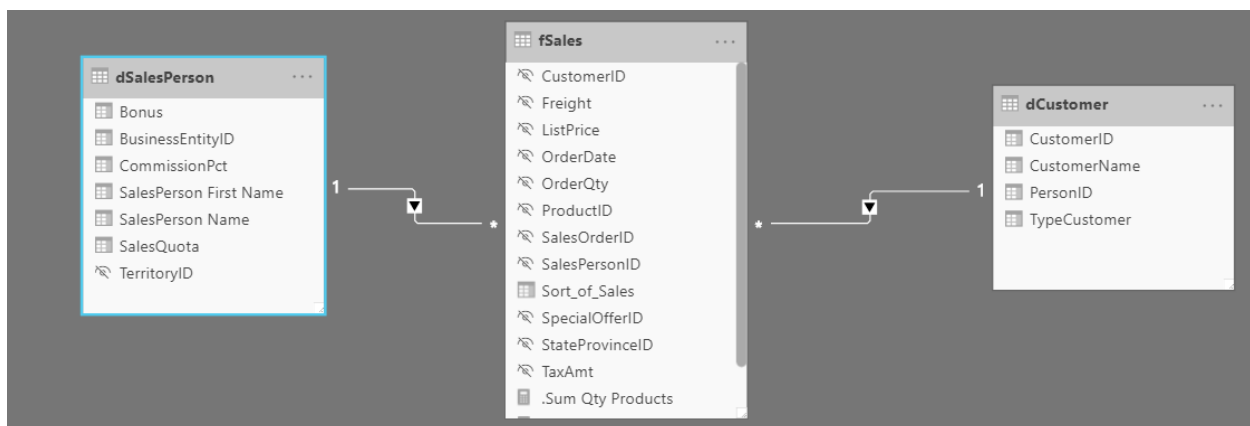
To understand circular relationships, you first need to understand dependencies.

For example, consider that you have the following calculated column **Total** in the Sales table.

```
Sales['TotalCost'] = Sales['Quantity'] * Sales['Price']
```

TotalCost depends on **Quantity** and **Price**, so if a change occurs in either quantity or price, a change will occur in **TotalCost** as well. This example outlines a dependency of a column on other columns, but you can also have dependencies between measures, tables, and relationships.

Consider the following relationships between **dSalesPerson**, **fSales**, and **dCustomer**. A change in **dCustomer** will result in a change in **fSales**, which results in changes in **dSalesPerson**. These types of dependencies can exist within relationships.



Exercise - Design a Data Model in Power BI

Lab story

In this lab, you'll commence developing the data model. It will involve creating relationships between tables, and then configuring table and column properties to improve the friendliness and usability of the data model. You'll also create hierarchies and create quick measures.

In this lab you learn how to:

- Create model relationships
- Configure table and column properties

- Create hierarchies

This lab should take approximately 45 minutes.

Get started

To complete this exercise, first open a web browser and enter the following URL to download the zip folder:

<https://github.com/MicrosoftLearning/PL-300-Microsoft-Power-BI-Data-Analyst/raw/Main/Allfiles/Labs/03-configure-data-model-in-power-bi-desktop/03-model-data.zip>

Extract the folder to the **C:\Users\Student\Downloads\03-model-data** folder.

Open the **03-Starter-Sales Analysis.pbix** file.

Note: You can dismiss the sign-in by selecting **Cancel**. Close any other informational windows. Select **Apply Later**, if prompted to apply changes.

Create model relationships

In this task, you'll create model relationships. The file was configured to not identify relationships between tables, which isn't the default setting, but is recommended to prevent extra work creating the correct relationships for your model.

Important: The labs use a shorthand notation to reference a field. It will look like this: **Product | Category**. In this example, **Product** is the table name and **Category** is the field name.

1. In Power BI Desktop, at the left, select the **Report view** icon.



2. To view all table fields, in the **Data** pane, right-click an empty area, and then select **Expand All**.

3. To create a table visual, in the **Data** pane, from inside the **Product** table, check the **Category** field.
4. To add another column to the table, in the **Data** pane, check the **Sales | Sales** field.
5. Notice that the table visual lists four product categories, and that the sales value is the same for each, and the same for the total.

Category	Sum of Sales
Accessories	77,548,570.20
Bikes	77,548,570.20
Clothing	77,548,570.20
Components	77,548,570.20
Total	77,548,570.20

*The issue is that the table is based on fields from different tables. The expectation is that each product category displays the sales for that category. However, because there isn't a model relationship between these tables, the **Sales** table isn't filtered. You'll now add a relationship to propagate filters between the tables.*

6. Select the **Model** view icon from the left navigation pane, and select **Manage Relationships**.



7. In the **Manage Relationships** window, notice that no relationships are yet defined. To create a relationship, select **New relationship**.
8. Configure the relationship from **Product** table to **Sales** table. Notice the following elements were automatically configured:
 - **ProductKey columns in each table are selected.** *The columns were selected because they share the same name and data type. You may need to find matching columns with different names in real data.*
 - **Cardinality type is One To Many (1:*)**. *The cardinality was automatically detected, because Power BI understands that the **ProductKey** column from the **Product** table contains unique values. One-to-many relationships are the most common cardinality, and all relationship you create in this lab will be this type.*
 - **Cross Filter Direction type is Single**. *Single filter direction means that filters propagate from the “one side” to the “many side”. In this case, it means filters applied to the **Product** table will propagate to the **Sales** table, but not in the opposite direction.*
 - **Make This Relationship Active is checked**. *Active relationships propagate filters. It’s possible to mark a relationship as inactive so filters don’t propagate. Inactive relationships can exist when there are multiple relationship paths between tables. In this case, model calculations can use special functions to activate them.*

← New relationship



Select tables and columns that are related.

From table

Product

Background C...	Category	Color	Font Color Fo...	Product	ProductKey	Subcategory
#000000	Components	Black	#FFFFFF	HL Road Fram...	210	Road Fra
#000000	Accessories	Black	#FFFFFF	Sport-100 Hel...	215	Helmets
#000000	Accessories	Black	#FFFFFF	Sport-100 Hel...	216	Helmets

To table

Sales

EmployeeKey	OrderDate	ProductKey	Quantity	ResellerKey	SalesOrderNu...	SalesTerrit
282	Friday, Augus...	235	2	312	SO43897	4
282	Friday, Augus...	351	2	312	SO43897	4
282	Friday, Augus...	348	2	312	SO43897	4

Cardinality

One to many (1:*)

Cross-filter direction

Single

☒ Make this relationship active

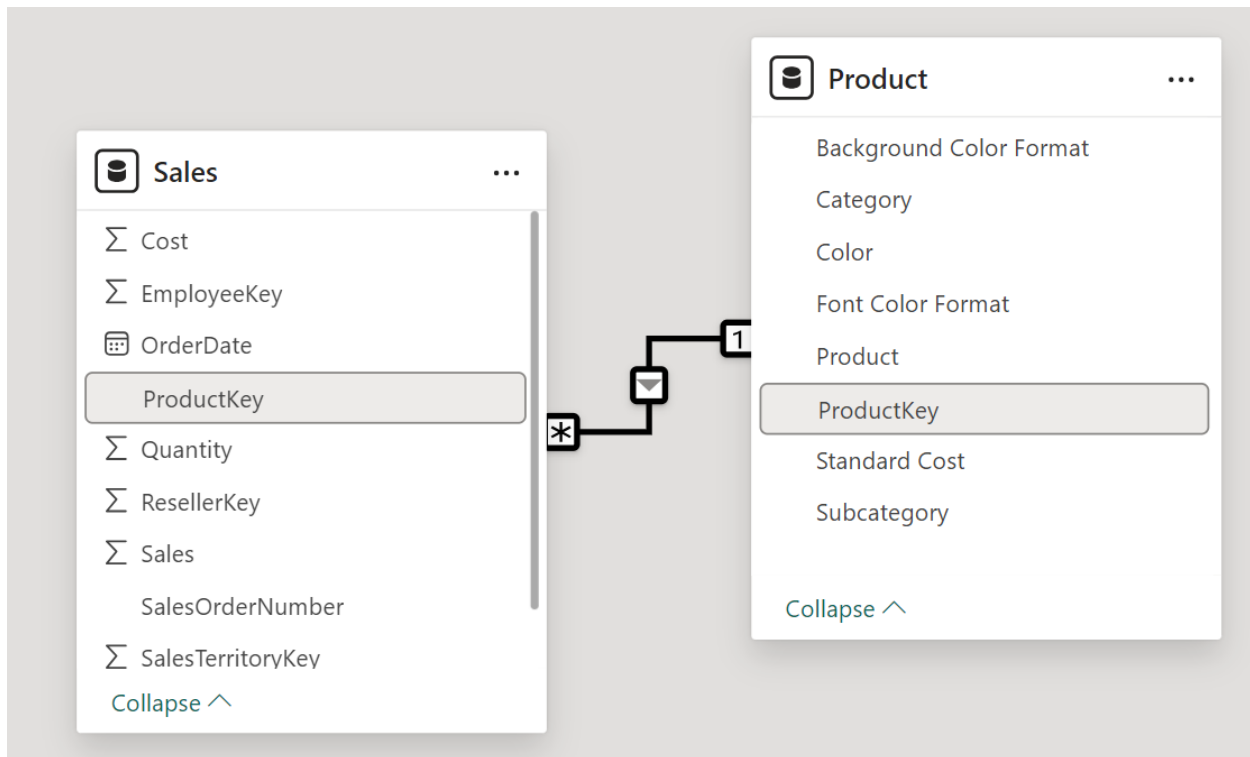
☐ Apply security filter in both directions

☐ Assume referential integrity

Save

Cancel

9. Select **OK**, notice in the **Manage Relationships** window that the new relationship is listed, and then select **Close**.



Notice there's now a connector between the two tables (*it doesn't matter if the tables are positioned next to each other*). - You can interpret the cardinality that is represented by the **1** and **(*)** indicators. - Filter direction is represented by the arrow head. - A solid line represents an active relationship; a dashed line represents an inactive relationship. - Hover the cursor over the relationship to highlight the related columns.

Create additional relationships

There's **an easier way to create a relationship**. In the model diagram, you can **drag and drop columns** to create a new relationship.

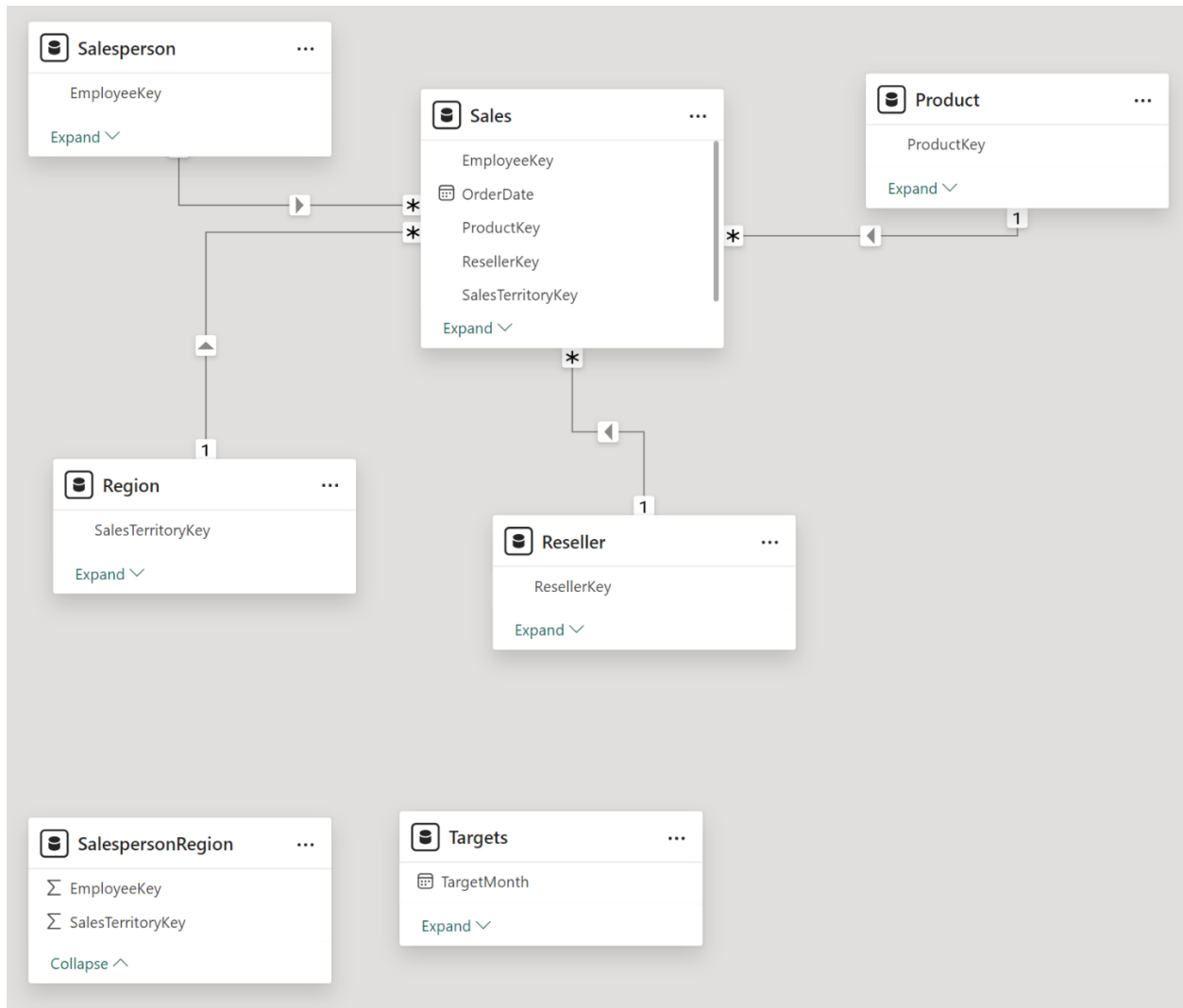
1. To create a new relationship using a different technique, from the **Reseller** table, drag the **ResellerKey** column onto the **ResellerKey** column of the **Sales** table.

Tip: *Sometime a column doesn't want to be dragged. If this situation arises, select a different column, and then select the column you intend to drag again, and then try again.* Ensure that you see the new relationship added to the diagram.

! [Drag ResellerKey from Reseller table onto Sales	ResellerKey] (/PL-300-Microsoft-Power-BI-Data-Analyst/Instructions/Labs/Linked_image_Files/03-drag-rela
---	--

2. Use the new technique to create the following two model relationships:

- **Region | SalesTerritoryKey to Sales | SalesTerritoryKey**
 - **Salesperson | EmployeeKey to Sales | EmployeeKey**
3. In the diagram, arrange the tables so that the **Sales** table is positioned in the center of the diagram, and the related tables are arranged about it. Position the disconnected tables to the side.



4. In the report view, notice that the table visual updated to display different values for each product category.

*Filters applied to the **Product** table now propagate to the **Sales** table.*

Category	Sum of Sales
Accessories	539,106.09
Bikes	64,069,033.16
Clothing	1,714,056.05
Components	11,226,374.90
Total	77,548,570.20

5. **Save** the Power BI Desktop file.

Configure the Product table

In this task, you'll configure the **Product** table with a hierarchy and display folder.

1. In **Model view > Data pane**, if necessary, **expand** the **Product table** to reveal all fields.
2. To create a hierarchy, in the Data pane, **right-click** the **Category** column, and then select **Create Hierarchy**.
3. Update the name to **Products** (right-click or double-click to rename).
4. To **add** the second **level to the hierarchy**, in the **Properties** pane, in the **Hierarchy dropdown** list, select **Subcategory** (you might need to **scroll down** inside the pane).
5. To **add the third level to the hierarchy**, in the **Hierarchy dropdown** list, select **Product**.
6. To complete the hierarchy design, select **Apply Level Changes**.

Hierarchy

Select a column to add level...



↕ Category (Category)



↕ Subcategory (Subcategory)

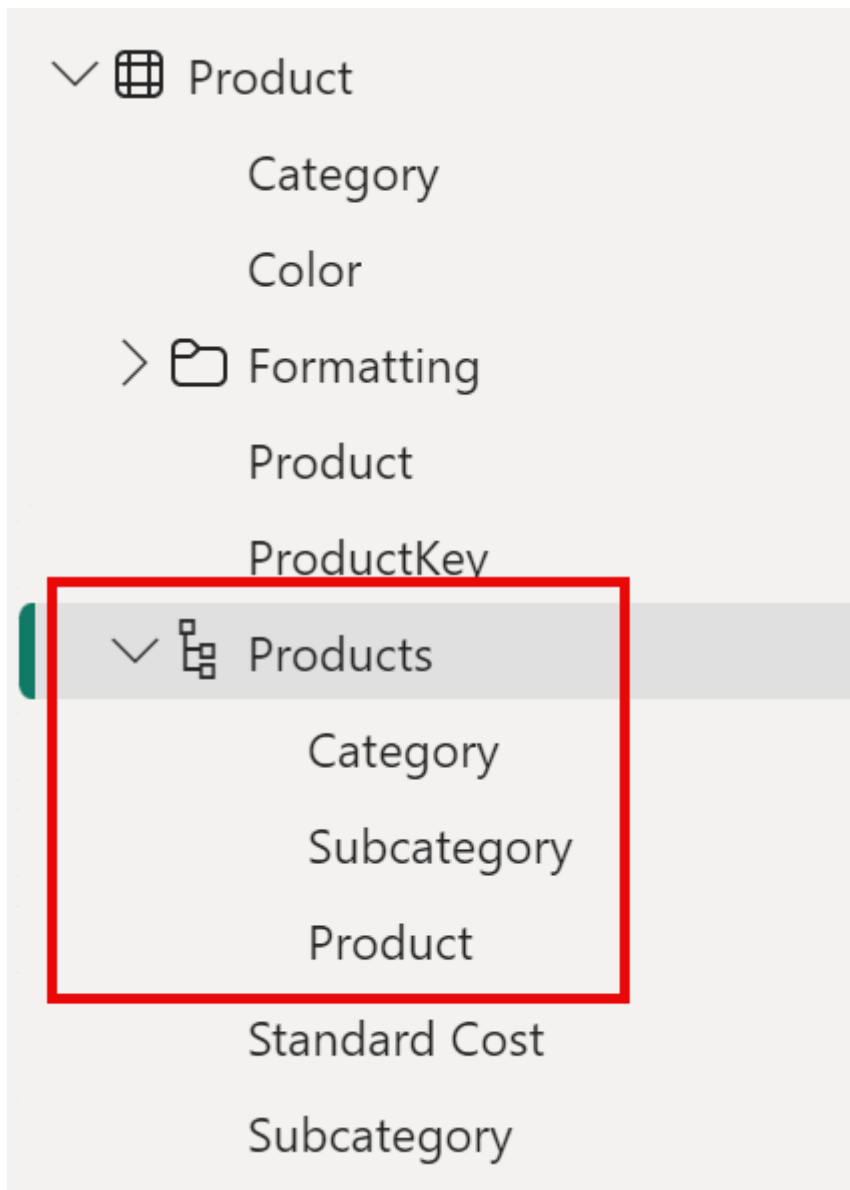


↕ Product (Product)

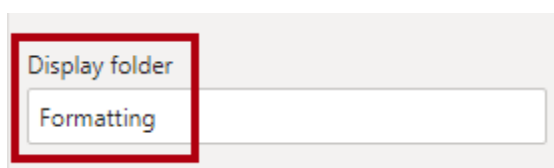


Apply Level Changes

7. In the **Data** pane, notice the **Products** hierarchy. To reveal the hierarchy levels, expand the **Products** hierarchy.

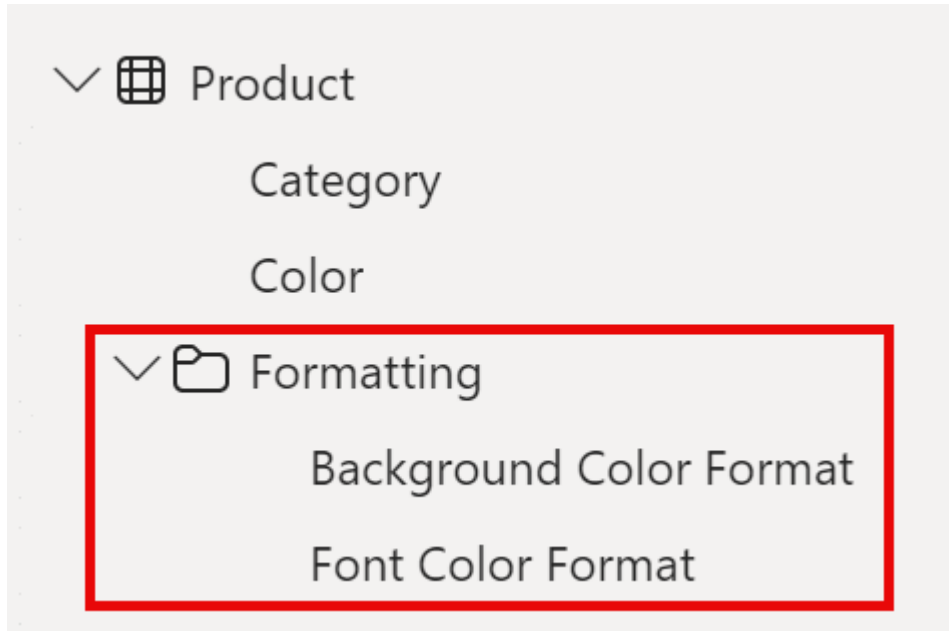


8. To organize columns into a display folder, in the **Data** pane, first select the **Background Color Format** column.
9. While pressing the **Ctrl** key, select the **Font Color Format** column.
10. In the **Properties** pane, in the **Display Folder** box, enter **Formatting**.



11. In the **Data** pane, notice that the two columns are now inside a folder.

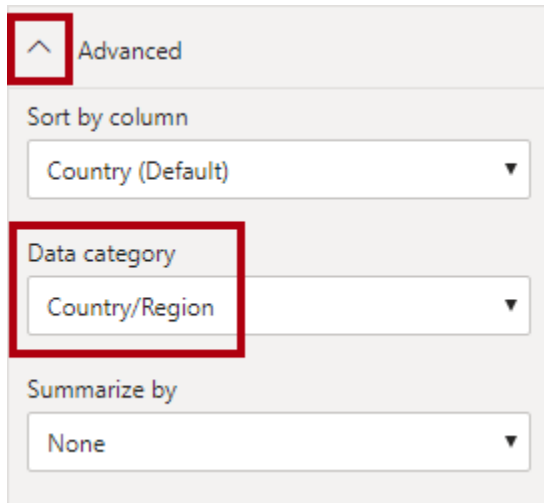
Display folders are a great way to organize tables, especially for tables that comprise many fields. They're logical presentation only.



Configure the Region table

In this task, you'll configure the **Region** table with a hierarchy and updated categories.

1. In the **Region** table, create a hierarchy named **Regions**, with the following three levels:
 - Group
 - Country
 - Region
2. Select the **Country** column (not the **Country** hierarchy level).
3. In the **Properties** pane, expand the **Advanced** section (at the bottom of the pane), and then in the **Data Category** dropdown list, select **Country/Region**.



Advanced

Sort by column

Country (Default) ▼

Data category

Country/Region ▼

Summarize by

None ▼

Data categorization can provide hints to the report designer. In this case, categorizing the column as country or region provides more accurate information to Power BI when it renders a map visualization.

Configure the Reseller table

In this task, you'll configure the **Reseller** table to add a hierarchy and update data categories.

1. In the **Reseller** table, create a hierarchy named **Resellers**, with the following two levels:
 - Business Type
 - Reseller
2. Create a second hierarchy named **Geography**, with the following four levels:
 - Country-Region
 - State-Province
 - City
 - Reseller
3. Set the **Data Category** for the following columns (not within the hierarchy):
 - Country-Region to **Country/Region**
 - State-Province to **State or Province**
 - City to **City**

Configure the Sales table

In this task, you'll configure the **Sales** table with updated descriptions, formatting, and summarization.

1. In the **Sales** table, select the **Cost** column.
2. In the **Properties** pane, in the **Description** box, enter: *Based on standard cost.*

*Descriptions can be applied to tables, columns, hierarchies, or measures. In the **Data** pane, description text is revealed in a tooltip when a report author hovers their cursor over the field.*

1. Select the **Quantity** column.
2. In the **Properties** pane, from inside the **Formatting** section, slide the **Thousands Separator** property to **Yes**.
3. Select the **Unit Price** column.
4. In the **Properties** pane, from inside the **Formatting** section, set the **Decimal Places** property to **2**.
5. In the **Advanced** group (you may need to scroll down to locate it), in the **Summarize By** dropdown list, select **Average**.

*By default, numeric columns will summarize by summing values together. This default behavior isn't suitable for a column like **Unit Price**, which represents a rate. Setting the default summarization to average will produce a meaningful result.*

Bulk update properties

In this task, you'll update multiple columns using single bulk updates. You'll use this approach to hide columns, and format column values.

1. From the **Model view > Data** pane, select the **Product | ProductKey** column.
2. While pressing the **Ctrl** key, select the following 13 columns (spanning multiple tables):
 - Region | SalesTerritoryKey
 - Reseller | ResellerKey
 - Sales | EmployeeKey
 - Sales | ProductKey
 - Sales | ResellerKey

- Sales | SalesOrderNumber
- Sales | SalesTerritoryKey
- Salesperson | EmployeeID
- Salesperson | EmployeeKey
- Salesperson | UPN
- SalespersonRegion | EmployeeKey
- SalespersonRegion | SalesTerritoryKey
- Targets | EmployeeID

3. In the **Properties** pane, slide the **Is Hidden** property to **Yes**.

The columns were hidden because they're either used by relationships or will be used in row-level security configuration or calculation logic.

*You'll use the **SalesOrderNumber** in a calculation in the **Create DAX Calculations in Power BI Desktop** lab.*

1. Multi-select the following three columns:

- Product | Standard Cost
- Sales | Cost
- Sales | Sales

2. In the **Properties** pane, from inside the **Formatting** section, set the **Decimal Places** property to **0** (zero).

Explore the model interface

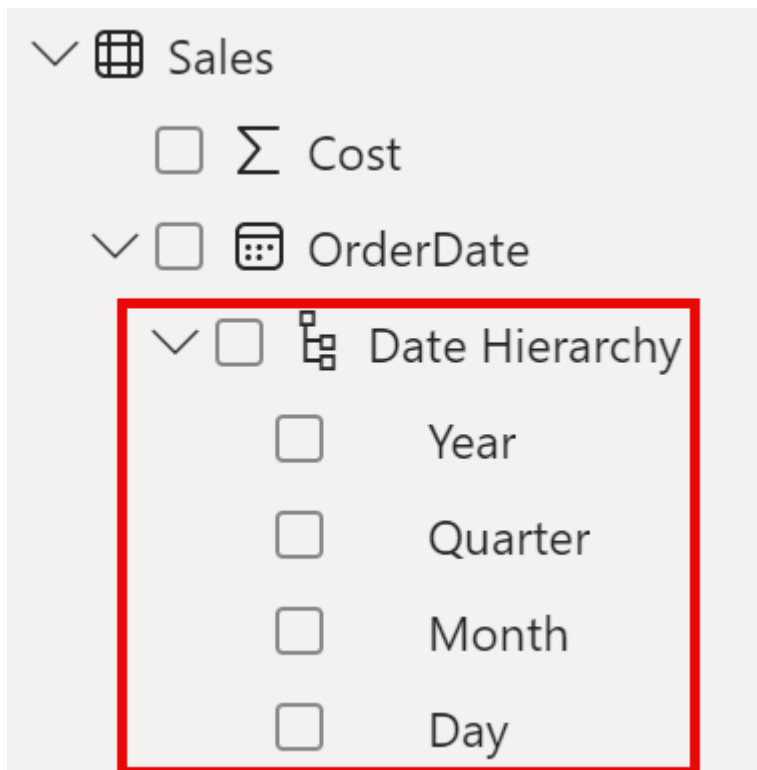
In this task you'll switch to Report view, review the data model interface, and configure the auto date/time setting.

1. Switch to **Report** view.

2. In the **Data** pane, notice the following:

- Columns, hierarchies and their levels are fields, which can be used to configure report visuals
- Only fields relevant to report authoring are visible

- The **SalespersonRegion** table isn't visible—because all of its fields are hidden
 - Spatial fields in the **Region** and **Reseller** table are adorned with a spatial icon
 - Fields adorned with the sigma symbol (Σ) will summarize, by default
 - A tooltip appears when hovering the cursor over the **Sales | Cost** field
3. Expand the **Sales | OrderDate** field, and then notice that it reveals a **Date Hierarchy**. The **Targets | TargetMonth** field delivers a similar hierarchy.



Important: These hierarchies weren't created by you. They were created automatically as a default setting. There's a problem, however. *The Adventure Works financial year commences on July 1 of each year. But, in these automatically created date hierarchies, the date hierarchy year commences on January 1 of each year.*

1. To turn off the **Auto date/time** setting, Navigate to **File > Options and Settings > Options**.
2. Under the **Current File** section, navigate to **Data Load > Time Intelligence**, and uncheck **Auto Date/Time**.

Time intelligence

☒ Auto date/time ⓘ [Learn more](#)

3. In the **Data** pane, notice that the date hierarchies are no longer available.

Create quick measures

In this task, you'll create two quick measures to calculate profit and profit margin. A quick measure creates the calculation formula for you. They're easy and fast to create for simple and common calculations.

1. In the **Data** pane, right-click the **Sales** table, and then select **New Quick Measure**.

Select

New measure

New column

New quick measure

2. In the **Quick Measures** window, in the **Calculation** dropdown list, from inside the **Mathematical Operations** group, select **Subtraction**.

Quick measure



Select a calculation to create a measure or describe the measure you need and we'll generate suggestions in DAX, which you can customize later.

Calculations



Suggestions with Copilot

Select a calculation



Quarter-to-date total

Month-to-date total

Year-over-year change

Quarter-over-quarter change

Month-over-month change

Rolling average

Totals

Running total

Total for category (filters applied)

Total for category (filters not applied)

Mathematical operations

Addition

Subtraction

3. In the **Data** pane of the **Quick Measures** window, expand the **Sales** table.
4. Drag the **Sales** field into the **Base Value** box.
5. Drag the **Cost** field into the **Value to Subtract** box, then **select Add**.

Calculations  Suggestions with Copilot

Subtraction 

Calculate the difference between two values. [Learn more](#)

Base value ⓘ

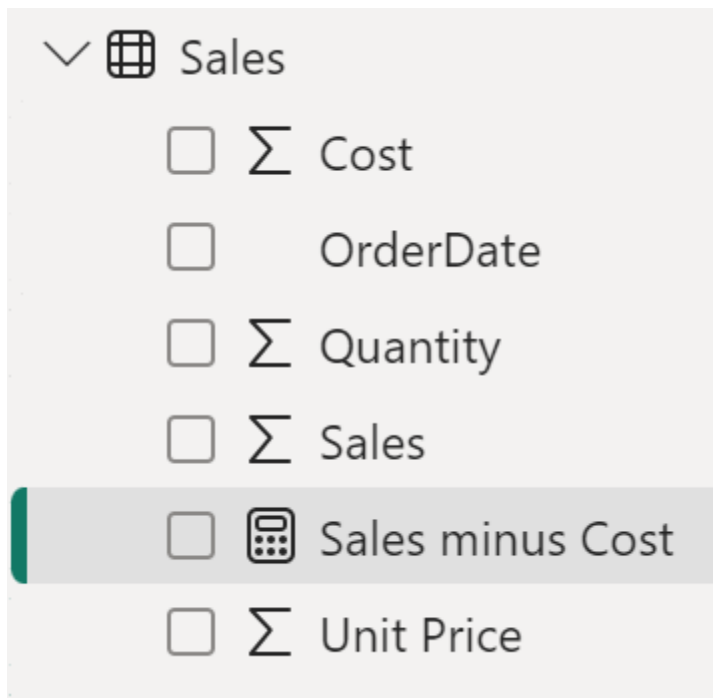
Sum of Sales   

Value to subtract ⓘ

Sum of Cost   

6. In the **Data** pane, inside the **Sales** table, notice that new measure.

Measures are indicated by the calculator icon.



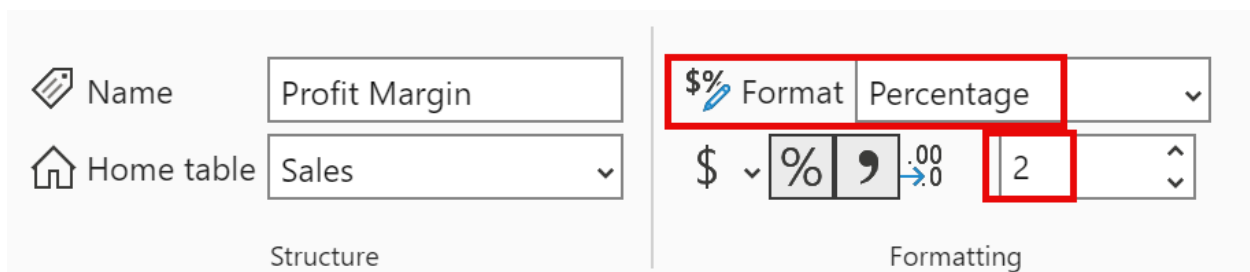
7. To rename the measure, right-click it, select **Rename**, then rename to **Profit**.

*Tip: To rename a field, you can also double-click it, or select it and press **F2**.*

8. In the **Sales** table, add a second quick measure, based on the following requirements:

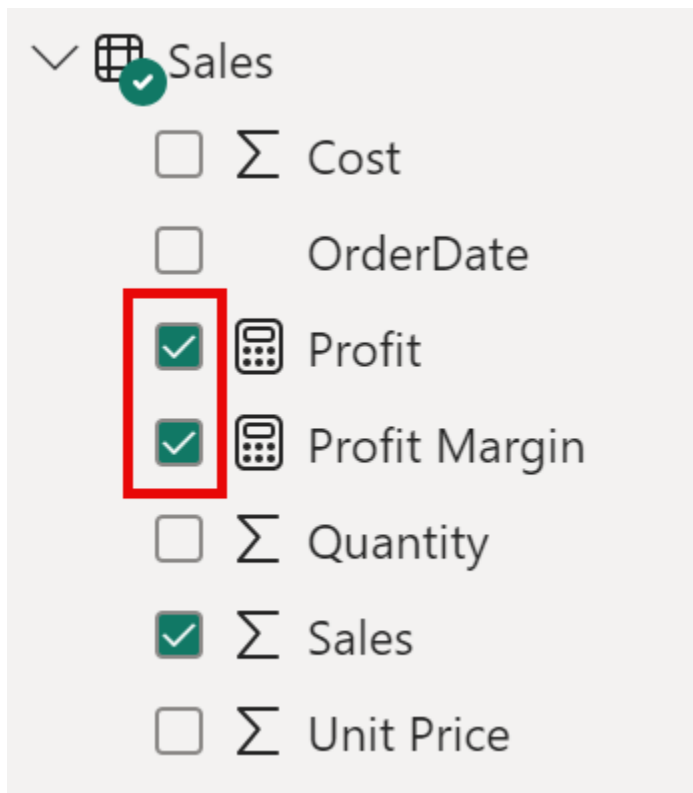
- Use the **Division** mathematical operation
- Set the **Numerator** to the **Sales | Profit** field
- Set the **Denominator** to **Sales | Sales** field
- Rename the measure as **Profit Margin**

9. Ensure the **Profit Margin** measure is selected, and then on the **Measure Tools** contextual ribbon, set the format to **Percentage**, with two decimal places.



10. To test the two measures, first select the **Table** visual.

11. In the **Data** pane, check the two measures.



12. Select and drag the right guide to widen the table visual.

The screenshot shows a table visual with the following data:

Category	Sales	Profit	Prc
Accessories	\$539,106	\$188,081	
Bikes	\$64,069,033	(\$435,679)	
Clothing	\$1,714,056	\$245,857	
Components	\$11,226,375	\$1,001,235	
Total	\$77,548,570	\$999,495	

A red rectangle highlights the rightmost column header 'Prc', and a red arrow points to the right, indicating the action to drag the column guide to widen the table.

13. Verify that the measures produce reasonable results that are correctly formatted.

Category	Sales	Profit	Profit Margin
Accessories	\$539,106	\$188,081	34.89 %
Bikes	\$64,069,033	(\$435,679)	-0.68 %
Clothing	\$1,714,056	\$245,857	14.34 %
Components	\$11,226,375	\$1,001,235	8.92 %
Total	\$77,548,570	\$999,495	1.29 %

Create a many-to-many relationship

In this task, you'll create a many-to-many relationship between the **Salesperson** table and the **Sales** table.

1. In Power BI Desktop, in Report view, in the **Data** pane, check the follow two fields to create a new **table** visual.
 - Salesperson | Salesperson
 - Sales | Sales

Salesperson	Sales
Amy Alberts	\$737,568
David Campbell	\$3,614,761
Garrett Vargas	\$3,486,102
Jae Pak	\$8,099,817
Jillian Carson	\$9,755,992
José Saraiva	\$5,536,439
Linda Mitchell	\$10,158,635
Lynn Tsoflias	\$1,216,917
Michael Blythe	\$8,952,751
Pamela Ansman-Wolfe	\$3,187,720
Rachel Valdez	\$1,681,208
Total	\$77,548,570

The table displays sales made by each salesperson. However, there's another relationship between salespeople and sales. *Some salespeople belong to one, two, or possibly more sales regions.* In addition, *sales regions can have multiple salespeople assigned to them.*

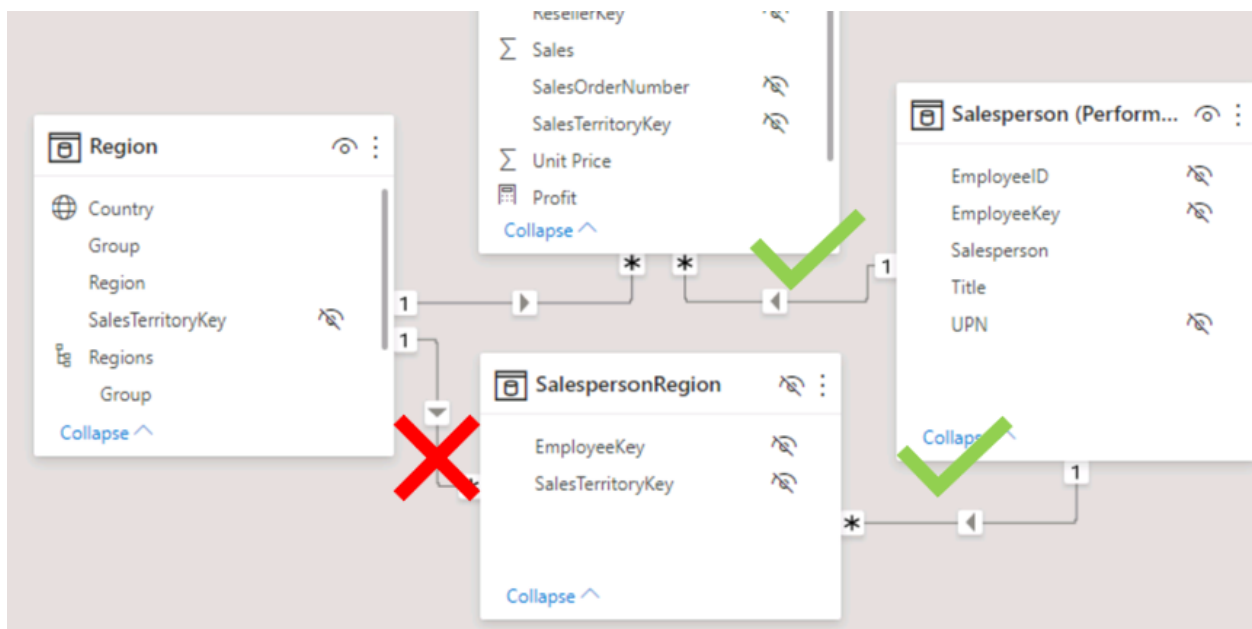
From a performance management perspective, a salesperson's sales (based on their assigned territories) need to be analyzed and compared with sales targets. You'll create relationships to support this analysis in the next exercise.

2. Notice that **Michael Blythe** has sold almost \$9 million.
3. Switch to **Model view**, then drag the **SalespersonRegion** table to position it between the **Region** and **Salesperson** tables.
4. Use the **drag-and-drop technique** to create the following two model relationships:
 - **Salesperson | EmployeeKey to SalespersonRegion | EmployeeKey**
 - **Region | SalesTerritoryKey to SalespersonRegion | SalesTerritoryKey**

The **SalespersonRegion** table can be considered to be a *bridging table*.

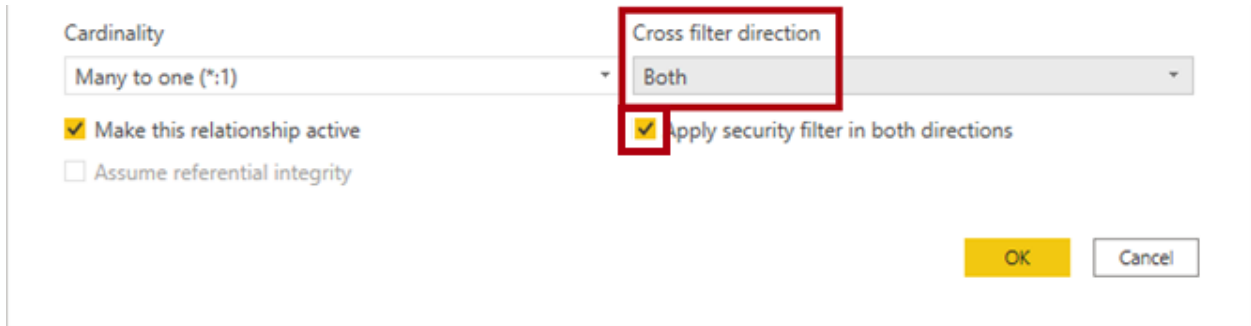
5. Switch to Report view, and then notice that the **visual hasn't updated**—the sales result for Michael Blythe hasn't changed.
6. Switch back to Model view, and then **follow the relationship filter directions** (arrowhead) from the **Salesperson** table.

Consider that the **Salesperson** table filters the **Sales** table. It also filters the **SalespersonRegion** table, but it **doesn't continue by propagating filters to the Region table** (the arrowhead is pointing the wrong direction).

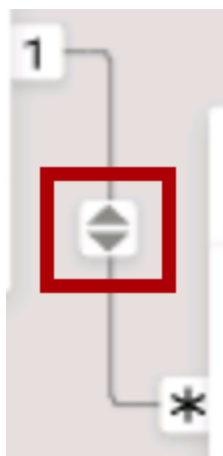


7. To edit the relationship between the **Region** and **SalespersonRegion** tables, **double-click the relationship**.
8. In the **Edit Relationship** window, in the **Cross Filter Direction** dropdown list, select **Both**.

9. Check the **Apply Security Filter in Both Directions** checkbox, then select OK.



10. Notice that the relationship has a **double arrowhead** now.



11. Switch to **Report view**, and then notice that the sales values have **still not changed**.

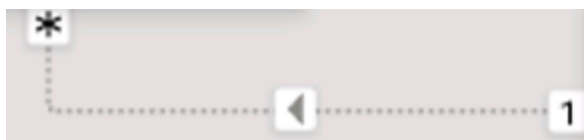
*The issue now relates to the fact that there are two possible filter propagation paths between the **Salesperson** and **Sales** tables. This ambiguity is internally resolved, based on a “least number of tables” assessment. To be clear, you shouldn’t design models with this type of ambiguity—the issue will be addressed in part later in this lab, and by the completion of the **Create DAX Calculations in Power BI Desktop** lab.*

12. Switch to Model view to **force filter propagation via the bridging table**. Edit (double-click) the relationship between the **Salesperson** and **Sales** tables.

13. In the **Edit Relationship** window, **uncheck the Make This Relationship Active** checkbox, and select **OK**.

The filter propagation will now follow the only active path.

14. In the diagram, notice that the **inactive relationship is represented by a dashed line**.



15. Switch to Report view, and then notice that the sales for Michael Blythe are now nearly \$22 million.

Salesperson	Sales
Amy Alberts	\$10,288,626
Brian Welcker	\$77,548,570
David Campbell	\$12,004,822
Garrett Vargas	\$13,875,633
Jae Pak	\$8,410,883
Jillian Carson	\$7,633,387
José Saraiva	\$13,875,633
Linda Mitchell	\$25,634,503
Lynn Tsoflias	\$1,391,025
Michael Blythe	\$21,987,348
Pamela Ansman-Wolfe	\$30,005,939
Total	\$77,548,570

16. Notice also, that the sales for each salesperson—if added—would exceed the table total.

It's a common observation of a many-to-many relationship due to the double, triple, etc. counting of regional sales results. Consider Brian Welcker, the second salesperson listed. His sales amount equals the total sales amount. It's the correct result due to the fact the he's the Director of Sales; his sales are measured by the sales of all regions.

*While the many-to-many relationship is now working, it's now not possible to analyze sales made by a salesperson (because the relationship is inactive). You'll be able to reactivate the relationship when you introduce a calculated table that will allow analyzing sales made in the sales region(s) assigned to the salesperson (for performance analysis) in the **Create DAX Calculations in Power BI Desktop** lab.*

17. Switch to Model view, and then in the diagram, select the **Salesperson** table.

18. In the **Properties** pane, in the **Name** box, replace the text with **Salesperson (Performance)**.

The renamed table now reflects its purpose: it's used to report and analyze the performance of salespeople based on the sales of their assigned sales regions.

Relate the Targets table

In this task, you'll create a relationship to the **Targets** table.

1. Create a relationship from the **Salesperson (Performance) | EmployeeID** column and the **Targets | EmployeeID** column.
2. In Report view, add the **Targets | Target** field to the table visual.
3. Resize the table visual so all columns are visible.

Salesperson	Sales	Target
Amy Alberts	\$10,288,626	\$19,450,000
Brian Welcker	\$77,548,570	\$221,700,000
David Campbell	\$12,004,822	\$19,625,000
Garrett Vargas	\$13,875,633	\$23,675,000
Jae Pak	\$8,410,883	\$13,575,000
Jillian Carson	\$7,633,387	\$13,675,000
José Saraiva	\$13,875,633	\$18,875,000
Linda Mitchell	\$25,634,503	\$40,850,000
Lynn Tsoflias	\$1,391,025	\$3,210,000
Michael Blythe	\$21,987,348	\$31,150,000
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000
Total	\$77,548,570	\$676,210,000

It's now possible to visualize sales and targets—but take care for two reasons. First, there's no filter on a time period, and so targets also include future target amounts. Second, targets aren't additive, and so the total shouldn't be displayed. They can either be disabled by formatting the visual or removed by using calculation logic.

Lab complete

Knowledge check

Summary

You have learned about modeling data in Power BI, which includes such topics as [creating common date tables](#), learning about and configuring [many-to-many relationships](#), resolving [circular relationships](#), designing [star schemas](#), and much more. These skills are crucial to the Power BI practitioner's toolkit so that it is [easier to build visuals](#) and hand off your report elements to other teams. With this foundation, you now have the ability to explore the many nuances of the semantic model.

Write DAX formulas for Power BI Desktop models

Introduction

By using Data Analysis Expressions (DAX), you can add three types of calculations to your semantic model:

- Calculated tables
- Calculated columns
- Measures

Note

DAX can also be used to define row-level security (RLS) rules, which are expressions that enforce filters over model tables. However, rules aren't considered to be model calculations so they're out of scope for this module. For more information, see [Row-level security \(RLS\) with Power BI](#).

Calculated tables

You can write a DAX formula to add a calculated table to your model. The formula can duplicate or transform existing model data, or create a series of data, to produce a new table. Calculated table data is always imported into your model, so it increases the model storage size and can prolong data refresh time.

Note

A calculated table can't connect to external data; you need to use Power Query to accomplish that task.

Calculated tables can be useful in various scenarios:

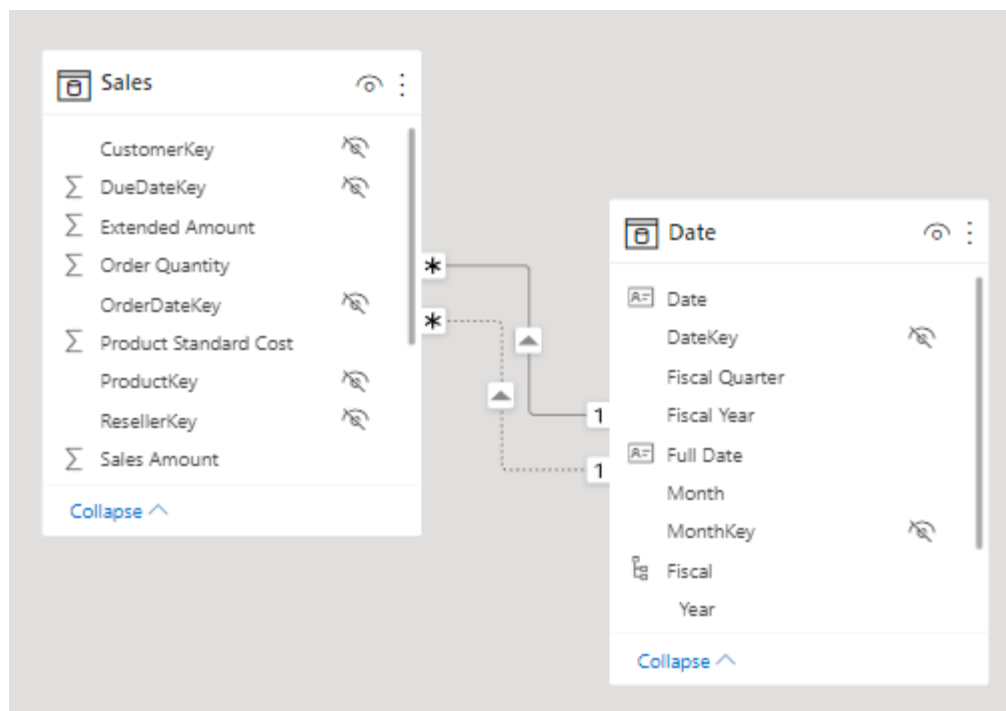
- Date tables
- Role-playing dimensions
- What-if analysis

Date tables

Date tables are required to apply special time filters known as *time intelligence*. DAX time intelligence functions only work correctly when a date table is set up. When your source data doesn't include a date table, you can create one as calculated tables by using the [CALENDAR](#) or [CALENDARAUTO](#) DAX functions.

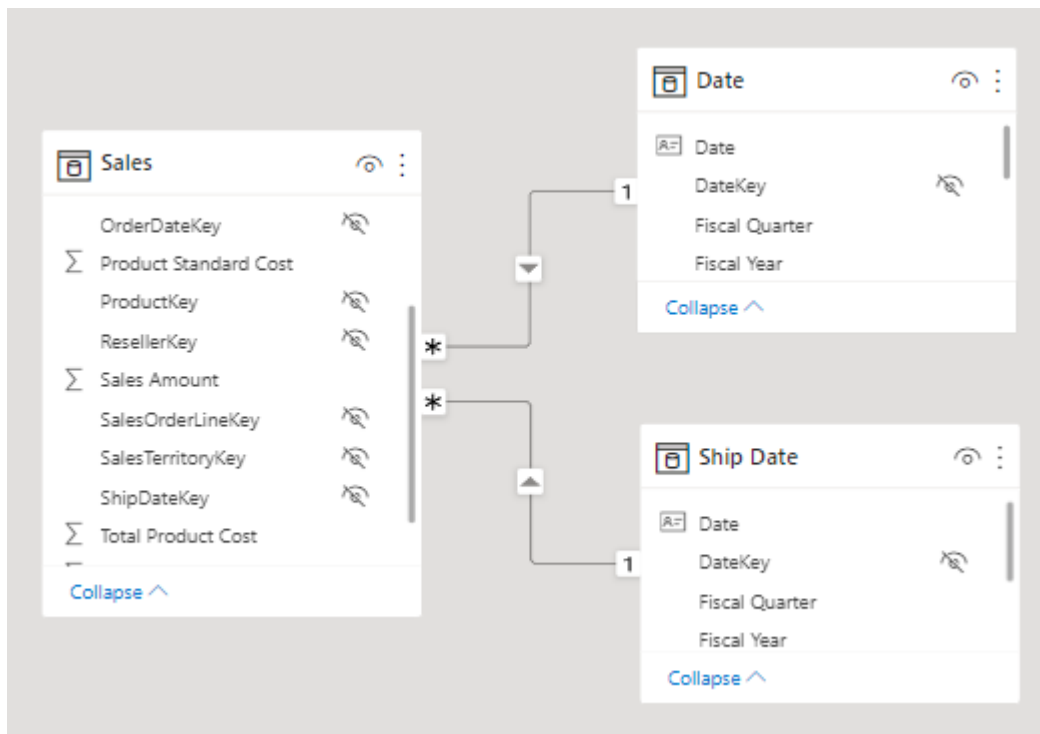
Role-playing dimensions

When two model tables have multiple relationships, it could be because your model has a role-playing dimension. For example, if you have a table named **Sales** that includes two date columns, **OrderDateKey** and **ShipDateKey**, both columns are related to the **Date** column in the **Date** table. In this case, the **Date** table is described as a role-playing dimension because it could play the role of *order date* or *ship date*.



Microsoft Power BI models only allow one active relationship between tables, which in the model diagram is indicated as a solid line. The active relationship is used by default to propagate filters, which in this case would be from the **Date** table to the **OrderDateKey** column in the **Sales** table. Any remaining relationships between the two tables are inactive. In a model diagram, the relationships are represented as dashed lines. Inactive relationships are only used when they're expressly requested in a calculated formula by using the **USERELATIONSHIP** DAX function.

Perhaps a better model design could have two date tables, each with an active relationship to the **Sales** table. Thus, report users can filter by order date or ship date, or both at the same time. A calculated table can duplicate the **Date** table data to create the **Ship Date** table.



What-if analysis

Power BI Desktop supports a feature called [What-if parameters](#). When you create a what-if parameter, a calculated table is automatically added to your model.

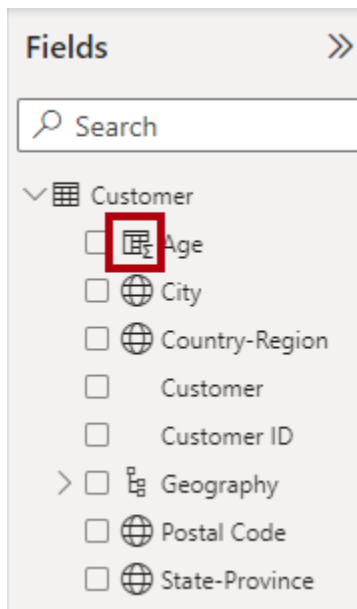
What-if parameters allow report users to select or filter by values that are stored in the calculated table. Measure formulas can use selected value(s) in a meaningful way. For example, a **what-if** parameter could allow the report user to select a hypothetical currency exchange rate, and a measure could divide revenue values (in a local currency) by the selected rate.

Notably, what-if calculated tables aren't related to other model tables because they're not used to propagate filters. For this reason, they're sometimes called *disconnected tables*.

Calculated columns

You can write a DAX formula to add a calculated column to any table in your model. The formula is evaluated for each table row and it returns a single value. When added to an Import storage mode table, the formula is evaluated when the semantic model is refreshed, and it increases the storage size of your model. When added to a DirectQuery storage mode table, the formula is evaluated by the underlying source database when the table is queried.

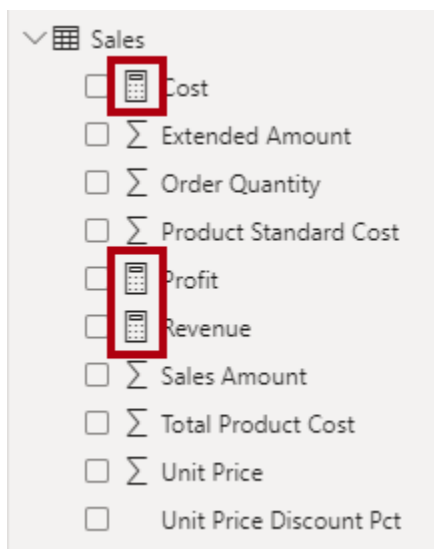
In the **Fields** pane, calculated columns are enhanced with a special icon. The following example shows a single calculated column in the **Customer** table called **Age**.



Measures

You can write a DAX formula to add a measure to any table in your model. The formula is concerned with achieving summarization over model data. Similar to a calculated column, the formula must return a single value. Unlike calculated columns, which are evaluated at data refresh time, measures are evaluated at query time. Their results are never stored in the model.

In the **Fields** pane, measures are shown with the calculator icon. The following example shows three measures in the **Sales** table: **Cost**, **Profit**, and **Revenue**.



Occasionally, measures can be described as **explicit measures**. To be clear, explicit measures are model calculations that are written in DAX and are commonly referred to as

simply *measures*. Yet, the concept of *implicit measures* exists, too. Implicit measures are columns that can be summarized by visuals in simplistic ways, like `count`, `sum`, `minimum`, `maximum`, and so on. You can identify implicit measures in the **Fields** pane because they're shown with the sigma symbol (Σ).

Note

Any column can be summarized when added to a visual. Therefore, whether they're shown with the sigma symbol or not, when they're added to a visual, they can be set up as implicit measures.

Additionally, no such concept as a *calculated measure* exists in tabular modeling. The word *calculated* is used to describe calculated tables and calculated columns, which distinguishes them from tables and columns that originate from Power Query. Power Query doesn't have the concept of an explicit measure.

Write DAX formulas

Each model calculation type, calculated table, calculated column, or measure is defined by its name, followed by the equals symbol (=), which is then followed by a DAX formula. Use the following template to create a model calculation:

DAX

```
<Calculation name> = <DAX formula>
```

For example, the definition of the **Ship Date** calculated table that duplicates the **Date** table data is:

DAX

```
Ship Date = 'Date'
```

A DAX formula consists of expressions that return a result. The result is either a table object or a scalar value. Calculated table formulas must return a table object; calculated column and measure formulas must return a scalar value (single value).

Formulas are assembled by using:

- DAX functions
- DAX operators
- References to model objects
- Constant values, like the number 24 or the literal text "FY" (abbreviation for fiscal year)

- DAX variables
- Whitespace

Tip

When entering DAX formulas in Power BI Desktop, you have the benefit of **IntelliSense**. IntelliSense is a code-completion aid that lists functions and model resources. When you select a DAX function, it also provides you with a definition and description. We recommend that you use IntelliSense to help you quickly build accurate formulas.

DAX functions

Similar to Microsoft Excel, DAX is a functional language meaning that formulas rely on functions to accomplish specific goals. Typically, DAX functions have arguments that allow passing in variables. Formulas can use many function calls and will often nest functions within other functions.

In a formula, function names must be followed by parentheses. Within the parentheses, variables are passed in.

Note

Some functions don't take arguments, or arguments might be optional.

Working with DAX functions is described later in this module.

DAX operators

Formulas also rely on operators, which can perform arithmetic calculations, compare values, work with strings, or test conditions.

DAX operators are described in more detail later in this module.

References to model objects

Formulas can only refer to three types of model objects: tables, columns, or measures. A formula can't refer to a hierarchy or a hierarchy level. (Recall that a hierarchy level is based on a column, so your formula can refer to a hierarchy level's column.)

Table references

When you reference a table in a formula, officially, the table name is enclosed within single quotation marks. In the following calculated table definition, notice that the **Date** table is enclosed within single quotation marks.

DAX

```
Ship Date = 'Date'
```

However, single quotation marks can be omitted when both of the following conditions are true:

3. The table name does not include embedded spaces.
4. The table name isn't a reserved word that's used by DAX. All DAX function names and operators are reserved words. *Date* is a DAX function name, which explains why, when you are referencing a table named **Date**, that you must enclose it within single quotation marks.

In the following calculated table definition, it's possible to omit the single quotation marks when referencing the **Airport** table:

DAX

```
Arrival Airport = Airport
```

Column references

When you reference a column in a formula, the column name must be enclosed within square brackets. Optionally, it can be preceded by its table name. For example, the following measure definition refers to the **Sales Amount** column.

DAX

```
Revenue = SUM([Sales Amount])
```

Because column names are unique within a table but not necessarily unique within the model, you can disambiguate the column reference by preceding it with its table name. This disambiguated column is known as a *fully qualified column*. Some DAX functions require passing in fully qualified columns.

Tip

To improve the readability of your formulas, we recommend that you always precede a column reference with its table name.

The previous example measure definition can be rewritten as:

DAX

```
Revenue = SUM(Sales[Sales Amount])
```

Measure references

When you reference a measure in a formula, like column name references, the measure name **must be enclosed within square brackets**. For example, the following measure definition **refers to the Revenue and Cost** measures.

DAX

```
Profit = [Revenue] - [Cost]
```

If you're a DAX beginner, the fact that **column and measure references are always enclosed within square brackets can cause confusion** when you're trying to read a formula. However, as you **become proficient with DAX fundamentals, you'll be able to determine which type** of object it is because, in DAX formulas, columns, and measures are used in different ways.

Tip

It's possible to precede a measure reference with its table name. However, **measures are a model-level object**. While **they're assigned to a home table, it's only a cosmetic relationship to logically organize measures in the Fields pane**.

Therefore, while we **recommend that you always precede a column reference with its table name, the inverse is true for measures**: We recommend that you never precede a measure reference with its table name.

For more information, see [Column and measure references](#).

DAX variables

Formulas **can declare DAX variables to store results**.

How and when to use DAX variables is described later in this module.

Whitespace

Whitespace refers to **characters that you can use to format your formulas** in a way that's **quick and simple to understand**. Whitespace characters include:

- Spaces
- Tabs
- Carriage returns

Whitespace is **optional and it doesn't modify your formula logic or negatively impact performance**. We strongly recommend that you adopt a format style and apply it consistently, and consider the following recommendations:

- Use **spaces between operators**.
- Use **tabs to indent nested function** calls.
- Use **carriage returns to separate function arguments**, especially when it's too long to fit on a single line. Formatting in this way makes it simpler to troubleshoot, especially when the formula is missing a parenthesis.
- **Err on the side of too much whitespace** than too little.

Tip

In the formula bar, **to enter a carriage return, press Shift+Enter**. Pressing **Enter** alone will commit your formula.

Consider the following measure definition that's **written in a single line** and that includes five DAX function calls:

DAX

```
Revenue YoY % = DIVIDE([Revenue] - CALCULATE([Revenue],
SAMEPERIODLASTYEAR('Date'[Date])),
CALCULATE([Revenue],
SAMEPERIODLASTYEAR('Date'[Date])))
```

The following example is the **same measure definition but now formatted**, which helps make it easier to read and understand:

DAX

```
Revenue YoY % =
DIVIDE(
    [Revenue]
    - CALCULATE(
        [Revenue],
        SAMEPERIODLASTYEAR('Date'[Date])
    ),
    CALCULATE(
        [Revenue],
        SAMEPERIODLASTYEAR('Date'[Date])
    )
)
```

Try formatting the measure on your own. Open the **Adventure Works DW 2020 M02.pbix** Power BI Desktop file and then, in the **Fields** pane, expand the **Sales** table and then select the **Revenue YoY %** measure. In the formula bar, use tab and carriage return characters to produce the same result as the previous example. When you add a carriage return, remember to press **Shift+Enter**.

This measure definition can be further improved for readability and performance, which will be explained later in this module.

Tip

An excellent formatting tool from another source that can help you format your calculations is **DAX Formatter**. This tool allows you to paste in your calculation and format it. You can then copy the formatted calculation to the clipboard and paste it back into Power BI Desktop.

DAX data types

Semantic model columns have a set data type, which ensures that all column values conform to that data type. Column data types are defined in Power Query, or in the case of calculated columns, it's inferred from the formula. Measure data types, similar to calculated column data types, are inferred from the formula.

Model data types aren't the same as DAX data types, though a direct relationship exists between them. The following table lists the model data types and DAX data types. Notice the supported range of values for each data type.

Model data type	DAX data type	Description
Whole number	64-bit integer	-2^{63} through $2^{63}-1$
Decimal number	64-bit real	Negative: -1.79×10^{308} through -2.23×10^{-308} - zero (0) - positive: 2.23×10^{-308} through 1.79×10^{308} - Limited to 17 decimal digits
Boolean	Boolean	TRUE or FALSE
Text	String	Unicode character string
Date	Date/time	Valid dates are all dates after January 1, 1900

Currency	Currency	-9.22 x 10 ¹⁴ through 9.22 x 10 ¹⁴ - limited to four decimal digits of fixed precision
N/A	BLANK	In some cases, it's the equivalent of a database (SQL) NULL

BLANK data type

The BLANK data type [deserves a special mention](#). DAX uses BLANK for both database NULL and for blank cells in Excel. BLANK doesn't mean zero. Perhaps it might be simpler to think of it as the *absence of a value*.

Two DAX functions are related to the BLANK data type: the **BLANK** DAX function returns BLANK, while the **ISBLANK** DAX function tests whether an expression evaluates to BLANK.

Work with DAX functions

The DAX function library [consists of hundreds of functions](#), each designed to accomplish a specific goal.

Because DAX originated with the Power Pivot add-in for Microsoft Excel 2010, [over 80 functions](#) are available that [can also be found in Excel](#). It was a deliberate design strategy by Microsoft to ensure that [Excel users can quickly become productive with DAX](#).

However, [many functions exist that you won't find in Excel because they're specific to data modeling](#):

- Relationship navigation functions
- Filter context modification functions
- Iterator functions
- Time intelligence functions
- Path functions

Tip

[To search for documentation](#) that is related to a DAX function, in a web search, [enter the keyword DAX followed by the function name](#).

For more information, see the [DAX function reference](#).

Functions that originate from Excel

The following sections consider several useful functions that [you might already be familiar with because they exist in Excel](#).

The **IF** DAX function tests whether a condition that's provided as the first argument is met. It returns one value if the condition is TRUE and returns the other value if the condition is FALSE. The function's syntax is:

DAX

```
IF(<logical_test>, <value_if_true>[, <value_if_false>])
```

Tip

A function argument is optional when documentation shows it enclosed within square brackets.

If **logical_test** evaluates to FALSE and **value_if_false** isn't provided, the function will return **BLANK**.

Many Excel summarization functions are available, including SUM, COUNT, AVERAGE, MIN, MAX, and many others. The only difference is that in DAX, you pass in a column reference, whereas in Excel, you pass in a range of cells.

Many Excel mathematic, text, date and time, information, and logical functions are available as well. For example, a small sample of Excel functions that are available in DAX include ABS, ROUND, SQRT, LEN, LEFT, RIGHT, UPPER, DATE, YEAR, MONTH, NOW, ISNUMBER, TRUE, FALSE, AND, OR, NOT, and IFERROR.

Functions that don't originate from Excel

Two useful DAX functions that aren't specific to modeling and that don't originate from Excel are DISTINCTCOUNT and DIVIDE.

DISTINCTCOUNT function

You can use the **DISTINCTCOUNT** DAX function to count the number of distinct values in a column. This function is especially powerful in an analytics solution. Consider that the count of customers is different from the count of *distinct* customers. The latter doesn't count repeat customers, so the difference is "How many customers" compared with "How many *different* customers."

DIVIDE function

You can use the **DIVIDE** DAX function to achieve division. You must pass in numerator and denominator expressions. Optionally, you can pass in a value that represents an alternate result. The DIVIDE function's syntax is:

DAX

```
DIVIDE(<numerator>, <denominator>[, <alternate_result>])
```


The DIVIDE function automatically handles division by zero cases. If an alternate result isn't passed in, and the denominator is zero or BLANK, the function returns BLANK. When an alternate result is passed in, it's returned instead of BLANK.

This function is convenient because it saves your expression from having to first test the denominator value. The function is also better optimized for testing the denominator value than the IF function. The performance gain is significant because checking for division by zero is expensive. What's more, using the DIVIDE function results in a more concise and elegant expression.

Tip

We recommend that you use the DIVIDE function whenever the denominator is an expression that could return zero or BLANK. In the case that the denominator is a constant value, we recommend that you use the divide operator (/), which is introduced later in this module. In this case, the division is guaranteed to succeed, and your expression will perform better because it will avoid unnecessary testing.

Use DAX operators

Your DAX formulas can use operators to create expressions that perform arithmetic calculations, compare values, work with strings, or test conditions.

Tip

Many DAX operators and precedence order are the same as those found in Excel.

Arithmetic operators

The following table lists the arithmetic operators.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Exponentiation

Remember, when you are dividing two expressions, and when the denominator could return zero or BLANK, it's more efficient and safer to use the DIVIDE DAX function.

Comparison operators

The following table lists the **comparison operators**, which are used to compare two values. The result is either TRUE or FALSE.

Operator	Description
=	Equal to
==	Strict equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<>	Not equal to

All comparison operators, except **strict equal to (==)**, treat BLANK as equal to the number zero, an empty string (""), the date December 30, 1899, or FALSE. It means that the expression `[Revenue] = 0` will be TRUE when the value of [Revenue] is either zero or BLANK. In contrast, `[Revenue] == 0` is TRUE only when the value of [Revenue] is zero.

Text concatenation operator

Use the **ampersand (&) character to connect, or concatenate, two text values** to produce one continuous text value. For example, consider the following calculated column definition:

DAX

```
Model Color = 'Product'[Model] & "-" & 'Product'[Color]
```

Logical operators

Use logical operators **to combine expressions that produce a single result**. The following

Operator	Description
&&	Creates an AND condition between two expressions where each has a Boolean result. If both expressions return TRUE, the combination of the expressions also returns TRUE; otherwise the combination returns FALSE.
(double pipe)	Creates an OR condition between two logical expressions. If either expression returns TRUE, the result is TRUE; only when both expressions are FALSE is the result FALSE.

IN	Creates a logical OR condition between each row that is being compared to a table. Note: The table constructor syntax uses braces.
NOT	Inverts the state of a Boolean expression (FALSE to TRUE, and vice versa).

An example that uses the **IN** logical operator is the **ANZ Revenue** measure definition, which uses the **CALCULATE** DAX function to enforce a specific filter of two countries/regions: Australia and New Zealand.

Note

You'll be introduced to the powerful CALCULATE function when you learn how to modify the filter context.

DAX

```
ANZ Revenue =
CALCULATE(
    [Revenue],
    Customer[Country-Region] IN {
        "Australia",
        "New Zealand"
    }
)
```

Operator precedence

When your DAX formula includes multiple operators, DAX uses rules to determine the evaluation order, which is known as an operator precedence. Operations are ordered according to the following table.

Operator	Description
^	Exponentiation
-	Sign (as in -1)
* and /	Multiplication and division
NOT	NOT
+ and -	Addition and subtraction

&	Concatenation of two strings of text
=,==,<,>,<=,>=,<>	Comparison

When the operators have equal precedence value, they're ordered from left to right.

In general, operator precedence is the same as what's found in Excel. If you need to override the evaluation order, then group operations within parentheses.

For example, consider the following calculated column definition:

DAX

```
Extended Amount = Sales[Order Quantity] * Sales[Unit Price] * 1 - [Unit Price Discount Pct]
```

This sample calculated column definition produces an incorrect result because multiplication happens before the subtraction. The following correct calculated column definition uses parentheses to ensure that the subtractions happen before the multiplications.

DAX

```
Extended Amount = Sales[Order Quantity] * Sales[Unit Price] * (1 - [Unit Price Discount Pct])
```

Tip

Remembering operator precedence rules can be challenging, especially for DAX beginners. Consequently, we recommend that you test your formulas thoroughly. When the formulas don't produce the correct result due to an incorrect evaluation order, you can experiment by adding parentheses to adjust the evaluation order. You can also add parentheses to improve the readability of your formulas.

For more information about DAX operators and precedence order, see [DAX operators](#).

Implicit conversion

When writing a DAX formula that uses operators to combine different data types, you don't need to explicitly convert types. Usually, DAX automatically identifies the data types of referenced model objects and performs implicit conversions where necessary to complete the specified operation.

However, some limitations might exist on the values that can be successfully converted. If a value or a column has a data type that's incompatible with the current operation, DAX returns an error. For example, the attempt to multiply a date value will create an error because it isn't logical.

BLANK is handled differently, depending on the operator that is used. It's handled similar to how Excel treats BLANK, but differently to how databases (SQL) treat NULL. BLANK is treated as zero when acted on by arithmetic operators and as an empty string when concatenated to a string.

Tip

Remembering how BLANK is handled can be challenging, especially for DAX beginners. Consequently, we recommend that you test your formulas thoroughly. When BLANKs create unexpected results, consider using the IF and ISBLANK DAX functions to test for BLANK, and then respond in an appropriate way.

Use DAX variables

You can declare DAX variables in your formula expressions. When you declare at least one variable, a RETURN clause is used to define the expression, which then refers to the variables.

We recommend that you use variables because they offer several benefits:

- Improving the readability and maintenance of your formulas.
- Improving performance because variables are evaluated once and only when or if they're needed.
- Allowing (at design time) straightforward testing of a complex formula by returning the variable of interest.

The following example shows a formula that declares a variable. The Revenue YoY % measure definition is rewritten to declare a variable that's assigned the value of the prior year's revenue.

DAX

```
Revenue YoY % =  
VAR RevenuePriorYear =  
    CALCULATE(  
        [Revenue],  
        SAMEPERIODLASTYEAR('Date'[Date])  
    )  
RETURN  
    DIVIDE(
```

```

    [Revenue] - RevenuePriorYear,
    RevenuePriorYear
)

```

Notice that the **RETURN** clause refers to the variable twice. This improved measure definition formula will run in at least half the time because it doesn't need to evaluate the prior year's revenue twice.

In the **Adventure Works DW 2020 M02.pbix** Power BI Desktop file, refactor the **Revenue YoY %** measure to produce the same result as the previous example.

For more information on using DAX variables, see [Use variables to improve your formulas](#).

Knowledge check

Summary

In this module, you learned how DAX can be used to enhance your model with calculations. Calculation types include calculated tables, calculated columns, and measures.

You also learned about DAX fundamentals, which are essential for writing accurate and efficient DAX formulas. The fundamentals covered how to write DAX formulas, and you were introduced to important topics about DAX data types, DAX functions, DAX operators, references to model objects, constants, and using DAX variables.

Add measures to Power BI Desktop models

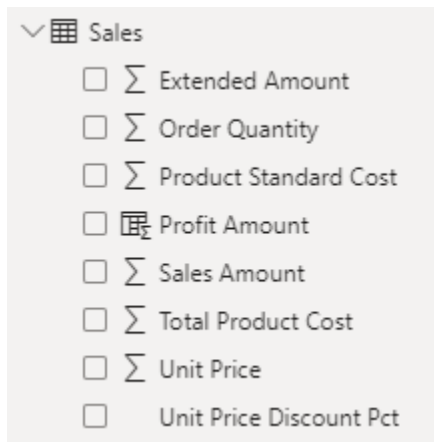
Introduction

Measures in Microsoft Power BI models are either *implicit* or *explicit*. Implicit measures are automatic behaviors that allow visuals to summarize model column data. Explicit measures, also known simply as *measures*, are calculations that you can add to your model. This module focuses on how you can use implicit measures.

In the **Fields** pane, a column that's shown with the sigma symbol (Σ) indicates two facts:

- It's a numeric column.
- It will summarize column values when it is used in a visual (when added to a field well that supports summarization).

In the following image, notice that the **Sales** table includes only fields that can be summarized, including the **Profit Amount** calculated column.



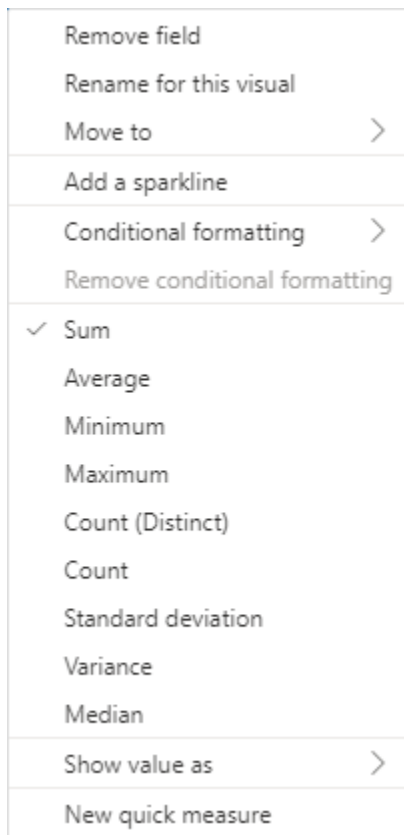
As a data modeler, you can control if and how the column summarizes by setting the **Summarization** property to **Don't summarize** or to a specific aggregation function. When you set the **Summarization** property to **Don't summarize**, the sigma symbol will no longer show next to the column in the **Fields** pane.

To observe how report authors can use implicit measures, you can first download and open the [Adventure Works DW 2020 M04.pbix](#) file.

In the report, from the **Sales** table, add the **Sales Amount** field to the matrix visual that groups fiscal year and month on its rows.

Fiscal Year	Sales Amount
FY2018	\$23,860,891.17
2017 Jul	\$1,423,357.32
2017 Aug	\$2,057,902.45
2017 Sep	\$2,523,947.55
2017 Oct	\$561,681.48
2017 Nov	\$4,764,920.16
2017 Dec	\$596,746.56

To determine how the column is summarized, in the visual fields pane, for the **Sales Amount** field, select the arrow and then review the context menu options.



Notice that the **Sum** aggregation function has a check mark next to it. This check mark indicates that the column is summarized by summing column values together. It's also possible to change the aggregation function by selecting any of the other options like average, minimum, and so on.

Next, add the **Unit Price** field to the matrix visual.

Fiscal Year	Sales Amount	Unit Price
FY2018	\$23,860,891.17	\$1,273.63
2017 Jul	\$1,423,357.32	\$1,817.15
2017 Aug	\$2,057,902.45	\$1,181.42
2017 Sep	\$2,523,947.55	\$1,030.56
2017 Oct	\$561,681.48	\$3,228.05
2017 Nov	\$4,764,920.16	\$1,009.71
2017 Dec	\$596,746.56	\$3,174.18

The default summarization is now set to **Average** (the modeler knows that it's inappropriate to sum unit price values together because they're rates, which are non-additive).

Remove field
Rename for this visual
Move >
Move to >
Add a sparkline
Conditional formatting >
Remove conditional formatting
Sum
✓ Average
Minimum
Maximum
Count (Distinct)
Count
Standard deviation
Variance
Median
Show value as >
New quick measure

Implicit measures allow the report author to start with a default summarization technique and lets them modify it to suit their visual requirements.

Numeric columns support the greatest range of aggregation functions:

- Sum
- Average
- Minimum
- Maximum
- Count (Distinct)
- Count
- Standard deviation
- Variance
- Median

Summarize non-numeric columns

Non-numeric columns can be summarized. However, the sigma symbol does not show next to non-numeric columns in the **Fields** pane because they don't summarize by default.

Text columns allow the following aggregations:

- First (alphabetically)
- Last (alphabetically)
- Count (Distinct)
- Count

Date columns allow the following aggregations:

- Earliest
- Latest
- Count (Distinct)
- Count

Boolean columns allow the following aggregations:

- Count (Distinct)
- Count

Benefits of implicit measures

Several benefits are associated with implicit measures. Implicit measures are simple concepts to learn and use, and they provide flexibility in the way that report authors visualize model data. Additionally, they mean less work for you as a data modeler because you don't have to create explicit calculations.

Limitations of implicit measures

Implicit measures do have limitations. Despite setting an appropriate summarization method, report authors could choose to aggregate a column in unsuitable ways. For example, in the matrix visual, you could modify the aggregate function of **Unit Price** to **Sum**.

Fiscal Year	Sales Amount	Sum of Unit Price
FY2018	\$23,860,891.17	\$13,905,519.77
2017 Jul	\$1,423,357.32	\$1,164,795.52
2017 Aug	\$2,057,902.45	\$1,115,258.56
2017 Sep	\$2,523,947.55	\$1,291,296.17
2017 Oct	\$561,681.48	\$561,681.48
2017 Nov	\$4,764,920.16	\$2,159,760.38
2017 Dec	\$596,746.56	\$596,746.56

The report visual obeys your setup, but it has now produced a **Sum of Unit Price** column, which [presents misleading data](#).

The most significant limitation of implicit measures is that they only work for simple scenarios, meaning that [they can only summarize column values that use a specific aggregation function](#). Therefore, in situations when you [need to calculate the ratio of each month's sales amount over the yearly sales amount](#), you'll need to produce an [explicit measure by writing a Data Analysis Expressions \(DAX\) formula](#) to achieve that more sophisticated requirement.

[Implicit measures don't work when the model is queried by using Multidimensional Expressions \(MDX\)](#). This [language expects explicit measures and can't summarize column data](#). It's used when a Power BI semantic model is queried by using [Analyze in Excel](#) or when a [Power BI paginated report](#) uses a query that is generated by the MDX graphical query designer.

Create simple measures

You can write a DAX formula to add a measure to any table in your model. [A measure formula must return a scalar or single value.](#)

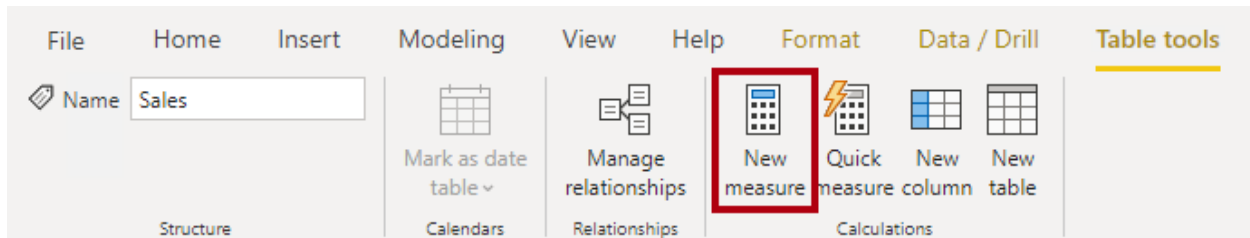
Note

In tabular modeling, no such concept as a calculated measure exists. The word *calculated* is used to describe calculated tables and calculated columns. It distinguishes them from tables and columns that originate from Power Query, which doesn't have the concept of an explicit measure.

[Measures don't store values](#) in the model. Instead, [they're used at query time to return summarizations](#) of model data. Additionally, [measures can't reference a table or column directly; they must pass the table or column into a function to produce a summarization.](#)

A [simple measure](#) is one that [aggregates the values of a single column](#); it [does what implicit measures do automatically](#).

In the next example, you will [add a measure to the Sales table](#). In the **Fields** pane, [select the Sales table](#). To [create a measure](#), in the **Table Tools** contextual ribbon, from inside the **Calculations** group, select **New measure**.



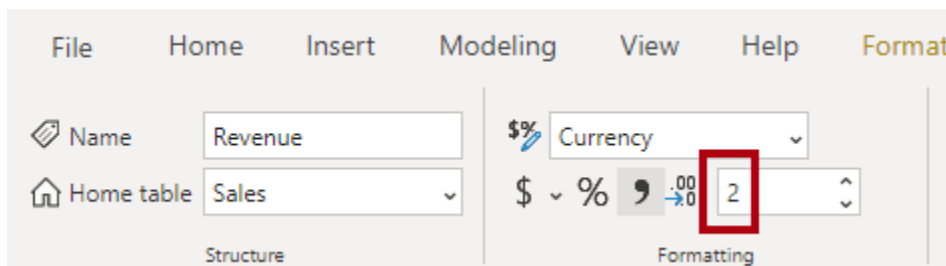
In the formula bar, enter the following measure definition and then [press Enter](#).

DAX

```
Revenue =  
SUM(Sales[Sales Amount])
```

The measure definition adds the **Revenue** measure to the **Sales** table. It [uses the SUM DAX function](#) to [sum the values of the Sales Amount column](#).

On the **Measure tools** contextual ribbon, inside the **Formatting** group, set the [decimal places to 2](#).



Tip

[Immediately after you create a measure, set the formatting options](#) to ensure well-presented and consistent values in all report visuals.

Now, add the **Revenue** measure to the matrix visual. Notice that [it produces the same result as the Sales Amount implicit measure](#).

In the matrix visual, [remove Sales Amount and Sum of Unit Price](#).

Next, [you will create more measures](#). Create the **Cost measure** by using the following measure definition, and then set the [format with two decimal places](#).

DAX

```
Cost =  
SUM(Sales[Total Product Cost])
```

Create the **Profit** measure, and then set the format with two decimal places.

DAX

```
Profit =  
SUM(Sales[Profit Amount])
```

Notice that the **Profit Amount** column is a calculated column. This topic will be discussed later in this module.

Next, create the **Quantity** measure and format it as a whole number with the thousands separator.

DAX

```
Quantity =  
SUM(Sales[Order Quantity])
```

Create three unit price measures and then set the format of each with two decimal places. Notice the different DAX aggregation functions that are used: **MIN**, **MAX**, and **AVERAGE**.

DAX

```
Minimum Price =  
MIN(Sales[Unit Price])
```

DAX

```
Maximum Price =  
MAX(Sales[Unit Price])
```

DAX

```
Average Price =  
AVERAGE(Sales[Unit Price])
```

Now, hide the **Unit Price** column, which results in report authors losing their ability to summarize the column except by using your measures.

Tip

Adding measures and hiding columns is how you, the data modeler, can limit summarization options.

Next, create the following two measures, which count the number of orders and order lines. Format both measures with zero decimal places.

DAX

```
Order Line Count =  
COUNT(Sales[SalesOrderLineKey])
```

DAX

```
Order Count =  
DISTINCTCOUNT('Sales Order'[Sales Order])
```

The **COUNT** DAX function counts the number of non-BLANK values in a column, while the **DISTINCTCOUNT** DAX function counts the number of distinct values in a column. Because an order can have one or more order lines, the **Sales Order** column will have duplicate values. A distinct count of values in this column will correctly count the number of orders.

Alternatively, you can choose the better way to write the **Order Line Count** measure. Instead of counting values in a column, it's semantically clearer to use the **COUNTROWS** DAX function. Unlike the previously introduced aggregation functions, which aggregate column values, the COUNTROWS function counts the number of rows for a table.

Modify the **Order Line Count** measure formula you created above to the following parameters:

DAX

```
Order Line Count =  
COUNTROWS(Sales)
```

Add each of the measures to the matrix visual.

All measures that you've created are considered simple measures because they aggregate a single column or single table.

Create compound measures

When a measure references one or more measures, it's known as a **compound measure**.

For this example, you will [modify the Profit measure](#) by using the following measure definition. Format the measure with two decimal places.

DAX

```
Profit =  
[Revenue] - [Cost]
```

Next, add the **Profit** measure to the matrix visual.

Now that your [model provides a way to summarize profit](#), you can [delete the Profit Amount](#) calculated column.

By removing this calculated column, you've [optimized the semantic model](#). Removing this columns [results in a decreased semantic model size](#) and [shorter data refresh times](#). The **Profit Amount** calculated column wasn't required because the **Profit** measure can directly produce the required result.

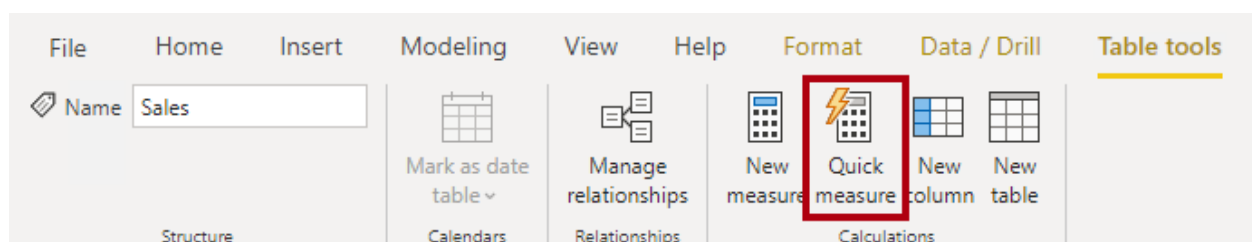
Create quick measures

Microsoft Power BI Desktop includes a feature named [Quick Measures](#). This feature helps you to [quickly perform common, powerful calculations by generating the DAX expression](#) for you.

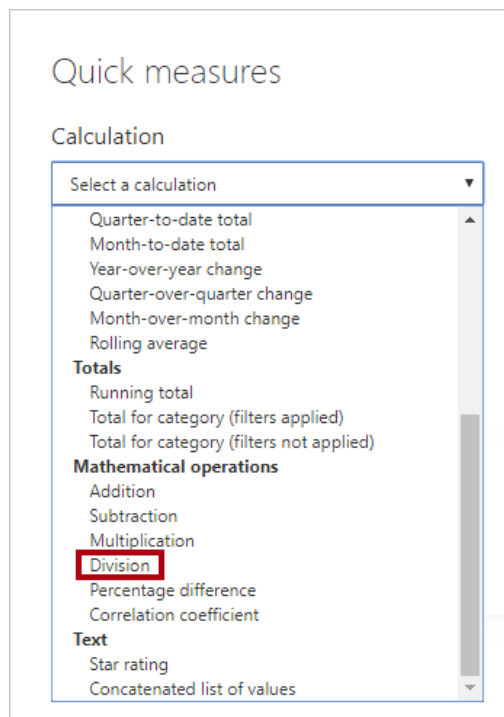
[Many categories of calculations and ways to modify each calculation are available](#) to fit your needs. Moreover, [you are able to see the DAX that's generated](#) by the quick measure and use it to jumpstart or expand your DAX knowledge.

In this next example, you'll [create another compound measure to calculate profit margin](#). However, this time, [you'll create it as a quick measure](#).

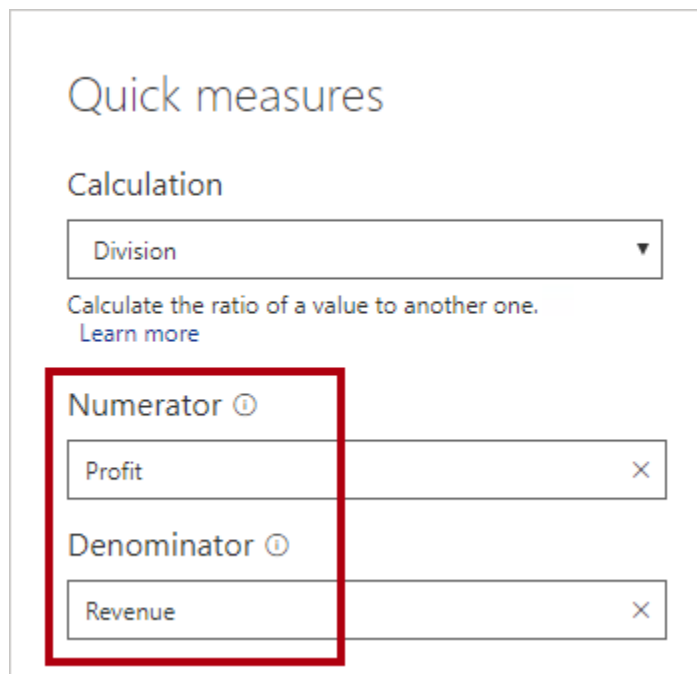
In the **Fields** pane, select the **Sales** table. On the **Table tools** contextual ribbon, from inside the **Calculations** group, select [Quick measure](#).



In the **Quick measures** window, in the [Calculation drop-down](#) list, locate the [Mathematical operations](#) group (you might need to scroll down the list) and then select [Division](#).



From the **Fields** list (in the **Quick measures** window), expand the **Sales** table and then drag the **Profit** measure into the **Numerator** box. Then, drag the **Revenue** measure into the **Denominator** box.



Select Add. In the **Fields** pane, notice the addition of the new compound measure. In the formula bar, review the measure definition.

DAX

```
Profit divided by Revenue =  
DIVIDE([Profit], [Revenue])
```

Note

After the quick measure has been created, you must apply any changes in the formula bar.

Rename the measure as **Profit Margin**, and then set the format to a percentage with two decimal places.

Add the **Profit Margin** measure to the matrix visual.

Compare calculated columns with measures

DAX beginners often experience a degree of confusion about calculated columns and measures. The following section reviews the similarities and differences between both.

Regarding **similarities between calculated columns and measures**, both are:

- **Calculations that you can add** to your semantic model.
- **Defined by using a DAX** formula.
- Referenced in DAX formulas by enclosing their **names within square brackets**.

The areas where calculated columns and measures differ include:

- **Purpose** - **Calculated columns extend a table** with a new column, while **measures define how to summarize** model data.
- **Evaluation** - **Calculated columns are evaluated by using row context** at data refresh time, while **measures are evaluated by using filter context** at query time. Filter context is introduced in a later module; it's an important topic to understand and master so that you can achieve complex summarizations.
- **Storage** - Calculated columns (in Import storage mode tables) **store a value for each row** in the table, but a **measure never stores values in the model**.
- **Visual use** - **Calculated columns** (like any column) **can be used to filter, group, or summarize** (as an implicit measure), whereas **measures are designed to summarize**.

Knowledge check

Exercise - Create DAX Calculations in Power BI Desktop

Lab story

In this lab you'll create calculated tables, calculated columns, and simple measures using Data Analysis Expressions (DAX).

In this lab you learn how to:

- Create calculated tables
- Create calculated columns
- Create measures

This lab should take approximately 45 minutes.

Get started

To complete this exercise, first open a web browser and enter the following URL to download the zip folder:

<https://github.com/MicrosoftLearning/PL-300-Microsoft-Power-BI-Data-Analyst/raw/Main/Allfiles/Labs/04-create-dax-calculations-in-power-bi-desktop\04-intro-dax.zip>

Extract the folder to the **C:\Users\Student\Downloads\04-intro-dax** folder.

Open the **04-Starter-Sales Analysis.pbix** file.

Note: You can dismiss the sign-in by selecting **Cancel**. Close any other informational windows. Select **Apply Later**, if prompted to apply changes.

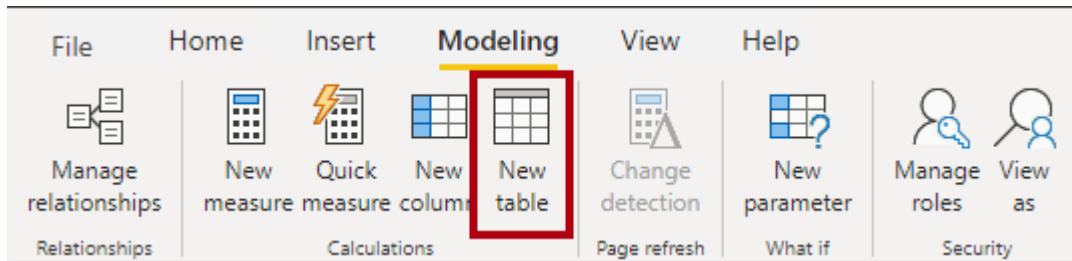
Create the Salesperson calculated table

In this task, you'll **create the Salesperson calculated table** (direct relationship to **Sales**).

A calculated table is created by first entering the table name, followed by the equals symbol (=), followed by a DAX formula that returns a table. The table name can't already exist in the data model.

The formula bar supports entering a valid DAX formula. It includes features like auto-complete, **Intellisense and color-coding**, enabling you to quickly and accurately enter the formula.

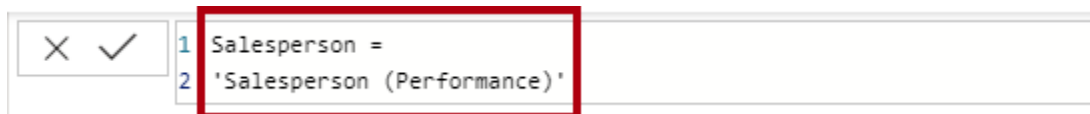
1. In Power BI Desktop, in Report view, on the **Modeling** ribbon, from inside the **Calculations** group, select **New Table**.



2. In the formula bar (which opens directly beneath the ribbon when creating or editing calculations), type **Salesperson** =, press **Shift+Enter**, type '**Salesperson (Performance)**', and then press **Enter**.

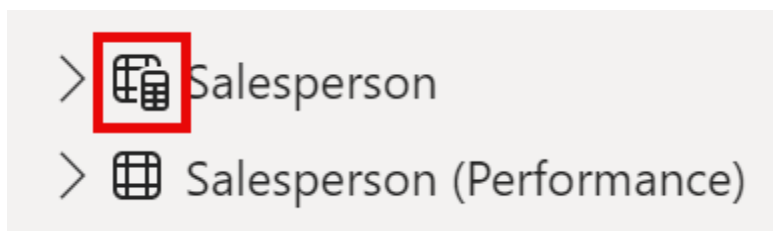
Note:

For your convenience, all DAX definitions in this lab can be copied from the snippets file, located in **04-intro-dax\Snippets.txt**.



*This table definition creates a copy of the **Salesperson (Performance)** table. It copies the data only, however model properties like visibility, formatting, etc. aren't copied.*

3. In the **Data** pane, notice that the **table icon has an additional calculator** in front of it (denoting a calculated table).



Note:

*Calculated tables are defined by using a DAX formula that returns a table. It's important to understand that **calculated tables increase the size of the data model because they materialize and store values**. They're **recomputed whenever formula dependencies are refreshed**, as will be the case for this data model when new (future) date values are loaded into tables.*

Unlike Power Query-sourced tables, **calculated tables can't be used to load data from external data sources**. They can only transform data based on what has already been loaded into the data model.

4. Switch to Model view, and notice that the **Salesperson** table is available (you may need to reset view to find table).
5. Create a relationship from the **Salesperson | EmployeeKey** column to the **Sales | EmployeeKey** column.
6. Right-click the inactive relationship between the **Salesperson (Performance)** and **Sales** tables, and then select **Delete**. When prompted to confirm the deletion, select **Yes**.
7. In the **Salesperson** table, multi-select the following columns, and then hide them (set the **Is Hidden** property to **Yes**):
 - EmployeeID
 - EmployeeKey
 - UPN
8. In the model diagram, select the **Salesperson** table.
9. In the **Properties** pane, in the **Description** box, enter: **Salesperson related to Sales**

You may **recall that descriptions appear as tooltips in the Data pane** when the user hovers their cursor over a table or field.

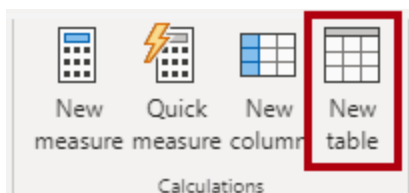
10. For the **Salesperson (Performance)** table, set the description to: **Salesperson related to region(s)**

*The data model now provides two alternatives when analyzing salespeople. The **Salesperson** table allows analyzing sales made by a salesperson, while the **Salesperson (Performance)** table allows analyzing sales made in the sales region(s) assigned to the salesperson.*

Create the Date table

In this task, you'll create the **Date** table.

1. Switch to Table view. On the **Home** ribbon tab, from inside the **Calculations** group, select **New Table**.

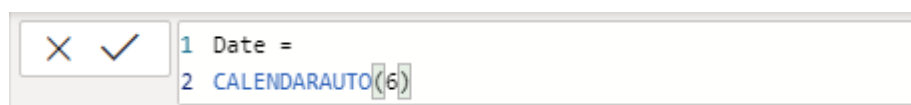


2. In the formula bar, enter the following DAX:

code

Date =

CALENDARAUTO(6)



The **CALENDARAUTO()** function returns a single-column table consisting of date values. The “auto” behavior scans all data model date columns to determine the earliest and latest date values stored in the data model. It then creates one row for each date within this range, extending the range in either direction to ensure full years of data is stored.

This function can take a single optional argument that is the last month number of a year. When omitted, the value is 12, meaning that December is the last month of the year. In this case, 6 is entered, meaning that June is the last month of the year.

3. Notice the column of date values which are formatted using US regional settings (that is, mm/dd/yyyy).

Date
07/01/2017 00:00:00
07/02/2017 00:00:00
07/03/2017 00:00:00
07/04/2017 00:00:00
07/05/2017 00:00:00
07/06/2017 00:00:00

4. At the bottom-left corner, in the status bar, notice the table statistics, confirming that 1826 rows of data have been generated, which represents five full years’ data.

TABLE: Date (1,826 rows)

Create calculated columns

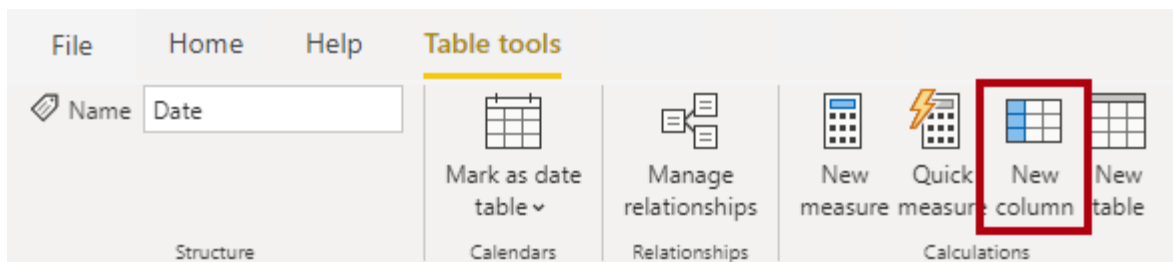
In this task, you’ll add more columns to enable filtering and grouping by different time periods. You’ll also create a calculated column to control the sort order of other columns.

Note:

For your convenience, all DAX definitions in this lab can be copied from the **Snippets.txt** file.

1. On the **Table Tools** contextual ribbon, from inside the **Calculations** group, select **New Column**.

A calculated column is created by first entering the column name, followed by the equals symbol (=), followed by a DAX formula that returns a single-value result. The column name can't already exist in the table.



2. In the formula bar, type the following (or copy from the snippets file), and then press **Enter**:

The formula uses the date's year value but adds one to the year value when the month is after June. It's how fiscal years at Adventure Works are calculated.

code

```
Year =  
"FY" & YEAR('Date'[Date]) + IF(MONTH('Date'[Date]) > 6, 1)
```

3. Use the snippets file definitions to create the following two calculated columns for the **Date** table:
 - Quarter
 - Month
4. Verify the new columns have been added.

Date ▼	Year ▼	Quarter ▼	Month ▼
7/1/2017 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
7/2/2017 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
7/3/2017 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
7/4/2017 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
7/5/2017 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
7/6/2017 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul
7/7/2017 12:00:00 AM	FY2018	FY2018 Q1	2017 Jul

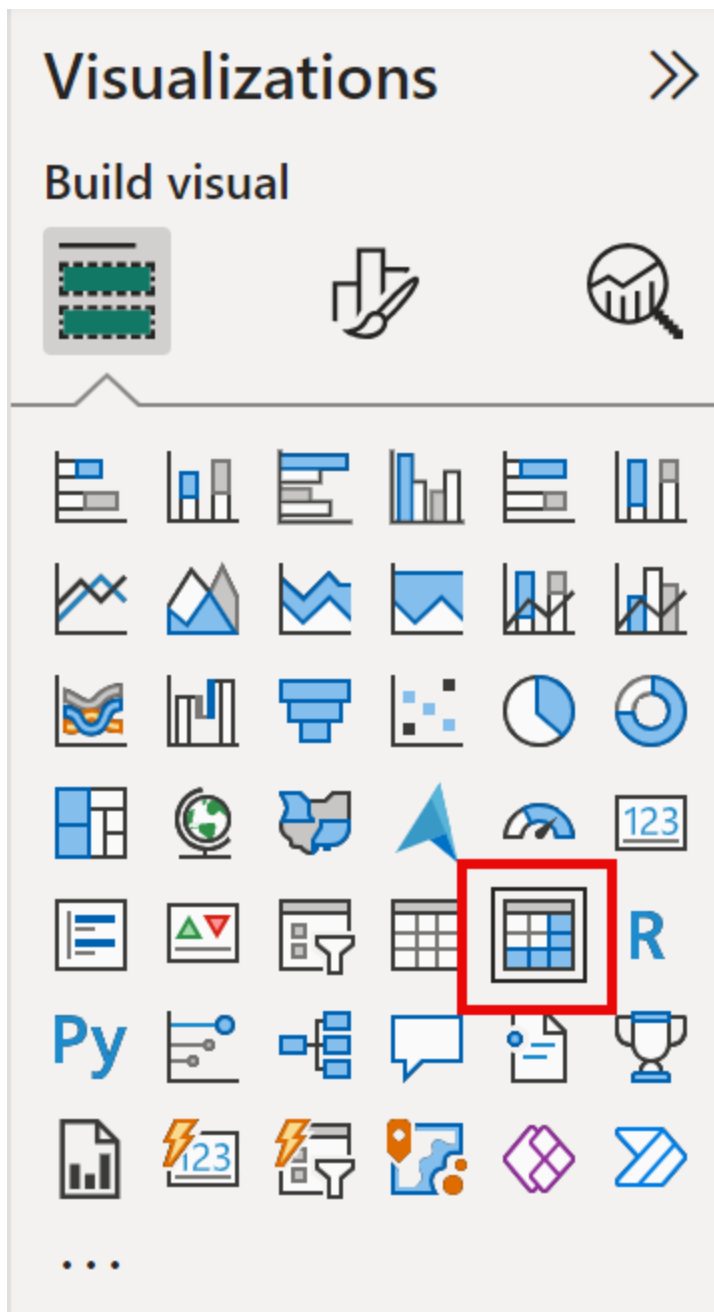
5. To validate the calculations, switch to Report view.
6. To create a new report page, select the plus icon next to Page 1.



7. To add a matrix visual to the new report page, in the **Visualizations** pane, select the matrix visual type.

Tip:

You can hover the cursor over each icon to reveal a tooltip describing the visual type.



8. In the **Data** pane, from inside the **Date** table, drag the **Year** field into the **Rows** well/area.

Visualizations >>

Build visual

Icons for various visualizations: Bar chart, Line chart, Pie chart, Map, etc.

Rows

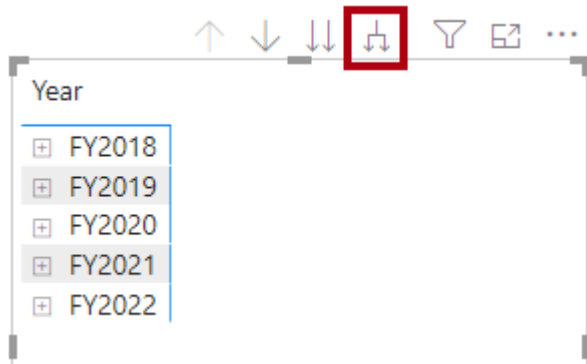
Year

Data

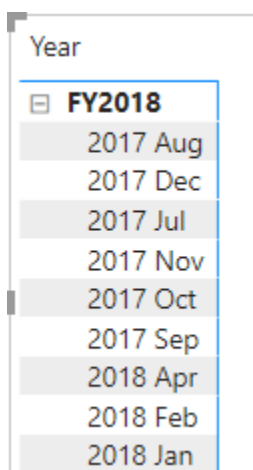
Search

- > Targets
- ✓ Date
 - ☐ Date
 - > ☐ Fiscal
 - ☐ Month
 - ☐ MonthKey
 - ☐ Quarter
 - ☒ Year
- > Product
- > Region
- > Reseller
- > Sales
- > Salesperson
- > Salesperson (Performance)

9. Drag the **Month** field into the **Rows** well/area, directly beneath the **Year** field.
10. At the top-right of the matrix visual (or bottom, depending on the location of the visual), select the forked-double arrow icon (which will expand all years down one level).



11. Notice that the years expand to months, and that the months are sorted alphabetically rather than chronologically.



By default, text values sort alphabetically, numbers sort from smallest to largest, and dates sort from earliest to latest.

12. To customize the **Month** field sort order, switch to Table view.

13. Add the **MonthKey** column to the **Date** table.

code

```
MonthKey =  
(YEAR('Date'[Date]) * 100) + MONTH('Date'[Date])
```

This formula computes a numeric value for each year/month combination.

14. In Table view, verify that the new column contains numeric values (for example, 201707 for July 2017, etc.).

Year	
☐	FY2018
	2017 Jul
	2017 Aug
	2017 Sep
	2017 Oct
	2017 Nov
	2017 Dec
	2018 Jan
	2018 Feb
	2018 Mar

Complete the Date table

In this task, you'll complete the design of the **Date** table by hiding a column and creating a hierarchy. You'll then create relationships to the **Sales** and **Targets** tables.

1. Switch to Model view. In the **Date** table, hide the **MonthKey** column (set **Is Hidden** to **Yes**).
2. On the **Data** right side pane, select the **Date** table, right select on the **Year** column, and select **create hierarchy**.
3. Rename newly created hierarchy to **Fiscal** by right select and **Rename**.
4. Add the follow two remaining fields to the Fiscal hierarchy by selecting them in the **Data** pane, right clicking, selecting **Add to hierarchy** -> **Fiscal**.
 - Quarter
 - Month

Fiscal	
📅	Year
📅	Quarter
📅	Month

5. Create the following two model relationships:

- **Date | Date to Sales | OrderDate**
- **Date | Date to Targets | TargetMonth**

The labs use a shorthand notation to reference a field. It will look like this: **Sales | Unit Price**. In this example, **Sales** is the table name and **Unit Price** is the field name.

6. Hide the following two columns:

- Sales | OrderDate
- Targets | TargetMonth

Mark the Date table

In this task, you'll mark the **Date** table as a date table.

1. Switch to Report view. In the **Data** pane, select the **Date** table (not the **Date** field).
2. On the **Table Tools** contextual ribbon, from inside the **Calendars** group, select **Mark as Date Table**.
3. In the **Mark as a Date Table** window, slide the **Mark as a Date Table** property to **Yes** and in the **Choose a date column** dropdown list, select **Date**. Select **Save**.

Mark as a date table



To enable the creation of date-related visuals, tables and quick measures using this table's date data, mark it as a date table.

Keep in mind any built-in date tables that are already associated with this table will be removed. Visuals or DAX expressions referring to them may break. [Learn more](#)

Mark as a date table



Choose a date column

Date

✔ Validated successfully

4. Save the Power BI Desktop file.

Power BI Desktop now understands that this table defines date (time). *This design approach for a date table is suitable when you don't have a date table in your data source.* If you have a data warehouse, it would be appropriate to load date data from its date dimension table rather than "redefining" date logic in your data model.

Create simple measures

In this task, you'll create simple measures. Simple measures aggregate values in a single column or count rows of a table.

1. In Report view, on **Page 2**, in the **Data** pane, drag the **Sales | Unit Price** field into the matrix visual.

Year	Unit Price
FY2018	\$748.68
2017 Jul	\$655.59
2017 Aug	\$758.93
2017 Sep	\$741.85
2017 Oct	\$677.45

2. In the visual fields pane (located beneath the **Visualizations** pane), in the **Values** field well/area, notice that **Unit Price** is listed as the **Average of Unit Price**. Select the down-arrow for **Unit Price**, and then notice the available menu options.

The screenshot shows the Power BI visual fields pane. The 'Values' field well contains 'Average of Unit Price'. A context menu is open, displaying various aggregation options. The 'Average' option is highlighted with a red box. The 'Rows' field well contains 'Year' and 'Month'. The 'Columns' field well is empty. The 'Visualizations' pane on the left shows a matrix visual.

Visualizations	Fields
Matrix	Year, Unit Price

Visual Fields Pane:

- Rows: Year, Month
- Columns: Add data fields here
- Values: Average of Unit Price

Context Menu Options:

- Remove field
- Rename for this visual
- Move to >
- Add a sparkline
- Conditional formatting >
- Remove conditional formatting
- Sum
- ✓ Average**
- Minimum
- Maximum
- Count (Distinct)
- Count
- Standard deviation
- Variance
- Median
- Show value as >
- New quick measure

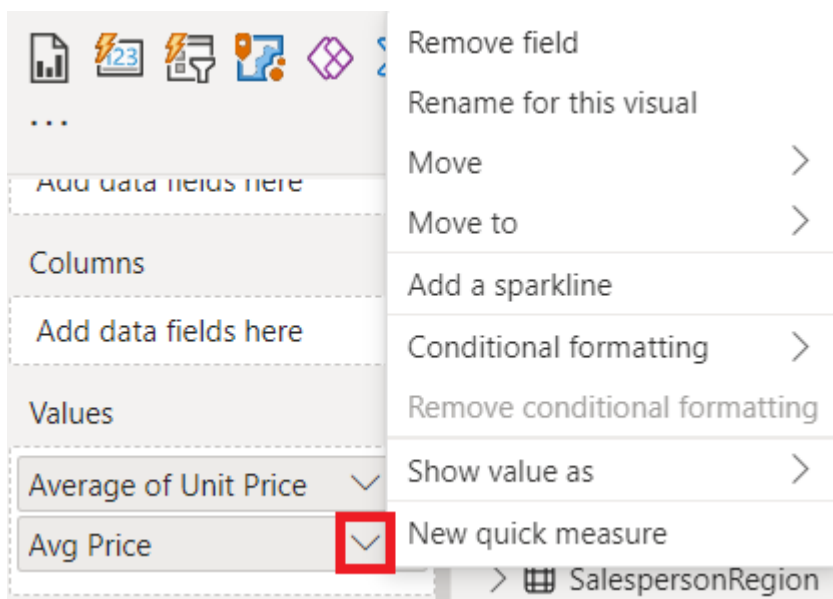
Visible numeric columns allow report authors at report design time to decide how column values will summarize (or not). It can result in inappropriate reporting. Some data modelers don't like leaving things to chance, however, and choose to hide these columns and instead expose aggregation logic defined in measures. It's the approach you'll now take in this lab.

3. To create a measure, in the **Data** pane, right-click the **Sales** table, and then select **New Measure**.
4. In the formula bar, add the following measure definition:

code

```
Avg Price =  
AVERAGE(Sales[Unit Price])
```

5. Add the **Avg Price** measure to the matrix visual, and notice that it produces the same result as the **Unit Price** column (but with different formatting).
6. In the **Values** well, open the context menu for the **Avg Price** field, and notice that it isn't possible to change the aggregation technique.



It's not possible to modify the aggregation behavior of a measure.

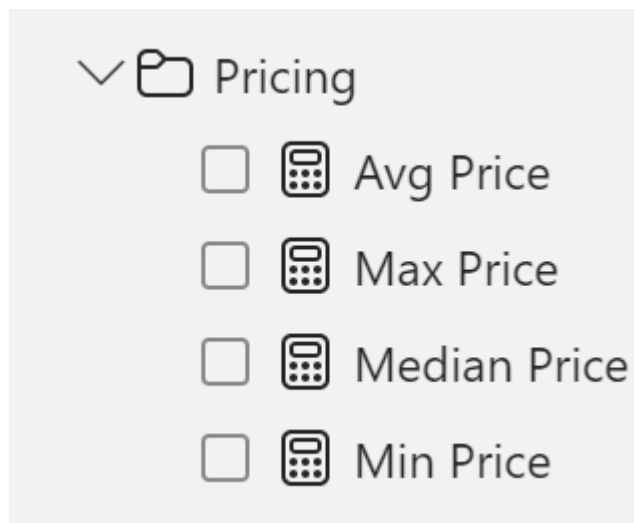
7. Use the snippets file definitions to create the following five measures for the **Sales** table:
 - Median Price
 - Min Price

- Max Price
- Orders
- Order Lines

The **DISTINCTCOUNT()** function used in the **Orders** measure **will count orders only once** (ignoring duplicates). The **COUNTROWS()** function used in the **Order Lines** measure operates over a table.

*In this case, the number of orders is calculated by counting the distinct **SalesOrderNumber** column values, while the number of order lines is simply the number of table rows (each row is a line of an order).*

- Switch to Model view, and then multi-select the four price measures: **Avg Price**, **Max Price**, **Median Price**, and **Min Price**.
- For the multi-selection of measures, configure the following requirements:
 - Set the format to two decimal places
 - Assign to a display folder named **Pricing**

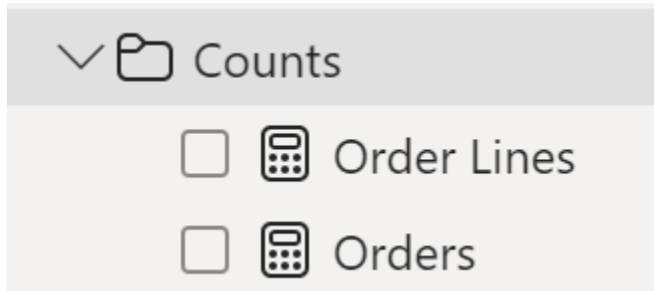


- Hide the **Unit Price** column.

*The **Unit Price** column is now not available to report authors. They must use the pricing measures you've added to the model. This design approach ensures that report authors won't inappropriately aggregate prices, for example, by summing them.*

- Multi-select the **Order Lines** and **Orders** measures, and then configure the following requirements:

- Set the format use the thousands separator
- Assign to a display folder named **Counts**



12. In Report view, in the **Values** well/area of the matrix visual, for the **Unit Price** field, select **X** to remove it.

13. Increase the size of the matrix visual to fill the page width and height.

14. Add the following five measures to the matrix visual:

- Median Price
- Min Price
- Max Price
- Orders
- Order Lines

15. Verify that the results look sensible and are correctly formatted.

Year	Avg Price	Median Price	Min Price	Max Price	Orders	Order Lines
FY2018	\$748.68	\$419.46	\$4.75	\$2,146.96	739	8,459
2017 Jul	\$655.59	\$419.46	\$5.19	\$2,146.96	38	352
2017 Aug	\$758.93	\$419.46	\$4.75	\$2,146.96	75	785
2017 Sep	\$741.85	\$419.46	\$5.19	\$2,146.96	60	593
2017 Oct	\$677.45	\$419.46	\$5.19	\$2,146.96	40	499
2017 Nov	\$752.31	\$419.46	\$5.01	\$2,146.96	90	1,106
2017 Dec	\$734.58	\$419.46	\$5.01	\$2,146.96	63	803

Create additional measures

In this task, you'll create more measures that use more complex formulas.

1. In Report view, select **Page 1** and review the table visual, noticing the total for the **Target** column.

Salesperson	Sales	Target
Amy Alberts	\$10,288,626	\$19,450,000
Brian Welcker	\$77,548,570	\$221,700,000
David Campbell	\$12,004,822	\$19,625,000
Garrett Vargas	\$13,875,633	\$23,675,000
Jae Pak	\$8,410,883	\$13,575,000
Jillian Carson	\$7,633,387	\$13,675,000
José Saraiva	\$13,875,633	\$18,875,000
Linda Mitchell	\$25,634,503	\$40,850,000
Lynn Tsoflias	\$1,391,025	\$3,210,000
Michael Blythe	\$21,987,348	\$31,150,000
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000
Total	\$77,548,570	\$676,210,000

2. Select the table visual, and then in the **Visualizations** pane, remove the **Target** field.
3. Rename the **Targets | Target** column as **Targets | TargetAmount**.

*Tip: There are several ways to rename the column in Report view: In the **Data** pane, you can right-click the column, and then select **Rename**—or, double-click the column, or press **F2**.*

4. Create the following measure on the **Targets** table:

code

```
Target =
IF(
    HASONEVALUE('Salesperson (Performance)'[Salesperson]),
    SUM(Targets[TargetAmount])
)
```

The **HASONEVALUE()** function tests whether a single value in the **Salesperson** column is filtered. When true, the expression returns the sum of target amounts (for just that salesperson). When false, BLANK is returned.

5. Format the **Target** measure for zero decimal places.

Tip:

*You can use the **Measure Tools** contextual ribbon.*

6. Hide the **TargetAmount** column.

Tip:

You can right-click the column in the **Data** pane, and then select **Hide**.

7. Add the **Target** measure to the table visual.
8. Notice that the **Target** column total is now BLANK.

Salesperson	Sales	Target
Amy Alberts	\$10,288,626	\$19,450,000
Brian Welcker	\$77,548,570	\$221,700,000
David Campbell	\$12,004,822	\$19,625,000
Garrett Vargas	\$13,875,633	\$23,675,000
Jae Pak	\$8,410,883	\$13,575,000
Jillian Carson	\$7,633,387	\$13,675,000
José Saraiva	\$13,875,633	\$18,875,000
Linda Mitchell	\$25,634,503	\$40,850,000
Lynn Tsoflias	\$1,391,025	\$3,210,000
Michael Blythe	\$21,987,348	\$31,150,000
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000
Total	\$77,548,570	

9. Use the snippets file definitions to create the following two measures for the **Targets** table:
 - Variance
 - Variance Margin
10. Format the **Variance** measure for zero decimal places.
11. Format the **Variance Margin** measure as percentage with two decimal places.
12. Add the **Variance** and **Variance Margin** measures to the table visual.
13. Resize the table visual so all columns and rows can be seen.

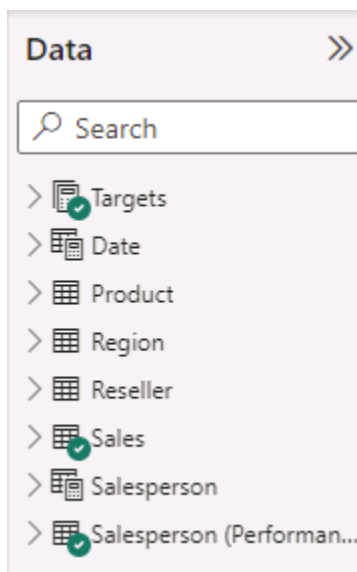
Salesperson	Sales	Target	Variance	Variance Margin
Amy Alberts	\$10,288,626	\$19,450,000	(\$9,161,374)	-47.10 %
Brian Welcker	\$77,548,570	\$221,700,000	(\$144,151,430)	-65.02 %
David Campbell	\$12,004,822	\$19,625,000	(\$7,620,178)	-38.83 %
Garrett Vargas	\$13,875,633	\$23,675,000	(\$9,799,367)	-41.39 %
Jae Pak	\$8,410,883	\$13,575,000	(\$5,164,117)	-38.04 %
Jillian Carson	\$7,633,387	\$13,675,000	(\$6,041,613)	-44.18 %
José Saraiva	\$13,875,633	\$18,875,000	(\$4,999,367)	-26.49 %
Linda Mitchell	\$25,634,503	\$40,850,000	(\$15,215,497)	-37.25 %
Lynn Tsoflias	\$1,391,025	\$3,210,000	(\$1,818,975)	-56.67 %
Michael Blythe	\$21,987,348	\$31,150,000	(\$9,162,652)	-29.41 %
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000	(\$23,844,061)	-44.28 %
Total	\$77,548,570			

While it appears all salespeople aren't meeting target, remember that the table visual isn't yet filtered by a specific time period. You'll produce sales performance reports that filter by a user-selected time period in the **Design a Report in Power BI Desktop** lab.

14. At the top-right corner of the **Data** pane, collapse and then expand open the pane.

Collapsing and reopening the pane resets the content.

15. Notice that the **Targets** table now appears at the top of the list.



Tables that comprise only visible measures are automatically listed at the top of the list.

Lab complete

Summary

In this module, you learned that Power BI **measures are either implicit or explicit**. Implicit measures are automatic behaviors that are supported by visuals, while explicit measures use DAX formulas that summarize model data.

Explicit measures are important because they allow you to create complex DAX formulas to achieve the precise calculations that your report visuals need. While you learned to create **simple and compound measures** in this module, in later modules you'll learn to create more powerful measures by using **filter modification functions and iterator functions**.

Add calculated tables and columns to Power BI Desktop models

Introduction

You can write a Data Analysis Expressions (DAX) formula to add a *calculated table* to your model. The **formula can duplicate or transform existing model data to produce a new table**.

Note

A calculated table can't connect to external data; you must use Power Query to accomplish that task.

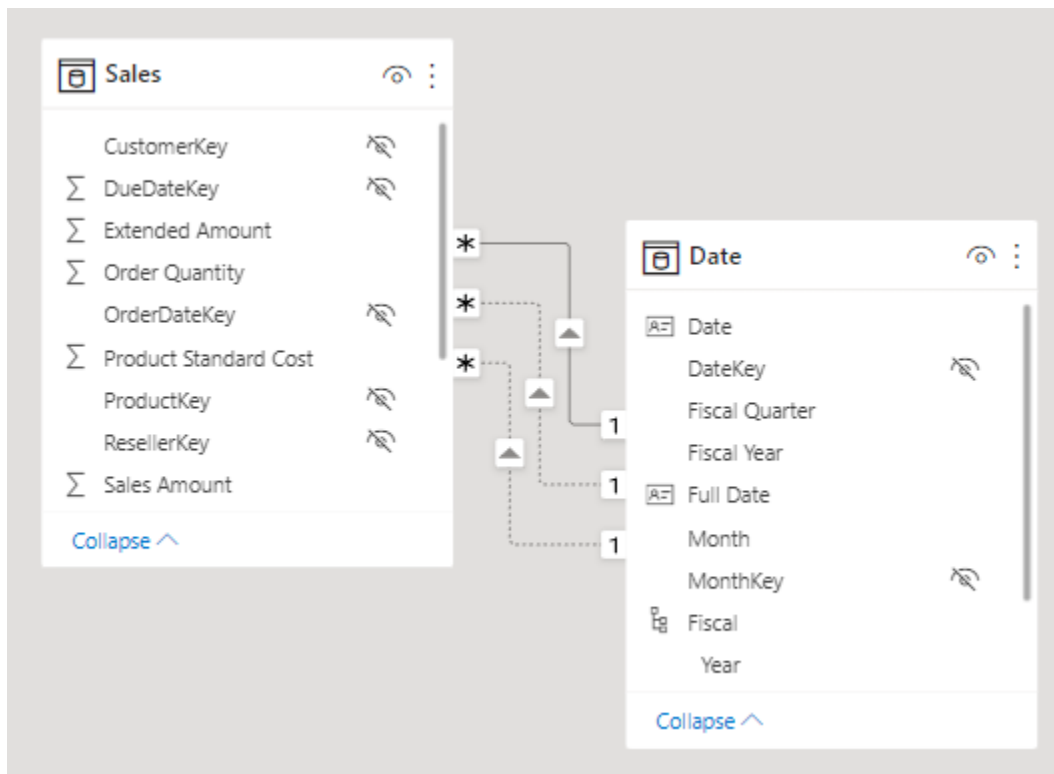
A **calculated table formula must return a table object**. The **simplest formula can duplicate an existing model table**.

Calculated tables have a cost: **They increase the model storage size and they can prolong the data refresh time**. The reason is because **calculated tables recalculate when** they have **formula dependencies to refreshed tables**.

Duplicate a table

The following section describes a common design challenge that can be solved by creating a calculated table. First, you should download and open the **Adventure Works DW 2020 M03.pbix** file and then switch to the model diagram.

In the model diagram, notice that the **Sales table has three relationships to the Date table**.



The model diagram shows three relationships because the Sales table stores sales data by order date, ship date, and due date. If you examine the **OrderDateKey**, **ShipDateKey**, and **DueDateKey** columns, notice that one relationship is represented by a solid line, which is the *active relationship*. The other relationships, which are represented by dashed lines, are *inactive relationships*.

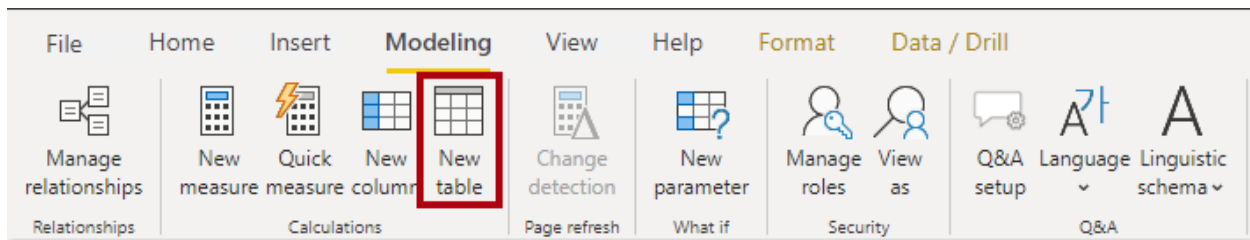
Note

Only one active relationship can exist between any two model tables.

In the diagram, hover the cursor over the active relationship to highlight the related columns, which is how you would interact with the model diagram to learn about related columns. In this case, the active relationship filters the **OrderDateKey** column in the **Sales** table. Thus, filters that are applied to the **Date** table will propagate to the **Sales** table to filter by order date; they'll never filter by ship date or due date.

The next step is to delete the two inactive relationships between the **Date** table and the **Sales** table. To delete a relationship, right-click it and then select **Delete** in the context menu. Make sure that you delete both inactive relationships.

Next, add a new table to allow report users to filter sales by ship date. Switch to Report view and then, on the **Modeling ribbon** tab, from inside the **Calculations** group, select **New table**.



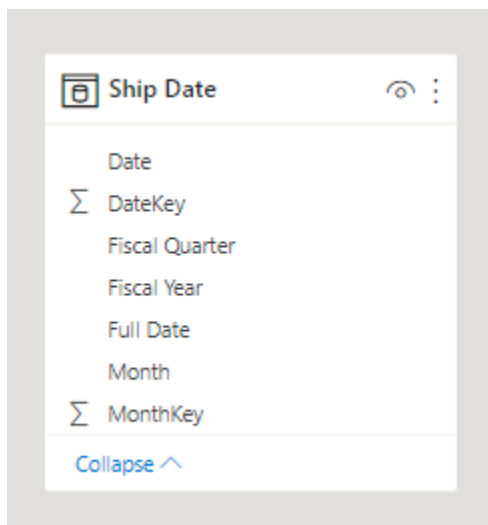
In the formula bar (located beneath the ribbon), enter the following calculated table definition and then press **Enter**.

DAX

```
Ship Date = 'Date'
```

The calculated table definition **duplicates the Date table data** to produce a **new table named Ship Date**. The **Ship Date** table has exactly the same columns and rows as the **Date** table. **When the Date table data refreshes, the Ship Date table recalculates, so they'll always be in sync.**

Switch to the model diagram, and then notice the addition of the **Ship Date** table.



Next, **create a relationship** between the **DateKey** column in the **Ship Date** table and the **ShipDateKey** column in the **Sales** table. You can create the relationship by dragging the **DateKey** column in the **Ship Date** table onto the **ShipDateKey** column in the **Sales** table.

A calculated table only duplicates data; it doesn't duplicate any model properties or objects like column visibility or hierarchies. You'll need to set them up for the new table, if required.

Tip

It's [possible to rename columns](#) of a calculated table. In this example, it's a good idea to rename columns so that they better describe their purpose. For example, the **Fiscal Year** column in the **Ship Date** table can be renamed as **Ship Fiscal Year**. Accordingly, when fields from the **Ship Date** table are used in visuals, [their names are automatically included in captions like the visual title or axis labels](#).

To complete the design of the **Ship Date** table, you can:

- [Rename the following columns](#):
 - **Date** as **Ship Date**
 - **Fiscal Year** as **Ship Fiscal Year**
 - **Fiscal Quarter** as **Ship Fiscal Quarter**
 - **Month** as **Ship Month**
 - **Full Date** as **Ship Full Date**
- Sort the **Ship Full Date** column by the **Ship Date** column.
- Sort the **Ship Month** column by the **MonthKey** column.
- Hide the **MonthKey** column.
- Create a hierarchy named **Fiscal** with the following levels:
 - Ship Fiscal Year
 - Ship Fiscal Quarter
 - Ship Month
 - Ship Full Date
- Mark the Ship Date table as a date table by using the **Ship Date** column.

Calculated tables are useful to work in scenarios when multiple relationships between two tables exist, as previously described. They can also be used to add a date table to your model. [Date tables are required to apply special time filters known as *time intelligence*](#).

Create a date table

In the next example, [a second calculated table will be created, this time by using the **CALENDAR** DAX function](#).

Create the **Due Date** calculated table by using the following definition.

DAX

```
Due Date = CALENDARAUTO(6)
```

The **CALENDARAUTO** DAX function takes a single optional argument, which is the last month number of the year, and returns a single-column table. If you don't pass in a month number, it's assumed to be 12 (for December). For example, at Adventure Works, their financial year ends on June 30 of each year, so the value 6 (for June) is passed in.

The function scans all date and date/time columns in your model to determine the earliest and latest stored date values. It then produces a complete set of dates that span all dates in your model, ensuring that full years of dates are loaded. For example, if the earliest date that is stored in your model is October 15, 2021, then the first date that is returned by the CALENDARAUTO function would be July 1, 2021. If the latest date that is stored in the model is June 15, 2022, then the last date that is returned by the CALENDARAUTO function would be June 30, 2022.

Effectively, the **CALENDARAUTO** function guarantees that the following requirements to mark a date table are met:

- The table must include a column of data type Date.
- The column must contain complete years.
- The column must not have missing dates.

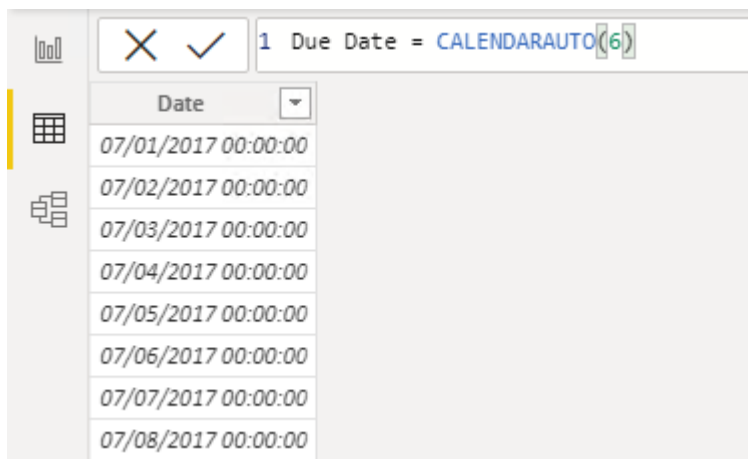
Tip

You can also create a date table by using the **CALENDAR** DAX function and passing in two date values, which represent the date range. The function generates one row for each date within the range. You can pass in static date values or pass in expressions that retrieve the earliest/latest dates from specific columns in your model.

Next, switch to data view, and then in the **Fields** pane, select the **Due Date** table. Now, review the column of dates. You might want to order them to see the earliest date in the first row by selecting the arrow inside the **Date** column header and then sorting in ascending order.

Note

Ordering or filtering columns doesn't change how the values are stored. These functions help you explore and understand the data.



Date
07/01/2017 00:00:00
07/02/2017 00:00:00
07/03/2017 00:00:00
07/04/2017 00:00:00
07/05/2017 00:00:00
07/06/2017 00:00:00
07/07/2017 00:00:00
07/08/2017 00:00:00

Now that the **Date** column is selected, review the message in the status bar (located in the lower-left corner). It describes how many rows that the table stores and how many distinct values are found in the selected column.

Table: Due Date (1,461 rows) Column: Date (1,461 distinct values)

When the table rows and distinct values are the same, it means that the column contains unique values. That factor is important for two reasons: It satisfies the requirements to mark a date table, and it allows this column to be used in a model relationship as the one-side.

The **Due Date** calculated table will recalculate each time a table that contains a date column refreshes. In other words, when a row is loaded into the **Sales** table with an order date of July 1, 2022, the **Due Date** table will automatically extend to include dates through to the end of the next year: June 30, 2023.

The **Due Date** table requires additional columns to support the known filtering and grouping requirements, specifically by year, quarter, and month.

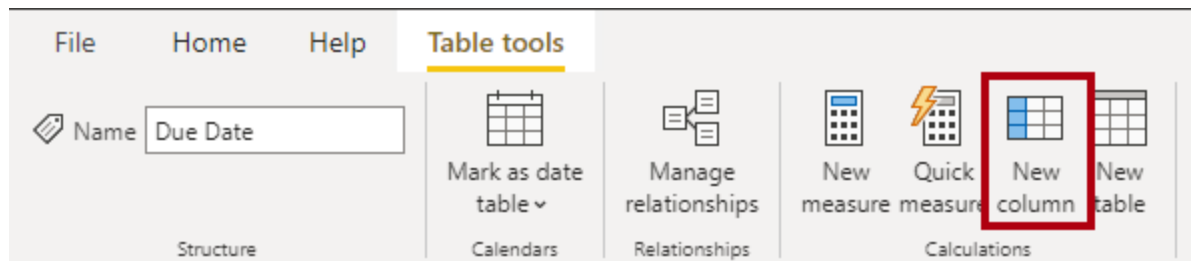
Create calculated columns

You can write a DAX formula to add a *calculated column* to any table in your model. A calculated column formula must return a scalar or single value.

Calculated columns in import models have a cost: They increase the model storage size and they can prolong the data refresh time. The reason is because calculated columns recalculate when they have formula dependencies to refreshed tables.

In data view, in the **Fields** pane, ensure that the **Due Date** table is selected. Before you create a calculated column, first rename the **Date** column to **Due Date**.

Now, you can add a calculated column to the **Due Date** table. To create a calculated column, in the **Table tools** contextual ribbon, from inside the **Calculations** group, select **New column**.



In the formula bar, enter the following calculated column definition and then press **Enter**.

DAX

```
Due Fiscal Year =  
"FY"  
    & YEAR('Due Date'[Due Date])  
    + IF(  
        MONTH('Due Date'[Due Date]) > 6,  
        1  
    )
```

The calculated column definition adds the **Due Fiscal Year** column to the **Due Date** table. The following steps describe how Microsoft Power BI evaluates the calculated column formula:

1. The addition operator (+) is evaluated before the text concatenation operator (&).
2. The **YEAR** DAX function returns the whole number value of the due date year.
3. The **IF** DAX function returns the value when the due date month number is 7-12 (July to December); otherwise, it returns **BLANK**. (For example, because the Adventure Works financial year is July-June, the last six months of the calendar year will use the next calendar year as their financial year.)
4. The year value is added to the value that is returned by the IF function, which is the value one or BLANK. If the value is BLANK, it's implicitly converted to zero (0) to allow the addition to produce the fiscal year value.
5. The literal text value "FY" concatenated with the fiscal year value, which is implicitly converted to text.

Add a second calculated column by using the following definition:

DAX

```
Due Fiscal Quarter =  
'Due Date'[Due Fiscal Year] & " Q"  
    & IF(  
        MONTH('Due Date'[Due Date]) <= 3,  
        3,  
        IF(  
            MONTH('Due Date'[Due Date]) <= 6,  
            4,  
            IF(  
                MONTH('Due Date'[Due Date]) <= 9,  
                1,  
                2  
            )  
        )  
    )  
)
```

The calculated column definition adds the **Due Fiscal Quarter** column to the **Due Date** table. The **IF** function returns the quarter number (Quarter 1 is July-September), and the result is concatenated to the **Due Fiscal Year** column value and the literal text Q.

Add a third calculated column by using the following definition:

DAX

```
Due Month =  
FORMAT('Due Date'[Due Date], "yyyy mmm")
```

The calculated column definition adds the **Due Month** column to the **Due Date** table. The **FORMAT** DAX function converts the **Due Date** column value to text by using a format string. In this case, the format string produces a label that describes the year and abbreviated month name.

Note

Many user-defined date/time formats exist. For more information, see [Custom date and time formats for the FORMAT function](#).

Add a fourth calculated column by using the following definition:

DAX

```
Due Full Date =  
FORMAT('Due Date'[Due Date], "yyyy mmm, dd")
```

Add a fifth calculated column by using the following definition:

DAX

```
MonthKey =  
(YEAR('Due Date'[Due Date]) * 100) + MONTH('Due Date'[Due Date])
```

The **MonthKey** calculated column [multiplies the due date year by the value 100](#) and then [adds the month number of the due date](#). It produces a numeric value that can be [used to sort the Due Month text values in chronological order](#).

Verify that the **Due Date** table has six columns. [The first column was added when the calculated table was created](#), and the other five columns were added as calculated columns.

1 MonthKey =

2 (YEAR('Due Date'[Due Date]) * 100) + MONTH('Due Date'[Due Date])

Due Date	Due Fiscal Year	Due Fiscal Quarter	Due Month	Due Full Date	MonthKey
07/01/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 01	201707
07/02/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 02	201707
07/03/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 03	201707
07/04/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 04	201707
07/05/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 05	201707
07/06/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 06	201707
07/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	2017 Jul, 07	201707

To complete the design of the **Due Date** table, you can:

- Sort the **Due Full Date** column by the **Due Date** column.
- Sort the **Due Month** column by the **MonthKey** column.
- Hide the **MonthKey** column.

- Create a hierarchy named **Fiscal** with the following levels:
 - Due Fiscal Year
 - Due Fiscal Quarter
 - Due Month
 - Due Full Date
- Mark the **Due Date** table as a date table by using the **Due Date** column.

Learn about row context

Now that you've created calculated columns, you can learn how their formulas are evaluated.

The formula for a calculated column is evaluated for each table row. Furthermore, it's evaluated within row context, which means *the current row*. Consider the **Due Fiscal Year** calculated column definition:

DAX

```
Due Fiscal Year =
"FY"
& YEAR('Due Date'[Due Date])
+ IF(
    MONTH('Due Date'[Due Date]) <= 6,
    1
)
```

When the formula is evaluated for each row, the 'Due Date'[Due Date] column reference returns the column value for *that row*. You might be familiar with this concept from working with formulas in Excel tables.

However, row context doesn't extend beyond the table. If your formula needs to reference columns in other tables, you have two options:

- If the tables are related, directly or indirectly, you can use the **RELATED** or **RELATEDTABLE** DAX function. The **RELATED** function retrieves the value at the one-side of the relationship, while the **RELATEDTABLE** retrieves values on the many-side. The **RELATEDTABLE** function returns a table object.
- When the tables aren't related, you can use the **LOOKUPVALUE** DAX function.

Generally, try to use the RELATED function whenever possible. It will usually perform better than the LOOKUPVALUE function due to the ways that relationship and column data is stored and indexed.

Now, add the following calculated column definition to the **Sales** table:

DAX

```
Discount Amount =  
(  
    Sales[Order Quantity]  
    * RELATED('Product'[List Price])  
) - Sales[Sales Amount]
```

The calculated column definition adds the **Discount Amount** column to the **Sales** table. Power BI evaluates the calculated column formula for each row of the **Sales** table. The values for the **Order Quantity** and **Sales Amount** columns are retrieved within row context. However, because the **List Price** column belongs to the **Product** table, the RELATED function is required to retrieve the list price value *for the sale product*.

Row context is used when calculated column formulas are evaluated. It's also used when a class of functions, known as *iterator functions*, are used. Iterator functions provide you with flexibility to create sophisticated summarizations. Iterator functions are described in a later module.

Choose a technique to add a column

There are three techniques that you can use to add columns to a model table:

- Add columns to a view or table (as a persisted column), and then source them in Power Query. This option only makes sense when your data source is a relational database and if you have the skills and permissions to do so. However, it's a good option because it supports ease of maintenance and allows reuse of the column logic in other models or reports.
- Add custom columns (using M) to Power Query queries.
- Add calculated columns (using DAX) to model tables.

Regardless of which technique you use, it results in the same outcome. Report users can't determine the origin of a column. Typically, they're not concerned with how the column was created but rather that it delivers the right data.

When multiple ways are available to add a column, you can consider using the approach that best fits your skills and that's supported by the language (M or DAX). However, the preference is to add custom columns in Power Query, whenever possible, because they load to the model in a more compact and optimal way.

When you need to add a column to a calculated table, make sure that you create a calculated column. Otherwise, we recommend that you only use a calculated column when the calculated column formula:

- Depends on summarized model data.
- Needs to use specialized modeling functions that are only available in DAX, such as the RELATED and RELATEDTABLE functions. Specialized functions can also include the DAX parent and child hierarchies, which are designed to naturalize a recursive relationship into columns, for example, in an employee table where each row stores a reference to the row of the manager (who is also an employee).

Knowledge check

Summary

In this module, you learned how to use DAX to extend your model designs with calculated tables and calculated columns. You also learned that calculated columns are evaluated within row context, and you learned how to correctly reference columns that belong to different tables. Finally, you learned that while many ways are available for adding new columns to your model, it's best to create Power Query custom columns whenever possible.

Modify DAX filter context in Power BI Desktop models

Introduction

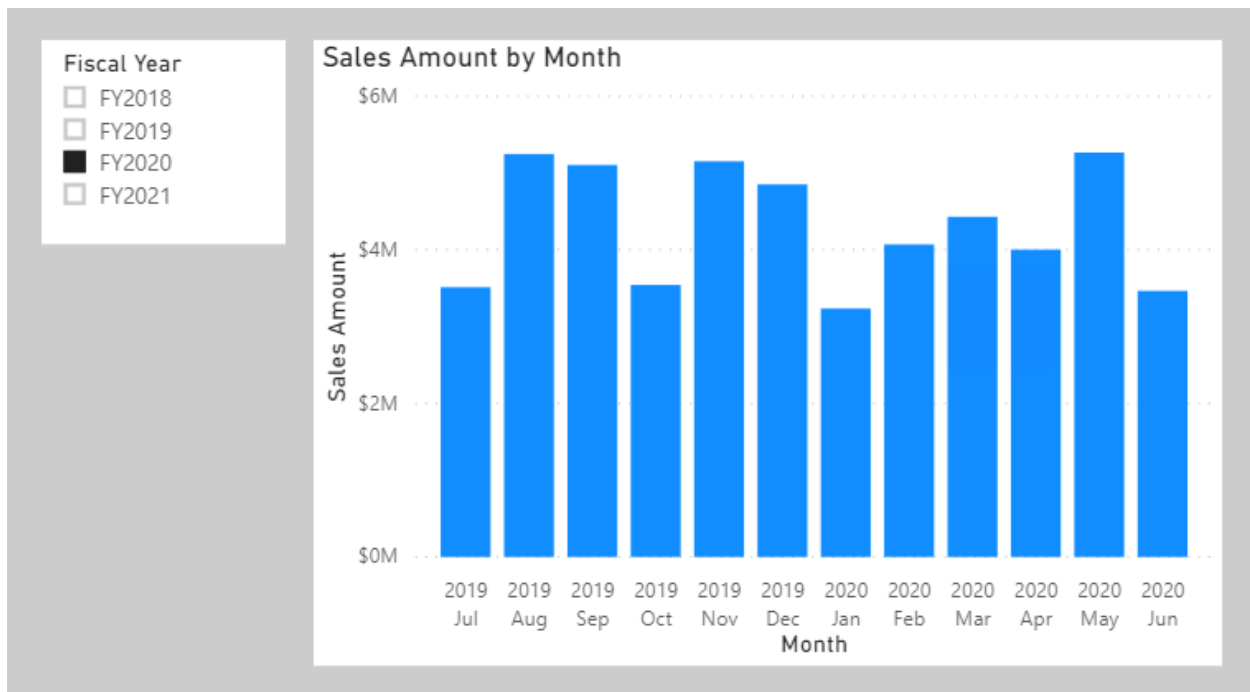
[Watch the following video to learn about filter context.](#)

Filter context describes the filters that are applied during the evaluation of a measure or measure expression. Filters can be applied directly to columns, like a filter on the Fiscal Year column in the Date table for the value FY2020. Additionally, filters can be applied indirectly, which happens when model relationships propagate filters to other tables. For example, the Sales table receives a filter through its relationship with the Date table, filtering the Sales table rows to those with an OrderDateKey column value in FY2020.

Note

Calculated tables and calculated columns aren't evaluated within filter context. Calculated columns are evaluated in row context, though the formula can transition the row context to filter context, if it needs to summarize model data. Context transition is described in Unit 5.

At report design time, filters are applied in the **Filters** pane or to report visuals. The **slicer visual** is an example of a visual whose only purpose is to filter the report page (and other pages when it's configured as a **synced slicer**). Report visuals, which perform grouping, also apply filters. They're implied filters; the difference is that the filter result is visible in the visual. For example, a stacked column chart visual can filter by fiscal year FY2020, group by month, and summarize sales amount. The fiscal year filter isn't visible in the visual result, yet the grouping, which results in a column for each month, behaves as a filter.



Not all filters are applied at report design time. Filters can be added when a report user interacts with the report. They can modify filter settings in the **Filters** pane, and they can cross-filter or cross-highlight visuals by selecting visual elements like columns, bars, or pie chart segments. These interactions apply additional filters to report page visuals (unless interactions have been disabled).

It's important to understand how filter context works. It guides you in defining the correct formula for your calculations. As you write more complex formulas, you'll identify times when you need to add, modify, or remove filters to achieve the desired result.

Consider an example that requires your formula to modify the filter context. Your objective is to produce a report visual that shows each sales region together with its revenue and revenue as a percentage of total revenue.

Region	Revenue	Revenue % Total Region
Australia	\$10,655,335.96	9.70%
Canada	\$16,355,770.46	14.89%
Central	\$7,909,009.01	7.20%
France	\$7,251,555.65	6.60%
Germany	\$4,878,300.38	4.44%
Northeast	\$6,939,374.48	6.32%
Northwest	\$16,084,942.55	14.65%
Southeast	\$7,879,655.07	7.18%
Southwest	\$24,184,609.60	22.02%
United Kingdom	\$7,670,721.04	6.99%
Total	\$109,809,274.20	100.00%

The **Revenue % Total Region** result is achieved by defining a measure expression that's the ratio of revenue divided by revenue for all regions. Therefore, for Australia, the ratio is 10,655,335.96 dollars divided by 109,809,274.20 dollars, which is 9.7 percent.

The numerator expression doesn't need to modify filter context; it should use the current filter context (a visual that groups by region applies a filter for that region). The denominator expression, however, needs to remove any region filters to achieve the result for all regions.

Tip

The key to writing complex measures is mastering these concepts:

- Understanding how filter context works.
- Understanding when and how to modify or remove filters to achieve a required result.
- Composing a formula to accurately and efficiently modify filter context.

Mastering these concepts takes practice and time. Rarely will students understand the concepts from the beginning of training. Therefore, be patient and persevere with the theory and activities. We recommend that you repeat this module at a later time to help reinforce key lessons.

The next unit introduces the [CALCULATE](#) DAX function. It's one of the most powerful DAX functions, allowing you to modify filter context when your formulas are evaluated.

Modify filter context

You can use the **CALCULATE** DAX function to modify filter context in your formulas. The [syntax for the CALCULATE function](#) is as follows:

DAX

```
CALCULATE(<expression>, [[<filter1>], <filter2>]...)
```

The function requires passing in an expression that returns a scalar value and as many filters as you need. The expression can be a measure (which is a named expression) or any expression that can be evaluated in filter context.

Filters can be Boolean expressions or table expressions. It's also possible to pass in filter modification functions that provide additional control when you're modifying filter context.

When you have multiple filters, they're evaluated by using the AND logical operator, which means that all conditions must be TRUE at the same time.

Note

The **CALCULATETABLE** DAX function performs exactly the same functionality as the **CALCULATE** function, except that it modifies the filter context that's applied to an expression that returns a table object. In this module, the explanations and examples use the **CALCULATE** function, but keep in mind that these scenarios could also apply to the **CALCULATETABLE** function.

Apply Boolean expression filters

A Boolean expression filter is an expression that evaluates to TRUE or FALSE. Boolean filters must abide by the following rules:

- They can reference only a single column.
- They cannot reference measures.
- They cannot use functions that scan or return a table that includes aggregation functions like **SUM**.

In this example, you will create a measure. First, download and open the [Adventure Works DW 2020 M06.pbix](#) file. Then add the following measure to the **Sales** table that filters the **Revenue** measure by using a Boolean expression filter for red products.

DAX

`Revenue Red = CALCULATE([Revenue], 'Product'[Color] = "Red")`

Add the **Revenue Red** measure to the table visual that is found on **Page 1** of the report.

Region	Revenue	Revenue Red
Australia	\$10,655,335.96	\$2,681,324.79
Canada	\$16,355,770.46	\$3,573,412.99
Central	\$7,909,009.01	\$1,585,997.34
France	\$7,251,555.65	\$1,051,014.15
Germany	\$4,878,300.38	\$670,607.30
Northeast	\$6,939,374.48	\$1,876,016.33
Northwest	\$16,084,942.55	\$2,292,905.61
Southeast	\$7,879,655.07	\$1,457,221.07
Southwest	\$24,184,609.60	\$5,345,637.47
United Kingdom	\$7,670,721.04	\$1,063,753.75
Total	\$109,809,274.20	\$21,597,890.81

In this next example, the following measure filters the **Revenue** measure by multiple colors. Notice the use of the **IN** operator followed by a list of color values.

DAX

`Revenue Red or Blue = CALCULATE([Revenue], 'Product'[Color] IN {"Red", "Blue"})`

The following measure filters the **Revenue** measure by expensive products. Expensive products are those with a list price greater than USD 1000.

DAX

`Revenue Expensive Products = CALCULATE([Revenue], 'Product'[List Price] > 1000)`

Apply table expression filters

A table expression filter applies a table object as a filter. It could be a reference to a model table; however, it's likely a DAX function that returns a table object.

Commonly, you'll use the **FILTER** DAX function to apply complex filter conditions, including those that can't be defined by a Boolean filter expression. The FILTER function is classed as an iterator function, and so you would pass in a table, or table expression, and an expression to evaluate for each row of that table.

The FILTER function returns a table object with exactly the same structure as one that the table passed in. Its rows are a subset of those rows that were passed in, meaning the rows where the expression evaluated as TRUE.

The following example shows a table filter expression that uses the FILTER function:

DAX

```
Revenue High Margin Products =  
CALCULATE(  
    [Revenue],  
    FILTER(  
        'Product',  
        'Product'[List Price] > 'Product'[Standard Cost] * 2  
    )  
)
```

In this example, the FILTER function filters all rows of the Product table that are in filter context. Each row for a product where its list price exceeds double its standard cost is displayed as a row of the filtered table. Therefore, the **Revenue** measure is evaluated for all products that are returned by the FILTER function.

All filter expressions that are passed in to the CALCULATE function are table filter expressions. A Boolean filter expression is a shorthand notation to improve the writing and reading experience. Internally, Microsoft Power BI translates Boolean filter expressions to table filter expressions, which is how it translates your **Revenue Red** measure definition.

DAX

```
Revenue Red =  
CALCULATE(  
    [Revenue],  
    FILTER(  
        'Product',  
        'Product'[Color] = "Red"  
    )  
)
```

Filter behavior

Two possible standard outcomes occur when you add filter expressions to the CALCULATE function:

- If the columns (or tables) aren't in filter context, then new filters will be added to the filter context to evaluate the CALCULATE expression.
- If the columns (or tables) are already in filter context, the existing filters will be overwritten by the new filters to evaluate the CALCULATE expression.

The following examples show how adding filter expressions to the CALCULATE function works.

Note

In each of the examples, no filters are applied to the table visual.

As in the previous activity, the **Revenue Red** measure was added to a table visual that groups by region and displays revenue.

Region	Revenue	Revenue Red
Australia	\$10,655,335.96	\$2,681,324.79
Canada	\$16,355,770.46	\$3,573,412.99
Central	\$7,909,009.01	\$1,585,997.34
France	\$7,251,555.65	\$1,051,014.15
Germany	\$4,878,300.38	\$670,607.30
Northeast	\$6,939,374.48	\$1,876,016.33
Northwest	\$16,084,942.55	\$2,292,905.61
Southeast	\$7,879,655.07	\$1,457,221.07
Southwest	\$24,184,609.60	\$5,345,637.47
United Kingdom	\$7,670,721.04	\$1,063,753.75
Total	\$109,809,274.20	\$21,597,890.81

Because no filter is applied on the **Color** column in the **Product** table, the evaluation of the measure adds a new filter to filter context. In the first row, the value of \$2,681,324.79 is for red products that were sold in the Australian region.

Switching the first column of the table visual from **Region** to **Color** will produce a different result because the **Color** column in the **Product** table is now in filter context.

Color	Revenue	Revenue Red Filter
Black	\$38,236,124.06	
Blue	\$9,602,850.97	
Multi	\$649,030.25	
NA	\$1,099,303.91	
Red	\$21,597,890.81	\$21,597,890.8068
Silver	\$19,777,339.95	
Silver/Black	\$147,483.91	
White	\$29,745.13	
Yellow	\$18,669,505.22	
Total	\$109,809,274.20	\$21,597,890.8068

The **Revenue Red** measure formula evaluates the **Revenue** measure by adding a filter on the **Color** column (to red) in the **Product** table. Consequently, in this visual that groups by color, the measure formula overwrites the filter context with a new filter.

This result might or might not be what you want. The next unit introduces the **KEEPFILTERS** DAX function, which is a filter modification function that you can use to preserve filters rather than overwrite them.

Use filter modifier functions

When using the **CALCULATE** function, you can pass in filter modification functions, which allow you to accomplish more than adding filters alone.

Remove filters

Use the **REMOVEFILTERS** DAX function as a **CALCULATE** filter expression to remove filters from filter context. It can remove filters from one or more columns or from all columns of a single table.

Note

The **REMOVEFILTERS** function is relatively new. In previous versions of DAX, you removed filters by using the **ALL DAX** function or variants including the **ALLEXCEPT** and the **ALLNOBLANKROW** DAX functions. These functions behave as both filter modifiers and as functions that return table objects of distinct values.

These functions are mentioned now because [you're likely to find documentation and formula examples](#) that remove filters by using them.

In the following example, you will add a new measure to the **Sales** table that evaluates the **Revenue** measure but does so by removing filters from the **Sales Territory** table. Format the measure as currency with two decimal places.

DAX

Revenue Total Region = CALCULATE([Revenue], REMOVEFILTERS('Sales Territory'))

Now, add the **Revenue Total Region** measure to the matrix visual that is found on **Page 2** of the report. The matrix visual will group by three columns from the Sales Territory table on the rows: **Group**, **Country**, and **Region**.

Reseller Revenue				
Group	Country	Region	Revenue	Revenue Total Region
Corporate HQ	Corporate HQ	Corporate HQ		\$80,450,596.98
		Total		\$80,450,596.98
	Total			\$80,450,596.98
Europe	France	France	\$4,607,537.94	\$80,450,596.98
		Total	\$4,607,537.94	\$80,450,596.98
	Germany	Germany	\$1,983,988.04	\$80,450,596.98
		Total	\$1,983,988.04	\$80,450,596.98
	United Kingdom	United Kingdom	\$4,279,008.83	\$80,450,596.98
		Total	\$4,279,008.83	\$80,450,596.98
	Total		\$10,870,534.80	\$80,450,596.98
North America	Canada	Canada	\$14,377,925.60	\$80,450,596.98
		Total	\$14,377,925.60	\$80,450,596.98
	United States	Central	\$7,906,008.18	\$80,450,596.98
		Northeast	\$6,932,842.01	\$80,450,596.98
		Northwest	\$12,435,076.00	\$80,450,596.98
		Southeast	\$7,867,416.23	\$80,450,596.98
		Southwest	\$18,466,458.79	\$80,450,596.98
		Total	\$53,607,801.21	\$80,450,596.98
	Total		\$67,985,726.81	\$80,450,596.98
Pacific	Australia	Australia	\$1,594,335.38	\$80,450,596.98
		Total	\$1,594,335.38	\$80,450,596.98
	Total		\$1,594,335.38	\$80,450,596.98
Total			\$80,450,596.98	\$80,450,596.98

Notice that **each Revenue Total Region value is the same**. It's the value of total revenue.

While this result on its own isn't useful, **when it's used as a denominator in a ratio, it calculates a percent of grand total**. Therefore, you will now overwrite the **Revenue Total Region** measure definition with the following definition. (This new

definition changes the measure name and declares two variables. Be sure to format the measure as a [percentage with two decimal places](#).)

DAX

```
Revenue % Total Region =
VAR CurrentRegionRevenue = [Revenue]
VAR TotalRegionRevenue =
    CALCULATE(
        [Revenue],
        REMOVEFILTERS('Sales Territory')
    )
RETURN
    DIVIDE(
        CurrentRegionRevenue,
        TotalRegionRevenue
    )
```

Verify that the matrix visual now displays the **Revenue % Total Region** values.

Reseller Revenue

Group	Country	Region	Revenue	Revenue % Total Region
Europe	France	France	\$4,607,537.94	5.73%
		Total	\$4,607,537.94	5.73%
	Germany	Germany	\$1,983,988.04	2.47%
		Total	\$1,983,988.04	2.47%
	United Kingdom	United Kingdom	\$4,279,008.83	5.32%
		Total	\$4,279,008.83	5.32%
Total		\$10,870,534.80	13.51%	
North America	Canada	Canada	\$14,377,925.60	17.87%
		Total	\$14,377,925.60	17.87%
	United States	Central	\$7,906,008.18	9.83%
		Northeast	\$6,932,842.01	8.62%
		Northwest	\$12,435,076.00	15.46%
		Southeast	\$7,867,416.23	9.78%
		Southwest	\$18,466,458.79	22.95%
		Total	\$53,607,801.21	66.63%
	Total		\$67,985,726.81	84.51%
	Pacific	Australia	Australia	\$1,594,335.38
Total			\$1,594,335.38	1.98%
Total		\$1,594,335.38	1.98%	
Total		\$80,450,596.98	100.00%	

You'll now create **another measure**, but this time, you will **calculate the ratio of revenue for a region divided by its country's or region's revenue**.

Before you complete this task, notice that the **Revenue % Total Region** value for the **Southwest region is 22.95 percent**. **Investigate the filter context** for this cell. Switch to data view and then, in the **Fields** pane, select the **Sales Territory** table.

Apply the following column filters:

- **Group** - North America
- **Country** - United States
- **Region** - Southwest

SalesTerritoryKey	Region	Country	Group
4	Southwest	United States	North America

Notice that the filters reduce the table to **only one row**. Now, while thinking about your new **objective to create a ratio of the region revenue over its country's revenue**, **clear the filter from the Region column**.

Sort ascending
Sort descending
Clear sort
Clear filter
Clear all filters
Text filters
Search
(Select all)
☐ Central
☐ Northeast
☐ Northwest
☐ Southeast
☒ Southwest
OK Cancel

Notice that five rows now exist, **each row belonging to the country United States**. Accordingly, **when you clear the Region column filters, while preserving filters on the Country and Group columns, you will have a new filter context that's for the region's country**.

In the following measure definition, notice how you can clear or remove a filter from a column. In DAX logic, it's a small and subtle change that's made to the **Revenue % Total Region** measure formula: The REMOVEFILTERS function now removes filters from the **Region** column instead of all columns of the **Sales Territory** table.

DAX

```
Revenue % Total Country =
VAR CurrentRegionRevenue = [Revenue]
VAR TotalCountryRevenue =
    CALCULATE(
        [Revenue],
        REMOVEFILTERS('Sales Territory'[Region])
    )
RETURN
    DIVIDE(
        CurrentRegionRevenue,
        TotalCountryRevenue
    )
```

Add the **Revenue % Total Country** measure to the **Sales** table and then format it as a percentage with two decimal places. Add the new measure to the matrix visual.

Reseller Revenue					
Group	Country	Region	Revenue	Revenue % Total Region	Revenue % Total Country
Europe	France	France	\$4,607,537.94	5.73%	100.00%
		Total	\$4,607,537.94	5.73%	100.00%
	Germany	Germany	\$1,983,988.04	2.47%	100.00%
		Total	\$1,983,988.04	2.47%	100.00%
	United Kingdom	United Kingdom	\$4,279,008.83	5.32%	100.00%
		Total	\$4,279,008.83	5.32%	100.00%
	Total		\$10,870,534.80	13.51%	100.00%
North America	Canada	Canada	\$14,377,925.60	17.87%	100.00%
		Total	\$14,377,925.60	17.87%	100.00%
	United States	Central	\$7,906,008.18	9.83%	14.75%
		Northeast	\$6,932,842.01	8.62%	12.93%
		Northwest	\$12,435,076.00	15.46%	23.20%
		Southeast	\$7,867,416.23	9.78%	14.68%
		Southwest	\$18,466,458.79	22.95%	34.45%
		Total	\$53,607,801.21	66.63%	100.00%
	Total		\$67,985,726.81	84.51%	100.00%
Pacific	Australia	Australia	\$1,594,335.38	1.98%	100.00%
		Total	\$1,594,335.38	1.98%	100.00%
	Total		\$1,594,335.38	1.98%	100.00%
Total			\$80,450,596.98	100.00%	100.00%

Notice that all values, except those values for United States regions, are 100 percent. The reason is because, at the Adventure Works company, the United States has regions, while all other countries/regions do not.

Note

Tabular models don't support ragged hierarchies, which are hierarchies with variable depths. Therefore, it's a common design approach to repeat parent (or other ancestor) values at lower levels of the hierarchy. For example, Australia doesn't have a region, so the country/region value is repeated as the region name. It's always better to store a meaningful value instead of BLANK.

The next example is last measure that you will create. Add the **Revenue % Total Group measure**, and then format it as a percentage with two decimal places. Then, add the new measure to the matrix visual.

DAX

```
Revenue % Total Group =  
VAR CurrentRegionRevenue = [Revenue]  
VAR TotalGroupRevenue =  
    CALCULATE(  
        [Revenue],  
        REMOVEFILTERS(  
            'Sales Territory'[Region],  
            'Sales Territory'[Country]  
        )  
    )  
RETURN  
    DIVIDE(  
        CurrentRegionRevenue,  
        TotalGroupRevenue  
    )
```

Reseller Revenue

Group	Country	Region	Revenue	Revenue % Total Region	Revenue % Total Country	Revenue % Total Group	
Europe	France	France	\$4,607,537.94	5.73%	100.00%	42.39%	
		Total	\$4,607,537.94	5.73%	100.00%	42.39%	
	Germany	Germany	\$1,983,988.04	2.47%	100.00%	18.25%	
		Total	\$1,983,988.04	2.47%	100.00%	18.25%	
	United Kingdom	United Kingdom	\$4,279,008.83	5.32%	100.00%	39.36%	
		Total	\$4,279,008.83	5.32%	100.00%	39.36%	
Total			\$10,870,534.80	13.51%	100.00%	100.00%	
North America	Canada	Canada	\$14,377,925.60	17.87%	100.00%	21.15%	
		Total	\$14,377,925.60	17.87%	100.00%	21.15%	
	United States	Central	\$7,906,008.18	9.83%	14.75%	11.63%	
		Northeast	\$6,932,842.01	8.62%	12.93%	10.20%	
		Northwest	\$12,435,076.00	15.46%	23.20%	18.29%	
		Southeast	\$7,867,416.23	9.78%	14.68%	11.57%	
		Southwest	\$18,466,458.79	22.95%	34.45%	27.16%	
		Total	\$53,607,801.21	66.63%	100.00%	78.85%	
	Total			\$67,985,726.81	84.51%	100.00%	100.00%
Pacific	Australia	Australia	\$1,594,335.38	1.98%	100.00%	100.00%	
		Total	\$1,594,335.38	1.98%	100.00%	100.00%	
	Total			\$1,594,335.38	1.98%	100.00%	100.00%
Total				\$80,450,596.98	100.00%	100.00%	100.00%

When you remove filters from the **Region** and **Country** columns in the **Sales Territory** table, the [measure will calculate the region revenue as a ratio](#) of its group's revenue.

Preserve filters

You can use the **KEEPFILTERS** DAX function as a [filter expression in the CALCULATE function to preserve filters](#).

To observe how to accomplish this task, [switch to Page 1](#) of the report. Then, [modify the Revenue Red measure definition to use the KEEPFILTERS function](#).

DAX

```
Revenue Red =
CALCULATE(
    [Revenue],
    KEEPFILTERS('Product'[Color] = "Red")
)
```

Color	Revenue	Revenue Red
Black	\$38,236,124.06	
Blue	\$9,602,850.97	
Multi	\$649,030.25	
NA	\$1,099,303.91	
Red	\$21,597,890.81	\$21,597,890.81
Silver	\$19,777,339.95	
Silver/Black	\$147,483.91	
White	\$29,745.13	
Yellow	\$18,669,505.22	
Total	\$109,809,274.20	\$21,597,890.81

In the table visual, notice that only one **Revenue Red** value exists. The reason is because the Boolean filter expression preserves existing filters on the **Color** column in the **Product** table. The reason why colors other than red are BLANK is because the filter contexts and the filter expressions are combined for these two filters. The color black and color red are intersected, and because both can't be TRUE at the same time, the expression is filtered by no product rows. It's only possible that both red filters can be TRUE at the same time, which explains why the one **Revenue Red** value is shown.

Use inactive relationships

An inactive model relationship can only propagate filters when the **USERELATIONSHIP** DAX function is passed as a filter expression to the **CALCULATE** function. When you use this function to engage an inactive relationship, the active relationship will automatically become inactive.

Review an example of a measure definition that uses an inactive relationship to calculate the **Revenue** measure by shipped dates:

DAX

```
Revenue Shipped =
CALCULATE (
    [Revenue],
    USERELATIONSHIP('Date'[DateKey], Sales[ShipDateKey])
)
```

Modify relationship behavior

You can modify the model relationship behavior when an expression is evaluated by passing the [CROSSFILTER](#) DAX function as a filter expression to the CALCULATE function. It's an advanced capability.

The **CROSSFILTER** function can modify filter directions (from both to single or from single to both) and even disable a relationship.

Examine filter context

The **VALUES** DAX function lets your formulas determine what values are in filter context.

The VALUES function syntax is as follows:

DAX

```
VALUES(<TableNameOrColumnName>)
```

The function requires passing in a table reference or a column reference. When you pass in a table reference, it returns a table object with the same columns that contain rows for what's in filter context. When you pass in a column reference, it returns a single-column table of unique values that are in filter context.

The function always returns a table object and it's possible for a table to contain multiple rows. Therefore, to test whether a specific value is in filter context, your formula must first test that the VALUES function returns a single row. Two functions can help you accomplish this task: the [HASONEVALUE](#) and the [SELECTEDVALUE](#) DAX functions.

The HASONEVALUE function returns TRUE when a given column reference has been filtered down to a single value.

The SELECTEDVALUE function simplifies the task of determining what a single value could be. When the function is passed a column reference, it'll return a single value, or when more than one value is in filter context, it'll return BLANK (or an alternate value that you pass to the function).

In the following example, you will use the HASONEVALUE function. Add the following measure, which calculates sales commission, to the **Sales** table. Note that, at Adventure Works, the commission rate is 10 percent of revenue for all countries/regions except the United States. In the United States, salespeople earn 15 percent commission. Format the measure as currency with two decimal places, and then add it to the table that is found on **Page 3** of the report.

DAX

```
Sales Commission =  
[Revenue]  
    * IF(  
        HASONEVALUE('Sales Territory'[Country]),  
        IF(  
            VALUES('Sales Territory'[Country]) = "United States",  
            0.15,  
            0.1  
        )  
    )
```

Region	Revenue	Sales Commission
Australia	\$10,655,335.96	\$1,065,533.60
Canada	\$16,355,770.46	\$1,635,577.05
Central	\$7,909,009.01	\$1,186,351.35
France	\$7,251,555.65	\$725,155.56
Germany	\$4,878,300.38	\$487,830.04
Northeast	\$6,939,374.48	\$1,040,906.17
Northwest	\$16,084,942.55	\$2,412,741.38
Southeast	\$7,879,655.07	\$1,181,948.26
Southwest	\$24,184,609.60	\$3,627,691.44
United Kingdom	\$7,670,721.04	\$767,072.10
Total	\$109,809,274.20	

Notice that the total **Sales Commission** result is BLANK. The reason is because multiple values are in filter context for the **Country** column in the **Sales Territory** table. In this case, the HASONEVALUE function returns FALSE, which results in the **Revenue** measure being multiplied by BLANK (a value multiplied by BLANK is BLANK). To produce a total, you will need to use an iterator function, which is explained later in this module.

Three other functions that you can use to test filter state are:

- [ISFILTERED](#) - Returns TRUE when a passed-in column reference is *directly* filtered.

- [ISCROSSFILTERED](#) - Returns TRUE when a passed-in column reference is *indirectly* filtered. A column is cross-filtered when a filter that is applied to another column in the same table, or in a related table, affects the reference column by filtering it.
- [ISINSCOPE](#) - Returns TRUE when a passed-in column reference is the level in a hierarchy of levels.

Return to **Page 2** of the report, and then modify the **Revenue % Total Country** measure definition to test that the **Region** column in the **Sales Territory** table is in scope. If it's not in scope, the measure result should be BLANK.

DAX

```
Revenue % Total Country =
VAR CurrentRegionRevenue = [Revenue]
VAR TotalCountryRevenue =
    CALCULATE(
        [Revenue],
        REMOVEFILTERS('Sales Territory'[Region])
    )
RETURN
    IF(
        ISINSCOPE('Sales Territory'[Region]),
        DIVIDE(
            CurrentRegionRevenue,
            TotalCountryRevenue
        )
    )
```

Reseller Revenue						
Group	Country	Region	Revenue	Revenue % Total Region	Revenue % Total Country	Revenue % Total Group
Europe	France	France	\$4,607,537.94	5.73%	100.00%	42.39%
		Total	\$4,607,537.94	5.73%		42.39%
	Germany	Germany	\$1,983,988.04	2.47%	100.00%	18.25%
		Total	\$1,983,988.04	2.47%		18.25%
	United Kingdom	United Kingdom	\$4,279,008.83	5.32%	100.00%	39.36%
		Total	\$4,279,008.83	5.32%		39.36%
	Total		\$10,870,534.80	13.51%		100.00%
North America	Canada	Canada	\$14,377,925.60	17.87%	100.00%	21.15%
		Total	\$14,377,925.60	17.87%		21.15%
	United States	Central	\$7,906,008.18	9.83%	14.75%	11.63%
		Northeast	\$6,932,842.01	8.62%	12.93%	10.20%
		Northwest	\$12,435,076.00	15.46%	23.20%	18.29%
		Southeast	\$7,867,416.23	9.78%	14.68%	11.57%
		Southwest	\$18,466,458.79	22.95%	34.45%	27.16%
		Total	\$53,607,801.21	66.63%		78.85%
	Total		\$67,985,726.81	84.51%		100.00%
Pacific	Australia	Australia	\$1,594,335.38	1.98%	100.00%	100.00%
		Total	\$1,594,335.38	1.98%		100.00%
	Total		\$1,594,335.38	1.98%		100.00%
Total			\$80,450,596.98	100.00%		100.00%

In the matrix visual, notice that **Revenue % Total Country** values are now only displayed when a region is in scope.

Perform context transition

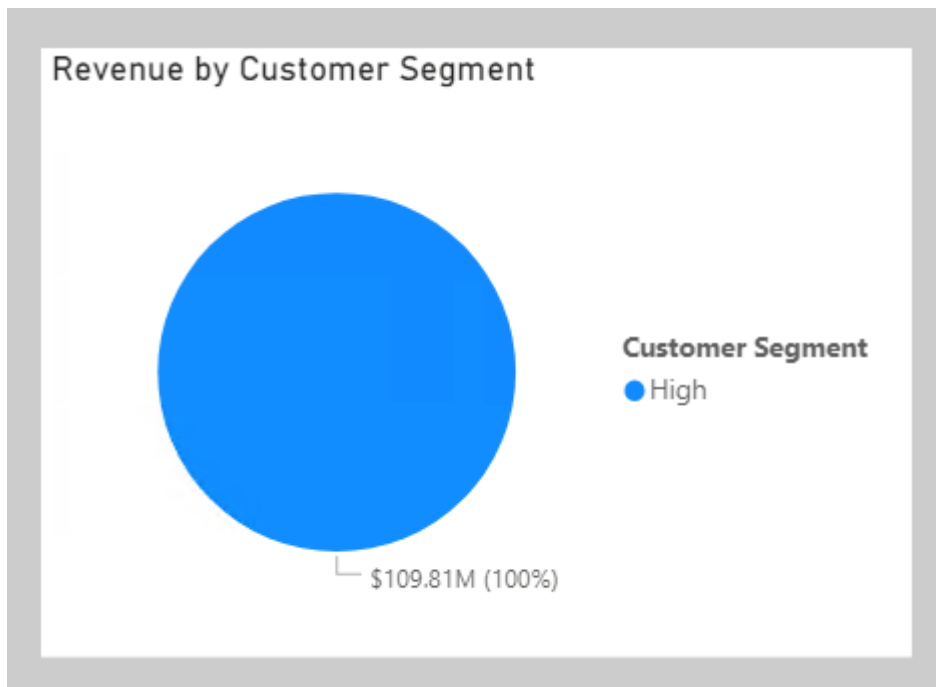
What happens when a measure or measure expression is evaluated within row context? This scenario can happen in a calculated column formula or when an expression in an iterator function is evaluated.

In the following example, you will add a calculated column to the **Customer** table to classify customers into a loyalty class. The scenario is simple: When the revenue that is produced by the customer is less than \$2500, the customer is classified as **Low**; otherwise they're classified as **High**.

DAX

```
Customer Segment =
VAR CustomerRevenue = SUM(Sales[Sales Amount])
RETURN
    IF(CustomerRevenue < 2500, "Low", "High")
```

On **Page 4** of the report, add the **Customer Segment** column as the legend of the pie chart.



Notice that only one **Customer Segment** value exists. The reason is because the calculated column formula produces an incorrect result: Each customer is assigned the value of **High** because the expression `SUM(Sales[Sales Amount])` isn't evaluated in a filter context. Consequently, each customer is assessed on the sum of *every* **Sales Amount** column value in the **Sales** table.

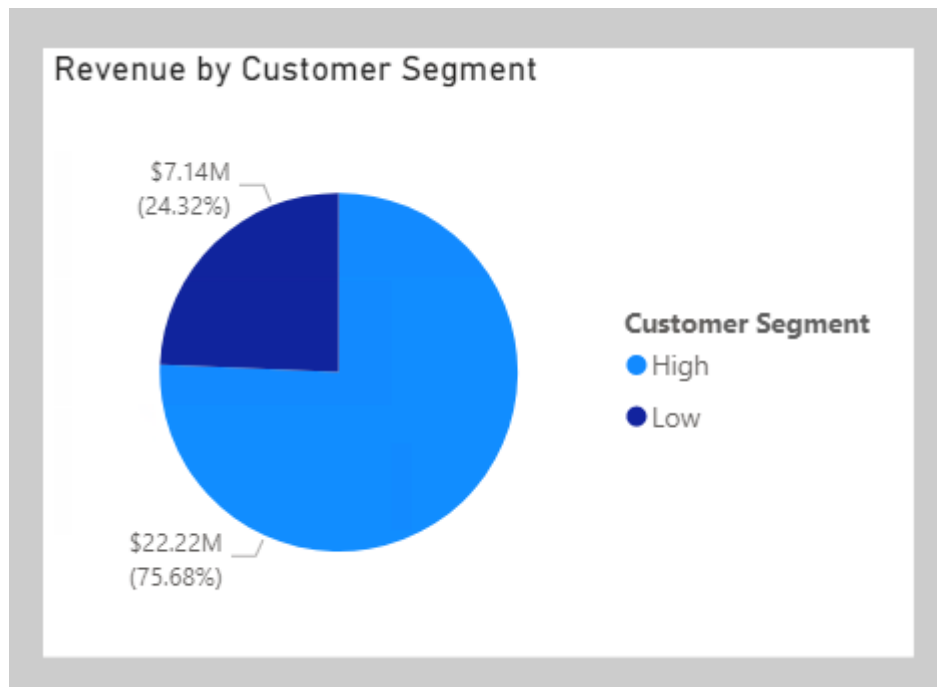
To force the evaluation of the `SUM(Sales[Sales Amount])` expression *for each customer*, a context transition must take place that applies the row context column values to filter context. You can accomplish this transition by using the `CALCULATE` function without passing in filter expressions.

Modify the calculated column definition so that it produces the correct result.

DAX

```
Customer Segment =  
VAR CustomerRevenue = CALCULATE(SUM(Sales[Sales Amount]))  
RETURN  
    IF(CustomerRevenue < 2500, "Low", "High")
```

In the pie chart visual add the new calculated column to the Legend well, verify that two pie segments now display.



In this case, the CALCULATE function applies row context values as filters, known as *context transition*. To be accurate, the process doesn't quite work that way when a unique column is on the table. When a unique column is on the table, you only need to apply a filter on that column to make the transition happen. In this case, Power BI applies a filter on the **CustomerKey** column for the value in row context.

If you reference measures in an expression that's evaluated in row context, context transition is automatic. Thus, you don't need to pass measure references to the CALCULATE function.

Modify the calculated column definition, which references the **Revenue** measure, and notice that it continues to produce the correct result.

DAX

```
Customer Segment =
VAR CustomerRevenue = [Revenue]
RETURN
    IF(CustomerRevenue < 2500, "Low", "High")
```

Now, you can complete the **Sales Commission** measure formula. To produce a total, you need to use an iterator function to iterate over all regions in filter context. The iterator function expression must use the CALCULATE function to transition the row context to the filter context. Notice that it no longer needs to test whether a single **Country** column value in the **Sales Territory** table is in filter context because it's known to be filtering by a single

country/region (because it's iterating over the regions in filter context and a region belongs to only one country/region).

Switch to **Page 3** of the report, and then modify the **Sales Commission** measure definition to use the SUMX iterator function:

DAX

```
Sales Commission =  
SUMX(  
    VALUES('Sales Territory'[Region]),  
    CALCULATE(  
        [Revenue]  
        * IF(  
            VALUES('Sales Territory'[Country]) = "United States",  
            0.15,  
            0.1  
        )  
    )  
)
```

The table visual now displays a sales commission total for all regions.

Region	Revenue	Sales Commission
Australia	\$10,655,335.96	\$1,065,533.60
Canada	\$16,355,770.46	\$1,635,577.05
Central	\$7,909,009.01	\$1,186,351.35
France	\$7,251,555.65	\$725,155.56
Germany	\$4,878,300.38	\$487,830.04
Northeast	\$6,939,374.48	\$1,040,906.17
Northwest	\$16,084,942.55	\$2,412,741.38
Southeast	\$7,879,655.07	\$1,181,948.26
Southwest	\$24,184,609.60	\$3,627,691.44
United Kingdom	\$7,670,721.04	\$767,072.10
Total	\$109,809,274.20	\$14,130,806.96

Knowledge check

Summary

In this module, you learned about filter context. You also learned why and how to work with the CALCULATE function to modify filter context by passing in filters. These filters can add or overwrite the filter context, and they can modify filter context when you are passing in special functions like REMOVEFILTERS or KEEPFILTERS. Additionally, you learned that the CALCULATE function can transition row context to filter context, which can be required when you are using measure expressions in calculated columns or iterator functions.

Use DAX time intelligence functions in Power BI Desktop models